

WebGPU Shading Language

W3C Candidate Recommendation Draft, 16 July 2026



▼ More details about this document

This version:

<https://www.w3.org/TR/2026/CRD-WGSL-20260716/>

Latest published version:

<https://www.w3.org/TR/WGSL/>

Editor's Draft:

<https://gpuweb.github.io/gpuweb/wgsl/>

Previous Versions:

<https://www.w3.org/TR/2026/CRD-WGSL-20260714/>

History:

<https://www.w3.org/standards/history/WGSL/>

Feedback:

public-gpu@w3.org with subject line “[WGSL] ... message topic ...” ([archives](#))

[GitHub](#)

Editors:

[Alan Baker](#) (Google)

[Mehmet Oguz Derin](#)

[David Neto](#) (Google)

Former Editors:

[Myles C. Maxfield](#) (Apple Inc.)

[dan sinclair](#) (Google)

Participate:

[File an issue](#) ([open issues](#))

Test Suite:

[WebGPU CTS shader/](#)

Copyright © 2026 World Wide Web Consortium. W3C[®] [liability](#), [trademark](#) and [permissive document license](#) rules apply.

Abstract

Shading language for WebGPU.

Status of this document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the [W3C standards and drafts index](#).

Feedback and comments on this specification are welcome. [GitHub Issues](#) are preferred for discussion on this specification. Alternatively, you can send comments to the GPU for the Web Working Group’s mailing-list, public-gpu@w3.org ([archives](#)). This draft highlights some of the pending issues that are still to be discussed in the working group. No decision has been taken on the outcome of these issues including whether they are valid.

This document was published by the [GPU for the Web Working Group](#) as a Candidate Recommendation Draft using the [Recommendation track](#). This document will remain a Candidate Recommendation at least until 28 February 2025.

The group expects to demonstrate implementation of each feature in at least two deployed browsers on top of modern GPU system APIs. The test suite will be used to build an implementation report.

Publication as a Candidate Recommendation does not imply endorsement by W3C and its Members. A Candidate Recommendation Draft integrates changes from the previous Candidate Recommendation that the Working Group intends to include in a subsequent Candidate Recommendation Snapshot.

This document is maintained and updated at any time. Some parts of this document are work in progress.

This document was produced by a group operating under the ‘[W3C Patent Policy](#)’. W3C maintains a [public list of any patent disclosures](#) made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains [Essential Claim\(s\)](#) must disclose the information in accordance with [section 6 of the W3C Patent Policy](#).

This document is governed by the [18 August 2025 W3C Process Document](#).

Table of Contents

1	Introduction
1.1	Overview
1.2	Syntax Notation
1.3	Mathematical Terms and Notation
2	WGSL Module
2.1	Shader Lifecycle
2.2	Errors
2.3	Diagnostics
2.3.1	Diagnostic Processing
2.3.2	Filterable Triggering Rules
2.3.3	Diagnostic Filtering
2.4	Limits
3	Textual Structure
3.1	Parsing
3.2	Blankspace and Line Breaks
3.3	Comments
3.4	Tokens
3.5	Literals
3.5.1	Boolean Literals
3.5.2	Numeric Literals
3.6	Keywords
3.7	Identifiers
3.7.1	Identifier Comparison
3.8	Context-Dependent Names
3.8.1	Attribute Names
3.8.2	Built-in Value Names
3.8.3	Diagnostic Rule Names
3.8.4	Diagnostic Severity Control Names
3.8.5	Extension Names

3.8.6	Interpolation Type Names
3.8.7	Interpolation Sampling Names
3.8.8	Swizzle Names
3.9	Template Lists
4	Directives
4.1	Extensions
4.1.1	Enable Extensions
4.1.2	Language Extensions
4.2	Global Diagnostic Filter
5	Declaration and Scope
6	Types
6.1	Type Checking
6.1.1	Type Rule Tables
6.1.2	Conversion Rank
6.1.3	Overload Resolution
6.2	Plain Types
6.2.1	Abstract Numeric Types
6.2.2	Boolean Type
6.2.3	Integer Types
6.2.4	Floating Point Types
6.2.5	Scalar Types
6.2.6	Vector Types
6.2.7	Matrix Types
6.2.8	Atomic Types
6.2.9	Array Types
6.2.10	Structure Types
6.2.11	Composite Types
6.2.12	Constructible Types
6.2.13	Fixed-Footprint Types
6.3	Buffer Types
6.4	Enumeration Types
6.4.1	Predeclared enumerants
6.5	Memory Views
6.5.1	Storable Types
6.5.2	Host-shareable Types
6.5.3	Reference and Pointer Types
6.5.4	Valid and Invalid Memory References
6.5.5	Originating Variable
6.5.6	Out-of-Bounds Access
6.5.7	Use Cases for References and Pointers
6.5.8	Forming Reference and Pointer Values
6.5.9	Comparison with References and Pointers in Other Languages
6.6	Texture and Sampler Types
6.6.1	Texel Formats
6.6.2	Sampled Texture Types
6.6.3	Multisampled Texture Types
6.6.4	External Sampled Texture Types
6.6.5	Storage Texture Types

6.6.6	Depth Texture Types
6.6.7	Sampler Type
6.7	AllTypes Type
6.8	Type Aliases
6.9	Type Specifier Grammar
6.10	Predeclared Types and Type-Generators Summary
7	Variable and Value Declarations
7.1	Variables vs Values
7.2	Value Declarations
7.2.1	const Declarations
7.2.2	override Declarations
7.2.3	let Declarations
7.3	var Declarations
7.4	Variable and Value Declaration Grammar Summary
8	Expressions
8.1	Early Evaluation Expressions
8.1.1	const Expressions
8.1.2	override Expressions
8.2	Indeterminate values
8.3	Literal Value Expressions
8.4	Parenthesized Expressions
8.5	Composite Value Decomposition Expressions
8.5.1	Vector Access Expression
8.5.1.1	Vector Single Component Selection
8.5.1.2	Vector Multiple Component Selection
8.5.1.3	Component Reference from Vector Memory View
8.5.2	Matrix Access Expression
8.5.3	Array Access Expression
8.5.4	Structure Access Expression
8.6	Logical Expressions
8.7	Arithmetic Expressions
8.8	Comparison Expressions
8.9	Bit Expressions
8.10	Function Call Expression
8.11	Variable Identifier Expression
8.12	Formal Parameter Expression
8.13	Address-Of Expression
8.14	Indirection Expression
8.15	Identifier Expressions for Value Declarations
8.16	Enumeration Expressions
8.17	Type Expressions
8.18	Expression Grammar Summary
8.19	Operator Precedence and Associativity
9	Statements
9.1	Compound Statement
9.2	Assignment Statement
9.2.1	Simple Assignment

9.2.2	Phony Assignment
9.2.3	Compound Assignment
9.3	Increment and Decrement Statements
9.4	Control Flow
9.4.1	If Statement
9.4.2	Switch Statement
9.4.3	Loop Statement
9.4.4	For Statement
9.4.5	While Statement
9.4.6	Break Statement
9.4.7	Break-If Statement
9.4.8	Continue Statement
9.4.9	Continuing Statement
9.4.10	Return Statement
9.4.11	Discard Statement
9.5	Function Call Statement
9.6	Statements Grammar Summary
9.7	Statements Behavior Analysis
9.7.1	Rules
9.7.2	Notes
9.7.3	Examples

10 Assertions

10.1	Const Assertion Statement
------	---------------------------

11 Functions

11.1	Declaring a User-defined Function
11.2	Function Calls
11.3	const Functions
11.4	Restrictions on Functions
11.4.1	Alias Analysis
11.4.1.1	Root Identifier
11.4.1.2	Aliasing

12 Attributes

12.1	align
12.2	binding
12.3	blend_src
12.4	builtin
12.5	const
12.6	diagnostic
12.7	group
12.8	id
12.9	interpolate
12.10	invariant
12.11	location
12.12	must_use
12.13	size
12.14	subgroup_size
12.15	workgroup_size

12.16 Shader Stage Attributes

12.16.1 vertex

12.16.2 fragment

12.16.3 compute

13 Entry Points

13.1 Shader Stages

13.2 Entry Point Declaration

13.2.1 Function Attributes for Entry Points

13.3 Shader Interface

13.3.1 Inter-stage Input and Output Interface

13.3.1.1 Built-in Inputs and Outputs

13.3.1.1.1 clip_distances

13.3.1.1.2 frag_depth

13.3.1.1.3 front_facing

13.3.1.1.4 global_invocation_id

13.3.1.1.5 global_invocation_index

13.3.1.1.6 instance_index

13.3.1.1.7 local_invocation_id

13.3.1.1.8 local_invocation_index

13.3.1.1.9 num_workgroups

13.3.1.1.10 position

13.3.1.1.11 primitive_index

13.3.1.1.12 sample_index

13.3.1.1.13 sample_mask

13.3.1.1.14 vertex_index

13.3.1.1.15 workgroup_id

13.3.1.1.16 workgroup_index

13.3.1.1.17 subgroup_invocation_id

13.3.1.1.18 subgroup_size

13.3.1.1.19 subgroup_id

13.3.1.1.20 num_subgroups

13.3.1.2 User-defined Inputs and Outputs

13.3.1.3 Input-output Locations

13.3.1.4 Interpolation

13.3.2 Resource Interface

13.3.3 Resource Layout Compatibility

13.3.4 Buffer Binding Determines Runtime-Sized Array Element Count

14 Memory

14.1 Memory Locations

14.2 Memory Access Mode

14.3 Address Spaces

14.4 Memory Layout

14.4.1 Alignment and Size

14.4.2 Structure Member Layout

14.4.3 Array Layout Examples

14.4.4 Internal Layout of Values

14.4.5 Address Space Layout Constraints

14.5 Memory Model

14.5.1 Memory Operation

14.5.2 Memory Model Reference

- 14.5.3 Scoped Operations
- 14.5.4 Memory Semantics
- 14.5.5 Private vs Non-private

15 Execution

- 15.1 Program Order Within an Invocation
- 15.2 Uniformity
 - 15.2.1 Terminology and Concepts
 - 15.2.2 Uniformity Analysis Overview
 - 15.2.3 Analyzing the Uniformity Requirements of a Function
 - 15.2.4 Pointer Desugaring
 - 15.2.5 Function-scope Variable Value Analysis
 - 15.2.6 Uniformity Rules for Statements
 - 15.2.7 Uniformity Rules for Function Calls
 - 15.2.8 Uniformity Rules for Expressions
 - 15.2.9 Annotating the Uniformity of Every Point in the Control-flow
 - 15.2.10 Examples
 - 15.2.10.1 Invalid textureSample Function Call
 - 15.2.10.2 Function-scope Variable Uniformity
 - 15.2.10.3 Composite Value Analysis Limitations
 - 15.2.10.4 Uniformity in a Loop
 - 15.2.10.5 User-defined Function Calls
- 15.3 Compute Shaders and Workgroups
- 15.4 Fragment Shaders and Helper Invocations
- 15.5 Subgroups
- 15.6 Collective Operations
 - 15.6.1 Barriers
 - 15.6.2 Derivatives
 - 15.6.3 Subgroup Operations
 - 15.6.4 Quad Operations
- 15.7 Floating Point Evaluation
 - 15.7.1 Overview of IEEE-754
 - 15.7.2 Differences from IEEE-754
 - 15.7.3 Floating Point Rounding and Overflow
 - 15.7.4 Floating Point Accuracy
 - 15.7.4.1 Accuracy of Concrete Floating Point Expressions
 - 15.7.4.2 Accuracy of AbstractFloat Expressions
 - 15.7.5 Reassociation and Fusion
 - 15.7.6 Floating Point Conversion
 - 15.7.7 Domains of Floating Point Expressions and Built-in Functions

16 Keyword and Token Summary

- 16.1 Keyword Summary
- 16.2 Reserved Words
- 16.3 Syntactic Tokens

17 Built-in Functions

- 17.1 Constructor Built-in Functions
 - 17.1.1 Zero Value Built-in Functions
 - 17.1.2 Value Constructor Built-in Functions
 - 17.1.2.1 array

17.1.2.2	bool
17.1.2.3	f16
17.1.2.4	f32
17.1.2.5	i32
17.1.2.6	mat2x2
17.1.2.7	mat2x3
17.1.2.8	mat2x4
17.1.2.9	mat3x2
17.1.2.10	mat3x3
17.1.2.11	mat3x4
17.1.2.12	mat4x2
17.1.2.13	mat4x3
17.1.2.14	mat4x4
17.1.2.15	Structures
17.1.2.16	u32
17.1.2.17	vec2
17.1.2.18	vec3
17.1.2.19	vec4
17.2	Bit Reinterpretation Built-in Functions
17.2.1	bitcast
17.3	Logical Built-in Functions
17.3.1	all
17.3.2	any
17.3.3	select
17.4	Array Built-in Functions
17.4.1	arrayLength
17.5	Numeric Built-in Functions
17.5.1	abs
17.5.2	acos
17.5.3	acosh
17.5.4	asin
17.5.5	asinh
17.5.6	atan
17.5.7	atanh
17.5.8	atan2
17.5.9	ceil
17.5.10	clamp
17.5.11	cos
17.5.12	cosh
17.5.13	countLeadingZeros
17.5.14	countOneBits
17.5.15	countTrailingZeros
17.5.16	cross
17.5.17	degrees
17.5.18	determinant
17.5.19	distance
17.5.20	dot
17.5.21	dot4U8Packed
17.5.22	dot4I8Packed

17.5.23	exp
17.5.24	exp2
17.5.25	extractBits (signed)
17.5.26	extractBits (unsigned)
17.5.27	faceForward
17.5.28	firstLeadingBit (signed)
17.5.29	firstLeadingBit (unsigned)
17.5.30	firstTrailingBit
17.5.31	floor
17.5.32	fma
17.5.33	fract
17.5.34	frexp
17.5.35	insertBits
17.5.36	inverseSqrt
17.5.37	ldexp
17.5.38	length
17.5.39	log
17.5.40	log2
17.5.41	max
17.5.42	min
17.5.43	mix
17.5.44	modf
17.5.45	normalize
17.5.46	pow
17.5.47	quantizeToF16
17.5.48	radians
17.5.49	reflect
17.5.50	refract
17.5.51	reverseBits
17.5.52	round
17.5.53	saturate
17.5.54	sign
17.5.55	sin
17.5.56	sinh
17.5.57	smoothstep
17.5.58	sqrt
17.5.59	step
17.5.60	tan
17.5.61	tanh
17.5.62	transpose
17.5.63	trunc
17.6	Derivative Built-in Functions
17.6.1	dpx
17.6.2	dpxCoarse
17.6.3	dpxFine
17.6.4	dpdy
17.6.5	dpdyCoarse
17.6.6	dpdyFine

17.6.7	fwidth
17.6.8	fwidthCoarse
17.6.9	fwidthFine
17.7	Texture Built-in Functions
17.7.1	textureDimensions
17.7.2	textureGather
17.7.3	textureGatherCompare
17.7.4	textureLoad
17.7.5	textureNumLayers
17.7.6	textureNumLevels
17.7.7	textureNumSamples
17.7.8	textureSample
17.7.9	textureSampleBias
17.7.10	textureSampleCompare
17.7.11	textureSampleCompareLevel
17.7.12	textureSampleGrad
17.7.13	textureSampleLevel
17.7.14	textureSampleBaseClampToEdge
17.7.15	textureStore
17.8	Atomic Built-in Functions
17.8.1	atomicLoad
17.8.2	atomicStore
17.8.3	Atomic Read-modify-write Arithmetic and Logical Functions
17.8.3.1	atomicAdd
17.8.3.2	atomicSub
17.8.3.3	atomicMax
17.8.3.4	atomicMin
17.8.3.5	atomicAnd
17.8.3.6	atomicOr
17.8.3.7	atomicXor
17.8.4	atomicExchange
17.8.5	atomicCompareExchangeWeak
17.9	Data Packing Built-in Functions
17.9.1	pack4x8snorm
17.9.2	pack4x8unorm
17.9.3	pack4xI8
17.9.4	pack4xU8
17.9.5	pack4xI8Clamp
17.9.6	pack4xU8Clamp
17.9.7	pack2x16snorm
17.9.8	pack2x16unorm
17.9.9	pack2x16float
17.10	Data Unpacking Built-in Functions
17.10.1	unpack4x8snorm
17.10.2	unpack4x8unorm
17.10.3	unpack4xI8
17.10.4	unpack4xU8
17.10.5	unpack2x16snorm
17.10.6	unpack2x16unorm

17.10.7	unpack2x16float
17.11	Synchronization Built-in Functions
17.11.1	storageBarrier
17.11.2	textureBarrier
17.11.3	workgroupBarrier
17.11.4	workgroupUniformLoad
17.12	Subgroup Built-in Functions
17.12.1	subgroupAdd
17.12.1.1	subgroupExclusiveAdd
17.12.1.2	subgroupInclusiveAdd
17.12.2	subgroupAll
17.12.3	subgroupAnd
17.12.4	subgroupAny
17.12.5	subgroupBallot
17.12.6	subgroupBroadcast
17.12.6.1	subgroupBroadcastFirst
17.12.7	subgroupElect
17.12.8	subgroupMax
17.12.9	subgroupMin
17.12.10	subgroupMul
17.12.10.1	subgroupExclusiveMul
17.12.10.2	subgroupInclusiveMul
17.12.11	subgroupOr
17.12.12	subgroupShuffle
17.12.12.1	subgroupShuffleDown
17.12.12.2	subgroupShuffleUp
17.12.12.3	subgroupShuffleXor
17.12.13	subgroupXor
17.13	Quad Operations
17.13.1	quadBroadcast
17.13.2	quadSwapDiagonal
17.13.3	quadSwapX
17.13.4	quadSwapY
17.14	Buffer View Built-in Functions
17.14.1	bufferView
17.14.2	bufferArrayView
17.14.3	bufferLength

18 Grammar for Recursive Descent Parsing

Appendix A: The text/wgsl Media Type

Conformance

Document conventions

Conformant Algorithms

Index

Terms defined by this specification

Terms defined by reference

References

Normative References

Non-Normative References

§ 1. Introduction

WebGPU Shading Language (WGSL) is the shader language for [\[WebGPU\]](#). That is, an application using the WebGPU API uses WGSL to express the programs, known as shaders, that run on the GPU.

EXAMPLE 1

```

// A fragment shader which lights textured geometry with point lights.

// Lights from a storage buffer binding.
struct PointLight {
    position : vec3f,
    color : vec3f,
}

struct LightStorage {
    pointCount : u32,
    point : array<PointLight>,
}

@group(0) @binding(0) var<storage> lights : LightStorage;

// Texture and sampler.
@group(1) @binding(0) var baseColorSampler : sampler;
@group(1) @binding(1) var baseColorTexture : texture_2d<f32>;

// Function arguments are values from the vertex shader.
@fragment
fn fragmentMain(@location(0) worldPos : vec3f,
                @location(1) normal : vec3f,
                @location(2) uv : vec2f) -> @location(0) vec4f {
    // Sample the base color of the surface from a texture.
    let baseColor = textureSample(baseColorTexture, baseColorSampler, uv);

    let N = normalize(normal);
    var surfaceColor = vec3f(0);

    // Loop over the scene point lights.
    for (var i = 0u; i < lights.pointCount; i++) {
        let worldToLight = lights.point[i].position - worldPos;
        let dist = length(worldToLight);
        let dir = normalize(worldToLight);

        // Determine the contribution of this light to the surface color.
        let radiance = lights.point[i].color * (1 / pow(dist, 2));
        let nDotL = max(dot(N, dir), 0);

        // Accumulate light contribution to the surface color.
        surfaceColor += baseColor.rgb * radiance * nDotL;
    }

    // Return the accumulated surface color.
    return vec4(surfaceColor, baseColor.a);
}

```

§ 1.1. Overview

WebGPU issues a unit of work to the GPU in the form of a [GPU command](#). WGSL is concerned with two kinds of GPU commands:

- a *draw command* executes a [render pipeline](#) in the context of [inputs](#), [outputs](#), and attached [resources](#).
- a *dispatch command* executes a [compute pipeline](#) in the context of [inputs](#) and attached [resources](#).

Both kinds of pipelines use shaders written in WGSL.

A *shader* is the portion of a WGSL program that executes a [shader stage](#) in a pipeline. A shader comprises:

- An [entry point function](#).
- The transitive closure of all called functions, starting with the entry point. This set includes both [user-defined](#) and [built-in](#) functions. (For a more rigorous definition, see "[functions in a shader stage](#)".)
- The set of variables and constants [statically accessed](#) by all those functions.
- The set of types used to define or analyze all those functions, variables, and constants.

NOTE: A WGSL program does not require an [entry point](#); however, such a program cannot be executed by the API because an entry point is required to create a [GPUProgrammableStage](#).

When executing a shader stage, the implementation:

- Computes the values of constants declared at [module-scope](#).
- Binds [resources](#) to variables in the shader's [resource interface](#), making the contents of those resources available to the shader during execution.
- Allocates memory for other [module-scope](#) variables, and populates that memory with the specified initial values.
- Populates the formal parameters of the entry point, if they exist, with the shader stage's inputs.
- Connects the entry point [return value](#), if one exists, to the shader stage's outputs.
- Then it invokes the entry point.

A WGSL program is organized into:

- [Directives](#), which specify module-level behavior controls.
- [Functions](#), which specify execution behavior.
- [Statements](#), which are declarations or units of executable behavior.
- [Literals](#), which are text representations for pure mathematical values.
- [Constants](#), each providing a name for a value computed at a specific time.
- [Variables](#), each providing a name for memory holding a value.
- [Expressions](#), each of which combines a set of values to produce a result value.
- [Types](#), each of which describes:
 - A set of values.
 - Constraints on supported expressions.
 - The semantics of those expressions.
- [Attributes](#), which modify an object to specify extra information such as:
 - Specifying the [interfaces](#) to [entry points](#).
 - Specifying [diagnostic filters](#).

NOTE: A WGSL program is currently composed of a single [WGSL module](#).

WGSL is an imperative language: behavior is specified as a sequence of statements to execute. Statements can:

- Declare [constants](#) or [variables](#).
- Modify the contents of variables.
- Modify execution order using structured programming constructs:
 - Selective execution: [if](#) (with optional `else if` and `else` clauses), [switch](#).
 - Repetition: [loop](#), [while](#), [for](#).
 - Escaping a nested execution construct: [continue](#), [break](#), [break if](#).
 - Refactoring: [function call](#) and [return](#).
- Evaluate expressions to compute values as part of the above behaviors.
- Check [assumptions](#) at [shader creation](#) time on [constant expressions](#).

WGSL is statically typed: each value computed by a particular expression is in a specific type, determined only by examining the program source.

WGSL has types describing [booleans](#) and numbers ([integers](#) and [floating point](#)). These types can be aggregated into [composites](#) ([vectors](#), [matrices](#), [arrays](#), and [structures](#)). WGSL has special types (e.g. [atomics](#)) that provide unique operations. WGSL describes the types that can be stored in [memory](#) as [memory views](#). WGSL provides commonly used rendering types in the form of [textures and samplers](#). These types have associated [built-in functions](#) to expose commonly provided GPU hardware for graphics rendering.

WGSL has very limited implicit conversions and promotions. It only provides implicit conversions and promotions from [abstract](#) types and [buffer](#) pointers passed as arguments to [function calls](#). Converting a value from one [concrete](#) numeric or boolean type to another requires an explicit [conversion](#), [value constructor](#), or [reinterpretation of bits](#); however, WGSL does provide some limited facility to promote [scalar](#) types to [vector](#) types. This also applies to [composite](#) types.

The work of a shader stage is partitioned into one or more *invocations*, each of which executes the entry point, but under slightly different conditions. Invocations in a shader stage share access to certain variables:

- All invocations in the stage share the resources in the shader interface.
- In a [compute shader](#), invocations in the same [workgroup](#) share variables in the [workgroup address space](#). Invocations in different workgroups do not share those variables.

However, the invocations act on different sets of shader stage inputs, including built-in inputs that provide an identifying value to distinguish an invocation from its peers. Each invocation has its own independent memory space in the form of variables in the [private](#) and [function](#) address spaces.

Invocations within a shader stage execute concurrently, and may often execute in parallel. The shader author is responsible for ensuring the dynamic behavior of the invocations in a shader stage:

- Meet the [uniformity](#) requirements of certain primitive operations, including texture sampling and control barriers.
- Coordinate potentially conflicting accesses to shared variables, to avoid data races.

WGSL sometimes permits several possible behaviors for a given feature. This is a portability hazard, as different implementations may exhibit the different behaviors. The design of WGSL aims to minimize such cases, but is constrained by feasibility, and goals for achieving high performance across a broad range of devices.

Behavioral requirements are actions the implementation will perform when processing or executing a WGSL program. They describe the implementation's obligations in the contract with the programmer. The specification explicitly states these obligations when they might not be otherwise obvious.

§ 1.2. Syntax Notation

Following syntax notation describes the conventions of the syntactic grammar of WGSL:

- Italic text on both sides of a rule indicates a syntax rule.
- Bold monospace text starting and ending with single quotes (') on the right-hand side of a rule indicates keywords and tokens.
- A colon (:) in regular text registers a syntax rule.
- A vertical bar (|) in regular text indicates alternatives.
- An question mark (?) in regular text indicates that previous keyword, token, rule, or group occurs zero or one times (is optional).
- An asterisk (*) in regular text indicates that previous keyword, token, rule, or group occurs zero or more times.
- A plus (+) in regular text indicates that previous keyword, token, rule, or group occurs one or more times.
- A matching pair of opening parenthesis (() and closing parenthesis ()) in regular text indicates a group of elements.

§ 1.3. Mathematical Terms and Notation

Angles:

- By convention, angles are measured in radians.
- The reference ray for measuring angles is the ray from the origin (0,0) toward $(+\infty, 0)$.
- Let θ be the angle subtended by a comparison ray and the reference ray. Then θ increases as the comparison ray moves counterclockwise.
- There are 2π radians in a complete circle.
- Examples:
 - The angle 0 points from the origin to the right, toward (1,0)
 - The angle 2π points from the origin to the right, toward (1,0)
 - The angle $\pi/4$ points from the origin to the point (1,1)
 - The angle $\pi/2$ points from the origin to the point (0,1)
 - The angle π points from the origin to the point (-1,0)
 - The angle $(3/2)\pi$ points from the origin to the point (0,-1)

A *hyperbolic angle* is a unitless area, not an angle in the traditional sense. Specifically:

- Consider the hyperbola $x^2 - y^2 = 1$ for $x > 0$.
- Let R be a ray from the origin to some point (x, y) on the hyperbola.
- Let a be twice the area enclosed by R , the x axis, and the curve of the hyperbola itself.
- Consider a to be positive when R is above the x axis, and negative when below.

Then the area a is a hyperbolic angle such that x is the hyperbolic cosine of a , and y is the hyperbolic sine of a .

Positive infinity, denoted by $+\infty$, is a unique value strictly greater than all real numbers.

Negative infinity, denoted by $-\infty$, is a unique value strictly lower than all real numbers.

The *extended real* numbers (also known as the affinely extended real numbers) is the set of real numbers together with $+\infty$ and $-\infty$. Computers use [floating point types](#) to approximately represent the extended reals, including values for both

infinities. See [§ 15.7 Floating Point Evaluation](#).

An **interval** is a contiguous set of numbers with a lower and upper bound. Depending on context, they are sets of integers, floating point numbers, real numbers, or [extended real](#) numbers.

- The closed interval $[a,b]$ is the set of numbers x such that $a \leq x \leq b$.
- The half-open interval $[a,b)$ is the set of numbers x such that $a \leq x < b$.
- The half-open interval $(a,b]$ is the set of numbers x such that $a < x \leq b$.

The **floor expression** is defined for [extended real](#) numbers x :

- $\lfloor +\infty \rfloor = +\infty$
- $\lfloor -\infty \rfloor = -\infty$
- for real number x , $\lfloor x \rfloor = k$, where k is the unique integer such that $k \leq x < k+1$

The **ceiling expression** is defined for [extended real](#) numbers x :

- $\lceil +\infty \rceil = +\infty$
- $\lceil -\infty \rceil = -\infty$
- for real number x , $\lceil x \rceil = k$, where k is the unique integer such that $k-1 < x \leq k$

The **truncate** function is defined for [extended real](#) numbers x :

- $\text{truncate}(+\infty) = +\infty$
- $\text{truncate}(-\infty) = -\infty$
- for real number x , computes the nearest whole number whose absolute value is less than or equal to the absolute value of x :
 - $\text{truncate}(x) = \lfloor x \rfloor$ if $x \geq 0$, and $\lceil x \rceil$ if $x < 0$.

The **roundUp** function is defined for positive integers k and n as:

- $\text{roundUp}(k, n) = \lceil n \div k \rceil \times k$

The **roundDown** function is defined for positive integers k and n as:

- $\text{roundDown}(k, n) = \lfloor n \div k \rfloor \times k$

The **transpose** of an c -column r -row matrix A is the r -column c -row matrix A^T formed by copying the rows of A as the columns of A^T :

- $\text{transpose}(A) = A^T$
- $\text{transpose}(A)_{i,j} = A_{j,i}$

The transpose of a column vector is defined by interpreting the column vector as a 1-row matrix. Similarly, the transpose of a row vector is defined by interpreting the row vector as a 1-column matrix.

§ 2. WGSL Module

A WGSL program is composed of a single WGSL module.

A module is a sequence of optional [directives](#) followed by [module scope declarations](#) and [assertions](#). A module is organized into:

- [Directives](#), which specify module-level behavior controls.
- [Functions](#), which specify execution behavior.
- [Statements](#), which are declarations or units of executable behavior.
- [Literals](#), which are text representations for pure mathematical values.
- [Variables](#), each providing a name for memory holding a value.
- [Constants](#), each providing a name for a value computed at a specific time.
- [Expressions](#), each of which combines a set of values to produce a result value.
- [Types](#), each of which describes:
 - A set of values.
 - Constraints on supported expressions.
 - The semantics of those expressions.
- [Attributes](#), which modify an object to specify extra information such as:
 - Specifying the [interfaces](#) to [entry points](#).
 - Specifying [diagnostic filters](#).

```
translation_unit :
    global_directive * ( global_decl | global_assert | ';' ) *
```

```
global_decl :
    global_variable_decl ';'
  | global_value_decl ';'
  | type_alias_decl ';'
  | struct_decl
  | function_decl
```

§ 2.1. Shader Lifecycle

There are four key events in the lifecycle of a WGSL program and the shaders it may contain. The first two correspond to the WebGPU API methods used to prepare a WGSL program for execution. The last two are the start and end of execution of a shader.

The events are:

1. *Shader module creation*

- This occurs when the WebGPU [createShaderModule\(\)](#) method is called. The source text for a WGSL program is provided at this time.

2. *Pipeline creation*

- This occurs when the WebGPU [createComputePipeline\(\)](#) method or the WebGPU [createRenderPipeline\(\)](#) method is invoked. These methods use one or more previously created shader modules, together with other configuration information.
- Only the code forming the [shader](#) of the specified [entry point](#) of the [GPUProgrammableStage](#) is considered during pipeline creation. That is, code unrelated to the entry point is effectively stripped before compilation.

- **NOTE:** Each [shader stage](#) is considered to be compiled separately and, thus, might include different portions of the module.

3. *Shader execution start*

- This occurs when a [draw](#) or [dispatch command](#) is issued to the GPU, begins executing the pipeline, and invokes the [shader stage entry point](#) function.

4. *Shader execution end*

- This occurs when all work in the shader completes:
 - all its [invocations](#) terminate, and
 - all accesses to [resources](#) complete, and
 - outputs, if any, are passed to downstream pipeline stages.

The events are ordered due to:

- data dependencies: shader execution requires a pipeline, and a pipeline requires a shader module.
- causality: the shader must start executing before it can finish executing.

§ 2.2. Errors

A WebGPU implementation may fail to process a shader for two reasons:

- A **program error** occurs if the shader does not satisfy the requirements of the WGSL or WebGPU specifications.
- An **uncategorized error** may occur even when all WGSL and WebGPU requirements have been satisfied. Possible causes include:
 - The shaders are too complex, exceeding the capabilities of the implementation, but in a way not easily captured by prescribed [limits](#). Simplifying the shaders may work around the issue.
 - A defect in the WebGPU implementation.

A processing error may occur during three phases in the shader lifecycle:

- A **shader-creation error** is an error feasibly detectable at [shader module creation](#) time. Detection relies only on the WGSL module source text and other information available to the `createShaderModule` API method. Statements in this specification that describe something the program *must* do generally produce a shader-creation error if those assertions are violated.
- A **pipeline-creation error** is an error detectable at [pipeline creation](#) time. Detection relies on the WGSL module source text and other information available to the particular pipeline creation API method. These errors are only triggered for code present in the [shader](#) of the [entry point](#) being compiled for the [GPUProgrammableStage](#).
- A **dynamic error** is an error occurring during shader execution. These errors may or may not be detectable.

NOTE: For example, a data race may not be detectable.

Each requirement [will](#) be checked at the earliest opportunity. That is:

- A shader-creation error results when failing to meet a requirement detectable at shader-creation time.
- A pipeline-creation error results when failing to meet a requirement detectable at pipeline-creation time, but not detectable earlier.

When unclear from context, this specification indicates whether failure to meet a particular requirement results in a shader-creation, pipeline-creation, or dynamic error.

The consequences of an error are as follows:

- A WGSL module with a [shader-creation error](#) or [pipeline-creation error](#) will not be incorporated into a [pipeline](#) and hence will not be executed.
- Detectable errors [will trigger](#) a [diagnostic](#).
- If a [dynamic error](#) occurs:
 - [Memory accesses will](#) be restricted to:
 - [shader stage inputs](#),
 - [shader stage outputs](#),
 - any part of a [resource](#) bound to a variable in the WGSL module, and
 - other variables declared in the WGSL module.
 - Otherwise, the program may not behave as described in the rest of this specification. Note: These effects may be non-local.

§ 2.3. Diagnostics

An implementation can generate [diagnostics](#) during [shader module creation](#) or [pipeline creation](#). A *diagnostic* is a message produced by the implementation for the benefit of the application author.

A diagnostic is created, or *triggered*, when a particular condition is met, known as the *triggering rule*. The place in the source text where the condition is met, expressed as a point or range in the source text, is known as the *triggering location*.

A [diagnostic](#) has the following properties:

- A [severity](#).
- A [triggering rule](#).
- A [triggering location](#).

The *severity* of a diagnostic is one of the following, ordered from greatest to least:

error

The diagnostic is an error. This corresponds to a [shader-creation error](#) or to a [pipeline-creation error](#).

warning

The diagnostic describes an anomaly that merits the attention of the application developer, but is not an error.

info

The diagnostic describes a notable condition that merits attention of the application developer, but is not an error or warning.

off

The diagnostic is disabled. It will not be conveyed to the application.

The name of a [triggering rule](#) is either:

- a [diagnostic_name_token](#), or
- two [diagnostic_name_token](#) tokens, separated by a period '.' (U+002E).

diagnostic_rule_name :
[diagnostic_name_token](#)

| `diagnostic_name_token '.' diagnostic_name_token`

§ 2.3.1. Diagnostic Processing

Triggered diagnostics [will](#) be processed as follows:

1. For each diagnostic *D*, find the [diagnostic filter](#) with the smallest [affected range](#) that contains *D*'s [triggering location](#), and which has the same [triggering rule](#).
 - If such a filter exists, apply it to *D*, updating *D*'s [severity](#).
 - Otherwise *D* remains unchanged.
2. Discard diagnostics that have severity [off](#).
3. If at least one remaining diagnostic *DI* has severity [info](#), then:
 - Other [info](#) diagnostics with same triggering rule *may* be discarded, leaving only the original diagnostic *DI*.
4. If at least one remaining diagnostic *DW* has severity [warning](#), then:
 - Other [info](#) or [warning](#) diagnostics with same triggering rule *may* be discarded, leaving only the original diagnostic *DW*.
5. If at least one remaining diagnostic has [error](#) severity, then:
 - Other diagnostics *may* be discarded, including other diagnostics with [error](#) severity.
 - A [program error](#) is generated.
 - The error is a [shader-creation error](#) if the diagnostic was triggered at [shader module creation](#) time.
 - The error is a [pipeline-creation error](#) if the diagnostic was triggered at [pipeline creation](#) time.
6. If processing during [shader module creation](#) time, the remaining diagnostics populate the [messages](#) member of the WebGPU [GPUCompilationInfo](#) object.
7. If processing during [pipeline creation](#), [error](#) diagnostics result in WebGPU validation failure when validating [GPUProgrammableStage](#).

NOTE: The rules allow an implementation to stop processing a WGSL module as soon as an error is detected. Additionally, an analysis for a particular kind of warning can stop on the first warning, and an analysis for a particular kind of info diagnostic can stop on the first occurrence. WGSL does not specify the order to perform different kinds of analyses, or an ordering within a single analysis. Therefore, for the same WGSL module, different implementations may report different instances of diagnostics with the same severity.

§ 2.3.2. Filterable Triggering Rules

Most [diagnostics](#) are unconditionally reported to the WebGPU application. Some kinds of diagnostics can be [filtered](#), in part by naming their [triggering rule](#). The following table lists the standard set of triggering rules that can be filtered.

Filterable diagnostic triggering rules

Filterable Triggering Rule	Default Severity	Triggering Location	Description
<i>derivative_uniformity</i>	error	The location of the call site for any builtin function that	A call to a builtin function computes derivatives, but uniformity analysis

Filterable Triggering Rule	Default Severity	Triggering Location	Description
		computes a derivative . That is, the location of a call to any of: - the derivative builtin functions textureSample textureSampleBias textureSampleCompare	cannot prove that the call occurs in uniform control flow. See § 15.2 Uniformity.
<i>subgroup_uniformity</i>	error	The location of the call site for any subgroup or quad built-in function.	A call to a subgroup or quad builtin function, but uniformity analysis cannot prove that the call occurs in uniform control flow. Additionally, when uniformity analysis cannot prove that the following parameter values are uniform: - delta in subgroupShuffleUp and subgroupShuffleDown - mask in subgroupShuffleXor See § 15.2 Uniformity.

Using an unrecognized triggering rule consisting of a single [diagnostic name-token](#) should trigger a warning from the user agent.

An implementation may support triggering rules not specified here, provided they are spelled using the multiple-token form of [diagnostic_rule_name](#). Using an unrecognized triggering rule spelled in the multiple-token form *may* itself trigger a diagnostic.

Future versions of this specification may remove a particular rule or weaken its default severity (i.e. replace its current default with a less severe default) and still be deemed as satisfying backward compatibility. For example, a future version of WGSL may change the default severity for [derivative_uniformity](#) from error to either warning or info. After such a change to the specification, previously valid programs would remain valid.

§ 2.3.3. Diagnostic Filtering

Once a [diagnostic](#) with a filterable [triggering rule](#) is [triggered](#), WGSL provides mechanisms to discard the diagnostic, or to modify its severity.

A *diagnostic filter* DF has three parameters:

- AR : a range of source text known as the *affected range*
- NS : a new [severity](#)
- TR : a [triggering rule](#)

Applying a diagnostic filter $DF(AR,NS,TR)$ to a diagnostic D has the following effect:

- If *D*'s [triggering location](#) is in *AR* and *D*'s [triggering rule](#) is *TR*, then set *D*'s [severity](#) property to *NS*.
- Otherwise, *D* is unchanged.

A **range diagnostic filter** is a [diagnostic filter](#) whose [affected range](#) is a specified range of source text. A range diagnostic filter is specified as a `@diagnostic` [attribute](#) at the start of the affected source range, as specified in the following table. A `@diagnostic` attribute **must not** appear anywhere else.

Placement of a range diagnostic filter

Placement	Affected Range
Beginning of a compound statement .	The compound statement.
Beginning of a function declaration .	The function declaration.
Beginning of an if statement.	The if statement: the if_clause and all associated else_if_clause and else_clause clauses, including all controlling condition expressions.
Beginning of a switch statement.	The switch statement: the selector expression and the switch_body .
Beginning of a switch_body .	The switch_body .
Beginning of a loop statement.	The loop statement.
Beginning of a while statement.	The while statement: both the condition expression and the loop body.
Beginning of a for statement.	The for statement: the for_header and the loop body.
Immediately before the opening brace (' <code>{</code> ') of the loop body of a loop , while , or for loop.	The loop body .
Beginning of a continuing_compound_statement .	The continuing_compound_statement .

NOTE: The following are also [compound statements](#): a [function body](#), a [case clause](#), a [default-alone clause](#), the bodies of [while](#) and [for](#) loops, and the bodies of [if_clause](#), [else_if_clause](#), and [else_clause](#).

EXAMPLE: RANGE DIAGNOSTIC FILTER ON TEXTURE SAMPLING

```
var<private> d: f32;
fn helper() -> vec4<f32> {
    // Disable the derivative_uniformity diagnostic in the
    // body of the "if".
    if (d < 0.5) @diagnostic(off, derivative_uniformity) {
        return textureSample(t,s,vec2(0,0));
    }
    return vec4(0.0);
}
```

A [global diagnostic filter](#) can be used to apply a diagnostic filter to the entire WGSL module.

EXAMPLE: GLOBAL DIAGNOSTIC FILTER FOR DERIVATIVE UNIFORMITY

```

diagnostic(off,derivative_uniformity);
var<private> d: f32;
fn helper() -> vec4<f32> {
  if (d < 0.5) {
    // The derivative_uniformity diagnostic is disabled here
    // by the global diagnostic filter.
    return textureSample(t,s,vec2(0,0));
  } else {
    // The derivative_uniformity diagnostic is set to 'warning' severity.
    @diagnostic(warning,derivative_uniformity) {
      return textureSample(t,s,vec2(0,0));
    }
  }
  return vec4(0.0);
}

```

Two [diagnostic filters](#) $DF(AR1,NS1,TR1)$ and $DF(AR2,NS2,TR2)$ **conflict** when:

- $(AR1 = AR2)$, and
- $(TR1 = TR2)$, and
- $(NS1 \neq NS2)$.

[Diagnostic filters](#) **must** not [conflict](#).

NOTE: Multiple [global diagnostic filters](#) are permitted when they do not [conflict](#).

WGSL's diagnostic filters are designed so their affected ranges nest perfectly. If the affected range of DF1 overlaps with the affected range of DF2, then either DF1's affected range is fully contained in DF2's affected range, or the other way around.

The **nearest enclosing diagnostic filter** for source location L and triggering rule TR , if one exists, is the diagnostic filter $DF(AR,NS,TR)$ where:

- L falls in the affected range AR , and
- If there is another filter $DF'(AR',NS',TR)$ where L falls in AR' , then AR is contained in AR' .

Because affected ranges nest, the nearest enclosing diagnostic:

- is a unique [range diagnostic filter](#),
- otherwise is one of a set of duplicate (non-[conflicting](#)) [global diagnostic filters](#),
- otherwise does not exist.

2.4. Limits

A WGSL implementation [will](#) support shaders that satisfy the following limits. A WGSL implementation may support shaders that go beyond the specified limits.

NOTE: A WGSL implementation should issue an error if it does not support a shader that goes beyond the specified limits.

Quantifiable shader complexity limits

Limit	Minimum supported value
Maximum number of members in a structure type	1023
Maximum nesting depth of a composite type	15
Maximum nesting depth of brace-enclosed statements in a function	127
Maximum number of parameters for a function	255
Maximum number of case selector values in a switch statement. The sum of, for each case statement, the number of case values, including the default clause .	1023
Maximum combined byte-size of all variables instantiated in the private address space that are statically accessed by a single shader	8192
Maximum combined byte-size of all variables instantiated in the function address space that are declared in a single function	8192
Maximum combined byte-size of all variables instantiated in the workgroup address space that are statically accessed by a single shader	
For the purposes of this limit, a fixed-footprint array is treated as a creation-fixed footprint array when substituting the override value.	16384
This maps the WebGPU maxComputeWorkgroupStorageSize limit into a standalone WGSL limit.	
Maximum byte-size of an immediate data variable .	
This maps the WebGPU maxImmediateSize limit into a standalone WGSL limit.	64
Maximum number of elements in value constructor expression of array type	2047

§ 3. Textual Structure

The `text/wgsl` media type is used to identify content as a WGSL module. See [Appendix A: The text/wgsl Media Type](#).

A WGSL module is Unicode text using the UTF-8 encoding, with no byte order mark (BOM).

WGSL module text consists of a sequence of Unicode [code points](#), grouped into contiguous non-empty sets forming:

- [comments](#)
- [tokens](#)
- [blankspaces](#)

The program text [must not](#) include a null code point (U+0000).

§ 3.1. Parsing

To parse a WGSL module:

1. Remove [comments](#):
 - Replace the first comment with a space code point (U+0020).
 - Repeat until no comments remain.
2. Find [template lists](#), using the [algorithm](#) in § 3.9 [Template Lists](#). This step disambiguates uses of '[<](#)' (U+003C) and '[>](#)' (U+003E) code points as template list delimiters versus other uses such as in comparison operators.
3. Parse the whole text, attempting to match the [translation_unit](#) grammar rule. Parsing uses a LALR(1) parser (one token of lookahead) [[DeRemer1969](#)], with the following customization:
 - Tokenization is interleaved with parsing, and is context-aware. When the parser requests the next token:
 - Consume and ignore an initial sequence of [blankspace](#) code points.
 - If the next code point is the start of a [template list](#), consume it and return [_template_args_start](#).
 - If the next code point is the end of a [template list](#), consume it and return [_template_args_end](#).
 - Otherwise:
 - A *token candidate* is any WGSL [token](#) formed from the non-empty prefix of the remaining unconsumed code points.
 - The token returned is the longest [token candidate](#) that is also a valid lookahead token for the current parser state. [[VanWyk2007](#)]

A [shader-creation error](#) results if:

- the entire source text cannot be converted into a finite sequence of valid tokens, or
- the [translation_unit](#) grammar rule does not match the entire token sequence.

NOTE: The procedure given above describes [template list discovery](#) as a discrete phase (step 2), performed before grammar-based parsing (step 3). This separation simplifies the specification. However, it may be inconvenient to map the above procedure to some tools because it inserts a distinct processing phase before tokenization.

An alternative approach is to interleave template list discovery with tokenization. In this approach, a synthetic token ([_disambiguate_template](#)) is placed in a grammar rule at any place where a template list may occur. When the scanner attempts to match the [_disambiguate_template](#) token:

- The scanner runs the [template list discovery](#) algorithm on the remaining text, recording the positions of template list delimiters.
- The scanner signals a successful match of the [_disambiguate_template](#) token, with an empty string as the associated text.

Future token scanning steps will use the recorded template list delimiter positions to yield [_template_args_start](#) and [_template_args_end](#) tokens as indicated.

This alternative approach is non-normative. The normative grammar includes [_disambiguate_template](#) tokens as a help to implementations using the alternative approach. A parser using the standard approach can either ignore the synthetic token, or equivalently, always successfully match it to an empty string.

§ 3.2. Blankspace and Line Breaks

Blankspace is any combination of one or more of code points from the Unicode [Pattern_White_Space](#) property. The following is the set of code points in [Pattern_White_Space](#):

- space (U+0020)
- horizontal tab (U+0009)
- line feed (U+000A)
- vertical tab (U+000B)
- form feed (U+000C)
- carriage return (U+000D)
- next line (U+0085)
- left-to-right mark (U+200E)
- right-to-left mark (U+200F)
- line separator (U+2028)
- paragraph separator (U+2029)

A **line break** is a contiguous sequence of [blankspace](#) code points indicating the end of a line. It is defined as the blankspace signalling a "mandatory break" as defined by [UAX14 Section 6.1 Non-tailorable Line Breaking Rules LB4](#) and [LB5](#). That is, a line break is any of:

- line feed (U+000A)
- vertical tab (U+000B)
- form feed (U+000C)
- carriage return (U+000D) when not also followed by line feed (U+000A)
- carriage return (U+000D) followed by line feed (U+000A)
- next line (U+0085)
- line separator (U+2028)
- paragraph separator (U+2029)

NOTE: Diagnostics that report source text locations in terms of line numbers should use [line breaks](#) to count lines.

§ 3.3. Comments

A **comment** is a span of text that does not influence the validity or meaning of a WGSL program, except that a comment can separate [tokens](#). Shader authors can use comments to document their programs.

A **line-ending comment** is a kind of [comment](#) consisting of the two code points `//` (U+002F followed by U+002F) and the code points that follow, up until but not including:

- the next [line break](#), or
- the end of the program.

A **block comment** is a kind of [comment](#) consisting of:

- The two code points `/*` (U+002F followed by U+002A)
- Then any sequence of:

- A [block comment](#), or
- Text that does not contain either `*/` (U+002A followed by U+002F) or `/*` (U+002F followed by U+002A)
- Then the two code points `*/` (U+002A followed by U+002F)

NOTE: Block comments can be nested. Since a block comment requires matching start and end text sequences, and allows arbitrary nesting, a block comment cannot be recognized with a regular expression. This is a consequence of the Pumping Lemma for Regular Languages.

EXAMPLE: COMMENTS

```
const f = 1.5; // This is line-ending comment.
const g = 2.5; /* This is a block comment
                 that spans lines.
                 /* Block comments can nest.
                 */
                 But all block comments must terminate.
                 */
```

§ 3.4. Tokens

A **token** is a contiguous sequence of code points forming one of:

- a [literal](#).
- a [keyword](#).
- a [reserved word](#).
- a [syntactic token](#).
- an [identifier](#).
- a [context-dependent name](#).

§ 3.5. Literals

A **literal** is one of:

- A **boolean literal**: either true or false.
- A **numeric literal**: either an [integer literal](#) or a [floating point literal](#), and is used to represent a number.

literal :

int_literal

| *float_literal*

| *bool_literal*

§ 3.5.1. Boolean Literals

EXAMPLE: BOOLEAN LITERALS

```
const a = true;
const b = false;
```

```
bool_literal :
    'true'
    | 'false'
```

§ 3.5.2. Numeric Literals

The form of a [numeric literal](#) is defined via pattern-matching.

An *integer literal* is:

- An integer specified as any of:
 - 0
 - A sequence of decimal digits, where the first digit is not 0.
 - 0x or 0X followed by a sequence of hexadecimal digits.
- Then an optional i or u suffix.

NOTE: A leading zero on a non-zero integer literal (e.g. 012) is forbidden, so as to avoid confusion with other languages' leading-zero-means-octal notation.

```
int_literal :
    decimal_int_literal
    | hex_int_literal
```

```
decimal_int_literal :
    /0[iu]?/
    | /[1-9][0-9]*[iu]?/
```

EXAMPLE: DECIMAL INTEGER LITERALS

```
const a = 1u;
const b = 123;
const c = 0;
const d = 0i;
```

```
hex_int_literal :
    /0[xX][0-9a-fA-F]+[iu]?/
```

EXAMPLE: HEXADECIMAL INTEGER LITERALS

```
const a = 0x123;
const b = 0X123u;
const c = 0x3f;
```

A **floating point literal** is either a [decimal floating point literal](#) or a [hexadecimal floating point literal](#).

```
float_literal :
    decimal_float_literal
    | hex_float_literal
```

A [floating point literal](#) has two logical parts: a significand to represent a fraction, and an optional exponent. Roughly, the value of the literal is the significand multiplied by a base value raised to the given exponent. A significand digit is **significant** if it is non-zero, or if there are significand digits to its left and to its right that are both non-zero. Significant digits are counted from left-to-right: the N 'th significant digit has $N-1$ significant digits to its left.

A **decimal floating point literal** is:

- A significand, specified as a sequence of digits, with an optional decimal point (.) somewhere among them. The significand represents a fraction in base 10 notation.
- Then an optional exponent suffix consisting of:
 - e or E.
 - Then an exponent specified as a decimal number with an optional leading sign (+ or -).
 - Then an optional f or h suffix.
- At least one of the decimal point, or the exponent, or the f or h suffix [must](#) be present. If none are, then the token is instead an [integer literal](#).

```
decimal_float_literal :
    /0[fh]/
    | /[1-9][0-9]*[fh]/
    | /[0-9]*\.[0-9]+([eE][+-]?[0-9]+)?[fh]?/
    | /[0-9]+\.[0-9]*([eE][+-]?[0-9]+)?[fh]?/
    | /[0-9]+[eE][+-]?[0-9]+[fh]?/
```

EXAMPLE: DECIMAL FLOATING POINT LITERALS

```
const a = 0.e+4f;
const b = 01.;
const c = .01;
const d = 12.34;
const f = .0f;
const g = 0h;
const h = 1e-3;
```

The mathematical value of a [decimal floating point literal](#) is computed as follows:

- Compute *effective_significand* from *significand*:
 - If *significand* has 20 or fewer [significant](#) digits, then *effective_significand* is *significand*.

- Otherwise:
 - Let *truncated_significand* be the same as *significand* except each digit to the right of the 20th significant digit is replaced with 0.
 - Let *truncated_significand_next* be the same as *significand* except:
 - the 20th significant digit is incremented by 1, and carries are propagated to the left as needed to ensure each digit remains in the range 0 through 9, and
 - each digit to the right of the 20th significant digit is replaced with 0.
 - Set *effective_significand* to either *truncated_significand* or *truncated_significand_next*. This is an implementation choice.
- The mathematical value of the literal is the mathematical value of *effective_significand* as a decimal fraction, multiplied by 10 to the power of the exponent. When no exponent is specified, an exponent of 0 is assumed.

NOTE: The decimal significand is truncated after 20 decimal digits, preserving approximately $\log(10)/\log(2) \times 20 \approx 66.4$ significant bits in the fraction.

A **hexadecimal floating point literal** is:

- A 0x or 0X prefix
- Then a significand, specified as a sequence of hexadecimal digits, with an optional hexadecimal point (.) somewhere among them. The significand represents a fraction in base 16 notation.
- Then an optional exponent suffix consisting of:
 - p or P
 - Then an exponent specified as a decimal number with an optional leading sign (+ or -).
 - Then an optional f or h suffix.
- At least one of the hexadecimal point, or the exponent [must](#) be present. If neither are, then the token is instead an [integer literal](#).

hex_float_literal :

```
/0[xX][0-9a-fA-F]*\.[0-9a-fA-F]+([pP][+-]?[0-9]+[fh]?)?/
| /0[xX][0-9a-fA-F]+\.[0-9a-fA-F]*([pP][+-]?[0-9]+[fh]?)?/
| /0[xX][0-9a-fA-F]+[pP][+-]?[0-9]+[fh]?/
```

EXAMPLE: HEXADECIMAL FLOATING POINT LITERALS

```
const a = 0xa.fp+2;
const b = 0x1P+4f;
const c = 0X.3;
const d = 0x3p+2h;
const e = 0X1.fp-4;
const f = 0x3.2p+2h;
```

The mathematical value of a [hexadecimal floating point literal](#) is computed as follows:

- Compute *effective_significand* from *significand*:
 - If *significand* has 16 or fewer [significant](#) digits, then *effective_significand* is *significand*.
 - Otherwise:

- Let *truncated_significand* be the same as *significand* except each digit to the right of the 16th significant digit is replaced with 0.
- Let *truncated_significand_next* be the same as *significand* except:
 - the 16th significant digit is incremented by 1, and carries are propagated to the left as needed to ensure each digit remains in the range 0 through f, and
 - each digit to the right of the 16th significant digit is replaced with 0.
- Set *effective_significand* to either *truncated_significand* or *truncated_significand_next*. This is an implementation choice.
- The mathematical value of the literal is the mathematical value of *effective_significand* as a hexadecimal fraction, multiplied by 2 to the power of the exponent. When no exponent is specified, an exponent of 0 is assumed.

NOTE: The hexadecimal significand is truncated after 16 hexadecimal digits, preserving approximately $4 \times 16 = 64$ significant bits in the fraction.

When a [numeric literal](#) has a suffix, the literal denotes a value in a specific [concrete scalar](#) type. Otherwise, the literal denotes a value one of the [abstract numeric types](#) defined below. In either case, the value denoted by the literal is its mathematical value after conversion to the target type, following the rules in [§ 15.7.6 Floating Point Conversion](#).

Mapping numeric literals to types

Numeric Literal	Suffix	Type	Examples
integer literal	i	i32	42i
integer literal	u	u32	42u
integer literal		AbstractInt	124
floating point literal	f	f32	42f 1e5f 1.2f 0x1.0p10f
floating point literal	h	f16	42h 1e5h 1.2h 0x1.0p10h
floating point literal		AbstractFloat	1e5 1.2 0x1.0p10

A [shader-creation error](#) results if:

- An [integer literal](#) with a i or u suffix cannot be represented by the target type.
- A [hexadecimal floating point literal](#) with a f or h suffix overflows or cannot be exactly represented by the target type.
- A [decimal floating point literal](#) with a f or h suffix overflows the target type.
- A [floating point literal](#) with a h suffix is used while the [f16 extension](#) is not enabled.

NOTE: The hexadecimal float value 0x1.00000001p0 requires 33 significand bits to be represented exactly, but [f32](#) only has 23 explicit significand bits.

NOTE: If you want to use an f suffix to force a hexadecimal float literal to be of type, the literal must also use a binary exponent. For example, write 0x1p0f. In comparison, 0x1f is a hexadecimal integer literal.

§ 3.6. Keywords

A **keyword** is a [token](#) which refers to a predefined language concept. See [§ 16.1 Keyword Summary](#) for the list of WGSL keywords.

§ 3.7. Identifiers

An **identifier** is a kind of [token](#) used as a name. See [§ 5 Declaration and Scope](#).

WGSL uses two grammar nonterminals to separate use cases:

- An [ident](#) is used to name a declared object.
- A [member_ident](#) is used to name a [member](#) of a [structure](#) type.

ident :
ident_pattern_token _disambiguate_template

member_ident :
ident_pattern_token

The form of an identifier is based on the [Unicode Standard Annex #31](#) for [Unicode Version 14.0.0](#), with the following elaborations.

Identifiers use the following profile described in terms of [UAX31 Grammar](#):

<Identifier> := <Start> <Continue>* (<Medial> <Continue>+)*

<Start> := XID_Start + U+005F

<Continue> := <Start> + XID_Continue

<Medial> :=

This means identifiers with non-ASCII code points like these are valid: Δέλτα, réflexion, Кызыл, ʼᄎX, 朝焼け, سلام, 검정, दाढ़ी, गुलाबी, фірмЛq.

With the following exceptions:

- An identifier **must not** have the same spelling as a [keyword](#) or as a [reserved word](#).
- An identifier **must not** be `_` (a single underscore, U+005F).
- An identifier **must not** start with `__` (two underscores, U+005F followed by U+005F).

ident_pattern_token :
/([_\\p{XID_Start}][\\p{XID_Continue}])|([\\p{XID_Start}])/u*

[Unicode Character Database for Unicode Version 14.0.0](#) includes non-normative listing with all valid code points of both [XID_Start](#) and [XID_Continue](#).

NOTE: The [return type](#) for some [built-in functions](#) are structure types whose name cannot be used WGSL source. Those structure types are described as if they were [predeclared](#) with a name starting with two underscores. The result value can be saved into newly declared `let` or `var` using type inferencing, or immediately have one of its members immediately extracted by name. See example usages in the description of `frexp` and `modf`.

§ 3.7.1. Identifier Comparison

Two WGSL identifiers are the same if and only if they consist of the same sequence of code points.

NOTE: This specification does not permit Unicode normalization of values for the purposes of comparison. Values that are visually and semantically identical but use different Unicode character sequences will not match. Content authors are advised to use the same encoding sequence consistently or to avoid potentially troublesome characters when choosing values. For more information, see [\[CHARMOD-NORM\]](#).

NOTE: A user agent should issue developer-visible warnings when the meaning of a WGSL module would change if all instances of an identifier are replaced with one of that identifier's homographs. (A homograph is a sequence of code points that may appear the same to a reader as another sequence of code points. Examples of mappings to detect homographs are the transformations, mappings, and matching algorithms mentioned in the previous paragraph. Two sequences of code points are homographs if the identifier can transform one into the other by repeatedly replacing a subsequence with its homograph.)

§ 3.8. Context-Dependent Names

A *context-dependent name* is a [token](#) used to name a concept, but only in specific grammatical contexts. The spelling of the token may be the same as an [identifier](#), but the token does not [resolve](#) to a declared object. This section lists the tokens used as [context-dependent names](#). The token must not be a [keyword](#) or [reserved word](#).

§ 3.8.1. Attribute Names

See [§ 12 Attributes](#).

The [attribute](#) names are:

- ['align'](#)
- ['binding'](#)
- ['builtin'](#)
- ['compute'](#)
- ['const'](#)
- ['diagnostic'](#)
- ['fragment'](#)
- ['group'](#)
- ['id'](#)
- ['interpolate'](#)
- ['invariant'](#)
- ['location'](#)
- ['blend_src'](#)
- ['must_use'](#)
- ['size'](#)
- ['vertex'](#)
- ['workgroup_size'](#)

- ['subgroup_size'](#)

§ 3.8.2. Built-in Value Names

A *built-in value name-token* is a [token](#) used in the name of a [built-in value](#).

See § 13.3.1.1 Built-in Inputs and Outputs.

```
builtin_value_name :  
    ident_pattern_token
```

The [built-in value](#) names are:

- ['vertex_index'](#)
- ['instance_index'](#)
- ['position'](#)
- ['front_facing'](#)
- ['frag_depth'](#)
- ['sample_index'](#)
- ['sample_mask'](#)
- ['local_invocation_id'](#)
- ['local_invocation_index'](#)
- ['global_invocation_id'](#)
- ['global_invocation_index'](#)
- ['workgroup_id'](#)
- ['workgroup_index'](#)
- ['num_workgroups'](#)
- ['subgroup_invocation_id'](#)
- ['subgroup_size'](#)
- ['primitive_index'](#)
- ['subgroup_id'](#)
- ['num_subgroups'](#)

§ 3.8.3. Diagnostic Rule Names

A *diagnostic name-token* is a [token](#) used in the name of a diagnostic [triggering rule](#).

See § 2.3 Diagnostics.

```
diagnostic_name_token :  
    ident_pattern_token
```

The pre-defined [diagnostic rule](#) names are:

- ['derivative_uniformity'](#)
- ['subgroup_uniformity'](#)

§ 3.8.4. Diagnostic Severity Control Names

The valid [diagnostic filter](#) severity control names are listed in [§ 2.3 Diagnostics](#) but have the same form as an [identifier](#):

```
severity_control_name :  
    ident_pattern_token
```

The [diagnostic filter](#) severity control names are:

- ['error'](#)
- ['warning'](#)
- ['info'](#)
- ['off'](#)

§ 3.8.5. Extension Names

The valid [enable-extension](#) names are listed in [§ 4.1.1 Enable Extensions](#) but in general have the same form as an [identifier](#):

```
enable_extension_name :  
    ident_pattern_token
```

The [enable-extension](#) names are:

- ['f16'](#)
- ['clip_distances'](#)
- ['dual_source_blending'](#)
- ['subgroups'](#)
- ['primitive_index'](#)
- ['subgroup_size_control'](#)

The valid [language extension](#) names are listed in [§ 4.1.2 Language Extensions](#) but in general have the same form as an [identifier](#):

```
language_extension_name :  
    ident_pattern_token
```

The [language extension](#) names are:

- ['readonly_and_readwrite_storage_textures'](#)
- ['packed_4x8_integer_dot_product'](#)
- ['unrestricted_pointer_parameters'](#)
- ['pointer_composite_access'](#)
- ['uniform_buffer_standard_layout'](#)
- ['subgroup_id'](#)
- ['subgroup_uniformity'](#)
- ['texture_and_sampler_let'](#)
- ['texture_formats_tier1'](#)

- ['linear_indexing'](#)
- ['immediate_address_space'](#)
- ['buffer_view'](#)

§ 3.8.6. Interpolation Type Names

An *interpolation type name-token* is a [token](#) used in the name of an [interpolation type](#) for [interpolate_type_name](#).

See [§ 13.3.1.4 Interpolation](#).

The [interpolation type](#) names are:

- ['perspective'](#)
- ['linear'](#)
- ['flat'](#)

§ 3.8.7. Interpolation Sampling Names

An *interpolation sampling name-token* is a [token](#) used in the name of an [interpolation sampling](#).

See [§ 13.3.1.4 Interpolation](#).

```
interpolate_sampling_name :  
    ident_pattern_token
```

The [interpolation sampling](#) names are:

- ['center'](#)
- ['centroid'](#)
- ['sample'](#)
- ['first'](#)
- ['either'](#)

§ 3.8.8. Swizzle Names

The [swizzle](#) names are used in [vector access expressions](#):

```
swizzle_name :  
    /[rgba]/  
  | /[rgba][rgba]/  
  | /[rgba][rgba][rgba]/  
  | /[rgba][rgba][rgba][rgba]/  
  | /[xyzw]/  
  | /[xyzw][xyzw]/  
  | /[xyzw][xyzw][xyzw]/  
  | /[xyzw][xyzw][xyzw][xyzw]/
```

§ 3.9. Template Lists

Template parameterization is a way to specify parameters that modify a general concept. To write a template parameterization, write the general concept, followed by a [template list](#).

Ignoring [comments](#) and [blankspace](#), a **template list** is:

- An initial '`<`' (U+003C) code point, then
- A [comma](#)-separated list of one or more **template parameters**, then
- An optional trailing [comma](#), then
- A terminating '`>`' (U+003E) code point.

The form of a [template parameter](#) is implicitly defined by the [template list discovery](#) algorithm below. Generally, they are names, expressions, or types.

NOTE: For example, the phrase `vec3<f32>` is a template parameterization where `vec3` is the general concept being modified, and `<f32>` is a template list containing one parameter, the [f32](#) type. Together, `vec3<f32>` denotes a specific [vector](#) type.

NOTE: For example, the phrase `var<storage, read_write>` modifies the general `var` concept with template parameters `storage` and `read_write`.

NOTE: For example, the phrase `array<vec4<f32>>` has two template parameterizations:

- `vec4<f32>` modifies the general `vec4` concept with template parameter `f32`.
- `array<vec4<f32>>` modifies the general `array` concept with template parameter `vec4<f32>`.

The '`<`' (U+003C) and '`>`' (U+003E) code points that delimit a template list are also used when spelling:

- A comparison operator in a [relational expression](#).
- A shift operator in a [shift expression](#).
- A [compound assignment operator](#) for performing a shift operation followed by an assignment.

The syntactic ambiguity is resolved in favour of template lists:

- Template lists are discovered in an early phase of parsing, before [declarations](#), [expressions](#), [statements](#) are parsed.
- During tokenization in a later phase, the initial '`<`' (U+003C) of a template list is mapped to a [__template_args_start](#) token, and the terminating '`>`' (U+003E) of a template list is mapped to a [__template_args_end](#) token.

The [template list discovery](#) algorithm is given below. It uses the following assumptions and properties:

1. A [template parameter](#) is an [expression](#), and therefore does not start with either a '`<`' (U+003C) or a '`=`' (U+003D) code point.
2. An expression does not contain code points '`;`' (U+003B), '`{`' (U+007B), or '`:`' (U+003A).
3. An expression does not contain an [assignment](#).
4. The only time a '`=`' (U+003D) code point appears is as part of a comparison operation, i.e. in one of '`<=`', '`>=`', '`==`', or '`!=`'. Otherwise, a '`=`' (U+003D) code point appears as part of an assignment.
5. Template list delimiters respect nested expressions formed by parentheses '`(...)`', and array indexing '`[...]`'. The start and end of a template list must appear at the same nesting level.

Algorithm: *Template list discovery***Input:** The program source text.**Record types:**Let *UnclosedCandidate* be a record type containing:

- *position*, a location in the source text
- *depth*, an integer, the expression nesting depth at *position*

Let *TemplateList* be a record type containing:

- *start_position*, the source location of the '<' (U+003C) code point that starts this template list.
- *end_position*, the source location of the '>' (U+003E) code point that ends this template list.

Output: *DiscoveredTemplateLists*, a list of *TemplateList* records.**Procedure:**

- Initialize *DiscoveredTemplateLists* to an empty list.
- Initialize a *Pending* variable to be an empty stack of *UnclosedCandidate* records.
- Initialize a *CurrentPosition* integer variable to 0. It encodes the position of the code point currently being examined, as a count of the number of code points after the start of the source text.
 - This variable will advance forward in the text while executing the algorithm. When the end of text is reached, terminate the algorithm immediately and have it return *DiscoveredTemplateLists*.
- Initialize a *NestingDepth* integer variable to 0.
- Repeat the following steps:
 - Advance *CurrentPosition* past [blankspace](#), [comments](#), and [literals](#).
 - If [ident_pattern_token](#) matches the text at *CurrentPosition*, then:
 - Advance *CurrentPosition* past the [ident_pattern_token](#).
 - Advance *CurrentPosition* past blankspace and comments, if present.
 - If '<' (U+003C) appears at *CurrentPosition*, then:
 - **NOTE:** This code point is a candidate for being the start of a template list. Save enough state so it can be matched against a terminating '>' (U+003E) appearing later in the input.
 - Push *UnclosedCandidate*(*position*=*CurrentPosition*,*depth*=*NestingDepth*) onto the *Pending* stack.
 - Advance *CurrentPosition* to the next code point.
 - If '<' (U+003C) appears at *CurrentPosition*, then:
 - **NOTE:** From assumption 1, no template parameter starts with '<' (U+003C), so the previous code point cannot be the start of a template list. Therefore the current and previous code point must be '<<' operator.
 - Pop the top entry from the *Pending* stack.
 - Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
 - If '=' (U+003D) appears at *CurrentPosition*, then:

- **NOTE:** From assumption 1, no template parameter starts with '=' (U+003C), so the previous code point cannot be the start of a template list. Assume the current and previous code point form a '<=' comparison operator. Skip over the '=' (U+003D) code point so a later step does not mistake it for an assignment.
- Pop the top entry from the *Pending* stack.
- Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
- Start the next iteration of the loop.
- If '>' (U+003E) appears at *CurrentPosition* then:
 - **NOTE:** This code point is a candidate for being the end of a template list.
 - If *Pending* is not empty, then let *T* be its top entry, and if *T.depth* equals *NestingDepth* then:
 - **NOTE:** This code point ends the current template list whose start is recorded in *T*.
 - Add *TemplateList*(*start_position*=*T.position*, *end_position*=*CurrentPosition*) to *DiscoveredTemplateLists*.
 - Pop *T* off the *Pending* stack.
 - Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
 - Otherwise, this code point does not end a template list:
 - Advance *CurrentPosition* past this code point.
 - If '=' (U+003D) appears at *CurrentPosition* then:
 - **NOTE:** Assume the current and previous code points form a '>=' comparison operator. Skip over the '=' (U+003D) code point so a later step does not mistake it for an assignment.
 - Advance *CurrentPosition* past this code point.
 - Start the next iteration of the loop.
- If '(' (U+0028) or '[' (U+005B) appears at *CurrentPosition* then:
 - **NOTE:** Enter a nested expression.
 - Add 1 to *NestingDepth*.
 - Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
- If ')' (U+0029) or ']' (U+005D) appears at *CurrentPosition* then:
 - **NOTE:** Exit a nested expression.
 - Pop entries from the *Pending* stack until it is empty, or until the its top entry has *depth* < *NestingDepth*.
 - Set *NestingDepth* to 0 or *NestingDepth* - 1, whichever is larger.
 - Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
- If '!' (U+0021) appears at *CurrentPosition* then:
 - Advance *CurrentPosition* past this code point.
 - If '=' (U+003D) appears at *CurrentPosition* then:

- **NOTE:** Assume the current and previous code points form a '!=' comparison operator. Skip over the '=' (U+003D) code point so a later step does not mistake it for an assignment.
- Advance *CurrentPosition* past this code point.
- Start the next iteration of the loop.
- If '=' (U+003D) appears at *CurrentPosition* then:
 - Advance *CurrentPosition* past this code point.
 - If '=' (U+003D) appears at *CurrentPosition* then:
 - **NOTE:** Assume the current and previous code points form a '==' comparison operator. Skip over the '=' (U+003D) code point so a later step does not mistake it for an assignment.
 - Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
 - **NOTE:** Assume this code point is part of an assignment, which cannot appear as part of an expression, and therefore cannot appear in a template list. Clear pending unclosed candidates.
 - Set *NestingDepth* to 0.
 - Remove all entries from the *Pending* stack.
 - Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
- If ';' (U+003B) or '{' (U+007B) or ':' (U+003A) appears at *CurrentPosition* then:
 - **NOTE:** These cannot appear in the middle of an expression, and therefore cannot appear in a template list. Clear pending unclosed candidates.
 - Set *NestingDepth* to 0.
 - Remove all entries from the *Pending* stack.
 - Advance *CurrentPosition* past this code point, and start the next iteration of the loop.
- If '&&' or '||' matches the text at *CurrentPosition* then:
 - **NOTE:** These are operators that have lower precedence than comparisons. Reject any pending unclosed candidates at the current expression level.
 - **NOTE:** With this rule, no template list will be found in the program fragment `a < b || c > d`. Instead it will be recognized as the short-circuiting disjunction of two comparisons.
 - Pop entries from the *Pending* stack until it is empty, or until its top entry has *depth* < *NestingDepth*.
 - Advance *CurrentPosition* past the two code points, and start the next iteration of the loop.
- Advance *CurrentPosition* past the current code point.

NOTE: The algorithm can be modified to find the source ranges for [template parameters](#), as follows:

- Modify *UnclosedCandidate* to add the following fields:
 - *parameters*, a list of source ranges of template parameters.
 - *parameter_start_position*, a source location.
- Modify *TemplateList* to add a field:
 - *parameters*, a list of source ranges of template parameters.
- When pushing a new *UnclosedCandidate* onto the *Pending* stack:
 - Set its *parameters* field to an empty list.
 - Set *parameter_start_position* to one code point past *CurrentPosition*.
- When adding a *TemplateList*, *TL*, to *DiscoveredTemplateLists*:
 - Let *T* be the top of the *Pending* stack, as in the original algorithm.
 - Push the source range starting at *T.parameter_start_position* and ending at *CurrentPosition*−1 onto *T.parameters*.
 - Prepare *TL* as in the original algorithm.
 - Set *TL.parameters* to *T.parameters*.
- Insert a check at the end the loop, just before advancing past the current code point:
 - If `'` (U+002C) appears at *CurrentPosition* and *Pending* is not empty, then:
 - Let *T* be the top of the *Pending* stack.
 - Push the source range starting at *T.parameter_start_position* and ending at *CurrentPosition*−1 onto *T.parameters*.
 - Set *T.parameter_start_position* to *CurrentPosition*+1

NOTE: The algorithm explicitly skips past literals because some numeric literals end in a letter, for example `1.0f`. The terminating `f` should not be mistaken as the start of an [ident pattern token](#).

NOTE: In the phrase `A ( B < C, D > ( E ) )`, the segment `< C, D >` is a [template list](#).

NOTE: The algorithm respects expression nesting: The start and end of a particular template list cannot appear at different expression nesting levels. For example, in `array<i32,select(2,3,a>b)>`, the template list has three parameters, where the last one is `select(2,3,a>b)`. The `'>'` in `a>b` does not terminate the template list because it is enclosed in a parenthesized part of the expression calling the `select` function.

NOTE: Both ends of a template list must appear within the same [indexing expression](#). For example `a[b<d]>()` does not contain a valid template list.

NOTE: In the phrase `A<B<<C>`, the phrase `B<<C` is parsed as `B` followed by the left-shift operator `'<<'` followed by `C`. The template discovery algorithm starts examining `B` then `'<'` (U+003C) but then sees that the next `'<'` (U+003C) code point cannot start a template argument, and so the `'<'` immediately after the `B` is not the start of a template list. The initial `'<'` and final `'>'` are the only template list delimiters, and it has template parameter `B<<C`.

NOTE: The phrase `A<B<=C>` is analyzed similarly to the previous note, so the phrase `B<=C` is parsed as `B` followed by the less-than-or-equal operator `'<='` followed by `C`. The template discovery algorithm starts examining `B` then `'<'` (U+003C) but then sees that the next `'='` (U+003D) code point cannot start a template argument, and so the `'<'` immediately after the `B` is not the start of a template list. The initial `'<'` and final `'>'` are the only template list delimiters, and it has template parameter `B<=C`.

NOTE: When examining the phrase `A<(B>=C)>`, there is one template list, starting at the first `'<'` (U+003C) code point and ending at the last `'>'` (U+003E) code point, and having template argument `B>=C`. After examining the first `'>'` (U+003C) code point (after `B`), the `'='` (U+003D) code point needs to be recognized specially so it isn't assumed to be part of an assignment.

NOTE: When examining the phrase `A<(B!=C)>`, there is one template list, starting at the first `'<'` (U+003C) code point and ending at the last `'>'` (U+003E) code point, and having template argument `B!=C`. After examining the `'!'` (U+0021) code point (after `'B'`), the `'='` (U+003D) code point needs to be recognized specially so it isn't assumed to be part of an assignment.

NOTE: When examining the phrase `A<(B==C)>`, there is one template list, starting at the first `'<'` (U+003C) code point and ending at the last `'>'` (U+003E) code point, and having template argument `B==C`. After examining the first `'='` (U+003D) code point (after `'B'`), the second `'='` (U+003D) code point needs to be recognized specially so neither are assumed to be part of an assignment.

After [template list discovery](#) completes, [parsing will](#) attempt to match each template list to the [template_list](#) grammar rule.

template_list :

template_args_start template_arg_comma_list template_args_end

template_arg_comma_list :

*template_arg_expression (`'.'` template_arg_expression) * `'.'` ?*

template_arg_expression :

expression

§ 4. Directives

A **directive** is a [token](#) sequence which modifies how a WGSL program is processed by a WebGPU implementation.

Directives are optional. If present, all directives [must](#) appear before any [declarations](#) or [const assertions](#).

global_directive :

diagnostic_directive

| *enable_directive*

| *requires_directive*

§ 4.1. Extensions

WGSL is expected to evolve over time.

An **extension** is a named grouping of a coherent set of modifications to the WGSL specification, consisting of any combination of:

- Addition of new concepts and behaviors via new syntax, including:
 - declarations, statements, attributes, and built-in functions.
- Removal of restrictions in the current specification or in previously published extensions.
- Syntax for reducing the set of permissible behaviors.
- Syntax for limiting the features available to a part of the program.
- A description of how the extension interacts with the existing specification, and optionally with other extensions.

Hypothetically, extensions could:

- Add numeric scalar types, such as different bit width integers.
- Add syntax to constrain floating point rounding mode.
- Add syntax to signal that a shader does not use atomic types.
- Add new kinds of statements.
- Add new built-in functions.
- Add syntax to constrain how shader invocations execute.
- Add new shader stages.

There are two kinds of extensions: [enable-extensions](#) and [language extensions](#).

§ 4.1.1. Enable Extensions

An **enable-extension** is an [extension](#) whose functionality is available only if:

- The implementation supports it, and
- The shader explicitly requests it via an [enable directive](#), and
- The corresponding WebGPU [GPUFeatureName](#) was one of the required features requested when creating the [GPUDevice](#).

[Enable-extensions](#) are intended to expose hardware functionality that is not universally supported.

An **enable directive** is a [directive](#) that turns on support for one or more enable-extensions. A [shader-creation error](#) results if the implementation does not support all the listed enable-extensions.

```
enable_directive :
    'enable' enable_extension_list ';'

enable_extension_list :
    enable_extension_name ( ',' enable_extension_name ) * ',' ?
```

Like other directives, if an [enable directive](#) is present, it must appear before all [declarations](#) and [const assertions](#). Extension names are not [identifiers](#): they do not [resolve](#) to [declarations](#).

The valid [enable-extensions](#) are listed in the following table.

Enable-extensions

WGSL enable-extension	WebGPU GPUFeatureName	Description
<i>f16</i>	"shader-f16"	The f16 type is valid to use in the WGSL module. Otherwise, using f16 (directly or indirectly) will result in a shader-creation error .
<i>clip_distances</i>	"clip-distances"	The built-in variable clip_distances is valid to use in the WGSL module. Otherwise, using clip_distances will result in a shader-creation error .
<i>dual_source_blending</i>	"dual-source-blending"	The attribute blend_src is valid to use in the WGSL module. Otherwise, using blend_src will result in a shader-creation error .
<i>subgroups</i>	"subgroups"	It is valid to use subgroup built-in values , subgroup built-in functions , and quad built-in functions in a WGSL module. Otherwise, any use will result in a shader-creation error .
<i>primitive_index</i>	"primitive-index"	The built-in variable primitive_index is valid to use in the WGSL module. Otherwise, using primitive_index will result in a shader-creation error .
<i>subgroup_size_control</i>	"subgroup-size-control"	The attribute subgroup_size is valid to use in the WGSL module. Otherwise, using subgroup_size will result in a shader-creation error . Must be enabled together with the subgroups extension. Otherwise, enabling subgroup_size_control will result in a shader-creation error .

EXAMPLE: USING HYPOTHETICAL ENABLE-EXTENSIONS

```

// Enable a hypothetical extension for arbitrary precision floating point types.
enable arbitrary_precision_float;
enable arbitrary_precision_float; // A redundant enable directive is ok.

// Enable a hypothetical extension to control the rounding mode.
enable rounding_mode;

// Assuming arbitrary_precision_float enables use of:
//   - a type f<E,M>
//   - as a type in function return, formal parameters and let-declarations
//   - as a value constructor from AbstractFloat
//   - operands to division operator: /
// Assuming @rounding_mode attribute is enabled by the rounding_mode enable directive.
@rounding_mode(round_to_even)
fn halve_it(x : f<8, 7>) -> f<8, 7> {
    let two = f<8, 7>(2);
    return x / 2; // uses round to even rounding mode.
}

```

§ 4.1.2. Language Extensions

A *language extension* is an [extension](#) which is automatically available if the implementation supports it. The program does not have to explicitly request it.

[Language extensions](#) embody functionality which could reasonably be supported on any WebGPU implementation. If the feature is not universally supported, that it is because some WebGPU implementation has not yet implemented it.

NOTE: For example, do-while loops could be a language extension.

The [wgpuLanguageFeatures](#) member of the WebGPU [GPU](#) object lists the set of [language extensions](#) supported by the implementation.

A *requires-directive* is a [directive](#) that *documents* the program's use of one or more [language extensions](#). It does not change the functionality exposed by the implementation. A [shader-creation error](#) results if the implementation does not support one of the required extensions.

A WGSL module *can* use a [requires-directive](#) to signal the potential for non-portability, and to signal the *intended* minimum bar for portability.

NOTE: Tooling outside of a WebGPU implementation could check whether all the [language extensions](#) used by a program are covered by [requires-directives](#) in the program.

requires_directive :

```
'requires' language_extension_list ';' ;
```

language_extension_list :

```
language_extension_name ( ',' language_extension_name ) * ',' ?
```

Like other directives, if a [requires-directive](#) is present, it must appear before all [declarations](#) and [const assertions](#). Extension names are not [identifiers](#): they do not [resolve](#) to [declarations](#).

Language extensions

WGSL language extension	Description
<i>readonly_and_readwrite_storage_textures</i>	Allows the use of read and read_write access modes with storage textures . Additionally, adds the textureBarrier built-in function.
<i>packed_4x8_integer_dot_product</i>	Supports using 32-bit integer scalars packing 4-component vectors of 8-bit integers as inputs to the dot product instructions with dot4U8Packed and dot4I8Packed built-in functions. Additionally, adds packing and unpacking instructions with packed 4-component vectors of 8-bit integers with pack4xI8 , pack4xU8 , pack4xI8Clamp , pack4xU8Clamp , unpack4xI8 , and unpack4xU8 built-in functions.
<i>unrestricted_pointer_parameters</i>	Removes the following restrictions from user-defined functions : For user-defined functions , a parameter of pointer type must be in one of the following address spaces: function private

WGSL language extension	Description
	Each argument of pointer type to a user-defined function must have the same memory view as its root identifier. NOTE: This means no vector, matrix, array, or structure access expressions can be applied to produce a memory view into the root identifier when traced from the argument back through all the let-declarations.
<i>pointer_composite_access</i>	Supports composite-value decomposition expressions where the root expression is a pointer, yielding a reference. For example, if <i>p</i> is a pointer to a structure with member <i>m</i>, then <i>p.m</i> is a reference to the memory locations for <i>m</i> inside the structure <i>p</i> points to. Similarly, if <i>pa</i> is a pointer to an array, then <i>pa[i]</i> is a reference to the memory locations for the <i>i</i>'th element of the array <i>pa</i> points to.
<i>uniform_buffer_standard_layout</i>	Allow buffers in the uniform address space to use the same memory layout constraints as other address spaces.
<i>subgroup_id</i>	Allows the use of the subgroup_id and num_subgroups built-in values when the subgroups extension is enabled.
<i>subgroup_uniformity</i>	Adds an additional scope, subgroup, for uniform control flow subgroup and quad built-in functions to be all invocations in the same subgroup. NOTE: The use of subgroup and quad built-in functions requires the subgroups extension to be enabled.
<i>texture_and_sampler_let</i>	Allows the effective-value-type of a let-declaration to be a texture or sampler type.
<i>texture_formats_tier1</i>	Supports additional texel formats: rgba16unorm, rgba16snorm, rg8unorm, rg8snorm, rg8uint, rg8sint, rg16unorm, rg16snorm, rg16uint, rg16sint, rg16float, r8unorm, r8snorm, r8uint, r8sint, r16unorm, r16snorm, r16uint, r16sint, r16float, rgb10a2unorm, rgb10a2uint, rg11b10float
<i>linear_indexing</i>	Supports the global_invocation_index and workgroup_index built-in values.
<i>immediate_address_space</i>	Enables the immediate address space, allowing variables to be declared with <code>var<immediate></code> and bound to small amounts of frequently updated data passed directly from the command encoder via the WebGPU API.

WGSL language extension	Description
	Enables the use of buffer types and the buffer_view built-in functions .
<i>buffer_view</i>	Allow the declaration of variables with an opaque store type that can be reinterpreted as other host-shareable types.

NOTE: The intent is that, over time, WGSL will define language extensions embodying all functionality in language extensions commonly supported at that time. In a [requires-directive](#), these serve as a shorthand for listing all those common features. They represent progressively increasing sets of functionality, and can be thought of as language versions, of a sort.

§ 4.2. Global Diagnostic Filter

A *global diagnostic filter* is a diagnostic filter whose [affected range](#) is the whole WGSL module. It is a [directive](#), thus appearing before any [module-scope](#) declarations. It is spelled like the attribute form, but without the leading @ (U+0040) code point, and with a terminating semicolon.

```
diagnostic_directive :
    'diagnostic' diagnostic_control ';' ;
```

§ 5. Declaration and Scope

A *declaration* associates an [identifier](#) with one of the following kinds of objects:

- a [type](#)
- a [type-generator](#)
- a [value](#)
- a [variable](#)
- a [function](#)
- a [formal parameter](#)
- an [enumerant](#)

In other words, a declaration introduces a *name* for an object.

A declaration is at *module scope* if the declaration appears in the program source, but outside the text of any other declaration.

A [function](#) declaration appears at module-scope. A function declaration contains declarations for formal parameters, if it has any, and it may contain [variable and value declarations](#) inside its [body](#). Those contained declarations are therefore not at module-scope.

NOTE: The only kind of declaration that contain another declaration is a [function declaration](#).

Certain objects are provided by the WebGPU implementation, and are treated as if they have been declared before the start of the WGSL module source. We say such objects are *predeclared*. For example, WGSL predeclares:

- [built-in functions](#),

- built-in types such as [i32](#) and [f32](#),
- built-in [type-generators](#) such as `array`, `ptr`, and `texture_2d`, and
- [enumerants](#) such as `read_write`, `workgroup`, and `rgba8unorm`.

The *scope* of a declaration is the set of program source locations where a declared identifier potentially denotes its associated object. We say the identifier is *in scope* (of the declaration) at those source locations.

Where a declaration appears determines its scope:

- Predeclared objects, and objects declared at module-scope, are [in scope](#) across the entire program source.
- Each [formal parameter](#) of a user-declared function is [in scope](#) across the corresponding [function body](#). See [§ 11.1 Declaring a User-defined Function](#).
- Otherwise, the scope is a span of text beginning immediately after the end of the declaration. For details, see [§ 7 Variable and Value Declarations](#).

Two declarations in the same WGSL source program [must not](#) simultaneously:

- introduce the same identifier name, and
- have the same end-of-scope.

NOTE: A predeclared object does not have a declaration in the WGSL source. So a user-specified declaration at module-scope or inside a function can have the same name as a predeclared object.

Identifiers are used as follows, distinguished by grammatical context:

- A token matching the [ident](#) grammar element is:
 - Used in a [declaration](#), as the name of the object being declared, or
 - Used as a name, denoting an object declared elsewhere. This is the common case.
- A token matching the [member_ident](#) grammar element is:
 - Used in a [structure](#) type [declaration](#), as the name of a [member](#), or
 - Used as a name, denoting a member of a structure value, or denoting a reference to a member of a structure. See [§ 8.5.4 Structure Access Expression](#).

When an [ident](#) token appears as a name denoting an object declared elsewhere, it [must](#) be [in scope](#) for some declaration. The object denoted by the identifier token is determined as follows:

- If the token is in scope for at least one non-[module-scope](#) declaration, then the token denotes the object associated with the nearest of those declarations.

NOTE: The nearest such declaration appears before the identifier token.

- Otherwise, if there is a module-scope declaration with that name, then the token denotes that declared object.

NOTE: The module-scope declaration may appear before **or after** the identifier token.

- Otherwise, if there is a [predeclared](#) object with that name, then the token denotes that object.

When the above algorithm is used to map an identifier to a declaration, we say the identifier *resolves* to that declaration. Similarly, we also say the identifier [resolves](#) to the declared object.

It is a [shader-creation error](#) if any [module scope](#) declaration is recursive. That is, no cycles can exist among the declarations:

Consider the directed graph where:

- Each node corresponds to a declaration D .
- There is an edge from declaration D to declaration T when the definition for D mentions an identifier which [resolves](#) to T .

This graph must not have a cycle.

NOTE: The [function body](#) is part of the [function declaration](#), thus functions must not be recursive, either directly or indirectly.

NOTE: Non-[module scope](#) identifier declarations must precede their uses in the text.

EXAMPLE: VALID AND INVALID DECLARATIONS

```

// Valid, user-defined variables can have the same name as a built-in function.
var<private> modf: f32 = 0.0;

// Valid, foo_1 is in scope for the entire program.
var<private> foo: f32 = 0.0; // foo_1

// Valid, bar_1 is in scope for the entire program.
var<private> bar: u32 = 0u; // bar_1

// Valid, my_func_1 is in scope for the entire program.
// Valid, foo_2 is in scope until the end of the function.
fn my_func(foo: f32) { // my_func_1, foo_2
    // Any reference to 'foo' resolves to the function parameter.

    // Invalid, modf resolves to the module-scope variable.
    let res = modf(foo);

    // Invalid, the scope of foo_2 ends at the of the function.
    var foo: f32; // foo_3

    // Valid, bar_2 is in scope until the end of the function.
    var bar: u32; // bar_2
    // References to 'bar' resolve to bar_2
    {
        // Valid, foo_4 is in scope until the end of the compound statement.
        var foo : f32; // foo_4

        // Valid, bar_3 is in scope until the end of the compound statement.
        var bar: u32; // bar_3
        // References to 'bar' resolve to bar_3

        // Invalid, bar_4 has the same end scope as bar_3.
        var bar: i32; // bar_4

        // Valid, i_1 is in scope until the end of the for loop
        for ( var i: i32 = 0; i < 10; i++ ) { // i_1
            // Invalid, i_2 has the same end scope as i_1.
            var i: i32 = 1; // i_2.
        }
    }

    // Invalid, bar_5 has the same end scope as bar_2.
    var bar: u32; // bar_5

    // Valid, later_def, a module scope declaration, is in scope for the entire program.
    var early_use : i32 = later_def;
}

// Invalid, bar_6 has the same scope as bar_1.
var<private> bar: u32 = 1u; // bar_6

// Invalid, my_func_2 has the same end scope as my_func_1.
fn my_func() { } // my_func_2

// Valid, my_foo_1 is in scope for the entire program.

```

```
fn my_foo( //my_foo_1
  // Valid, my_foo_2 is in scope until the end of the function.
  my_foo: i32 // my_foo_2
) { }

var<private> later_def : i32 = 1;
```

EXAMPLE: SHADOWING PREDECLARED OBJECTS

```
// This declaration hides the predeclared 'min' built-in function.
// Since this declaration is at module-scope, it is in scope over the entire
// source. The built-in function is no longer accessible.
fn min() -> u32 { return 0; }

const rgba8unorm = 12; // This shadows the predeclared 'rgba8unorm' enumerant.
```

§ 6. Types

Programs calculate values.

In WGSL, a **type** is a set of values, and each value belongs to exactly one type. A value's type determines the syntax and semantics of operations that can be performed on that value.

For example, the mathematical number 1 corresponds to these distinct values in WGSL:

- the 32-bit signed integer value 1i,
- the 32-bit unsigned integer value 1u,
- the 32-bit floating point value 1.0f,
- the 16-bit floating point value 1.0h if the [f16 extension](#) is enabled,
- the [AbstractInt](#) value 1, and
- the [AbstractFloat](#) value 1.0

WGSL treats these as different because their machine representation and operations differ.

Some types are **opaque**, meaning their values have an implementation-defined representation that cannot be directly inspected, decomposed, or manipulated within a WGSL program.

A type is either [predeclared](#), or created in WGSL source via a [declaration](#).

Some types are expressed as [template parameterizations](#). A **type-generator** is a [predeclared](#) object which, when parameterized with a [template list](#), denotes a type. For example, the type `atomic<u32>` combines the type-generator `atomic` with template list `<u32>`.

We distinguish between the *concept* of a type and the *syntax* in WGSL to denote that type. In many cases the spelling of a type in this specification is the same as its WGSL syntax. For example:

- the set of 32-bit unsigned integer values is spelled `u32` in this specification, and also in a WGSL module.
- the spelling is different for structure types, or types containing structures.

Some WGSL types are only used for analyzing a source program and for determining the program's runtime behavior. This specification will describe such types, but they do not appear in WGSL source text.

NOTE: [Reference types](#) are not written in WGSL modules. See [§ 6.5.3 Reference and Pointer Types](#).

§ 6.1. Type Checking

A WGSL value is computed by evaluating an expression. An *expression* is a segment of source text parsed as one of the WGSL grammar rules whose name ends with "expression". An expression E can contain *subexpressions* which are expressions properly contained in the outer expression E . A *top-level expression* is an expression that is not itself a subexpression. See [§ 8.18 Expression Grammar Summary](#).

The particular value produced by an expression evaluation depends on:

- *static context*: the source text surrounding the expression, and
- *dynamic context*: the state of the invocation evaluating the expression, and the execution context in which the invocation is running.

The values that may result from evaluating a particular expression will always belong to a specific WGSL type, known as the *static type* of the expression. The rules of WGSL are designed so that the static type of an expression depends only on the expression's [static context](#).

A *type assertion* is a mapping from some WGSL source expression to a WGSL type. The notation

$$e : T$$

is a type assertion meaning T is the static type of WGSL expression e .

NOTE: A type assertion is a statement of fact about the text of a program. It is not a runtime check.

Statements often use expressions, and may place requirements on the static types of those expressions. For example:

- The condition expression of an if statement [must](#) be of type [bool](#).
- In a let declaration with an explicit type specified, the initializer expression [must](#) evaluate to that type.

Type checking a successfully parsed WGSL module is the process of mapping each expression to its static type, and verifying that type requirements of each statement are satisfied. If type checking fails, a special case of a [shader-creation error](#), called a *type error*, results.

Type checking can be performed by recursively applying [type rules](#) to syntactic phrases, where a *syntactic phrase* is either an [expression](#) or a [statement](#). A *type rule* describes how the [static context](#) for a [syntactic phrase](#) determines the static type for expressions contained within that phrase. A [type rule](#) has two parts:

- A *conclusion*.
 - If the phrase is an expression, the conclusion is a [type assertion](#) for the expression.
 - If the phrase is a statement, the conclusion is a set of [type assertions](#), one for each of the statement's [top-level expressions](#).
 - In both cases, the [syntactic phrases](#) are specified schematically, using *italicized* names to denote subexpressions or other syntactically-determined parameters.
- *Preconditions*, consisting of:
 - For expressions:

- Type assertions for subexpressions, when it has subexpressions. Each may be satisfied directly, or via a [feasible automatic conversion](#) (as defined in § 6.1.2 Conversion Rank). WGSL [will](#) evaluate all [const-expressions](#) required to determine the [static types](#) of a program.
- How the expression is used in a statement.
- For statements:
 - The syntactic form of the statement, and
 - Type assertions for [top-level expressions](#) in the statement.
- Conditions on the other schematic parameters, if any.
- Optionally, other static context.

Type rules may have *type parameters* in their preconditions and conclusions. When a type rule's conclusion or preconditions contain type parameters, we say it is *parameterized*. When they do not, we say the rule is *fully elaborated*. We can make a fully elaborated type rule from a parameterized one by substituting a type for each of its type parameters, using the same type for all occurrences of a given parameter in the rule. An assignment of types to a rule's type parameters is called a *substitution*.

For example, here is the type rule for [logical negation](#) (an expression of the form `!e`):

Precondition	Conclusion
$e: T$ T is bool or <code>vecN<bool></code>	$!e: T$

This is a parameterized rule, because it contains the type parameter T , which can represent any one of four types `bool`, `vec2<bool>`, `vec3<bool>`, or `vec4<bool>`. Applying the substitution that maps T to `vec3<bool>` produces the fully elaborated type rule:

Precondition	Conclusion
$e: \text{vec3<bool>}$	$!e: \text{vec3<bool>}$

Each fully elaborated rule we can produce from a parameterized rule by applying some substitution that meets the rule's other conditions is called an *overload* of the parameterized rule. For example, the boolean negation rule has four overloads, because there are four possible ways to assign a type to its type parameter T .

NOTE: In other words, a parameterized type rule provides the pattern for a collection of fully elaborated type rules, each one produced by applying a different substitution to the parameterized rule.

A *type rule applies to a syntactic phrase* when:

- The rule's conclusion matches a valid parse of the [syntactic phrase](#), and
- The rule's preconditions are satisfied.

A [parameterized](#) type rule applies to an expression if there exists a [substitution](#) producing a [fully elaborated](#) type rule that applies to the expression.

Consider the expression, `1u+2u`. It has two [literal subexpressions](#): `1u` and `2u`, both of type `u32`. The [top-level expression](#) is an addition. Referring to the rules in § 8.7 Arithmetic Expressions, the type rule for addition applies to the expression, because:

- $1u+2u$ matches a parse of the form $e1+e2$, with $e1$ standing for $1u$ and $e2$ standing for $2u$, and
- $e1$ is of type `u32`, and
- $e2$ is of type `u32`, and
- we can substitute `u32` for the type parameter T in the type rule, resulting in a [fully elaborated](#) rule that applies to the entire expression.

When analyzing a [syntactic phrase](#), three cases may occur:

- No type rules apply to the expression. This results in a [type error](#).
- Exactly one [fully elaborated](#) type rule applies to the expression. In this case, the rule's [conclusion](#) is asserted, determining the static type for the expression.
- More than one type rule applies. That is, the preconditions for more than one [overload](#) are satisfied. In this case the tie-breaking procedure described in [§ 6.1.3 Overload Resolution](#) is used.
 - If overload resolution succeeds, a single [overload](#) is determined to apply to the expression. The [type assertions](#) in the [conclusion](#) for that overload are asserted, and therefore determine the types for the expression or expressions in the [syntactic phrase](#).
 - If overload resolution fails, a [type error](#) results.

Continuing the example above, only one type rule applies to the expression $1u+2u$, and so type checking accepts the conclusion of that type rule, which is that $1u+2u$ is of type `u32`.

A WGSL source program is *well-typed* when:

- The static type can be determined for each expression in the program by applying the type rules, and
- The type requirements for each statement are satisfied.

Otherwise there is a [type error](#) and the source program is not a valid WGSL module.

WGSL is a *statically typed language* because type checking a WGSL module [will](#) either succeed or discover a type error, while only having to inspect the program source text.

§ 6.1.1. Type Rule Tables

The WGSL [type rules](#) for expressions are organized into *type rule tables*, with one row per type rule.

The *semantics of an expression* is the effect of evaluating that expression, and is primarily the production of a result value. The *Description* column of the type rule that applies to an expression will specify the expression's semantics. The semantics usually depends on the values of the type rule parameters, including the assumed values of any subexpressions. Sometimes the semantics of an expression includes effects other than producing a result value, such as the non-result-value effects of its subexpressions.

EXAMPLE: SIDE-EFFECT OF AN EXPRESSION

```
fn foo(p : ptr<function, i32>) -> i32 {
    let x = *p;
    *p += 1;
    return x;
}

fn bar() {
    var a: i32;
    let x = foo(&a); // the call to foo returns a value
                    // and updates the value of a
}
```

§ 6.1.2. Conversion Rank

When a type assertion $e:T$ is used as a [type rule precondition](#), it is satisfied when:

- e is already of type T , or
- e is of type S , and type S is [automatically convertible](#) to type T , as defined below.

The rule is codified by the **ConversionRank** function over pairs of types, defined in the table below. The [ConversionRank](#) function expresses the preference and feasibility of automatically converting a value of one type (Src) to another type ($Dest$). Lower ranks are more desirable.

A **feasible automatic conversion** converts a value from type Src to type $Dest$, and is allowed when [ConversionRank](#)($Src, Dest$) is finite. Such conversions are value-preserving, subject to limitations described in [§ 15.7 Floating Point Evaluation](#).

NOTE: Automatic conversions only occur in two kinds of situations. First, when converting a [const-expression](#) to its corresponding typed numeric value that can be used on the GPU. Second, when a load from a reference-to-memory occurs, yielding the value stored in that memory.

NOTE: A conversion of infinite rank is infeasible, i.e. not allowed.

NOTE: When no conversion is performed, the conversion rank is zero.

ConversionRank from one type to another

Src	Dest	ConversionRank(Src, Dest)	Description
T	T	0	Identity. No conversion performed.
ref< AS, T, AM > for address space AS , and where access mode AM is read or read_write .	T	0	Apply the Load Rule to load a value from a memory reference.

Src	Dest	ConversionRank(Src, Dest)	Description
AbstractFloat	f32	1	See § 15.7.6 Floating Point Conversion
AbstractFloat	fl6	2	See § 15.7.6 Floating Point Conversion
AbstractInt	i32	3	Identity if the value is in i32 . Produces a shader-creation error otherwise.
AbstractInt	u32	4	Identity if the value is in u32 . Produces a shader-creation error otherwise.
AbstractInt	AbstractFloat	5	See § 15.7.6 Floating Point Conversion
AbstractInt	f32	6	Behaves as AbstractInt to AbstractFloat , and then AbstractFloat to f32
AbstractInt	fl6	7	Behaves as AbstractInt to AbstractFloat , and then AbstractFloat to fl6
$\text{vec}N\langle S \rangle$	$\text{vec}N\langle T \rangle$	$\text{ConversionRank}(S, T)$	Inherit conversion rank from component type.
$\text{mat}CxR\langle S \rangle$	$\text{mat}CxR\langle T \rangle$	$\text{ConversionRank}(S, T)$	Inherit conversion rank from component type.
$\text{array}\langle S, N \rangle$	$\text{array}\langle T, N \rangle$	$\text{ConversionRank}(S, T)$	Inherit conversion rank from component type. Note: Only

Src	Dest	ConversionRank(Src, Dest)	Description
			fixed-size arrays may have an abstract component type.
<code>__frexp_result_abstract</code>	<code>__frexp_result_f32</code>	1	
<code>__frexp_result_abstract</code>	<code>__frexp_result_f16</code>	2	
<code>__frexp_result_vecN_abstract</code>	<code>__frexp_result_vecN_f32</code>	1	
<code>__frexp_result_vecN_abstract</code>	<code>__frexp_result_vecN_f16</code>	2	
<code>__modf_result_abstract</code>	<code>__modf_result_f32</code>	1	
<code>__modf_result_abstract</code>	<code>__modf_result_f16</code>	2	
<code>__modf_result_vecN_abstract</code>	<code>__modf_result_vecN_f32</code>	1	
<code>__modf_result_vecN_abstract</code>	<code>__modf_result_vecN_f16</code>	2	
<code>ptr<AS, buffer<N1>, AM></code> where Src is the type of an argument to a function call	<code>ptr<AS, buffer<N2>, AM></code> where <i>N1</i> and <i>N2</i> are both const-expressions and Dest is the type of the formal parameter for the argument in Src	1 if $N2 < N1$, infinity otherwise	Convert a fixed-size buffer pointer argument to a smaller sized pointer parameter. ¹
<code>ptr<AS, buffer<N>, AM></code> where Src is the type of an argument to a function call	<code>ptr<AS, buffer, AM></code> where Dest is the type of the formal parameter for the argument in Src	1	Convert a fixed-size buffer pointer argument to a runtime-sized buffer pointer parameter. ¹
<i>S</i>	<i>T</i> where above cases don't apply	infinity	There are no automatic conversions between other types.

1. These conversions require the [unrestricted pointer parameters](#) and [buffer_view](#) language features.

The type *T* is the *concretization* of type *S* if:

- *T* is [concrete](#), and
- *T* is not a reference type, and
- ConversionRank(*S*, *T*) is finite, and
- For any other non-reference type *T2*, ConversionRank(*S*, *T2*) > ConversionRank(*S*, *T*).

The *concretization of a value* e of type T is the value resulting from applying, to e , the feasible conversion that maps T to the concretization of T .

NOTE: Conversion to [f32](#) is always preferred over [f16](#), therefore automatic conversion will only ever produce an [f16](#) value if the [f16 extension](#) is enabled in the module.

§ 6.1.3. Overload Resolution

When more than one [type rule applies to a syntactic phrase](#), a tie-breaking procedure is used to determine which one should take effect. This procedure is called *overload resolution*, and assumes type checking has already succeeded in finding static types for [subexpressions](#).

Consider a [syntactic phrase](#) P , and all [type rules that apply](#) to P . The overload resolution algorithm calls these type rules *overload candidates*. For each candidate:

- Its preconditions have been met either directly or through [automatic conversion](#).
- Its [conclusion](#) has:
 - A syntactic form matching a valid parse of P , and
 - A [type assertion](#) corresponding to each [top-level expression](#) in P .

Overload resolution for P proceeds as follows, with the goal of finding a single most [preferable overload candidate](#):

1. For each candidate C , enumerate conversion ranks for subexpressions in the syntactic phrase. The candidate's preconditions have been met, and so for the i 'th subexpression in the P :
 - Its static type has been computed.
 - There is a [feasible automatic conversion](#) from the expression's static type to the type required by the corresponding type assertion in the preconditions. Let $C.R(i)$ be the [ConversionRank](#) of that conversion.
2. Eliminate any candidate where one of its subexpressions resolves to an [abstract](#) type after [feasible automatic conversions](#), but another of the candidate's subexpressions is not a [const-expression](#).

NOTE: As a consequence, if any subexpression in the phrase is not a [const-expression](#), then all subexpressions in the phrase must have a [concrete](#) type.

3. Rank candidates: Given two overload candidates $C1$ and $C2$, $C1$ is *preferred* over $C2$ if:
 - For each expression position i in P , $C1.R(i) \leq C2.R(i)$.
 - That is, each expression conversion required to apply $C1$ to P is at least as preferable as the corresponding expression conversion required to apply $C2$ to P .
 - There is at least one expression position i where $C1.R(i) < C2.R(i)$.
 - That is, there is at least one expression conversion required to apply $C1$ that is strictly more preferable than the corresponding conversion required to apply $C2$.
4. If there is a single candidate C which is [preferred](#) over all the others, then overload resolution succeeds, yielding the candidate type rule C . Otherwise, overload resolution fails.

§ 6.2. Plain Types

[Plain types](#) are types for the machine representation of boolean values, numbers, vectors, matrices, or aggregations of such values.

A *plain type* is either a [scalar](#) type, an [atomic](#) type, or a [composite](#) type.

NOTE: Plain types in WGSL are similar to Plain-Old-Data types in C++, but also include atomic types and abstract numeric types.

§ 6.2.1. Abstract Numeric Types

These types cannot be spelled in WGSL source. They are only used by [type checking](#).

Certain expressions are evaluated at [shader-creation time](#), and with a numeric range and precision that may be larger than directly implemented by the GPU.

WGSL defines two *abstract numeric types* for these evaluations:

- The *AbstractInt* type is the set of integers representable in the 64-bit two's complement format, with the sign bit in the most significant bit position.
- The *AbstractFloat* type is the set of finite floating point numbers representable in the [IEEE-754 binary64](#) (double precision) format.

An evaluation of an expression in one of these types [must not](#) overflow or produce an infinite or NaN value.

A type is *abstract* if it is an abstract numeric type or contains an abstract numeric type. A type is *concrete* if it is not abstract.

A [numeric literal](#) without a suffix denotes a value in an [abstract numeric type](#):

- An [integer literal](#) without an i or u suffix denotes an [AbstractInt](#) value.
- A [floating point literal](#) without an f or h suffix denotes an [AbstractFloat](#) value.

Example: The expression `log2(32)` is analyzed as follows:

- `log2(32)` is parsed as a function call to the `log2` builtin function with operand [AbstractInt](#) value 32.
- There is no overload of `log2` with an [integer scalar](#) formal parameter.
- Instead [overload resolution](#) applies, considering three possible overloads and [feasible automatic conversions](#):
 - [AbstractInt](#) to [AbstractFloat](#). (Conversion rank 4)
 - [AbstractInt](#) to [f32](#). (Conversion rank 5)
 - [AbstractInt](#) to [f16](#). (Conversion rank 6)
- The resulting computation occurs as an [AbstractFloat](#) (e.g. `log2(32.0)`).

Example: The expression `1 + 2.5` is analyzed as follows:

- `1 + 2.5` is parsed as an addition operation with subexpressions [AbstractInt](#) value 1, and [AbstractFloat](#) value 2.5.
- There is no overload for `e+f` where `e` is integer type and `f` is floating point.
- However, using feasible automatic conversions, there are three potential overloads:
 - 1 is converted to [AbstractFloat](#) value 1.0 (rank 4) and 2.5 remains an [AbstractFloat](#) (rank 0).
 - 1 is converted to [f32](#) value 1.0f (rank 5) and 2.5 is converted to [f32](#) value 2.5f (rank 1).

- 1 is converted to [f16](#) value 1.0f (rank 6) and 2.5 is converted to [f16](#) value 2.5h (rank 2).
- The first overload is the [preferable candidate](#) and type checking succeeds.
- The resulting computation occurs as an [AbstractFloat](#) $1.0 + 2.5$.

Example: `let x = 1 + 2.5;`

- This example is similar to the above, except that `x` cannot resolve to an [abstract numeric type](#).
- Therefore, there are two viable overload candidates: addition using [f32](#) or [f16](#).
- The [preferable candidate](#) uses [f32](#).
- The effect of the declaration is as if it were written `let x : f32 = 1.0f + 2.5f;`.

Example: `1u + 2.5` results in a [shader-creation error](#):

- The `1u` term is an expression of type [u32](#).
- The `2.5` term is an expression of type [AbstractFloat](#).
- There are no valid overload candidates:
 - There is no feasible automatic conversion from a GPU-materialized [integer scalar](#) type to a floating point type.
 - No type rule matches $e+f$ with e in an [integer scalar](#) type, and f in a floating point type.

Example: `-1 * i32(-2147483648)` does not result in a [shader-creation error](#):

- The `-1` term is an expression of type [AbstractInt](#).
- The `i32(-2147483648)` term is an expression of type [i32](#).
- There is no overload of the multiply operator for these two types and the [i32](#) term cannot be up-converted to [AbstractInt](#).
- The only feasible automatic conversion is converting the [AbstractInt](#) to [i32](#) so:
 - The resulting computation is a multiplication operation between [i32](#).
 - [i32](#) operations use two's complement arithmetic and have defined overflow behavior.
 - Wrapping occurs.

EXAMPLE: TYPE INFERENCE FOR LITERALS

```

// Explicitly-typed unsigned integer literal.
var u32_1 = 1u; // variable holds a u32

// Explicitly-typed signed integer literal.
var i32_1 = 1i; // variable holds a i32

// Explicitly-typed floating point literal.
var f32_1 = 1f; // variable holds a f32

// Explicitly-typed unsigned integer literal cannot be negated.
var u32_neg = -1u; // invalid: unary minus does not support u32

// When a concrete type is required, but no part of the statement or
// expression forces a particular concrete type, an integer literal is
// interpreted as an i32 value:
//   Initializer for a let-declaration must be constructible (or pointer).
//   The most preferred automatic conversion from AbstractInt to a constructible type
//   is AbstractInt to i32, with conversion rank 2. So '1' is inferred as i32.
let some_i32 = 1; // like let some_i32: i32 = 1i;

// Inferred from declaration type.
var i32_from_type : i32 = 1; // variable holds i32. AbstractInt to i32, conversion rank 2
var u32_from_type : u32 = 1; // variable holds u32. AbstractInt to u32, conversion rank 3

// Unsuffixed integer literal can convert to floating point when needed:
//   Automatically convert AbstractInt to f32, with conversion rank 5.
var f32_promotion : f32 = 1; // variable holds f32

// Invalid: no feasible conversion from floating point to integer
var i32_demotion : i32 = 1.0; // Invalid

// Inferred from expression.
var u32_from_expr = 1 + u32_1; // variable holds u32
var i32_from_expr = 1 + i32_1; // variable holds i32

// Values must be representable.
let u32_too_large : u32 = 1234567890123456890; // invalid, overflow
let i32_too_large : i32 = 1234567890123456890; // invalid, overflow
let u32_large : u32 = 2147483649; // valid
let i32_large : i32 = 2147483649; // invalid, overflow
let f32_out_of_range1 = 0x1p500; // invalid, out of range
let f32_hex_lost_bits = 0x1.0000000001p0; // invalid, not exactly representable in f32

// Minimum integer: unary negation over AbstractInt, then infer i32.
// Most preferred conversion from AbstractInt to a constructible type (with lowest
// conversion rank) is AbstractInt to i32.
let i32_min = -2147483648; // has type i32

// Invalid. Select AbstractInt to i32 as above, but the value is out of
// range, producing shader-creation error.
let i32_too_large_2 = 2147483648; // Invalid.

// Subexpressions can resolve to AbstractInt and AbstractFloat.
// The following examples are all valid and the value of the variable is 6u.
var u32_expr1 = (1 + (1 + (1 + (1 + 1)))) + 1u;

```

```

var u32_expr2 = 1u + (1 + (1 + (1 + (1 + 1))));
var u32_expr3 = (1 + (1 + (1 + (1u + 1)))) + 1;
var u32_expr4 = 1 + (1 + (1 + (1 + (1u + 1))));

// Inference based on built-in function parameters.

// Most-preferred candidate is clamp(i32,i32,i32)->i32
let i32_clamp = clamp(1, -5, 5);
// Most preferred candidate is clamp(u32,u32,u32).
// Literals use automatic conversion AbstractInt to u32.
let u32_clamp = clamp(5, 0, u32_from_expr);
// Most preferred candidate is clamp(f32,f32,f32)->f32
// literals use automatic conversion AbstractInt to f32.
let f32_clamp = clamp(0, f32_1, 1);

// The following examples all promote to f32 with an initial value of 10f.
let f32_promotion1 = 1.0 + 2 + 3 + 4;
let f32_promotion2 = 2 + 1.0 + 3 + 4;
let f32_promotion3 = 1f + ((2 + 3) + 4);
let f32_promotion4 = ((2 + (3 + 1f)) + 4);

// Type rule violations.

// Invalid, the initializer can only resolve to f32:
// No feasible automatic conversion from AbstractFloat to u32.
let mismatch : u32 = 1.0;

// Invalid. There is no overload of clamp that allows mixed sign parameters.
let ambiguous_clamp = clamp(1u, 0, 1i);

// Inference completes at the statement level.

// Initializer for a let-declaration must be constructible (or pointer).
// The most preferred automatic conversion from AbstractInt to a constructible type
// is AbstractInt to i32, with conversion rank 2. So '1' is inferred as i32.
let some_i32 = 1; // like let some_i32: i32 = 1i;

let some_f32 : f32 = some_i32; // Type error: i32 cannot be assigned to f32

// Another overflow case
let overflow_u32 = (1 - 2) + 1u; // invalid, -1 is out of range of u32

// Ideal value out of range of 32-bits, but brought back into range
let out_and_in_again = (0xffffffff / 8);

// Similar, but invalid
let out_of_range = (0xffffffff / 8u); // requires computation is done in 32-bits,
// making 0xffffffff out of range.

```

§ 6.2.2. Boolean Type

The *bool* type contains the values true and false.

Boolean literal type rules

Precondition	Conclusion	Description
	true: bool	The true value.
	false: bool	The false value.

§ 6.2.3. Integer Types

The ***u32*** type is the set of 32-bit unsigned integers.

The ***i32*** type is the set of 32-bit signed integers. It uses the two's complement representation, with the sign bit in the most significant bit position.

[Expressions](#) on [concrete](#) integer types that overflow produce a result that is modulo 2^{bitwidth}

Extreme values for integer types

Type	Lowest value	Highest value
i32	i32(-2147483648)	2147483647i
	i32(-0x80000000)	0x7fffffffî
u32	0u	4294967295u
	0x0u	0xffffffffu

NOTE: [AbstractInt](#) is also an integer type.

§ 6.2.4. Floating Point Types

The ***f32*** type is the set of 32-bit floating point values of the [IEEE-754 binary32](#) (single precision) format. See [§ 15.7 Floating Point Evaluation](#) for details.

The ***f16*** type is the set of 16-bit floating point values of the [IEEE-754 binary16](#) (half precision) format. It is a [shader-creation error](#) if the ***f16*** type is used unless the program contains the `enable f16;` directive to enable the [f16 extension](#). See [§ 15.7 Floating Point Evaluation](#) for details.

The following table lists certain extreme values for floating point types. Each has a corresponding negative value.

Extreme values for floating point types

Type	Smallest positive subnormal	Smallest positive normal	Largest positive finite	Largest finite power of 2
------	---	--	-------------------------	---------------------------

Type	Smallest positive subnormal	Smallest positive normal	Largest positive finite	Largest finite power of 2
f32	1.40129846432481707092e-45f	1.17549435082228750797e-38f	3.40282346638528859812e+38f	0x1p+127f
f16	0x1p-149f	0x1p-126f	0x1.fffffep+127f	0x1p+15h

NOTE: [AbstractFloat](#) is also a floating point type.

u32	0x1p-14h	0x1.ffffp+15h		
-----	----------	---------------	--	--

The *scalar* types are [bool](#), [AbstractInt](#), [AbstractFloat](#), [i32](#), [u32](#), [f32](#), and [f16](#).

The *numeric scalar* types are [AbstractInt](#), [AbstractFloat](#), [i32](#), [u32](#), [f32](#), and [f16](#).

The *integer scalar* types are [AbstractInt](#), [i32](#), and [u32](#).

A *scalar conversion* maps a value in one scalar type to a value in a different scalar type. Generally the result value is close to the original value, within the limitations of the destination type. Scalar conversions occur either:

- By explicitly invoking a [value constructor](#), or
- When converting a [const-expression](#) in an [abstract numeric type](#) to another type, via a [feasible automatic conversion](#).

§ 6.2.6. Vector Types

A *vector* is a grouped sequence of 2, 3, or 4 [scalar](#) components.

Type	Description
$\text{vec}_{N<T>}$	Vector of N components of type T . N must be in $\{2, 3, 4\}$ and T must be one of the scalar types. We say T is the <i>component type</i> of the vector.

A vector is a *numeric vector* if its component type is a [numeric scalar](#).

Key use cases of a vector include:

- to express both a direction and a magnitude.
- to express a position in space.
- to express a color in some color space. For example, the components could be intensities of red, green, and blue, while the fourth component could be an alpha (opacity) value.

Many operations on vectors (and [matrices](#)) act *component-wise*, i.e. the result is formed by operating on each scalar component independently.

EXAMPLE: VECTOR

```
vec2<f32> // is a vector of two f32s.
```

EXAMPLE: COMPONENT-WISE ADDITION

```
let x : vec3<f32> = a + b; // a and b are vec3<f32>
// x[0] = a[0] + b[0]
// x[1] = a[1] + b[1]
// x[2] = a[2] + b[2]
```

WGSL also [predeclares](#) the following [type aliases](#):

Predeclared alias	Original type	Restrictions
<i>vec2i</i>	vec2<i32>	
<i>vec3i</i>	vec3<i32>	
<i>vec4i</i>	vec4<i32>	
<i>vec2u</i>	vec2<u32>	
<i>vec3u</i>	vec3<u32>	
<i>vec4u</i>	vec4<u32>	
<i>vec2f</i>	vec2<f32>	
<i>vec3f</i>	vec3<f32>	
<i>vec4f</i>	vec4<f32>	
<i>vec2h</i>	vec2<f16>	
<i>vec3h</i>	vec3<f16>	Requires the f16 extension .
<i>vec4h</i>	vec4<f16>	

§ 6.2.7. Matrix Types

A *matrix* is a grouped sequence of 2, 3, or 4 floating point vectors.

Type	Description
matCxR<T>	Matrix of <i>C</i> columns and <i>R</i> rows of type <i>T</i> , where <i>C</i> and <i>R</i> are both in {2, 3, 4}, and <i>T</i> must be f32 , f16 , or AbstractFloat . Equivalently, it can be viewed as <i>C</i> column vectors of type vecR<T>.

The key use case for a matrix is to embody a linear transformation. In this interpretation, the vectors of a matrix are treated as column vectors.

The product operator (*) is used to either:

- scale the transformation by a scalar magnitude.
- apply the transformation to a vector.
- combine the transformation with another matrix.

See [§ 8.7 Arithmetic Expressions](#).

EXAMPLE: MATRIX

```
mat2x3<f32> // This is a 2 column, 3 row matrix of 32-bit floats.  
             // Equivalently, it is 2 column vectors of type vec3<f32>.
```

WGSL also [predeclares](#) the following [type aliases](#):

Predeclared alias	Original type	Restrictions
<i>mat2x2f</i>	mat2x2<f32>	
<i>mat2x3f</i>	mat2x3<f32>	
<i>mat2x4f</i>	mat2x4<f32>	
<i>mat3x2f</i>	mat3x2<f32>	
<i>mat3x3f</i>	mat3x3<f32>	
<i>mat3x4f</i>	mat3x4<f32>	
<i>mat4x2f</i>	mat4x2<f32>	
<i>mat4x3f</i>	mat4x3<f32>	
<i>mat4x4f</i>	mat4x4<f32>	
<i>mat2x2h</i>	mat2x2<f16>	
<i>mat2x3h</i>	mat2x3<f16>	
<i>mat2x4h</i>	mat2x4<f16>	
<i>mat3x2h</i>	mat3x2<f16>	
<i>mat3x3h</i>	mat3x3<f16>	Requires the f16 extension .
<i>mat3x4h</i>	mat3x4<f16>	
<i>mat4x2h</i>	mat4x2<f16>	
<i>mat4x3h</i>	mat4x3<f16>	
<i>mat4x4h</i>	mat4x4<f16>	

§ 6.2.8. Atomic Types

An *atomic type* encapsulates a [concrete integer scalar](#) type such that:

- atomic objects provide [certain guarantees](#) to concurrent observers, and
- the only valid operations on atomic objects are the [atomic builtin functions](#).

Type	Description
<code>atomic<T></code>	Atomic of type <i>T</i> . <i>T</i> must be either u32 or i32 .

An expression [must not](#) evaluate to an atomic type.

Atomic types may only be instantiated by variables in the [workgroup](#) address space or by [storage buffer](#) variables with a [read_write](#) access mode. The [memory scope](#) of operations on the type is determined by the [address space](#) it is instantiated in. Atomic types in the [workgroup](#) address space have a [memory scope](#) of Workgroup, while those in the [storage](#) address space have a memory scope of QueueFamily.

An *atomic modification* is any [operation](#) on an atomic object which sets the content of the object. The operation counts as a modification even if the new value is the same as the object's existing value.

In WGSL, atomic modifications are mutually ordered, for each object. That is, during execution of a shader stage, for each atomic object *A*, all agents observe the same order of modification operations applied to *A*. The ordering for distinct atomic objects may not be related in any way; no causality is implied. Note that variables in [workgroup](#) space are shared within a [workgroup](#), but are not shared between different workgroups.

§ 6.2.9. Array Types

An *array* is an indexable sequence of element values.

Type	Description
<code>array<E,N></code>	A <i>fixed-size array</i> with <i>N</i> elements of type <i>E</i> . <i>N</i> is called the <i>element count</i> of the array.
<code>array<E></code>	A <i>runtime-sized</i> array of elements of type <i>E</i> . These may only appear in specific contexts.

The first element in an array is at index 0, and each successive element is at the next integer index. See [§ 8.5.3 Array Access Expression](#).

An expression [must not](#) evaluate to a runtime-sized array type.

The element count expression *N* of a fixed-size array is subject to the following constraints:

- It [must](#) be an [override-expression](#).
- It [must](#) evaluate to a [concrete integer scalar](#).
- If *N* is not greater than zero:
 - It is a [shader-creation error](#) if *N* is a [const-expression](#).
 - Otherwise, it is a [pipeline-creation error](#).

NOTE: The element count value is fully determined at [pipeline creation](#) time if N depends on any [override-declarations](#), and [shader module creation](#) otherwise.

NOTE: To qualify for type-equivalency, any override expression that is not a const expression must be an **identifier**. See [Workgroup variables sized by overridable constants](#)

The number of elements in a runtime-sized array is determined by the size of buffer binding associated with the corresponding [storage buffer](#) variable. See [§ 13.3.4 Buffer Binding Determines Runtime-Sized Array Element Count](#).

An array element type [must](#) be one of:

- a [scalar](#) type
- a [vector](#) type
- a [matrix](#) type
- an [atomic](#) type
- an [array](#) type having a [creation-fixed footprint](#)
- a [structure](#) type having a [creation-fixed footprint](#).

NOTE: An array element type cannot be a [buffer](#) type.

NOTE: The element type must be a [plain type](#).

Two array types are the same if and only if all of the following are true:

- They have the same element type.
- Their element count specifications match, i.e. one of the following is true:
 - They are both runtime-sized.
 - They are both fixed-sized with [creation-fixed footprint](#), and equal-valued element counts, even if one is signed and the other is unsigned. (Signed and unsigned values are comparable in this case because element counts are always positive.)
 - They are both fixed-sized with element counts specified as identifiers [resolving](#) to the same declaration of a [pipeline-overridable](#) constant.

EXAMPLE: EXAMPLE FIXED-SIZE ARRAY TYPES, NON-OVERRIDABLE ELEMENT COUNT

```
// array<f32,8> and array<i32,8> are different types:
// different element types
var<private> a: array<f32,8>;
var<private> b: array<i32,8>;
var<private> c: array<i32,8u>; // array<i32,8> and array<i32,8u> are the same type

const width = 8;
const height = 8;

// array<i32,8>, array<i32,8u>, and array<i32,width> are the same type.
// Their element counts evaluate to 8.
var<private> d: array<i32,width>;

// array<i32,height> and array<i32,width> are the same type.
var<private> e: array<i32,width>;
var<private> f: array<i32,height>;
```

NOTE: The only valid use of an array type sized by an overridable constant is as a [memory view](#) in the [workgroup](#) address space. This includes the [store type](#) of a workgroup variable. See [§ 7 Variable and Value Declarations](#).

EXAMPLE: WORKGROUP VARIABLES SIZED BY OVERRIDABLE CONSTANTS

```
override blockSize = 16;

var<workgroup> odds: array<i32,blockSize>;
var<workgroup> evens: array<i32,blockSize>; // Same type

// None of the following have the same type as 'odds' and 'evens'.

// Different type: Not the identifier 'blockSize'
var<workgroup> evens_0: array<i32,16>;
// Different type: Uses arithmetic to express the element count.
var<workgroup> evens_1: array<i32,(blockSize * 2 / 2)>;
// Different type: Uses parentheses, not just an identifier.
var<workgroup> evens_2: array<i32,(blockSize)>;

// An invalid example, because the overridable element count may only occur
// at the outer level.
// var<workgroup> both: array<array<i32,blockSize>,2>;

// An invalid example, because the overridable element count is only
// valid for workgroup variables.
// var<private> bad_address_space: array<i32,blockSize>;
```

§ 6.2.10. Structure Types

A **structure** is a named grouping of named **member** values.

Type	Description
<code>struct <i>AStructName</i> { $M_I : T_I$, ... $M_N : T_N$, }</code>	A declaration of a structure type named by the identifier <i>AStructName</i> and having <i>N</i> members, where member <i>i</i> is named by the identifier M_i and is of the type T_i. <i>N</i> must be at least 1. Two members of the same structure type must not have the same name.

Structure types are declared at [module scope](#). Elsewhere in the program source, a structure type is denoted by its identifier name. See [§ 5 Declaration and Scope](#).

Two structure types are the same if and only if they have the same name.

A structure member type [must](#) be one of:

- a [scalar](#) type
- a [vector](#) type
- a [matrix](#) type
- an [atomic](#) type
- a [fixed-size array](#) type with [creation-fixed footprint](#)
- a [runtime-sized](#) array type, but only if it is the last member of the structure
- a [structure](#) type that has a [creation-fixed footprint](#)

NOTE: A structure member type cannot be a [buffer](#) type.

NOTE: All user-declared structure types are [concrete](#).

NOTE: Each member type must be a [plain type](#).

Some consequences of the restrictions structure member and array element types are:

- A pointer, texture, or sampler [must not](#) appear in any level of nesting within an array or structure.
- When a [runtime-sized](#) array is part of a larger type, it may only appear as the last element of a structure, which itself cannot be part of an enclosing array or structure.

EXAMPLE: STRUCTURE

```
// A structure with three members.
struct Data {
    a: i32,
    b: vec2<f32>,
    c: array<i32,10>, // last comma is optional
}

// Declare a variable storing a value of type Data.
var<private> some_data: Data;
```

struct_decl :

'struct' *ident struct_body_decl*

struct_body_decl :

'{' *struct_member* (**','** *struct_member*) * **','** ? **'}'**

struct_member :

attribute * *member_ident* **':'** *type_specifier*

The following attributes can be applied to structure members:

- [align](#)
- [builtin](#)
- [location](#)
- [blend_src](#)
- [interpolate](#)
- [invariant](#)
- [size](#)

Attributes [builtin](#), [location](#), [blend_src](#), [interpolate](#), and [invariant](#) are [IO attributes](#). An [IO attribute](#) on a member of a structure *S* has effect only when *S* is used as the type of a [formal parameter](#) or [return type](#) of an [entry point](#). See § 13.3.1 [Inter-stage Input and Output Interface](#).

Attributes [align](#) and [size](#) are [layout attributes](#), and may be required if the structure type is used to define a [uniform buffer](#) or a [storage buffer](#). See § 14.4 [Memory Layout](#).

EXAMPLE: STRUCTURE DECLARATION

```
struct my_struct {
    a: f32,
    b: vec4<f32>
}
```

EXAMPLE: STRUCTURE USED TO DECLARE A BUFFER

```
// Runtime Array
alias RTArr = array<vec4<f32>>>;
struct S {
    a: f32,
    b: f32,
    data: RTArr
}
@group(0) @binding(0) var<storage> buffer: S;
```

§ 6.2.11. Composite Types

A type is **composite** if it has internal structure expressed as a composition of other types. The internal parts do not overlap, and are called **components**. A composite value may be decomposed into its components. See [§ 8.5 Composite Value Decomposition Expressions](#).

The composite types are:

- [vector](#) type
- [matrix](#) type
- [array](#) type
- [structure](#) type

For a composite type T , the **nesting depth** of T , written $NestDepth(T)$ is:

- 1 for a vector type
- 2 for a matrix type
- $1 + NestDepth(E)$ for an array type with element type E
- $1 + \max(NestDepth(M_1), \dots, NestDepth(M_N))$ if T is a structure type with member types $M_1, \dots, M_N$

§ 6.2.12. Constructible Types

Many kinds of values can be created, loaded, stored, passed into functions, and returned from functions. We call these [constructible](#).

A type is **constructible** if it is one of:

- a [scalar](#) type
- a [vector](#) type
- a [matrix](#) type
- a [fixed-size array](#) type, if it has [creation-fixed footprint](#) and its element type is constructible.
- a [structure](#) type, if all its members are constructible.

NOTE: All constructible types have a [creation-fixed footprint](#).

NOTE: Atomic types and runtime-sized array types are not constructible. Composite types containing atomics and runtime-sized arrays are not constructible.

§ 6.2.13. Fixed-Footprint Types

The *memory footprint* of a variable is the number of [memory locations](#) used to store the contents of the variable. The memory footprint of a variable depends on its [store type](#) and becomes finalized at some point in the [shader lifecycle](#). Most variables are sized very early, at [shader creation](#) time. Some variables may be sized later, at [pipeline creation](#) time, and others as late as the [start of shader execution](#).

A type has a *creation-fixed footprint* if its [concretization](#) has a size that is fully determined at [shader creation](#) time.

A type has a *fixed footprint* if its size is fully determined at [pipeline creation](#) time.

NOTE: All [concrete](#) creation-fixed footprint and fixed footprint types are [storable](#).

NOTE: Pipeline creation depends on shader creation, so a type with [creation-fixed footprint](#) also has [fixed footprint](#).

The types with [creation-fixed footprint](#) are:

- a [scalar](#) type
- a [vector](#) type
- a [matrix](#) type
- an [atomic](#) type
- a [fixed-size array](#) type, when:
 - its [element count](#) is a [const-expression](#).
- a [structure](#) type, if all its members have [creation-fixed footprint](#).
- a [fixed-size buffer](#) type, when:
 - its size is a [const-expression](#).

NOTE: A [constructible](#) type has a [creation-fixed footprint](#).

The types with a [fixed footprint](#) are any of:

- a type with [creation-fixed footprint](#)
- a [fixed-size array](#) type (without further constraining its [element count](#))
- a [fixed-size buffer](#) type (without further constraining its size)

NOTE: The only valid use of a fixed-size array with an element count that is an [override-expression](#) that is not a [const-expression](#) is as a [memory view](#) in the [workgroup](#) address space. This includes the [store type](#) of a workgroup variable.

NOTE: A fixed-footprint type may contain an [atomic](#) type, either directly or indirectly, while a [constructible](#) type cannot.

NOTE: Fixed-footprint types exclude [runtime-sized](#) arrays, runtime-sized buffers, and any structure that contains a runtime-sized array.

§ 6.3. Buffer Types

These types require the [buffer_view](#) language feature.

A *buffer* value denotes memory whose contents can be interpreted as another [host-shareable](#) type. Buffer values are otherwise [opaque](#). They are useful when the data in a variable needs to be accessed as multiple different types or the shader author wishes to sub-divide one variable into multiple logical variables.

Buffer types are not [constructible](#) and can only be accessed after interpreting their data as a non-opaque, [host-shareable](#) type via the [buffer_view built-in functions](#).

Type	Description
<code>buffer<N></code>	A <i>fixed-size buffer</i> with N bytes of storage. N must be an override-expression .
<code>buffer</code>	A <i>runtime-sized buffer</i> .

An [expression must not](#) evaluate to a buffer type.

Fixed-size buffers may only be instantiated by variables in the [storage](#), [uniform](#), and [workgroup](#) address spaces. In the storage and uniform address spaces, the size [must](#) be a [const-expression](#). Runtime-sized buffers can only be instantiated by variables in the storage address space.

If N is not greater than 0:

- It is a [shader-creation error](#) if N is a [const-expression](#).
- Otherwise, It is a [pipeline-creation error](#).

If the [f16 extension](#) is enabled and N is not evenly divisible by 2:

- It is a [shader-creation error](#) if N is a [const-expression](#).
- Otherwise, It is a [pipeline-creation error](#).

If the [f16 extension](#) is not enabled and N is not evenly divisible by 4:

- It is a [shader-creation error](#) if N is a [const-expression](#).
- Otherwise, It is a [pipeline-creation error](#).

Two buffer types are the same if and only if any of the following conditions hold:

- They are both runtime-sized.
- They are both fixed-sized with a [creation-fixed footprint](#), and equal-valued sizes, even if one is signed and the other is unsigned. (Signed and unsigned values are comparable in this case because sizes are always positive.)
- They are both fixed-sized with sizes specified as identifiers resolving to the same declaration of a [pipeline-overridable](#) constant.

§ 6.4. Enumeration Types

An *enumeration* type is a limited set of named values. An enumeration is used to distinguish among the set of possibilities for a specific concept, such as the set of valid [texel formats](#).

An *enumerant* is one of the named values in an [enumeration](#). Each [enumerant](#) is distinct from all other enumerants, and distinct from all other kinds of values.

There is no mechanism for declaring new enumerants or new enumeration types in WGSL source.

NOTE: Enumerants are used as [template parameters](#).

NOTE: There is no way to copy or to create an alternative name for an enumerant:

- A [variable or value](#) declaration cannot have an enumeration as its [store type](#) or its [effective-value-type](#).
- A function formal parameter cannot be an enumeration type, in part because enumerations are not [constructible](#).

§ 6.4.1. Predeclared enumerants

The following table lists the [enumeration](#) types in WGSL, their [predeclared enumerants](#) and the required [language extensions](#) for the [enumeration](#) types. The enumeration types exist, but cannot be spelled in WGSL source.

Predeclared enumerants

Enumeration (Cannot be spelled in WGSL)	Predeclared enumerant	Required language extension
access mode	read	
	write	
	read_write	
address space NOTE: The handle address space is never written in a WGSL source.	function	
	private	
	workgroup	
	uniform	
	storage	
texel format	rgba8unorm	
	rgba8snorm	
	rgba8uint	
	rgba8sint	
	rgba16unorm	texture_formats_tier1
	rgba16snorm	texture_formats_tier1

Enumeration (Cannot be spelled in WGSL)	Predeclared enumerant	Required language extension
	rgba16uint	
	rgba16sint	
	rgba16float	
	rg8unorm	texture_formats_tier1
	rg8snorm	texture_formats_tier1
	rg8uint	texture_formats_tier1
	rg8sint	texture_formats_tier1
	rg16unorm	texture_formats_tier1
	rg16snorm	texture_formats_tier1
	rg16uint	texture_formats_tier1
	rg16sint	texture_formats_tier1
	rg16float	texture_formats_tier1
	r32uint	
	r32sint	
	r32float	
	rg32uint	
	rg32sint	
	rg32float	
	rgba32uint	
	rgba32sint	
	rgba32float	
	bgra8unorm	
	r8unorm	texture_formats_tier1
	r8snorm	texture_formats_tier1
	r8uint	texture_formats_tier1
	r8sint	texture_formats_tier1
	r16unorm	texture_formats_tier1
	r16snorm	texture_formats_tier1
	r16uint	texture_formats_tier1

Enumeration (Cannot be spelled in WGSL)	Predeclared enumerant	Required language extension
	r16sint	texture_formats_tier1
	r16float	texture_formats_tier1
	rgb10a2unorm	texture_formats_tier1
	rgb10a2uint	texture_formats_tier1
	rg11b10float	texture_formats_tier1

§ 6.5. Memory Views

In addition to calculating with [plain](#) values, a WGSL program will also often read values from [memory](#) or write values to memory, via [memory access](#) operations. Each memory access is performed via a [memory view](#).

A *memory view* comprises:

- a set of [memory locations](#) in a particular [address space](#),
- a [memory model reference](#),
- an interpretation of the contents of those locations as a WGSL [type](#), known as the *store type*, and
- an [access mode](#).

The access mode of a memory view [must](#) be supported by the address space. See [§ 7 Variable and Value Declarations](#).

§ 6.5.1. Storable Types

The value contained in a [variable must](#) be of a [storable](#) type. A storable type may have an explicit representation defined by WGSL, as described in [§ 14.4.4 Internal Layout of Values](#), or it may be [opaque](#), such as for [textures](#) and [samplers](#).

A type is *storable* if it is both [concrete](#) and one of:

- a [scalar](#) type
- a [vector](#) type
- a [matrix](#) type
- an [atomic](#) type
- an [array](#) type
- a [structure](#) type
- a [texture](#) type
- a [sampler](#) type
- a [buffer](#) type

NOTE: That is, the storable types are the [concrete plain types](#), texture types, sampler types, and buffer types.

§ 6.5.2. Host-shareable Types

Host-shareable types are used to describe the contents of buffers which are shared between the host and the GPU, or copied between host and GPU without format translation. When used for this purpose, the type may additionally have [layout attributes](#) applied as described in [§ 14.4 Memory Layout](#). As described in [§ 7.3 var Declarations](#), the [store type](#) of [uniform buffer](#) and [storage buffer](#) variables [must](#) be host-shareable.

A type is *host-shareable* if it is both [concrete](#) and one of:

- a [numeric scalar](#) type
- a [numeric vector](#) type
- a [matrix](#) type
- an [atomic](#) type
- a [fixed-size array](#) type, if it has [creation-fixed footprint](#) and its element type is host-shareable
- a [runtime-sized](#) array type, if its element type is host-shareable
- a [structure](#) type, if all its members are host-shareable
- a [buffer](#) type

NOTE: Restrictions on the types of inter-stage inputs and outputs are described in [§ 13.3.1 Inter-stage Input and Output Interface](#) and subsequent sections. Those types are also sized, but the counting differs.

NOTE: [Textures and samplers](#) can also be shared between the host and the GPU, but their contents are [opaque](#). The host-shareable types in this section are specifically for use in [storage](#) and [uniform](#) buffers.

§ 6.5.3. Reference and Pointer Types

WGSL has two kinds of types for representing [memory views](#): [reference types](#) and [pointer types](#).

Constraint	Type	Description
AS is an address space , T is a storable type, AM is an access mode	$\text{ref}\langle AS, T, AM \rangle$	The <i>reference type</i> identified with the set of memory views for memory locations in AS holding values of type T, supporting memory accesses described by mode AM. Here, T is the store type. Reference types are not written in WGSL source; instead they are used to analyze a WGSL module.
AS is an address space , T is a storable type, AM is an access mode	$\text{ptr}\langle AS, T, AM \rangle$	The <i>pointer type</i> identified with the set of memory views for memory locations in AS holding values of type T, supporting memory accesses described by mode AM. Here, T is the store type. Pointer types may appear in WGSL source.

Two pointer types are the same if and only if they have the same address space, store type, and access mode.

When *analyzing* a WGSL module, reference and pointer types are fully parameterized by an address space, a storable type, and an access mode. In code examples in this specification, the comments show this fully parameterized form.

However, in WGSL *source* text:

- Reference types [must not](#) appear.
- Pointer types may appear.
 - A pointer type is spelled with parameterization by:
 - [address space](#),
 - [store type](#), and
 - sometimes by [access mode](#), as specified in § 14.3 Address Spaces.
 - If a pointer type appears in the program source, it [must](#) also be valid to [declare a variable](#) or return that type from a [function call](#), somewhere in the program, with the pointer type's [address space](#), [store type](#), and [access mode](#).

NOTE: This restriction forbids the declaration of certain [type aliases](#) and function [formal parameters](#) that can never be used at runtime. Without the restriction, it would be valid to declare an alias to a pointer type, but never be able to create a pointer value of that type. Similarly, it would be valid to declare a function with a pointer formal parameter, but never be able to call that function.

EXAMPLE: POINTER TYPE

```
fn my_function(
  /* 'ptr<function,i32,read_write>' is the type of a pointer value that references
     memory for keeping an 'i32' value, using memory locations in the 'function'
     address space. Here 'i32' is the store type.
     The implied access mode is 'read_write'.
     See "Address Space" section for defaults. */
  ptr_int: ptr<function,i32>,

  // 'ptr<private,array<f32,50>,read_write>' is the type of a pointer value that
  // refers to memory for keeping an array of 50 elements of type 'f32', using
  // memory locations in the 'private' address space.
  // Here the store type is 'array<f32,50>'.
  // The implied access mode is 'read_write'.
  // See the "Address space section for defaults.
  ptr_array: ptr<private, array<f32, 50>>
) { }
```

Reference types and pointer types are both sets of memory views: a particular memory view is associated with a unique reference value and also a unique pointer value:

Each pointer value p of type $\text{ptr}\langle AS, T, AM \rangle$ corresponds to a unique reference value r of type $\text{ref}\langle AS, T, AM \rangle$, and vice versa, where p and r describe the same memory view.

§ 6.5.4. Valid and Invalid Memory References

A [reference](#) value is either [valid](#) or [invalid](#).

References are formed as described in detail in [§ 6.5.8 Forming Reference and Pointer Values](#). Generally, a *valid reference* is formed by:

- naming a variable, or
- applying the [indirection](#) (unary `*`) operation to a [valid pointer](#), or
- a [named component expression](#) where the [base](#) is a valid [memory view](#), or
- an [indexing expression](#) where the [base](#) is a valid [memory view](#), and using an [in-bounds index](#).

Generally, an *invalid memory reference* is formed by:

- applying the indirection operator to an [invalid pointer](#), or
- a [named component expression](#) where the [base](#) is an invalid memory reference, or
- an [indexing expression](#) where the base is a [memory view](#), and either:
 - the [base](#) is an invalid memory reference, or
 - the index is [out-of-bounds](#).

An *invalid pointer* is formed by:

- applying the [address-of](#) operator (unary `&`) to an [invalid memory reference](#), or
- calling a [bufferView](#) or [bufferArrayView](#) built-in function when the [memory view](#) of the resulting pointer would otherwise include [memory locations outside the bounds](#) of the memory view of buffer pointer argument.

A *valid pointer* is a pointer that corresponds to a [valid reference](#).

6.5.5. Originating Variable

The *originating variable* for a reference value R is defined as follows:

- It is the variable, when R is a variable.
- It is the originating variable of the pointer value P , when R is the application of the indirection operator (unary `*`) on P .
- It is the originating variable of the [base](#), when R is a [named component expression](#) or an [indexing expression](#).

The [originating variable](#) of a pointer value is defined as the originating variable of the corresponding reference value.

NOTE: The originating variable is a dynamic concept. The originating variable for a formal parameter of a function depends on the [call sites](#) for the function. Different call sites may supply pointers into different originating variables.

A [valid reference](#) always corresponds to a non-empty memory view for some or all of the memory locations for some variable.

NOTE: A reference can correspond to memory locations inside a variable, and still be invalid. This can occur when an index is too large for the type being indexed, but the referenced locations would be inside a subsequent sibling data member.

In the following example, the reference `the_particle.position[i]` is valid if and only if `i` is 0 or 1. When `i` is 2, the reference will be an [invalid memory reference](#), but would otherwise correspond the memory locations for `the_particle.color_index`.

EXAMPLE: INVALID MEMORY REFERENCE STILL INSIDE A VARIABLE

```
struct Particle {
    position: vec2f,
    velocity: vec2f,
    color_index: i32,
}

@group(0) @binding(0)
var<storage,read_write> the_particle: Particle;

fn particle_velocity_component(p: Particle, i: i32) -> f32 {
    return the_particle.velocity[i]; // A valid reference when i is 0 or 1.
}
```

§ 6.5.6. Out-of-Bounds Access

An operation that [accesses](#) an [invalid memory reference](#) is an *out-of-bounds access*.

An out-of-bounds access is a program defect, because if it *were* performed as written, it would typically:

- read or write [memory locations](#) outside of a variable, or
- interpret the contents of those locations as the wrong [store type](#), or
- cause an unintended data race.

For this reason, an implementation [will not](#) perform the access as written. Executing an [out-of-bounds access](#) generates a [dynamic error](#).

NOTE: An example of interpreting the store type incorrectly occurs in [the example from the previous section](#). When `i` is 2, the expression `the_particle.velocity[i]` has type `ref<storage,f32,read_write>`, meaning it is a memory view with `f32` as its [store type](#). However, the memory locations are allocated to the `color_index` member, so the stored value is actually of type `i32`.

NOTE: An out-of-bounds access causes a dynamic error, which allows for many possible outcomes.

Those outcomes include, but are not limited to, the following:

Trap

The shader invocation immediately terminates, and [shader stage outputs](#) are set to zero values.

Invalid Load

[Loads](#) from an invalid reference may return one of:

- when the [originating variable](#) is a [uniform buffer](#) or a [storage buffer](#), the value from any [memory location\(s\)](#) of the WebGPU [GPUBuffer](#) bound to the originating variable
- when the originating variable is not a [uniform buffer](#) or [storage buffer](#), a value from any [memory location\(s\)](#) in the originating variable
- the [zero value](#) for store type of the reference
- if the loaded value is a vector, the value (0, 0, 0, x), where x is:
 - 0, 1, or the maximum positive value for integer components
 - 0.0 or 1.0 for floating-point components

Invalid Store

[Stores](#) to an invalid reference may do one of:

- when the [originating variable](#) is a [storage buffer](#), store the value to any [memory location\(s\)](#) of the WebGPU [GPUBuffer](#) bound to the [originating variable](#)
- when the [originating variable](#) is not a [storage buffer](#), store the value to any [memory locations\(s\)](#) in the originating variable
- not be executed.

A data race may occur if an invalid load or store is redirected to access different locations inside a variable in a shared address space. For example, the accesses of several concurrently executing invocations may be redirected to the first element in an array. If at least one access is a write, and they are not otherwise synchronized, then the result is a data race, and hence a dynamic error.

An out-of-bounds access invalidates the assumptions of [uniformity analysis](#). For example, if an invocation terminates early due to an out-of-bounds access, then it can no longer participate in collective operations. In particular, a call to [workgroupBarrier](#) may hang the shader, and derivatives may yield invalid results.

§ 6.5.7. Use Cases for References and Pointers

References and pointers are distinguished by how they are used:

- The type of a [variable](#) is a reference type.
- The [address-of](#) operation (unary &) converts a reference value to its corresponding pointer value.
- The [indirection](#) operation (unary *) converts a pointer value to its corresponding reference value.
- A [let-declaration](#) can be of pointer type, but not of reference type.
- A [formal parameter](#) can be of pointer type, but not of reference type.
- A [simple assignment](#) statement performs a [write access](#) to update the contents of memory via a reference, where:
 - The [left-hand side](#) of the assignment statement [must](#) be of reference type, with access mode [write](#) or [read_write](#).
 - The [right-hand side](#) of the assignment statement [must](#) evaluate to the store type of the left-hand side.

- The **Load Rule**: Inside a function, a reference is automatically dereferenced (read from) to satisfy type rules:
 - In a function, when a reference expression r with store type T is used in a statement or an expression, where
 - r has an access mode of [read](#) or [read_write](#), and
 - The only potentially matching type rules require r to have a value of type T , then
 - That type rule requirement is considered to have been met, and
 - The result of evaluating r in that context is the value (of type T) stored in the memory locations referenced by r at the time of evaluation. That is, a [read access](#) is performed to produce the result value.

Defining references in this way enables simple idiomatic use of variables:

EXAMPLE: REFERENCE TYPES ENABLE SIMPLE USE OF VARIABLES

```
@compute @workgroup_size(1)
fn main() {
  // 'i' has reference type ref<function,i32,read_write>
  // The memory locations for 'i' store the i32 value 0.
  var i: i32 = 0;

  // 'i + 1' can only match a type rule where the 'i' subexpression is of type i32.
  // So the expression 'i + 1' has type i32, and at evaluation, the 'i' subexpression
  // evaluates to the i32 value stored in the memory locations for 'i' at the time
  // of evaluation.
  let one: i32 = i + 1;

  // Update the value in the locations referenced by 'i' so they hold the value 2.
  i = one + 1;

  // Update the value in the locations referenced by 'i' so they hold the value 5.
  // The evaluation of the right-hand-side occurs before the assignment takes effect.
  i = i + 3;
}
```


EXAMPLE: RETURNING A REFERENCE RETURNS THE VALUE LOADED VIA THE REFERENCE

```

var<private> age: i32;
fn get_age() -> i32 {
    // The type of the expression in the return statement must be 'i32' since it
    // must match the declared return type of the function.
    // The 'age' expression is of type ref<private,i32,read_write>.
    // Apply the Load Rule, since the store type of the reference matches the
    // required type of the expression, and no other type rule applies.
    // The evaluation of 'age' in this context is the i32 value loaded from the
    // memory locations referenced by 'age' at the time the return statement is
    // executed.
    return age;
}

fn caller() {
    age = 21;
    // The copy_age constant will get the i32 value 21.
    let copy_age: i32 = get_age();
}

```

Defining pointers in this way enables two key use cases:

- Using a let-declaration with pointer type, to form a short name for part of the contents of a variable.
- Using a formal parameter of a function to refer to the memory of a variable that is accessible to the [calling function](#).
 - The call to such a function [must](#) supply a pointer value for that operand. This often requires using an [address-of](#) operation (unary &) to get a pointer to the variable's contents.

EXAMPLE: USING A POINTER AS A SHORT NAME FOR PART OF A VARIABLE

```

struct Particle {
    position: vec3<f32>,
    velocity: vec3<f32>
}
struct System {
    active_index: i32,
    timestep: f32,
    particles: array<Particle,100>
}
@group(0) @binding(0) var<storage,read_write> system: System;

@compute @workgroup_size(1)
fn main() {
    // Form a pointer to a specific Particle in storage memory.
    let active_particle = &system.particles[system.active_index];

    let delta_position: vec3<f32> = (*active_particle).velocity * system.timestep;
    let current_position: vec3<f32> = (*active_particle).position;
    (*active_particle).position = delta_position + current_position;
}

```

EXAMPLE: USING A POINTER AS A FORMAL PARAMETER

```

fn add_one(x: ptr<function,i32>) {
    /* Update the locations for 'x' to contain the next higher integer value,
       (or to wrap around to the largest negative i32 value).
       On the left-hand side, unary '*' converts the pointer to a reference that
       can then be assigned to. It has a read_write access mode, by default.
       /* On the right-hand side:
       - Unary '*' converts the pointer to a reference, with a read_write
         access mode.
       - The only matching type rule is for addition (+) and requires '*x' to
         have type i32, which is the store type for '*x'. So the Load Rule
         applies and '*x' evaluates to the value stored in the memory for '*x'
         at the time of evaluation, which is the i32 value for 0.
       - Add 1 to 0, to produce a final value of 1 for the right-hand side. */
       Store 1 into the memory for '*x'. */
    *x = *x + 1;
}

@compute @workgroup_size(1)
fn main() {
    var i: i32 = 0;

    // Modify the contents of 'i' so it will contain 1.
    // Use unary '&' to get a pointer value for 'i'.
    // This is a clear signal that the called function has access to the memory
    // for 'i', and may modify it.
    add_one(&i);
    let one: i32 = i; // 'one' has value 1.
}

```

§ 6.5.8. Forming Reference and Pointer Values

A reference value is formed in one of the following ways:

- The [identifier resolving](#) to an [in-scope](#) variable v denotes the reference value for v 's memory.
- Use the [indirection](#) (unary `*`) operation on a pointer.
- Use a [named component expression](#) on a [memory view](#) to a composite:
 - Given a memory view with a [vector](#) store type, appending a single-letter vector access phrase results in a reference to the named component of the vector. See [§ 8.5.1.3 Component Reference from Vector Memory View](#).
 - Given a memory view with a [structure](#) store type, appending a member access phrase results in a reference to the named member of the structure. See [§ 8.5.4 Structure Access Expression](#).
- Use an [indexing expression](#) on a [memory view](#) to a composite:
 - Given a memory view with a [vector](#) store type, appending an array index access phrase results in a reference to the indexed component of the vector. See [§ 8.5.1.3 Component Reference from Vector Memory View](#).
 - Given a memory view with a [matrix](#) store type, appending an array index access phrase results in a reference to the indexed column vector of the matrix. See [§ 8.5.2 Matrix Access Expression](#).
 - Given a memory view with an [array](#) store type, appending an array index access phrase results in a reference to the indexed element of the array. See [§ 8.5.3 Array Access Expression](#).

In all cases, the [access mode](#) of the result is the same as the access mode of the original reference.

EXAMPLE: COMPONENT REFERENCE FROM A COMPOSITE REFERENCE



```

struct S {
    age: i32,
    weight: f32
}
var<private> person: S;
// Elsewhere, 'person' denotes the reference to the memory underlying the variable,
// and will have type ref<private,S,read_write>.

fn f() {
    var uv: vec2<f32>;
    // For the remainder of this function body, 'uv' denotes the reference
    // to the memory underlying the variable, and will have type
    // ref<function,vec2<f32>,read_write>.

    // Evaluate the left-hand side of the assignment:
    //   Evaluate 'uv.x' to yield a reference:
    //   1. First evaluate 'uv', yielding a reference to the memory for
    //   the 'uv' variable. The result has type ref<function,vec2<f32>,read_write>.
    //   2. Then apply the '.x' vector access phrase, yielding a reference to
    //   the memory for the first component of the vector pointed at by the
    //   reference value from the previous step.
    //   The result has type ref<function,f32,read_write>.
    // Evaluating the right-hand side of the assignment yields the f32 value 1.0.
    // Store the f32 value 1.0 into the storage memory locations referenced by uv.x.
    uv.x = 1.0;

    // Evaluate the left-hand side of the assignment:
    //   Evaluate 'uv[1]' to yield a reference:
    //   1. First evaluate 'uv', yielding a reference to the memory for
    //   the 'uv' variable. The result has type ref<function,vec2<f32>,read_write>.
    //   2. Then apply the '[1]' array index phrase, yielding a reference to
    //   the memory for second component of the vector referenced from
    //   the previous step. The result has type ref<function,f32,read_write>.
    // Evaluating the right-hand side of the assignment yields the f32 value 2.0.
    // Store the f32 value 2.0 into the storage memory locations referenced by uv[1].
    uv[1] = 2.0;

    var m: mat3x2<f32>;
    // When evaluating 'm[2]':
    // 1. First evaluate 'm', yielding a reference to the memory for
    // the 'm' variable. The result has type ref<function,mat3x2<f32>,read_write>.
    // 2. Then apply the '[2]' array index phrase, yielding a reference to
    // the memory for the third column vector pointed at by the reference
    // value from the previous step.
    // Therefore the 'm[2]' expression has type ref<function,vec2<f32>,read_write>.
    // The 'let' declaration is for type vec2<f32>, so the declaration
    // statement requires the initializer to be of type vec2<f32>.
    // The Load Rule applies (because no other type rule can apply), and
    // the evaluation of the initializer yields the vec2<f32> value loaded
    // from the memory locations referenced by 'm[2]' at the time the declaration
    // is executed.
    let p_m_col2: vec2<f32> = m[2];

    var A: array<i32,5>;
    // When evaluating 'A[4]'

```

```

// 1. First evaluate 'A', yielding a reference to the memory for
//   the 'A' variable. The result has type ref<function,array<i32,5>,read_write>.
// 2. Then apply the '[4]' array index phrase, yielding a reference to
//   the memory for the fifth element of the array referenced by
//   the reference value from the previous step.
//   The result value has type ref<function,i32,read_write>.
// The let-declaration requires the right-hand-side to be of type i32.
// The Load Rule applies (because no other type rule can apply), and
// the evaluation of the initializer yields the i32 value loaded from
// the memory locations referenced by 'A[4]' at the time the declaration
// is executed.
let A_4_value: i32 = A[4];

// When evaluating 'person.weight'
// 1. First evaluate 'person', yielding a reference to the memory for
//   the 'person' variable declared at module scope.
//   The result has type ref<private,S,read_write>.
// 2. Then apply the '.weight' member access phrase, yielding a reference to
//   the memory for the second member of the memory referenced by
//   the reference value from the previous step.
//   The result has type ref<private,f32,read_write>.
// The let-declaration requires the right-hand-side to be of type f32.
// The Load Rule applies (because no other type rule can apply), and
// the evaluation of the initializer yields the f32 value loaded from
// the memory locations referenced by 'person.weight' at the time the
// declaration is executed.
let person_weight: f32 = person.weight;

// Alternatively, references can also be formed from pointers using
// the same syntax.

let uv_ptr = &uv;
// For the remainder of this function body, 'uv_ptr' denotes a pointer
// to the memory underlying 'uv', and will have the type
// ptr<function,vec2<f32>,read_write>.

// Evaluate the left-hand side of the assignment:
//   Evaluate '*uv_ptr' to yield a reference:
//   1. First evaluate 'uv_ptr', yielding a pointer to the memory for
//      the 'uv' variable. The result has type ptr<function,vec2<f32>,read_write>.
//   2. Then apply the indirection expression operator, yielding a
//      reference to memory for 'uv'.
// Evaluating the right-hand side of the assignment yields the vec2<f32> value (1.0, 2.0).
// Store the value (1.0, 2.0) into the storage memory locations referenced by uv.
*uv_ptr = vec2f(1.0, 2.0);

// Evaluate the left-hand side of the assignment:
//   Evaluate 'uv_ptr.x' to yield a reference:
//   1. First evaluate 'uv_ptr', yielding a pointer to the memory for
//      the 'uv' variable. The result has type ptr<function,vec2<f32>,read_write>.
//   2. Then apply the '.x' vector access phrase, yielding a reference to
//      the memory for the first component of the vector pointed at by the
//      reference value from the previous step.
//      The result has type ref<function,f32,read_write>.
// Evaluating the right-hand side of the assignment yields the f32 value 1.0.
// Store the f32 value 1.0 into the storage memory locations referenced by uv.x.
uv_ptr.x = 1.0;

```

```

// Evaluate the left-hand side of the assignment:
//   Evaluate 'uv_ptr[1]' to yield a reference:
//   1. First evaluate 'uv_ptr', yielding a pointer to the memory for
//       the 'uv' variable. The result has type ptr<function,vec2<f32>,read_write>.
//   2. Then apply the '[1]' array index phrase, yielding a reference to
//       the memory for second component of the vector referenced from
//       the previous step. The result has type ref<function,f32,read_write>.
// Evaluating the right-hand side of the assignment yields the f32 value 2.0.
// Store the f32 value 2.0 into the storage memory locations referenced by uv[1].
uv_ptr[1] = 2.0;

let m_ptr = &m;
// When evaluating 'm_ptr[2]':
// 1. First evaluate 'm_ptr', yielding a pointer to the memory for
//    the 'm' variable. The result has type ptr<function,mat3x2<f32>,read_write>.
// 2. Then apply the '[2]' array index phrase, yielding a reference to
//    the memory for the third column vector pointed at by the reference
//    value from the previous step.
//    Therefore the 'm[2]' expression has type ref<function,vec2<f32>,read_write>.
// The 'let' declaration is for type vec2<f32>, so the declaration
// statement requires the initializer to be of type vec2<f32>.
// The Load Rule applies (because no other type rule can apply), and
// the evaluation of the initializer yields the vec2<f32> value loaded
// from the memory locations referenced by 'm[2]' at the time the declaration
// is executed.
let p_m_col2: vec2<f32> = m_ptr[2];

let A_ptr = &A;
// When evaluating 'A[4]'
// 1. First evaluate 'A', yielding a pointer to the memory for
//    the 'A' variable. The result has type ptr<function,array<i32,5>,read_write>.
// 2. Then apply the '[4]' array index phrase, yielding a reference to
//    the memory for the fifth element of the array referenced by
//    the reference value from the previous step.
//    The result value has type ref<function,i32,read_write>.
// The let-declaration requires the right-hand-side to be of type i32.
// The Load Rule applies (because no other type rule can apply), and
// the evaluation of the initializer yields the i32 value loaded from
// the memory locations referenced by 'A[4]' at the time the declaration
// is executed.
let A_4_value: i32 = A_ptr[4];

let person_ptr = &person;
// When evaluating 'person.weight'
// 1. First evaluate 'person_ptr', yielding a pointer to the memory for
//    the 'person' variable declared at module scope.
//    The result has type ptr<private,S,read_write>.
// 2. Then apply the '.weight' member access phrase, yielding a reference to
//    the memory for the second member of the memory referenced by
//    the reference value from the previous step.
//    The result has type ref<private,f32,read_write>.
// The let-declaration requires the right-hand-side to be of type f32.
// The Load Rule applies (because no other type rule can apply), and
// the evaluation of the initializer yields the f32 value loaded from
// the memory locations referenced by 'person.weight' at the time the

```

```
// declaration is executed.  
let person_weight: f32 = person_ptr.weight;  
}
```

A pointer value is formed in one of the following ways:

- Use the [address-of](#) (unary &) operator on a reference.
 - The result is a [valid pointer](#) if and only if the original reference is [valid](#).
 - The [originating variable](#) of a valid result is defined as the originating variable of the reference.
- If a function [formal parameter](#) has pointer type, then when the function is invoked at runtime the uses of the formal parameter denote the pointer value provided to the corresponding operand at the [call site](#) in the [calling function](#).
 - The value denoted by the formal parameter (at runtime) is a [valid pointer](#) if and only if the pointer value at the call site is [valid](#).
 - The [originating variable](#) of a [valid pointer](#) formal parameter (at runtime) is defined as the originating variable of the pointer operand at the call site.
- The result of a [bufferView](#) or [bufferArrayView](#) built-in function call.

In all cases, the [access mode](#) of the result is the same as the access mode of the original pointer.

EXAMPLE: POINTER FROM A VARIABLE

```
// Declare a variable in the private address space, for storing an f32 value.
var<private> x: f32;

fn f() {
  // Declare a variable in the function address space, for storing an i32 value.
  var y: i32;

  // The name 'x' resolves to the module-scope variable 'x',
  // and has reference type ref<private,f32,read_write>.
  // Applying the unary '&' operator converts the reference to a pointer.
  // The access mode is the same as the access mode of the original variable, so
  // the fully specified type is ptr<private,f32,read_write>. But read_write
  // is the default access mode for function address space, so read_write does not
  // have to be spelled in this case
  let x_ptr: ptr<private,f32> = &x;

  // The name 'y' resolves to the function-scope variable 'y',
  // and has reference type ref<private,i32,read_write>.
  // Applying the unary '&' operator converts the reference to a pointer.
  // The access mode defaults to 'read_write'.
  let y_ptr: ptr<function,i32> = &y;

  // A new variable, distinct from the variable declared at module scope.
  var x: u32;

  // Here, the name 'x' resolves to the function-scope variable 'x' declared in
  // the previous statement, and has type ref<function,u32,read_write>.
  // Applying the unary '&' operator converts the reference to a pointer.
  // The access mode defaults to 'read_write'.
  let inner_x_ptr: ptr<function,u32> = &x;
}
```

§ 6.5.9. Comparison with References and Pointers in Other Languages

This section is informative, not normative.

References and pointers in WGSL are more restricted than in other languages. In particular:

- In WGSL a reference can't directly be declared as an alias to another reference or variable, either as a variable or as a formal parameter.
- In WGSL pointers and references are not [storable](#). That is, the content of a WGSL [variable declaration](#) may not contain a pointer or a reference.
- In WGSL a function [must not](#) return a pointer or reference.
- In WGSL there is no way to convert between integer values and pointer values.
- In WGSL there is no way to forcibly change the type of a pointer value into another pointer type.
 - A composite component reference expression is different: it takes a reference to a composite value and yields a reference to one of the components or elements inside the composite value. These are considered different references in WGSL, even though they may have the same machine address at a lower level of implementation abstraction.

- In WGSL there is no way to forcibly change the type of a reference value into another reference type.
- In WGSL there is no way to change the access mode of a pointer or reference.
 - By comparison, C++ automatically converts a non-const pointer to a const pointer, and has a `const_cast` to convert a const value to a non-const value.
- In WGSL there is no way to allocate new memory from a "heap".
- In WGSL there is no way to explicitly destroy a variable. The memory for a WGSL variable becomes inaccessible only when the variable goes out of scope.

NOTE: From the above rules, it is not possible to form a "dangling" pointer, i.e. a pointer that does not reference the memory for a "live" originating variable. A [memory view](#) may be an [invalid memory reference](#), but it [will never](#) access [memory locations](#) not associated with the [originating variable](#) or buffer.

§ 6.6. Texture and Sampler Types

A ***texel*** is a scalar or vector used as the smallest independently accessible element of a [texture](#). The word *texel* is short for *texture element*.

A ***texture*** is a collection of texels supporting special operations useful for rendering. In WGSL, those operations are invoked via texture builtin functions. See [§ 17.7 Texture Built-in Functions](#) for a complete list.

A WGSL texture corresponds to a WebGPU [GPUTexture](#).

A texture has the following features:

texel format

The data representation of each texel. See [§ 6.6.1 Texel Formats](#).

dimensionality

The number of dimensions in the grid coordinates, and how the coordinates are interpreted. The number of dimensions is 1, 2, or 3. Most textures use cartesian coordinates. Cube textures have six square faces, and are sampled with a three dimensional coordinate interpreted as a direction vector from the origin toward the cube centered on the origin.

See [GPUTextureViewDimension](#).

size

The extent of grid coordinates along each dimension. This is a function of [mip level](#).

mip level count

The mip level count is at least 1 for [sampled textures](#) and [depth textures](#), and equal to 1 for [storage textures](#).

Mip level 0 contains a full size version of the texture. Each successive mip level contains a filtered version of the previous mip level at half the size (within rounding) of the previous mip level.

When sampling a texture, an explicit or implicitly-computed level-of-detail is used to select the mip levels from which to read texel data. These are then combined via filtering to produce the sampled value.

arrayed

Whether the texture is arrayed.

- A non-arrayed texture is a grid of texels.
- An arrayed texture is a homogeneous array of grids of texels.

array size

The number of homogeneous grids, if the texture is [arrayed](#).

sample count

The number of samples, if the texture is [multisampled](#).

Each texel in a texture is associated with a unique *logical texel address*, which is an integer tuple having:

- A [mip level](#) in $[0, \text{mip level count})$.
- A number of components, controlled by the [dimensionality](#), with each component value in $[0, S_i)$, where S_i is the [size](#) in the i 'th component.
- An array index in $[0, \text{array size})$, if the texture is [arrayed](#). Note that [size](#) is a function of [mip level](#).
- A sample index in $[0, \text{sample count})$, if the texture is [multisampled](#).

A texture's physical organization is typically optimized for rendering operations. To achieve this, many details are hidden from the programmer, including data layouts, data types, and internal operations that cannot be expressed directly in the shader language.

As a consequence, a shader does not have direct access to the texel memory within a [texture variable](#). Instead, access is mediated through an [opaque](#) handle:

- Within the shader:
 - Declare a module-scope variable where the [store type](#) is one of the texture types described in later sections. The variable stores an opaque handle to the underlying texture memory, and is automatically placed in the [handle](#) address space.
 - Inside a function, call one of the texture builtin functions, and provide the texture variable or function parameter as the builtin function's first parameter.
- When constructing the WebGPU pipeline, the texture variable's store type and binding [must](#) be compatible with the corresponding bind group layout entry.

In this way, the set of supported operations for a texture type is determined by the availability of [texture built-in functions](#) having a [formal parameter](#) with that texture type.

NOTE: The handle stored by a texture variable cannot be changed by the shader. That is, the variable is read-only, even if the underlying texture to which it provides access may be mutable (e.g. a write-only [storage texture](#)).

The *texture types* are the set of types defined in:

- [§ 6.6.2 Sampled Texture Types](#)
- [§ 6.6.3 Multisampled Texture Types](#)
- [§ 6.6.4 External Sampled Texture Types](#)
- [§ 6.6.5 Storage Texture Types](#)
- [§ 6.6.6 Depth Texture Types](#)

A [sampler](#) is an [opaque](#) handle that controls how [texels](#) are accessed from a [sampled texture](#) or a [depth texture](#).

A WGSL sampler maps to a WebGPU [GPUSampler](#).

Texel access is controlled via several properties of the sampler:

addressing mode

Controls how texture boundaries and out-of-bounds coordinates are resolved. The addressing mode for each texture dimension can be set independently. See WebGPU [GPUAddressMode](#).

filter mode

Controls which texels are accessed to produce the final result. Filtering can either use the nearest texel or interpolate between multiple texels. Multiple filter modes can be set independently. See WebGPU [GPUFilterMode](#).

LOD clamp

Controls the min and max levels of details that are accessed.

comparison

Controls the type of comparison done for [comparison sampler](#). See WebGPU [GPUCompareFunction](#).

max anisotropy

Controls the maximum anisotropy value used by the sampler.

Samplers cannot be created in WGSL modules and their state (e.g. the properties listed above) are immutable within a shader and can only be set by the WebGPU API.

It is a [pipeline-creation error](#) if a filtering sampler (i.e. any sampler using interpolative filtering) is used with texture that has a non-filterable format.

NOTE: The handle stored by a sampler variable cannot be changed by the shader.

§ 6.6.1. Texel Formats

In WGSL, certain texture types are parameterized by texel format.

A *texel format* is characterized by:

channels

Each channel contains a scalar. A texel format has up to four channels: r, g, b, and a, normally corresponding to the concepts of red, green, blue, and alpha channels.

channel format

The number of bits in the channel, and how those bits are interpreted.

Each texel format in WGSL corresponds to a WebGPU [GPUTextureFormat](#) with the same name.

Only certain texel formats are used in WGSL source code. The channel formats used to define those texel formats are listed in the **Channel Formats** table. The second last column specifies the conversion from the stored channel bits to the value used in the shader. This is also known as the *channel transfer function*, or CTF. The third column specifies the conversion from the shader value to the stored channel bits. This is also known as the *inverse channel transfer function*, or ICTF. The last column specifies the required [language extension](#) for the texel format.

NOTE: The channel transfer function for 8unorm maps $\{0, \dots, 255\}$ to the floating point interval $[0.0, 1.0]$.

NOTE: The channel transfer function for 8snorm maps $\{-128, \dots, 127\}$ to the floating point interval $[-1.0, 1.0]$.

Channel Formats

Channel format	Number of stored bits	Interpretation of stored bits	Shader type	Shader value (Channel Transfer Function)	Write value τ (Inverse Channel Transfer Function)
8unorm	8	unsigned integer $v \in \{0, \dots, 255\}$	f32	$v \div 255$	$\max(0, \min(1, \tau))$
8snorm	8	signed integer $v \in \{-128, \dots, 127\}$	f32	$v \div 127$	$\max(-1, \min(1, \tau))$

Channel format	Number of stored bits	Interpretation of stored bits	Shader type	Shader value (Channel Transfer Function)	Write value T (Inverse Channel Transfer Function)
8uint	8	unsigned integer $v \in \{0, \dots, 255\}$	u32	v	$\min(255, T)$
8sint	8	signed integer $v \in \{-128, \dots, 127\}$	i32	v	$\max(-128, \min(127, T))$
16unorm	16	unsigned integer $v \in \{0, \dots, 65535\}$	f32	$v \div 65535$	$\max(0, \min(1, T))$
16snorm	16	signed integer $v \in \{-32768, \dots, 32767\}$	f32	$v \div 32767$	$\max(-1, \min(1, T))$
16uint	16	unsigned integer $v \in \{0, \dots, 65535\}$	u32	v	$\min(65535, T)$
16sint	16	signed integer $v \in \{-32768, \dots, 32767\}$	i32	v	$\max(-32768, \min(32767, T))$
16float	16	IEEE-754 binary16 16-bit floating point value v	f32	v	$\text{quantizeToF16}(T)$
32uint	32	32-bit unsigned integer value v	u32	v	T
32sint	32	32-bit signed integer value v	i32	v	T
32float	32	IEEE-754 binary32 32-bit floating point value v	f32	v	T
2unorm	2	unsigned integer $v \in \{0, \dots, 3\}$	f32	$v \div 3$	$\max(0, \min(1, T))$
2uint	2	unsigned integer $v \in \{0, \dots, 3\}$	u32	v	$\min(3, T)$
10unorm	10	unsigned integer $v \in \{0, \dots, 1023\}$	f32	$v \div 1023$	$\max(0, \min(1, T))$
10uint	10	unsigned integer $v \in \{0, \dots, 1023\}$	u32	v	$\min(1023, T)$

Channel format	Number of stored bits	Interpretation of stored bits	Shader type	Shader value (Channel Transfer Function)	Write value T (Inverse Channel Transfer Function)
10float	10	10-bit floating point value: 5 bits of biased exponent, 5 bits of fraction v	f32	v	$\max(0, T)$
11float	11	11-bit floating point value: 5 bits of biased exponent, 6 bits of fraction v	f32	v	$\max(0, T)$

The texel formats listed in the [Texel Formats for Storage Textures](#) table correspond to the [WebGPU plain color formats](#) which support the WebGPU [STORAGE_BINDING](#) usage with at least one [access mode](#). These texel formats are used to parameterize the [storage texture](#) types defined in [§ 6.6.5 Storage Texture Types](#).

When the texel format does not have all four channels, then:

- When reading the texel, the [channel transfer function](#) is applied [component-wise](#):
 - If the texel format has no green channel, then the second component of the shader value is 0.
 - If the texel format has no blue channel, then the third component of the shader value is 0.
 - If the texel format has no alpha channel, then the fourth component of the shader value is 1.
- When writing the texel, the [inverse channel transfer function](#) is applied [component-wise](#) and shader value components for missing channels are ignored.

The last column in the table below uses the format-specific [channel transfer function](#) from the [channel formats](#) table.

Texel Formats for Storage Textures

Texel format	Channel format	Channels in memory order	Corresponding shader value	Required language extension
<i>rgba8unorm</i>	8unorm	r, g, b, a	$\text{vec4}\langle\text{f32}\rangle(\text{CTF}(r), \text{CTF}(g), \text{CTF}(b), \text{CTF}(a))$	
<i>rgba8snorm</i>	8snorm	r, g, b, a	$\text{vec4}\langle\text{f32}\rangle(\text{CTF}(r), \text{CTF}(g), \text{CTF}(b), \text{CTF}(a))$	
<i>rgba8uint</i>	8uint	r, g, b, a	$\text{vec4}\langle\text{u32}\rangle(\text{CTF}(r), \text{CTF}(g), \text{CTF}(b), \text{CTF}(a))$	
<i>rgba8sint</i>	8sint	r, g, b, a	$\text{vec4}\langle\text{i32}\rangle(\text{CTF}(r), \text{CTF}(g), \text{CTF}(b), \text{CTF}(a))$	
<i>rgba16unorm</i>	16unorm	r, g, b, a	$\text{vec4}\langle\text{f32}\rangle(\text{CTF}(r), \text{CTF}(g), \text{CTF}(b), \text{CTF}(a))$	texture_formats_tier1

Texel format	Channel format	Channels in memory order	Corresponding shader value	Required language extension
<i>rgba16snorm</i>	16snorm	r, g, b, a	<code>vec4<f32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	texture_formats_tier1
<i>rgba16uint</i>	16uint	r, g, b, a	<code>vec4<u32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	
<i>rgba16sint</i>	16sint	r, g, b, a	<code>vec4<i32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	
<i>rgba16float</i>	16float	r, g, b, a	<code>vec4<f32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	
<i>rg8unorm</i>	8unorm	r, g	<code>vec4<f32>(CTF(r), CTF(g), 0.0, 1.0)</code>	texture_formats_tier1
<i>rg8snorm</i>	8snorm	r, g	<code>vec4<f32>(CTF(r), CTF(g), 0.0, 1.0)</code>	texture_formats_tier1
<i>rg8uint</i>	8uint	r, g	<code>vec4<u32>(CTF(r), CTF(g), 0u, 1u)</code>	texture_formats_tier1
<i>rg8sint</i>	8sint	r, g	<code>vec4<i32>(CTF(r), CTF(g), 0, 1)</code>	texture_formats_tier1
<i>rg16unorm</i>	16unorm	r, g	<code>vec4<f32>(CTF(r), CTF(g), 0.0, 1.0)</code>	texture_formats_tier1
<i>rg16snorm</i>	16snorm	r, g	<code>vec4<f32>(CTF(r), CTF(g), 0.0, 1.0)</code>	texture_formats_tier1
<i>rg16uint</i>	16uint	r, g	<code>vec4<u32>(CTF(r), CTF(g), 0u, 1u)</code>	texture_formats_tier1
<i>rg16sint</i>	16sint	r, g	<code>vec4<i32>(CTF(r), CTF(g), 0, 1)</code>	texture_formats_tier1
<i>rg16float</i>	16float	r, g	<code>vec4<f32>(CTF(r), CTF(g), 0.0, 1.0)</code>	texture_formats_tier1
<i>r32uint</i>	32uint	r	<code>vec4<u32>(CTF(r), 0u, 0u, 1u)</code>	
<i>r32sint</i>	32sint	r	<code>vec4<i32>(CTF(r), 0, 0, 1)</code>	
<i>r32float</i>	32float	r	<code>vec4<f32>(CTF(r), 0.0, 0.0, 1.0)</code>	
<i>rg32uint</i>	32uint	r, g	<code>vec4<u32>(CTF(r), CTF(g), 0u, 1u)</code>	
<i>rg32sint</i>	32sint	r, g	<code>vec4<i32>(CTF(r), CTF(g), 0, 1)</code>	
<i>rg32float</i>	32float	r, g	<code>vec4<f32>(CTF(r), CTF(g), 0.0, 1.0)</code>	
<i>rgba32uint</i>	32uint	r, g, b, a	<code>vec4<u32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	
<i>rgba32sint</i>	32sint	r, g, b, a	<code>vec4<i32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	
<i>rgba32float</i>	32float	r, g, b, a	<code>vec4<f32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	
<i>bgra8unorm</i>	8unorm	b, g, r, a	<code>vec4<f32>(CTF(r), CTF(g), CTF(b), CTF(a))</code>	

Texel format	Channel format	Channels in memory order	Corresponding shader value	Required language extension
<i>r8unorm</i>	8unorm	r	vec4<f32>(CTF(r), 0.0, 0.0, 1.0)	texture_formats_tier1
<i>r8snorm</i>	8snorm	r	vec4<f32>(CTF(r), 0.0, 0.0, 1.0)	texture_formats_tier1
<i>r8uint</i>	8uint	r	vec4<u32>(CTF(r), 0u, 0u, 1u)	texture_formats_tier1
<i>r8sint</i>	8sint	r	vec4<i32>(CTF(r), 0, 0, 1)	texture_formats_tier1
<i>r16unorm</i>	16unorm	r	vec4<f32>(CTF(r), 0.0, 0.0, 1.0)	texture_formats_tier1
<i>r16snorm</i>	16snorm	r	vec4<f32>(CTF(r), 0.0, 0.0, 1.0)	texture_formats_tier1
<i>r16uint</i>	16uint	r	vec4<u32>(CTF(r), 0u, 0u, 1u)	texture_formats_tier1
<i>r16sint</i>	16sint	r	vec4<i32>(CTF(r), 0, 0, 1)	texture_formats_tier1
<i>r16float</i>	16float	r	vec4<f32>(CTF(r), 0.0, 0.0, 1.0)	texture_formats_tier1
<i>rgb10a2unorm</i>	r, g, b: 10unorm a: 2unorm	r, g, b, a	vec4<f32>(CTF(r), CTF(g), CTF(b), CTF(a))	texture_formats_tier1
<i>rgb10a2uint</i>	r, g, b: 10uint a: 2uint	r, g, b, a	vec4<u32>(CTF(r), CTF(g), CTF(b), CTF(a))	texture_formats_tier1
<i>rg11b10float</i>	r, g: 11float b: 10float	r, g, b	vec4<f32>(CTF(r), CTF(g), CTF(b), 1.0)	texture_formats_tier1

WGSL [predeclares](#) an [enumerant](#) for each of the texel formats in the table.

§ 6.6.2. Sampled Texture Types

A *sampled texture* is capable of being [accessed](#) in conjunction with a [sampler](#). It can also be accessed without the use of a sampler. Sampled textures only allow [read accesses](#).

The [texel format](#) is the [format](#) attribute of the [GPUTexture](#) bound to the texture variable. WebGPU [validates](#) compatibility between the texture, the [sampleType](#) of the bind group layout, and the [sampled type](#) of the texture variable.

The texture is parameterized by a *sampled type* and [must](#) be f32, i32, or u32.

Type	Dimensionality	Arrayed
<i>texture_1d<T></i>	1D	No
<i>texture_2d<T></i>	2D	No

Type	Dimensionality	Arrayed
<i>texture_2d_array</i> < <i>T</i> >	2D	Yes
<i>texture_3d</i> < <i>T</i> >	3D	No
<i>texture_cube</i> < <i>T</i> >	Cube	No
<i>texture_cube_array</i> < <i>T</i> > <i>T</i> is the sampled type. - The parameterized type for the images is the type after conversion from sampling. E.g. you can have an image with texels with 8bit unorm components, but when you sample them you get a 32-bit float result (or vec-of-f32).	Cube	Yes

§ 6.6.3. Multisampled Texture Types

A **multisampled texture** has a [sample count](#) of 1 or more. Despite the name, it cannot be used with a [sampler](#). It effectively stores multiple [texels](#) worth of data per [logical texel address](#) if the sample index is ignored.

The [texel format](#) is the [format](#) attribute of the [GPUTexture](#) bound to the texture variable. WebGPU [validates](#) compatibility between the texture, the [sampleType](#) of the bind group layout, and the [sampled type](#) of the texture variable.

`texture_multisampled_2d` is parameterized by a [sampled type](#) and [must](#) be f32, i32, or u32.

Type	Dimensionality	Arrayed
<i>texture_multisampled_2d</i> < <i>T</i> >	2D	No
<i>texture_depth_multisampled_2d</i>	2D	No

- T* is the [sampled type](#).

§ 6.6.4. External Sampled Texture Types

An **External texture** is an [opaque](#) two-dimensional float-[sampled texture](#) type similar to `texture_2d<f32>` but potentially with a different representation. It can be read using [textureLoad](#) or [textureSampleBaseClampToEdge](#) built-in functions, which handle these different representations.

See [WebGPU § 6.4 GPUExternalTexture](#).

Type	Dimensionality	Arrayed
<i>texture_external</i>	2D	No

§ 6.6.5. Storage Texture Types

A **storage texture** supports [accessing](#) individual texel values without the use of a sampler.

- A *write-only storage texture* supports [writing](#) individual texels, with automatic conversion of the shader value to a stored texel value.
- A *read-only storage texture* supports [reading](#) individual texels, with automatic conversion of the stored texel value to a shader texel value.
- A *read-write storage texture* supports [reading](#) and [writing](#) individual texels, with automatic conversion between shader and stored texel values.

A storage texture type [must](#) be parameterized by one of the [texel formats for storage textures](#). The texel format determines the conversion function as specified in [§ 6.6.1 Texel Formats](#).

When writing texels to a storage texture the *inverse* of the conversion function is used to convert the shader value to the stored texel.

Type	Dimensionality	Arrayed
<i>texture_storage_1d</i> <Format, Access>	1D	No
<i>texture_storage_2d</i> <Format, Access>	2D	No
<i>texture_storage_2d_array</i> <Format, Access>	2D	Yes
<i>texture_storage_3d</i> <Format, Access>	3D	No

- *Format* [must](#) be an [enumerant](#) for one of the [texel formats for storage textures](#)
- *Access* [must](#) be an [enumerant](#) for one of the [access modes](#).
- No [shader-creation error](#) occurs due to an invalid combination of *Format* and *Access*. Combinations of *Format* with *Access* are checked in the [shader binding validation](#) step during pipeline creation. An invalid combination [will](#) result in a [pipeline-creation error](#).

§ 6.6.6. Depth Texture Types

A *depth texture* is capable of being [accessed](#) in conjunction with a [sampler_comparison](#). It can also be accessed without the use of a sampler. Depth textures only allow [read accesses](#).

The [texel format](#) of the texture is defined in the [GPUTextureBindingLayout](#).

Type	Dimensionality	Arrayed
<i>texture_depth_2d</i>	2D	No
<i>texture_depth_2d_array</i>	2D	Yes
<i>texture_depth_cube</i>	Cube	No
<i>texture_depth_cube_array</i>	Cube	Yes

§ 6.6.7. Sampler Type

A *sampler* mediates access to a [sampled texture](#) or a [depth texture](#), by performing a combination of:

- coordinate transformation.
- optionally modifying mip-level selection.
- for a [sampled texture](#), optionally filtering retrieved texel values.
- for a [depth texture](#), determining the comparison function applied to the retrieved texel.

A *sampler types* are:

- [sampler](#)
- [sampler_comparison](#)

Type	Description
<i>sampler</i>	Sampler. Mediates access to a sampled texture .
<i>sampler_comparison</i>	Comparison sampler. Mediates access to a depth texture .

Samplers are parameterized when created in the WebGPU API. They cannot be modified by a WGSL module.

Samplers can only be used by the [texture built-in functions](#).

```
sampler
sampler_comparison
```

§ 6.7. AllTypes Type

The *AllTypes* type is the set of all WGSL [types](#).

There is no way to write the AllTypes type in WGSL source.

See [§ 6.10 Predeclared Types and Type-Generators Summary](#) for the list of all [predeclared](#) types and [type-generators](#).

NOTE: A type is not a value in an ordinary sense. It is not data that is manipulated by a shader at runtime.

Instead, the AllTypes type exists so [type checking](#) rules will apply to any phrase that *may* contain an ordinary value. WGSL makes the rules consistent by defining a type to be a kind of value, and allowing an expression to denote a type.

The motivating case is a [template parameter](#), which in various contexts may denote several kinds of things, including a [type](#), an [enumerant](#), or a [plain](#) value. In particular, the [template_arg_expression](#) grammar rule expands to the [expression](#) grammar nonterminal.

§ 6.8. Type Aliases

A *type alias* declares a new name for an existing type. The declaration [must](#) appear at [module scope](#), and its [scope](#) is the entire program.

When type *T* is defined as a [type alias](#) for a structure type *S*, all properties of the members of *S*, including attributes, carry over to the members of *T*.

NOTE: If the type being aliased supports [value constructor](#) built-in functions, then those functions can be called via the alias name instead of the original type specifier name, when the type alias is [in scope](#) and it is otherwise valid to call the function.

type_alias_decl :
'alias' ident '=' type_specifier

EXAMPLE: TYPE ALIAS

```
alias Arr = array<i32, 5>;

alias RTArr = array<vec4<f32>>;

alias single = f32;    // Declare an alias for f32
const pi_approx: single = 3.1415;
fn two_pi() -> single {
    return single(2) * pi_approx;
}
```

§ 6.9. Type Specifier Grammar

See [§ 8.17 Type Expressions](#).

type_specifier :
template_elaborated_ident

template_elaborated_ident :
ident template_list ?

NOTE: An [expression](#) can also denote a type, by expanding via the [primary_expression](#) grammar rule to [template_elaborated_ident](#), and via [parenthesization](#).

§ 6.10. Predeclared Types and Type-Generators Summary

The [predeclared types](#) that can be spelled in WGSL source are:

- [bool](#)
- [f16](#)
- [f32](#)
- [i32](#)
- [sampler](#)
- [sampler_comparison](#)
- [texture_depth_2d](#)
- [texture_depth_2d_array](#)
- [texture_depth_cube](#)

- [texture_depth_cube_array](#)
- [texture_depth_multisampled_2d](#)
- [texture_external](#)
- [u32](#)

WGSL also predeclares the return types for the [fexp](#), [modf](#), and [atomicCompareExchangeWeak](#) built-in functions. However, they cannot be spelled in WGSL source.

The predeclared [type-generators](#) are listed in the following table:

Predeclared type generators

Predeclared type-generator	Cross-reference
array	See § 6.2.9 Array Types
atomic	See § 6.2.8 Atomic Types
mat2x2	See § 6.2.7 Matrix Types , which also lists predeclared aliases for matrix types. NOTE: These are also used in value constructor expressions to create matrices.
mat2x3	
mat2x4	
mat3x2	
mat3x3	
mat3x4	
mat4x2	
mat4x3	
mat4x4	
ptr	See § 6.5.3 Reference and Pointer Types
texture_1d	See § 6.6.2 Sampled Texture Types
texture_2d	
texture_2d_array	
texture_3d	
texture_cube	
texture_cube_array	
texture_multisampled_2d	See § 6.6.3 Multisampled Texture Types
texture_storage_1d	See § 6.6.5 Storage Texture Types
texture_storage_2d	
texture_storage_2d_array	

Predeclared type-generator	Cross-reference
texture_storage_3d	
vec2	See § 6.2.6 Vector Types, which also lists predeclared aliases for vector types.
vec3	NOTE: These are also used in value constructor expressions to create vectors.
vec4	

§ 7. Variable and Value Declarations

[Variable](#) and [value](#) declarations provide names for data values.

A **value declaration** creates a name for a value, and that value is immutable once it has been declared. The four kinds of value declarations are `const`, `override`, `let`, and formal parameter declarations, further described below (see § 7.2 Value Declarations).

A **variable declaration** creates a name for [memory locations](#) for storing a value; the value stored there may be updated, if the variable has a [read_write](#) access mode. There is one kind of variable declaration, `var`, but it has options for [address space](#) and [access modes](#) in various combinations, described below (see § 7.3 var Declarations).

NOTE: A value declaration does not have associated memory locations. For example, no WGSL expression can form a pointer to the value.

A declaration appearing outside of any function definition is at [module scope](#). Its name is [in scope](#) for the entire program.

A declaration appearing within a function definition is in **function scope**. The name is available for use in the statement immediately after its declaration until the end of the brace-delimited list of statements immediately enclosing the declaration. A function-scope declaration is a [dynamic context](#).

NOTE:
Variable and value declarations have a similar overall syntax. The following non-normative illustration shows the general form of variable and value declarations, where [...] denotes optional parts, ...* denotes zero or more repetitions of the preceding, and ...+ denotes one or more repetitions of the preceding. For specific syntactic rules, see the respective sections for the elements.

```
// Specific value declarations.
    const    name [: type] = initializer ;
[attribute]* override name [: type] [= initializer];
    let      name [: type] = initializer ;

// General variable form.
[attribute]* var[<address_space[, access_mode]>] name [: type] [= initializer];

// Specific variable declarations.
// Function scope.
    var[<function>] name [: type] [= initializer];

// Module scope.
    var<private>    name [: type] [= initializer];
    var<workgroup>  name : type;
[attribute]+ var<uniform>    name : type;
[attribute]+ var          name : texture_type;
[attribute]+ var          name : sampler_type;
[attribute]+ var<storage[, access_mode]> name : type;
```

Each such declaration must have an explicitly specified type or an initializer. Both a type and an initializer may be specified. Each such declaration determines the type for the associated data value, known as the *effective-value-type* for the declaration. The effective-value-type of the declaration is:

- The declared type, if explicitly specified.
- Otherwise, if the initializer expression has type T:
 - For a const declaration, the effective-value-type is T itself.
 - For a override, let, or var declaration, the effective-value-type is the [concretization](#) of T.

Each kind of value or variable declaration may place additional constraints on the form of the initializer expression, if present, and on the effective-value-type.

Variable and Value Declaration Feature Summary.

Declaration	Mutability	Scope	Effective-value-type ¹	Initializer Support	Initializer Expression ²	Part Reso Inter
const	Immutable	Module or function	Constructible (Concrete or abstract)	Required	const-expression	No
override	Immutable	Module	Concrete scalar	Optional ³	const-expression or override-expression	No ⁴

Declaration	Mutability	Scope	Effective-value-type ¹	Initializer Support	Initializer Expression ²	Part Reso Inter
let	Immutable	Function	Concrete constructible or pointer type . Additionally, if the texture_and_sampler_let feature is supported, texture or sampler types.	Required	const-expression , override-expression , or runtime expression	No
var < storage , read> var < storage >	Immutable	Module	Concrete host-shareable	Disallowed		Yes. stora buffe
var < storage , read_write> ^{5,6}	Mutable	Module	Concrete host-shareable	Disallowed		Yes. stora buffe
var < uniform >	Immutable	Module	Concrete constructible host-shareable or fixed-size buffer	Disallowed		Yes. unifo buffe
var < immediate >	Immutable	Module	Concrete constructible host-shareable , excluding arrays and structures containing array members	Disallowed		Yes. imme data
var ⁵	Immutable ⁷	Module	Texture	Disallowed		Yes. textu resou
var	Immutable	Module	Sampler	Disallowed		Yes. samp resou
var < workgroup > ^{6,8}	Mutable	Module	Concrete plain type with a fixed footprint or a fixed-size buffer ⁹	Disallowed ¹⁰		No
var < private >	Mutable	Module	Concrete constructible	Optional ¹⁰	const-expression or override-expression	No
var < function > var	Mutable	Function	Concrete constructible	Optional ¹⁰	const-expression , override-expression , or runtime expression	No

Declaration	Mutability	Scope	Effective-value-type ¹	Initializer Support	Initializer Expression ²	Part Reso Inter
-------------	------------	-------	---	---------------------	-------------------------------------	-----------------

1. Only [const-declarations](#) can be [abstract](#) types, and only when the type is not explicitly specified.
2. The type of the expression must be [feasibly converted](#) to the [effective-value-type](#).
3. If an initializer is not specified, a value must be provided at [pipeline-creation time](#).
4. [Override-declarations](#) are part of the shader interface, but are not bound resources.
5. [Storage buffers](#) and [storage textures](#) with an access mode other than [read](#) cannot be [statically accessed](#) in a [vertex shader stage](#). See WebGPU [createBindGroupLayout\(\)](#).
6. [Atomic types](#) can only appear in mutable storage buffers or workgroup variables.
7. The data in [storage textures](#) with a [write](#) or [read_write access mode](#) is mutable, but can only be modified via [textureStore](#) built-in function. The variable itself cannot be modified.
8. Variables in the [workgroup](#) address space can only be [statically accessed](#) in a [compute shader stage](#).
9. The [element count](#) of the outermost [array](#) or the size of the [buffer](#) may be an [override-expression](#).
10. If there is no initializer, the variable is [default initialized](#).

§ 7.1. Variables vs Values

[Variable declarations](#) are the only mutable data in a WGSL module. [Value declarations](#) are always immutable. Variables can be the basis of [reference](#) and [pointer](#) values because variables have associated [memory locations](#), whereas a value declaration cannot be the basis of a pointer or reference value.

Using variables is generally more expensive than using value declarations, because using a variable requires extra operations to [read](#) or [write](#) to the [memory locations](#) associated with the variable.

Generally speaking, an author should prefer using declarations in the following order, with the most preferred option listed first:

- [const-declaration](#)
- [override-declaration](#)
- [let-declaration](#)
- [variable declaration](#)

This will generally result in the best overall performance of a shader.

§ 7.2. Value Declarations

When an [identifier resolves](#) to a [value declaration](#), the identifier denotes that value.

WGSL provides multiple kinds of value declarations. The value for each kind of declaration is fixed at a different point in the [shader lifecycle](#). The different kinds of value declarations and when their values are fixed are:

- [const-declarations](#), at [shader-creation time](#)
- [override-declarations](#), at [pipeline-creation time](#)

- [let-declarations](#), when they are executed
- [formal parameter](#) declarations, when the associated [function call](#) argument is executed

NOTE: [Formal parameters](#) are described in [§ 11 Functions](#).

§ 7.2.1. const Declarations

A **const-declaration** specifies a name for a data value that is fixed at [shader-creation time](#). Each const-declaration requires an initializer. A const-declaration can be declared in [module](#) or [function](#) scope. The initializer expression **must** be a [const-expression](#). The type of a const-declaration must be a [concrete](#) or [abstract constructible](#) type. const-declarations are the only declarations where the [effective-value-type](#) may be [abstract](#).

NOTE: Since [abstract numeric types](#) cannot be spelled in WGSL, they can only be used via type inference.

EXAMPLE: CONST-DECLARATIONS AT MODULE SCOPE

```
const a = 4;           // AbstractInt with a value of 4.
const b : i32 = 4;     // i32 with a value of 4.
const c : u32 = 4;     // u32 with a value of 4.
const d : f32 = 4;     // f32 with a value of 4.
const e = vec3(a, a, a); // vec3 of AbstractInt with a value of (4, 4, 4).
const f = 2.0;         // AbstractFloat with a value of 2.
const g = mat2x2(a, f, a, f); // mat2x2 of AbstractFloat with a value of:
                        // ((4.0, 2.0), (4.0, 2.0)).
                        // The AbstractInt a converts to AbstractFloat.
                        // An AbstractFloat cannot convert to AbstractInt.
const h = array(a, f, a, f); // array of AbstractFloat with 4 components:
                        // (4.0, 2.0, 4.0, 2.0).
```

§ 7.2.2. override Declarations

An **override-declaration** specifies a name for a [pipeline-overrideable](#) constant value. An override-declaration **must** only be declared at [module scope](#). The value of a **pipeline-overrideable** constant is fixed at [pipeline-creation time](#). The value is one provided by the WebGPU pipeline-creation method, if specified, and otherwise is the value of its [concretized](#) initializer expression. The [effective-value-type](#) of an override-declaration must be a [concrete scalar](#) type.

An initializer expression is optional. If present, it must be an [override-expression](#) and represents the **pipeline-overrideable constant default value**. If no initializer is specified, it is a [pipeline-creation error](#) if a value is not provided at pipeline-creation time.

If the declaration has an [id](#) attribute applied, the literal operand is known as the **pipeline constant ID**, and must be a unique integer between 0 and 65535 inclusive. That is, two override-declarations must not use the same pipeline constant ID.

The application can specify its own value for an override-declaration at [pipeline-creation time](#). The pipeline creation API accepts a mapping from overrideable constants to a value of the constant's type. The constant is identified by a **pipeline-overrideable constant identifier string**, which is the base-10 representation of the [pipeline constant ID](#) if specified, and otherwise the declared [name](#) of the constant.

EXAMPLE: MODULE CONSTANTS, PIPELINE-OVERRIDEABLE

```

@id(0)    override has_point_light: bool = true; // Algorithmic control
@id(1200) override specular_param: f32 = 2.3;    // Numeric control
@id(1300) override gain: f32;                    // Must be overridden
          override width: f32 = 0.0;              // Specified at the API level using
                                                  // the name "width".
          override depth: f32;                    // Specified at the API level using
                                                  // the name "depth".
                                                  // Must be overridden.
          override height = 2 * depth;            // The default value
                                                  // (if not set at the API level),
                                                  // depends on another
                                                  // overridable constant.

```

§ 7.2.3. let Declarations

A **let-declaration** specifies a name for a value that is fixed each time the statement is executed at runtime. A let-declaration must only be declared in [function scope](#), and as such, is a [dynamic context](#). A let-declaration must have an initializer expression. The value is the [concretized](#) value of the initializer. The [effective-value-type](#) of a let-declaration [must](#) be either a [concrete constructible](#) type or a [pointer type](#). If the [texture_and_sampler_let](#) feature is supported, the effective-value-type may also be a [texture](#) or [sampler](#) type.

NOTE: Since [buffer](#) types are not [constructible](#), only pointers to buffers can be let-declarations.

EXAMPLE: LET-DECLARED CONSTANTS AT FUNCTION SCOPE

```

// 'blockSize' denotes the i32 value 1024.
let blockSize: i32 = 1024;

// 'row_size' denotes the u32 value 16u. The type is inferred.
let row_size = 16u;

```

§ 7.3. var Declarations

A **variable** is a named reference to memory that can contain a value of a particular [storable](#) type.

Two types are associated with a variable: its [store type](#) (the type of value that may be placed in the referenced memory) and its [reference type](#) (the type of the variable itself). If a variable has store type T, [address space](#) AS, and [access mode](#) AM, then its reference type is `ref<AS, T, AM>`. The store type of a variable is always [concrete](#).

A variable declaration:

- Specifies the variable's [name](#).
- Determines the variable's [address space](#), [store type](#), and [access mode](#). Together these comprise the variable's [reference type](#).
 - The store type is the [effective-value-type](#) of the variable's declaration.

- Ensures the execution environment allocates memory for a value of the store type, in the specified address space, supporting the given access mode, for the [lifetime](#) of the variable.
- Optionally has an initializer expression if the variable is in the [private](#) or [function](#) address spaces. If present, the initializer must evaluate to the variable's store type. If present, the initializer for a [private](#) variable must be a [const-expression](#) or an [override-expression](#). Variables in address spaces other than [function](#) or [private](#) **must not** have an initializer.

When an [identifier resolves](#) to a variable declaration, the identifier is an expression denoting the reference [memory view](#) for the variable's memory, and its type is the variable's [reference type](#). See [§ 8.11 Variable Identifier Expression](#).

If the [address space](#) or [access mode](#) for a variable declaration are specified in program source, they are written as a [template list](#) after the var keyword:

- The [address space](#) is specified first, as one of the [predeclared](#) address space [enumerants](#).
- The [access mode](#) is specified second, if present, as one of the [predeclared](#) address mode [enumerants](#).
 - The address space must be specified if the access mode is specified.

Variables in the [private](#), [storage](#), [uniform](#), [workgroup](#), and [handle](#) address spaces must only be declared in [module scope](#), while variables in the [function](#) address space must only be declared in [function scope](#). The address space [must](#) be specified for all address spaces except handle and function. The handle address space [must not](#) be specified. Specifying the function address space is optional.

The [access mode](#) always has a default value, and except for variables in the [storage](#) address space, [must not](#) be specified in the WGSL source. See [§ 14.3 Address Spaces](#).

A variable in the [uniform](#) address space is a **uniform buffer** variable. Its [store type](#) must be a [host-shareable constructible](#) type or a [fixed-size buffer](#) type, and must satisfy the [address space layout constraints](#).

A variable in the [storage](#) address space is a **storage buffer** variable. Its [store type](#) must be a [host-shareable](#) type and must satisfy the [address space layout constraints](#). The variable may be declared with a [read](#) or [read_write](#) access mode; the default is read.

A variable in the [immediate](#) address space is an **immediate data** variable. Its [store type must](#) be a [host-shareable constructible](#) type, excluding arrays and structures that contain array members. Each [entry point must statically access](#) at most one immediate data variable. The value of an immediate variable is set via [setImmediates](#) commands recorded by the WebGPU API command encoder, and remains constant during shader execution. The variable size is limited by the pipeline layout's [immediateSize](#) configuration.

The [AccessibleSlots](#) exclude padding bytes between structure members and at the end of structures. Only the accessible slots need to be initialized via the WebGPU API before the immediate data variable is accessed in a shader.

A **texture resource** is a variable whose [effective-value-type](#) is a [texture type](#). It is declared at [module scope](#). It holds an [opaque](#) handle which is used to access the underlying grid of [texels](#) in a [texture](#). The handle itself is in the [handle](#) address space and is always read-only. In many cases the underlying texels are read-only, and we say the texture variable immutable. For [write-only storage textures](#) and [read-write storage textures](#), the underlying texels are mutable, and by convention we say the texture variable is mutable.

A **sampler resource** is a variable whose [effective-value-type](#) is a [sampler type](#). It is declared at [module scope](#), exists in the [handle](#) address space, and is immutable.

As described in [§ 13.3.2 Resource Interface](#), uniform buffers, storage buffers, textures, and samplers form the [resource interface of a shader](#).

The **lifetime** of a variable is the period during shader execution for which the [memory locations](#) are associated with the variable. The lifetime of a [module scope](#) variable is the entire execution of the shader stage. There is an independent version of a variable in the [private](#) and [function](#) address spaces for each invocation. [Function-scope](#) variables are a [dynamic context](#). The lifetime of a function-scope variable is determined by its scope:

- It starts when control enters the variable's declaration.
- It ends when the name is no longer [in scope](#) of any part of the [dynamic context](#). That is, the lifetime includes any functions [called](#) while the name is in scope.

Two [resource](#) variables may have [overlapping memory](#) locations, but it is a [dynamic error](#) if either of those variables is mutable. Other variables with overlapping lifetimes [will](#) not have overlapping memory locations. When a variable's lifetime ends, its memory may be used for another variable.

NOTE: WGSL ensures the contents of a variable are only observable during the variable's lifetime.

When a variable in the [private](#), [function](#), or [workgroup](#) address spaces is created, it [will](#) have an initial value. If no initializer is specified the initial value is the **default initial value**. The initial values are computed as follows:

- For variables in the function address space:
 - The [zero value](#) of the [store type](#), if the variable declaration did not specify an initializer.
 - Otherwise it is the result of evaluating the [concretized](#) initializer expression at that point in program execution.
- For variables in the private address space:
 - The [zero value](#) of the [store type](#), if the variable declaration did not specify an initializer.
 - Otherwise it is the result of evaluating the [concretized](#) initializer expression. The initializer must be an [override-expression](#), and so its value is fixed no later than [pipeline-creation time](#).
- For variables in the workgroup address space:
 - When the [store type](#) is [constructible](#), the [zero value](#) for the store type.
 - If the [store type](#) is an [atomic type](#), the [zero value](#) is that of the underlying type ([concrete integer scalar](#)).
 - If the [store type](#) is a [buffer](#) type, the [zero value](#) is all zero bytes.
 - Otherwise, if the [store type](#) is not [constructible](#), the [zero value](#) is determined by recursively applying these rules to each [component](#) of the [composite](#) until a [constructible](#) type is encountered.

- **NOTE:** This commonly occurs when using an [array](#) with a [pipeline-overridable element count](#) or a [composite](#) that contains an [atomic type](#).

Variables in other address spaces are [resources](#) set by bindings in the [draw command](#) or [dispatch command](#).

Consider the following snippet of WGSL:

EXAMPLE: VARIABLE INITIAL VALUES

```
var i: i32;          // Initial value is 0. Not recommended style.
loop {
  var twice: i32 = 2 * i; // Re-evaluated each iteration.
  i++;
  if i == 5 { break; }
}
```

The loop body will execute six times. Variable `i` will take on values 0, 1, 2, 3, 4, 5, and variable `twice` will take on values 0, 2, 4, 6, and 8.

Consider the following snippet of WGSL:

EXAMPLE: READING A VARIABLE MULTIPLE TIMES

```
var x: f32 = 1.0;
let y = x * x + x + 1;
```

Because `x` is a variable, all accesses to it turn into load and store operations. However, it is expected that either the browser or the driver optimizes this intermediate representation such that the redundant loads are eliminated.

EXAMPLE: MODULE SCOPE VARIABLE DECLARATIONS

```
var<private> decibels: f32;
var<workgroup> worklist: array<i32,10>;

struct Params {
    specular: f32,
    count: i32
}

// Uniform buffer. Always read-only, and has more restrictive layout rules.
@group(0) @binding(2)
var<uniform> param: Params;    // A uniform buffer

// A storage buffer, for reading and writing
@group(0) @binding(0)
var<storage,read_write> pbuf: array<vec2<f32>>;

// Textures and samplers are always in "handle" space.
@group(0) @binding(1)
var filter_params: sampler;
```

EXAMPLE: ACCESS MODES FOR BUFFERS

```
// Storage buffers
@group(0) @binding(0)
var<storage,read> buf1: Buffer;    // Can read, cannot write.
@group(0) @binding(0)
var<storage> buf2: Buffer;         // Can read, cannot write.
@group(0) @binding(1)
var<storage,read_write> buf3: Buffer; // Can both read and write.

struct ParamsTable {weight: f32}

// Uniform buffer. Always read-only, and has more restrictive layout rules.
@group(0) @binding(2)
var<uniform> params: ParamsTable; // Can read, cannot write.
```

EXAMPLE: FUNCTION SCOPE VARIABLES AND CONSTANTS

```
fn f() {
    var<function> count: u32; // A variable in function address space.
    var delta: i32;           // Another variable in the function address space.
    var sum: f32 = 0.0;       // A function address space variable with initializer.
    var pi = 3.14159;         // Infer the f32 store type from the initializer.
}
```

§ 7.4. Variable and Value Declaration Grammar Summary

variable_or_value_statement :

variable_decl

| *variable_decl* '=' *expression*

| 'let' *optionally_typed_ident* '=' *expression*

| 'const' *optionally_typed_ident* '=' *expression*

variable_decl :

'var' *disambiguate_template template_list* ? *optionally_typed_ident*

optionally_typed_ident :

ident (':' *type_specifier*) ?

global_variable_decl :

attribute * *variable_decl* ('=' *expression*) ?

global_value_decl :

'const' *optionally_typed_ident* '=' *expression*

| *attribute* * 'override' *optionally_typed_ident* ('=' *expression*) ?

§ 8. Expressions

[Expressions](#) specify how values are computed.

The different kinds of value expressions provide a tradeoff between when they are evaluated and how expressive they can be. The sooner the evaluation, the more constrained the operations, but also the more places the value can be used. This tradeoff leads to different flexibility with each kind of value declaration. [const-expressions](#) and [override-expressions](#) are evaluated prior to execution on the GPU, so only the result of the computation of the expression is necessary in the final GPU code. Additionally, because [const-expressions](#) are evaluated at [shader-creation time](#) they can be used in more situations than [override-expressions](#), for example, to size [arrays](#) in [function scope variables](#). A *runtime expression* is an expression that is neither a [const-expression](#) nor an [override-expression](#). A runtime expression is computed on the GPU during shader execution. While runtime expressions can be used by fewer grammar elements, they can be computed from a larger class of expressions, for example, other runtime values.

§ 8.1. Early Evaluation Expressions

WGSL defines two types of expressions that can be evaluated before runtime:

- [const-expressions](#), at [shader-creation time](#)
- [override-expressions](#), at [pipeline-creation time](#)

§ 8.1.1. const Expressions

Expressions that can be evaluated at [shader-creation time](#) are called *const-expressions*. An expression is a const-expression if all its [identifiers resolve](#) to:

- [const-declarations](#), or
- [const-functions](#), or
- [type aliases](#), or
- [structure](#) names.

The type of a const expression [must resolve](#) to a type with a [creation-fixed footprint](#).

NOTE: [Abstract types](#) can be the inferred type of a const-expression.

A const-expression E [will](#) be evaluated if and only if:

- E is [top-level expression](#),
- E is a [subexpression](#) of an expression $OuterE$, and $OuterE$ [will](#) be evaluated, and evaluation of $OuterE$ requires E to be evaluated,
- E is a [subexpression](#) of an expression $OuterE$ such that evaluation of E is required to determine the [static type](#) of $OuterE$, or
- E is a [subexpression](#) of an expression $OuterE$ such that $OuterE$ requires E to be evaluated to produce a [shader-creation error](#) (e.g. [integer division](#)).

NOTE: The evaluation rule implies that short-circuiting operators `&&` and `||` guard evaluation of their right-hand side subexpressions unless there is a [subexpression](#) that requires evaluation to determine a [static type](#).

A const-expression may be evaluated by the CPU implementing the WebGPU API methods. Therefore accuracy requirements for operations on [AbstractFloat](#) values are *no more strict* than required for common WebGPU runtime environments, such as WebAssembly [\[WASM-CORE-2\]](#) and ECMAScript [\[ECMASCRIPT\]](#). Accuracy requirements for [concrete](#) floating point types (such as f32) are specified in § 15.7.4.1 [Accuracy of Concrete Floating Point Expressions](#).

Example: (42) is analyzed as follows:

- The term 42 is the [AbstractInt](#) value 42.
- Surrounding that term with parentheses produces a new expression (42) that is of type [AbstractInt](#) with value 42.

Example: -5 is analyzed as follows:

- The term 5 is the [AbstractInt](#) value 5.
- Preceding that term with '-' produces a new expression -5 that is of type [AbstractInt](#) with value -5.

Example: -2147483648 is analyzed as follows:

- The term 2147483648 is the [AbstractInt](#) value 2147483648. Note that this value does **not** fit in a 32-bit signed integer.
- Preceding that term with '-' produces a new expression -2147483648 that is of type [AbstractInt](#) with value -2147483648.

Example: `const minint = -2147483648;` is analyzed as follows:

- As above, `-2147483648` evaluates to a [AbstractInt](#) value `-2147483648`.
- A [const-declaration](#) allows the initializer to be an [abstract numeric type](#).
- The result is that `minint` is declared to be the [AbstractInt](#) value `-2147483648`.

Example: `let minint = -2147483648;` is analyzed as follows:

- As above, `-2147483648` evaluates to a [AbstractInt](#) value `-2147483648`.
- A [let-declaration](#) requires the initializer to be a [concrete constructible](#) type or a pointer type.
- The `let`-declaration does not have an explicit type, so [overload resolution](#) is used. The overload candidates that apply use [feasible automatic conversions](#) from [AbstractInt](#) to either [i32](#), [u32](#), or [f32](#). The one of lowest rank is to [i32](#), and so [AbstractInt](#) `-2147483648` value is converted to the [i32](#) value `-2147483648`.
- The result is that `minint` is declared to be the `i32` value `-2147483648`.

Example: `false && (10i < i32(5 * 1000 * 1000 * 1000))` is analyzed as follows:

- The entire expression is a `const`-expression.
- However, the short-circuiting rules of the `&&` operator apply: the left-hand side evaluates to `false`, and so the right-hand side is *not evaluated*.
- Evaluation of `i32(5 * 1000 * 1000 * 1000)` would have caused a [shader-creation error](#) because the [AbstractInt](#) value `5000000000` overflows the [i32](#) type.

Example: `false && array<u32, 1 + 2>(0, 1, 2)[0] == 0`

- The entire expression is a `const`-expression.
- Type checking requires the `e1 : bool && e2 : bool`:
 - `false` is a `bool` value.
 - Type checking proceeds on the right-hand side and eventually evaluates `1 + 2` in the array element count expression.
- `1 + 2` evaluates to an `i32` value of `3`.
 - The array is typed `array<u32, 3i>`.
- Neither the array access expression nor the equality operator are evaluated.

§ 8.1.2. override Expressions

Expressions that can be evaluated at [pipeline creation](#) time are called *override-expressions*. An expression is an *override-expression* if all its [identifiers resolve](#) to:

- [override-declarations](#), or
- [const-declarations](#), or
- [const-functions](#), or
- [type aliases](#), or
- [structure](#) names.

NOTE: All [const-expressions](#) are also *override-expressions*.

Override-expressions other than const-expressions are only validated or evaluated during [pipeline creation](#), and only after any API-provided values are substituted for [override-declarations](#). If an [override-declaration](#) has its value substituted via the API, its initializer expression, if present, will not be evaluated. Otherwise, an override-expression *E* [will](#) be evaluated if and only if:

- *E* forms part of the [shader](#) for the [entry point](#) specified by the [GPUProgrammableStage](#), and:
- Any of the following are true:
 - *E* is a [top-level expression](#) (after value substitution), or
 - *E* is a [subexpression](#) of an expression *OuterE*, and *OuterE* [will](#) be evaluated, and evaluation of *OuterE* requires *E* to be evaluated, or
 - *E* is a [subexpression](#) of an expression *OuterE* such that *OuterE* requires *E* to be evaluated to produce a [pipeline-creation error](#) (e.g. [integer division](#)).

NOTE: Not all override-expressions may be usable as the initializer for an [override-declaration](#), because such initializers must [resolve](#) to a [concrete scalar](#) type.

Example: `override x = 42;` is analyzed as follows:

- The term 42 is the [AbstractInt](#) value 42.
- An [override-declaration](#) requires a [concrete scalar](#) type.
- 42 is converted to [i32](#) via a [feasible automatic conversion](#).

Example: `let y = x + 1;` is analyzed as follows:

- From above, *x* has a type of [i32](#).
- The expression `x + 1` is an override-expression because it is composed of an [override-declaration](#) and an [integer literal](#).
- The expression has a type of [i32](#) and is evaluated at [pipeline creation](#) time. Its value depends on whether or not *x* is overridden at pipeline creation time.

Example: `vec3(x,x,x)` is analyzed as follows:

- From above, *x* is an [override-declaration](#) with the type [i32](#).
- `vec3(x,x,x)` is an override-expression because the only identifiers [resolve](#) to override-declarations.
- The type of the expression is a [vector](#) of 3 components of [i32](#) (`vec3<i32>`).

EXAMPLE: SHADER-CREATION ERRORS FROM OVERRIDE-EXPRESSIONS

```
override a : i32 = 0;
override b = a / 0; // shader-creation error,
                    // regardless of attempting to override c
```

EXAMPLE: PIPELINE-CREATION ERRORS FROM OVERRIDE-EXPRESSIONS

```

override a : i32 = 0;
override b = 1 / a;

// b is a part of the frag1 shader. When compiling frag1 into a pipeline
// the following cases may occur:
// * if b is overridden, no error occurs.
// * if a is overridden to a non-zero value, no error occurs.
// * if a is 0 and b is not overridden, a pipeline-creation error occurs.
@fragment
fn frag1() {
    _ = b;
}

// b is not part of the frag2 shader. When compiling frag2 into a pipeline
// no errors occur even if b is not overridden and the value of a is 0.
@fragment
fn frag2() {
}

```

8.2. Indeterminate values

In limited cases, an evaluation of a [runtime expression](#) can occur using unsupported values for its [subexpressions](#).

In such a case, the result of that evaluation is an *indeterminate value* of the expression's [static type](#), meaning some arbitrary implementation-chosen value of the static type.

A distinct value may be produced for each unique [dynamic context](#) in which the expression is evaluated. For example, if the evaluation occurs once per iteration of a loop, a distinct value may be computed for each loop iteration.

NOTE: If the type is a floating point type and the implementation supports NaN values, then the indeterminate value produced at runtime may be a NaN value.

EXAMPLE: INDETERMINATE VALUE EXAMPLE

```
fn fun() {
  var extracted_values: array<i32,2>;
  const v = vec2<i32>(0,1);

  for (var i: i32 = 0; i < 2 ; i++) {
    // A runtime-expression used to index a vector, but outside the
    // indexing bounds of the vector, produces an indeterminate value
    // of the vector component type.
    let extract = v[i+5];

    // Now 'extract' is any value of type i32.

    // Save it for later.
    extracted_values[i] = extract;

    if extract == extract {
      // This is always executed
    }
    if extract < 2 {
      // This might be executed, but might not be executed.
      // Even though the original vector components are 0 and 1,
      // the extracted value might not be either of those values.
    }
  }
  if extracted_values[0] == extracted_values[1] {
    // This might be executed, but might not be executed.
  }
}

fn float_fun(runtime_index: u32) {
  const v = vec2<f32>(0,1); // A vector of floating point values

  // As in the previous example, 'float_extract' is an indeterminate value.
  // Since it is a floating point type, it may be a NaN.
  let float_extract: f32 = v[runtime_index+5];

  if float_extract == float_extract {
    // This *might not* be executed, because:
    // - 'float_extract' may be NaN, and
    // - a NaN is never equal to any other floating point number,
    //   even another NaN.
  }
}
```

§ 8.3. Literal Value Expressions

Scalar literal type rules

Precondition	Conclusion	Description
	true: bool	true boolean value.

Precondition	Conclusion	Description
	false: bool	false boolean value.
e is an integer literal with no suffix	e : AbstractInt	Abstract integer literal value.
e is a floating point literal with no suffix	e : AbstractFloat	Abstract float literal value.
e is an integer literal with i suffix	e : i32	32-bit signed integer literal value.
e is an integer literal with u suffix	e : u32	32-bit unsigned integer literal value.
e is an floating point literal with f suffix	e : f32	32-bit floating point literal value.
e is an floating point literal with h suffix	e : f16	16-bit floating point literal value.

§ 8.4. Parenthesized Expressions

Parenthesized expression type rules

Precondition	Conclusion	Description
$e : T$	$(e) : T$	Evaluates to e . Use parentheses to isolate an expression from the surrounding text.

§ 8.5. Composite Value Decomposition Expressions

This section describes expressions for getting a [component](#) of a [composite](#) value, and for getting a [reference](#) to a [component](#) from a [memory view](#) to the containing composite value. For this discussion, the composite value, or the memory view to the composite value, is known as the **base**.

There are two ways of doing so:

named component expression

The expression for the [base](#) B is followed by a period '.' (U+002D), and then the name of the component.

- This is supported when B is of [vector](#) or [structure](#) type, or a memory view to a vector or structure type.
- The valid names depend on B 's type.

indexing expression

The expression for the base is followed by '[' (U+005B), then the expression for an index then ']' (U+005D).

- The base may be a [vector](#), [matrix](#), or [fixed-size array](#) type, or a memory view to a vector, matrix, fixed-size array, or [runtime-sized](#) array type.
- The index expression [must](#) be of [integer scalar](#) type.

Syntactically, these two forms are embodied by uses of the [component_or_swizzle_specifier](#) grammar rule.

The index value i of an [indexing expression](#) is an **in-bounds index** if $0 \leq i < N$, where N is the number of components (elements) of the composite type:

- N is the number of components of the [vector](#) type, when the base is a vector or a [memory view](#) to a vector.
- N is the number of columns of the [matrix](#) type, when the base is a matrix or a memory view to a matrix.

- N is the [element count](#) of the [fixed-size array](#) type, when the base is a fixed-size array or a memory view to a fixed-size array.
- N is [NRuntime](#) for the base, when the base is a memory view to a [runtime-sized](#) array.

The index value is an *out-of-bounds index* when it is not an [in-bounds index](#). An out-of-bounds index is often a programming defect, and will often cause a [error](#). See below for details.

Additionally, vector types support a [swizzling](#) syntax for creating a new vector value from the components of another vector.

§ 8.5.1. Vector Access Expression

Accessing components of a vector can be done either:

- Using array subscripting (e.g. `v[2]`), or
- Using a *swizzle* name, a [context-dependent name](#) written as a sequence of convenience names, each mapping to a component of the source vector.
 - The color set of convenience names: `r`, `g`, `b`, `a` for vector components 0, 1, 2, and 3 respectively.
 - The dimensional set of convenience names: `x`, `y`, `z`, `w` for vector components 0, 1, 2, and 3, respectively.

The convenience names are accessed using the `.` notation. (e.g. `color.bgra`).

The convenience letterings [must not](#) be mixed. For example, you cannot use `.rybw`.

A convenience letter [must not](#) access a component past the end of the vector.

The convenience letterings can be applied in any order, including duplicating letters as needed. The provided number of letters [must](#) be between 1 and 4. That is, using convenience letters can only produce a scalar type or a valid vector type.

The result type depends on the number of letters provided. Assuming a `vec4<f32>`

Accessor Result type

```
r          f32
rg         vec2<f32>
rgb        vec3<f32>
rgba       vec4<f32>
```

EXAMPLE 43

```
var a: vec3<f32> = vec3<f32>(1., 2., 3.);
var b: f32 = a.y;           // b = 2.0
var c: vec2<f32> = a.bb;    // c = (3.0, 3.0)
var d: vec3<f32> = a.zyx;   // d = (3.0, 2.0, 1.0)
var e: f32 = a[1];         // e = 2.0
```

§ 8.5.1.1. Vector Single Component Selection

Vector decomposition: single component selection

Precondition	Conclusion	Description
--------------	------------	-------------

Precondition	Conclusion	Description
$e: \text{vec}N\langle T \rangle$	$e.x: T$ $e.r: T$	Select the first component of e This is a single-letter swizzle .
$e: \text{vec}N\langle T \rangle$	$e.y: T$ $e.g: T$	Select the second component of e This is a single-letter swizzle .
$e: \text{vec}N\langle T \rangle$ N is 3 or 4	$e.z: T$ $e.b: T$	Select the third component of e This is a single-letter swizzle .
$e: \text{vec}4\langle T \rangle$	$e.w: T$ $e.a: T$	Select the fourth component of e This is a single-letter swizzle .
$e: \text{vec}N\langle T \rangle$ $i: \text{i}32$ or $\text{u}32$ T is concrete	$e[i]: T$	Select the i^{th} component of vector The first component is at index $i=0$. If i is outside the range $[0, N-1]$: - It is a shader-creation error if i is a const-expression. - It is a pipeline-creation error if i is an override-expression. - Otherwise an indeterminate value for T may be returned.
$e: \text{vec}N\langle T \rangle$ $i: \text{i}32$ or $\text{u}32$ T is abstract i is a const-expression	$e[i]: T$	Select the i^{th} component of vector The first component is at index $i=0$. It is a shader-creation error if i is outside the range $[0, N-1]$. NOTE: When an abstract vector value e is indexed by an expression that is not a const-expression , then the vector is concretized before the index is applied.

8.5.1.2. Vector Multiple Component Selection

The expressions in this section are all multi-letter [swizzles](#). Each forms a [vector](#) from the components of another vector.

A multi-letter [swizzle](#) cannot appear on the left-hand side of an [assignment](#): The left-hand side of an assignment must be of [reference type](#), but a multi-letter swizzle expression always yields a value of [vector](#) type.

Vector decomposition: multiple component selection

Precondition	Conclusion	Description
$e: \text{vec}N\langle T \rangle$ or $\text{ptr}\langle AS, \text{vec}N\langle T, AM \rangle \rangle$ I is the letter x, y, z, or w	$e.I: \text{vec}2\langle T \rangle$	Computes the two-component vector with first component $e.I$, and second component $e.J$.

Precondition	Conclusion	Description
J is the letter x, y, z, or w AM is read or read_write		Letter z is valid only when N is 3 or 4. Letter w is valid only when N is 4. If e is a pointer, an indirection is first applied and then the load rule is invoked.
e: $\text{vecN}<T>$ or $\text{ptr}<AS, \text{vecN}<T, AM>>$ I is the letter r, g, b, or a J is the letter r, g, b, or a AM is read or read_write	$e.IJ$: $\text{vec2}<T>$	Computes the two-component vector with first component $e.I$, and second component $e.J$. Letter b is valid only when N is 3 or 4. Letter a is valid only when N is 4. If e is a pointer, an indirection is first applied and then the load rule is invoked.
e: $\text{vecN}<T>$ or $\text{ptr}<AS, \text{vecN}<T, AM>>$ I is the letter x, y, z, or w J is the letter x, y, z, or w K is the letter x, y, z, or w AM is read or read_write	$e.IJK$: $\text{vec3}<T>$	Computes the three-component vector with first component $e.I$, second component $e.J$, and third component $e.K$. Letter z is valid only when N is 3 or 4. Letter w is valid only when N is 4. If e is a pointer, an indirection is first applied and then the load rule is invoked.
e: $\text{vecN}<T>$ or $\text{ptr}<AS, \text{vecN}<T, AM>>$ I is the letter r, g, b, or a J is the letter r, g, b, or a K is the letter r, g, b, or a AM is read or read_write	$e.IJK$: $\text{vec3}<T>$	Computes the three-component vector with first component $e.I$, second component $e.J$, and third component $e.K$. Letter b is only valid when N is 3 or 4. Letter a is only valid when N is 4. If e is a pointer, an indirection is first applied and then the load rule is invoked.
e: $\text{vecN}<T>$ or $\text{ptr}<AS, \text{vecN}<T, AM>>$ I is the letter x, y, z, or w J is the letter x, y, z, or w K is the letter x, y, z, or w L is the letter x, y, z, or w AM is read or read_write	$e.IJKL$: $\text{vec4}<T>$	Computes the four-component vector with first component $e.I$, second component $e.J$, third component $e.K$, and fourth component $e.L$. Letter z is valid only when N is 3 or 4. Letter w is valid only when N is 4. If e is a pointer, an indirection is first applied and then the load rule is invoked.
e: $\text{vecN}<T>$ or $\text{ptr}<AS, \text{vecN}<T, AM>>$ I is the letter r, g, b, or a J is the letter r, g, b, or a K is the letter r, g, b, or a L is the letter r, g, b, or a AM is read or read_write	$e.IJKL$: $\text{vec4}<T>$	Computes the four-component vector with first component $e.I$, second component $e.J$, third component $e.K$, and fourth component $e.L$. Letter b is only valid when N is 3 or 4. Letter a is only valid when N is 4. If e is a pointer, an indirection is first applied and then the load rule is invoked.

NOTE: In the table above, [reference types](#) are implicitly handled via the [load rule](#).

§ 8.5.1.3. Component Reference from Vector Memory View

The expressions in this section form a [memory view](#) of a single component of a [vector](#) from the memory view of the whole vector.

The WGSL [type rules](#) imply that such expressions can appear:

- on the left-hand side of an [assignment](#), to write to that component in memory, or
- any place where a value of the vector component type can appear. In this case the [Load Rule](#) applies, loading the vector component from memory and yielding that component as the result.

A [write access](#) to component of a vector **may** access all of the [memory locations](#) associated with that vector.

NOTE: This means accesses to different components of a vector in memory by different invocations must be synchronized if at least one access is a [write access](#). See [§ 17.11 Synchronization Built-in Functions](#).

Getting a reference to a component from a memory view to a vector

Precondition	Conclusion	Description
$r: \text{ref}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$ or $\text{ptr}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$	$r.x: \text{ref}\langle AS, T, AM \rangle$ $r.r: \text{ref}\langle AS, T, AM \rangle$	Compute a reference to the first component of the vector referenced by the memory view r. The originating variable of the resulting reference is the same as the originating variable of r.
$r: \text{ref}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$ or $\text{ptr}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$	$r.y: \text{ref}\langle AS, T, AM \rangle$ $r.g: \text{ref}\langle AS, T, AM \rangle$	Compute a reference to the second component of the vector referenced by the memory view r. The originating variable of the resulting reference is the same as the originating variable of r.
$r: \text{ref}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$ or $\text{ptr}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$ N is 3 or 4	$r.z: \text{ref}\langle AS, T, AM \rangle$ $r.b: \text{ref}\langle AS, T, AM \rangle$	Compute a reference to the third component of the vector referenced by the memory view r. The originating variable of the resulting reference is the same as the originating variable of r.
$r: \text{ref}\langle AS, \text{vec4}\langle T \rangle, AM \rangle$ or $\text{ptr}\langle AS, \text{vec4}\langle T \rangle, AM \rangle$	$r.w: \text{ref}\langle AS, T, AM \rangle$ $r.a: \text{ref}\langle AS, T, AM \rangle$	Compute a reference to the fourth component of the vector referenced by the memory view r. The originating variable of the resulting reference is the same as the originating variable of r.
$r: \text{ref}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$ or $\text{ptr}\langle AS, \text{vecN}\langle T \rangle, AM \rangle$ $i: \text{i32 or u32}$	$r[i]: \text{ref}\langle AS, T, AM \rangle$	Compute a reference to the i^{th} component of the vector referenced by the memory view r. If i is outside the range $[0, N-1]$: • It is a shader-creation error if i is a const-expression. • It is a pipeline-creation error if i is an override-expression. • Otherwise, the expression evaluates to an invalid memory reference. The originating variable of the resulting reference is the same as the originating variable of r.

§ 8.5.2. Matrix Access Expression

Column vector extraction

Precondition	Conclusion	Description
e : $\text{matCxR}<T>$ i : i32 or u32 T is concrete	$e[i]$: $\text{vecR}<T>$	The result is the i^{th} column vector of e. If i is outside the range $[0, C-1]$: - It is a shader-creation error if i is a const-expression. - It is a pipeline-creation error if i is an override-expression. - Otherwise, an indeterminate value for $\text{vecR}<T>$ may be returned.

e : $\text{matCxR}<T>$ i : i32 or u32 T is abstract i is a const-expression	$e[i]$: $\text{vecR}<T>$	The result is the i^{th} column vector of e. It is a shader-creation error if i is outside the range $[0, C-1]$. NOTE: When an abstract matrix value e is indexed by an expression that is not a const-expression, then the matrix is concretized before the index is applied.
--	---------------------------	--

Getting a reference to a column vector from a memory view to a matrix

Precondition	Conclusion	Description
r : $\text{ref}<AS, \text{matCxR}<T>, AM>$ or $\text{ptr}<AS, \text{matCxR}<T>, AM>$ i : i32 or u32	$r[i]$: $\text{ref}<AS, \text{vecR}<T>, AM>$	Compute a reference to the i^{th} column vector of the matrix referenced by the memory view r. If i is outside the range $[0, C-1]$: - It is a shader-creation error if i is a const-expression. - It is a pipeline-creation error if i is an override-expression. - Otherwise, the expression evaluates to an invalid memory reference. The originating variable of the resulting reference is the same as the originating variable of r.

§ 8.5.3. Array Access Expression

Array element extraction

Precondition	Conclusion	Description
--------------	------------	-------------

Precondition	Conclusion	Description
e : array< T , N > i : i32 or u32 T is concrete	$e[i] : T$	The result is the value of the i^{th} element of the array value e. If i is outside the range $[0, N-1]$: - It is a shader-creation error if i is a const-expression. - It is a pipeline-creation error if i is an override-expression. - Otherwise, an indeterminate value for T may be returned.
e : array< T , N > i : i32 or u32 T is abstract i is a const-expression	$e[i] : T$	The result is the value of the i^{th} element of the array value e. It is a shader-creation error if i is outside the range $[0, N-1]$. NOTE: When an abstract array value e is indexed by an expression that is not a const-expression, then the array is concretized before the index is applied. <i>Getting a reference to an array element from a memory view to an array.</i>
Precondition	Conclusion	Description
r : ref< AS , array< T , N >, AM > or ptr< AS , array< T , N >, AM > i : i32 or u32	$r[i] : \text{ref}<AS, T, AM>$	Compute a reference to the i^{th} element of the array referenced by the memory view r. If i is outside the range $[0, N-1]$: - It is a shader-creation error if i is a const-expression. - It is a pipeline-creation error if i is an override-expression. - Otherwise, the expression evaluates to an invalid memory reference. The originating variable of the resulting reference is the same as the originating variable of r.
r : ref< AS , array< T >, AM > or ptr< AS , array< T >, AM > i : i32 or u32	$r[i] : \text{ref}<AS, T, AM>$	Compute a reference to the i^{th} element of the runtime-sized array referenced by the memory view r. If at runtime the array has N elements, and i is outside the range $[0, N-1]$, then the expression evaluates to an invalid memory reference. If i is a signed integer, and i is less than 0: - It is a shader-creation error if i is a const-expression. - It is a pipeline-creation error if i is an override-expression. The originating variable of the resulting reference is the same as the originating variable of r.

§ 8.5.4. Structure Access Expression

Structure member extraction

Precondition	Conclusion	Description
S is a structure type M is the identifier name of a member of S , having type T $e: S$	$e.M: T$	The result is the value of the member with name M from the structure value e .

Getting a reference to a structure member from a memory view to a structure

Precondition	Conclusion	Description
S is a structure type M is the identifier name of a member of S , having type T $r: \text{ref}\langle AS, S, AM \rangle$ or $\text{ptr}\langle AS, S, AM \rangle$	$r.M: \text{ref}\langle AS, T, AM \rangle$	Given a memory view to a structure, the result is a reference to the structure member with identifier name M . The originating variable of the resulting reference is the same as the originating variable of r .

§ 8.6. Logical Expressions

Unary logical operations

Precondition	Conclusion	Description
$e: T$ T is bool or $\text{vec}N\langle \text{bool} \rangle$	$!e: T$	Logical negation. The result is true when e is false and false when e is true. Component-wise when T is a vector.

Binary logical expressions

Precondition	Conclusion	Description
$e1: \text{bool}$ $e2: \text{bool}$	$e1 \mid\mid e2: \text{bool}$	Short-circuiting "or". Yields true if either $e1$ or $e2$ are true; evaluates $e2$ only if $e1$ is false.
$e1: \text{bool}$ $e2: \text{bool}$	$e1 \&\& e2: \text{bool}$	Short-circuiting "and". Yields true if both $e1$ and $e2$ are true; evaluates $e2$ only if $e1$ is true.
$e1: T$ $e2: T$ T is bool or $\text{vec}N\langle \text{bool} \rangle$	$e1 \mid e2: T$	Logical "or". Component-wise when T is a vector. Evaluates both $e1$ and $e2$.

Precondition	Conclusion	Description
$e1: T$ $e2: T$ T is bool or $\text{vecN}<\text{bool}>$	$e1 \ \& \ e2: T$	Logical "and". Component-wise when T is a vector. Evaluates both $e1$ and $e2$.

§ 8.7. Arithmetic Expressions

Unary arithmetic expressions

Precondition	Conclusion	Description
$e: T$ T is AbstractInt, AbstractFloat, i32, f32, f16, $\text{vecN}<\text{AbstractInt}>$, $\text{vecN}<\text{AbstractFloat}>$, $\text{vecN}<\text{i32}>$, $\text{vecN}<\text{f32}>$, or $\text{vecN}<\text{f16}>$	$-e: T$	Negation. Component-wise when T is a vector. If T is an integer scalar type and e evaluates to the largest negative value, then the result is e .

Binary arithmetic expressions

Precondition	Conclusion	Description
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S , or $\text{vecN}<S>$	$e1 + e2: T$	Addition. Component-wise when T is a vector. If T is a floating point type, the scalar domain is the set of all pairs of extended reals (x,y) except: $(-\infty, +\infty)$ $(+\infty, -\infty)$
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S , or $\text{vecN}<S>$	$e1 - e2: T$	Subtraction Component-wise when T is a vector. If T is a floating point type, the scalar domain is the set of all pairs of extended reals (x,y) except: $(-\infty, -\infty)$ $(+\infty, +\infty)$
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S , or $\text{vecN}<S>$	$e1 * e2: T$	Multiplication. Component-wise when T is a vector. If T is a floating point type, the scalar domain is the set of all pairs of extended reals (x,y) except: $(0, -\infty)$ $(0, +\infty)$ $(-\infty, 0)$ $(+\infty, 0)$

Precondition	Conclusion	Description
		Division. Component-wise when T is a vector. If T is a signed integer scalar type, evaluates to: - If $e2$ is zero: - It is a shader-creation error if $e2$ is a const-expression. - It is a pipeline-creation error if $e2$ is an override-expression. - Otherwise, $e1$. - If $e1$ is most negative value in T, and $e2$ is -1: - It is a shader-creation error if $e1$ and $e2$ are const-expressions. - It is a pipeline-creation error if $e1$ and $e2$ are override-expressions. - Otherwise, $e1$. truncate(x) otherwise, where x is the real-valued quotient $e1 \div e2$.
$e1 : T$ $e2 : T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S, or vecN<S>	$e1 / e2 : T$	NOTE: The need to ensure truncation behavior may require an implementation to perform more operations than when computing an unsigned division. Use unsigned division when both operands are known to have the same sign. If T is an unsigned integer scalar type, evaluates to: - If $e2$ is zero: - It is a shader-creation error if $e2$ is a const-expression. - It is a pipeline-creation error if $e2$ is an override-expression. - Otherwise, $e1$. - Otherwise, the integer q such that $e1 = q \times e2 + r$, where $0 \leq r < e2$. If T is a floating point type, the scalar domain is the set of all pairs of extended reals (x,y) except: $(0,0)$ $(-\infty, -\infty)$ $(-\infty, +\infty)$ $(+\infty, -\infty)$ $(+\infty, +\infty)$
$e1 : T$ $e2 : T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S, or vecN<S>	$e1 \% e2 : T$	Remainder. Component-wise when T is a vector. If T is a signed integer scalar type, evaluates $e1$ and $e2$ once, and evaluates to: - if $e2$ is zero: - It is a shader-creation error if $e2$ is a const-expression.

Precondition	Conclusion	Description
		- It is a pipeline-creation error if $e2$ is an override-expression. - Otherwise, 0.
		- If $e1$ is the most negative value in T, and $e2$ is -1: - It is a shader-creation error if $e1$ and $e2$ are const-expressions. - It is a pipeline-creation error if $e1$ and $e2$ are override-expressions. - Otherwise, 0. - Otherwise, $e1 - \text{truncate}(e1 \div e2) \times e2$ where the quotient is computed as a real value.

NOTE: When non-zero, the result has the same sign as $e1$.

NOTE: The need to ensure consistent behavior may require an implementation to perform more operations than when computing an unsigned remainder.

If T is an unsigned [integer scalar](#) type, evaluates to:

- if $e2$ is zero:
 - It is a [shader-creation error](#) if $e2$ is a [const-expression](#).
 - It is a [pipeline-creation error](#) if $e2$ is an [override-expression](#).
 - Otherwise, 0.
- Otherwise, the integer r such that $e1 = q \times e2 + r$, where q is an integer and $0 \leq r < e2$.

If T is a floating point type, the result is equal to:

$e1 - e2 * \text{trunc}(e1 / e2)$.

If T is a floating point type, the scalar [domain](#) is the set of all pairs of [extended reals](#) (x,y) except:

- Cases outside the domain of x / y :
 - $(0,0)$
 - $(-\infty, -\infty)$
 - $(-\infty, +\infty)$
 - $(+\infty, -\infty)$
 - $(+\infty, +\infty)$
- Additional cases outside the domain of $y * \text{trunc}(x / y)$:
 - y is infinite, and x is finite, implying $\text{trunc}(x / y)$ is 0.
 - y is 0, and x is infinite, implying $\text{trunc}(x / y)$ is infinite.

Binary arithmetic expressions with mixed scalar and vector operands

Preconditions	Conclusions	Semantics
S is one of AbstractInt, AbstractFloat, f32, f16, i32, u32 V is $\text{vec}N<S>$ $es: S$ $ev: V$	$ev + es: V$	$ev + V(es)$
	$es + ev: V$	$V(es) + ev$
	$ev - es: V$	$ev - V(es)$
	$es - ev: V$	$V(es) - ev$
	$ev * es: V$	$ev * V(es)$
	$es * ev: V$	$V(es) * ev$
	$ev / es: V$	$ev / V(es)$
	$es / ev: V$	$V(es) / ev$
	$ev \% es: V$	$ev \% V(es)$
	$es \% ev: V$	$V(es) \% ev$

Matrix arithmetic

Preconditions	Conclusions	Semantics
$e1, e2: \text{mat}CxR<T>$ T is AbstractFloat, f32, or f16	$e1 + e2: \text{mat}CxR<T>$	Matrix addition: results are computed component-wise , column i of the result is $e1[i] + e2[i]$
	$e1 - e2: \text{mat}CxR<T>$	Matrix subtraction: results are computed component-wise , column i of the result is $e1[i] - e2[i]$
$m: \text{mat}CxR<T>$ $s: T$ T is AbstractFloat, f32, or f16	$m * s: \text{mat}CxR<T>$	Component-wise scaling: $(m * s)[i][j]$ is $m[i][j] * s$
	$s * m: \text{mat}CxR<T>$	Component-wise scaling: $(s * m)[i][j]$ is $m[i][j] * s$
$m: \text{mat}CxR<T>$ $v: \text{vec}C<T>$ T is AbstractFloat, f32, or f16	$m * v: \text{vec}R<T>$	Linear algebra matrix-column-vector product: Component i of the result is $\text{dot}(\text{transpose}(m)[i], v)$
$m: \text{mat}CxR<T>$ $v: \text{vec}R<T>$ T is AbstractFloat, f32, or f16	$v * m: \text{vec}C<T>$	Linear algebra row-vector-matrix product: transpose ($\text{transpose}(m) * \text{transpose}(v)$)
$e1: \text{mat}KxR<T>$ $e2: \text{mat}CxK<T>$ T is AbstractFloat, f32, or f16	$e1 * e2: \text{mat}CxR<T>$	Linear algebra matrix product.

§ 8.8. Comparison Expressions

Comparisons

Precondition	Conclusion	Description
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, bool, i32, u32, f32, or f16 T is S or $\text{vecN}<S>$ TB is $\text{vecN}<\text{bool}>$ if T is a vector, otherwise TB is bool	$e1 == e2: TB$	Equality. Component-wise when T is a vector.
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, bool, i32, u32, f32, or f16 T is S or $\text{vecN}<S>$ TB is $\text{vecN}<\text{bool}>$ if T is a vector, otherwise TB is bool	$e1 != e2: TB$	Inequality. Component-wise when T is a vector.
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S , or $\text{vecN}<S>$ TB is $\text{vecN}<\text{bool}>$ if T is a vector, otherwise TB is bool	$e1 < e2: TB$	Less than. Component-wise when T is a vector.
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S , or $\text{vecN}<S>$ TB is $\text{vecN}<\text{bool}>$ if T is a vector, otherwise TB is bool	$e1 <= e2: TB$	Less than or equal. Component-wise when T is a vector.
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S , or $\text{vecN}<S>$ TB is $\text{vecN}<\text{bool}>$ if T is a vector, otherwise TB is bool	$e1 > e2: TB$	Greater than. Component-wise when T is a vector.
$e1: T$ $e2: T$ S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S , or $\text{vecN}<S>$ TB is $\text{vecN}<\text{bool}>$ if T is a vector, otherwise TB is bool	$e1 >= e2: TB$	Greater than or equal. Component-wise when T is a vector.

§ 8.9. Bit Expressions

Unary bitwise operations

Precondition	Conclusion	Description
$e: T$ S is AbstractInt, i32, or u32 T is S or $\text{vecN}\langle S \rangle$	$\sim e : T$	Bitwise complement on e . Each bit in the result is the opposite of the corresponding bit in e . Component-wise when T is a vector.

Binary bitwise operations

Precondition	Conclusion	Description
$e1: T$ $e2: T$ S is AbstractInt, i32, or u32 T is S or $\text{vecN}\langle S \rangle$	$e1 \mid e2: T$	Bitwise-or. Component-wise when T is a vector.
$e1: T$ $e2: T$ S is AbstractInt, i32, or u32 T is S or $\text{vecN}\langle S \rangle$	$e1 \& e2: T$	Bitwise-and. Component-wise when T is a vector.
$e1: T$ $e2: T$ S is AbstractInt, i32, or u32 T is S or $\text{vecN}\langle S \rangle$	$e1 \wedge e2: T$	Bitwise-exclusive-or. Component-wise when T is a vector.

Bit shift expressions

Precondition	Conclusion	Description
$e1: T$ $e2: TS$ S is i32 or u32 T is S or $\text{vecN}\langle S \rangle$ TS is u32 when T is S , otherwise TS is $\text{vecN}\langle \text{u32} \rangle$	$e1 \ll e2: T$	Shift left (shifted value is concrete): Shift $e1$ left, inserting zero bits at the least significant positions, and discarding the most significant bits. The number of bits to shift is the value of $e2$, modulo the bit width of $e1$. If $e2$ is greater than or equal to the bit width of $e1$, then: - It is a shader-creation error if $e2$ is a const-expression. - It is a pipeline-creation error if $e2$ is an override-expression. When both $e1$ and $e2$ are known before shader execution start, the result must not overflow: - If T is a signed integer type, and the $e2+1$ most significant bits of $e1$ do not have the same bit value, then:

Precondition	Conclusion	Description
		◦ It is a shader-creation error if $e1$ and $e2$ are const-expressions. ◦ It is a pipeline-creation error if $e1$ and $e2$ are override-expressions. • If T is an unsigned integer type, and any of the $e2$ most significant bits of $e1$ are 1, then: ◦ It is a shader-creation error if $e1$ and $e2$ are const-expressions. ◦ It is a pipeline-creation error if $e1$ and $e2$ are override-expressions. Component-wise when T is a vector.
$e1: T$ $e2: TS$ T is AbstractInt or $vecN<AbstractInt>$ TS is u32 when T is AbstractInt, otherwise TS is $vecN<u32>$	$e1 \ll e2: T$	Shift left (shifted value abstract): Shift $e1$ left, inserting zero bits at the least significant positions, and discarding the most significant bits. The number of bits to shift is the value of $e2$. The $e2+1$ most significant bits of $e1$ must have the same bit value. Otherwise overflow would occur. NOTE: This condition means all the discarded bits must be the same as the sign bit of the original value, and the same as the sign bit of the final value. Component-wise when T is a vector.

Precondition	Conclusion	Description
		Shift right (shifted value is concrete).
		Shift $e1$ right, discarding the least significant bits.
		If S is an unsigned type, insert zero bits at the most significant positions.
		If S is a signed type:
$e1: T$ $e2: TS$ S is i32 or u32 T is S or $\text{vecN}<S>$ TS is u32 when T is S , otherwise TS is $\text{vecN}<\text{u32}>$	$e1 \gg e2: T$	- If $e1$ is negative, each inserted bit is 1, and so the result is also negative. - Otherwise, each inserted bit is 0. The number of bits to shift is the value of $e2$, modulo the bit width of $e1$. If $e2$ is greater than or equal to the bit width or $e1$, then: - It is a shader-creation error if $e2$ is a const-expression. - It is a pipeline-creation error if $e2$ is an override-expression. Component-wise when T is a vector.
		Shift right (abstract).
		Shift $e1$ right, discarding the least significant bits.
		If $e1$ is negative, each inserted bit is 1, and so the result is also negative. Otherwise, each inserted bit is 0.
$e1: T$ $e2: TS$ T is AbstractInt or $\text{vecN}<\text{AbstractInt}>$	$e1 \gg e2: T$	A function call expression executes a function call where the called function has a return type. If the called function does not return a value, a function call statement should be used instead. See § 9.5 Function Call Statement. Component-wise when T is a vector.

8.10. AbstractInt Call Expression

8.10.1. [AbstractInt](#) Call Expression

8.10.2. [AbstractInt](#) Call Statement

8.11. Variable Identifier Expression

§ 8.11. Variable Identifier Expression

Getting a reference from a variable name

Precondition	Conclusion	Description
v is an identifier resolving to an in-scope variable declared in address space AS with store type T and access mode AM	$v: \text{ref} \langle AS, T, AM \rangle$	Result is a reference to the memory for the named variable v .

§ 8.12. Formal Parameter Expression

Getting the value of an identifier declared as a formal parameter to a function

Precondition	Conclusion	Description
a is an identifier resolving to an in-scope formal parameter declaration with type T	$a: T$	Result is the value supplied for the corresponding function call operand at the call site invoking this instance of the function.

§ 8.13. Address-Of Expression

The ***address-of*** operator converts a reference to its corresponding pointer.

Getting a pointer from a reference

Precondition	Conclusion	Description
r : $\text{ref}\langle AS, T, AM \rangle$	$\&r: \text{ptr}\langle AS, T, AM \rangle$	Result is the pointer value corresponding to the same memory view as the reference value r. If r is an invalid memory reference, then the resulting pointer is also an invalid memory reference. It is a shader-creation error if AS is the handle address space. It is a shader-creation error if r is a reference to a vector component.

§ 8.14. Indirection Expression

The ***indirection*** operator converts a pointer to its corresponding reference.

Getting a reference from a pointer

Precondition	Conclusion	Description
p : $\text{ptr}\langle AS, T, AM \rangle$	$*p: \text{ref}\langle AS, T, AM \rangle$	Result is the reference value corresponding to the same memory view as the pointer value p. If p is an invalid memory reference, then the resulting reference is also an invalid memory reference.

§ 8.15. Identifier Expressions for Value Declarations

Getting the value of a const-, override-, or let-declared identifiers

Precondition	Conclusion	Description
c is an identifier resolving to an in-scope const-declaration with type T	$c: T$	Result is the value computed for the initializer expression. The expression is a const-expression , and is evaluated at shader-creation time.
c is an identifier resolving to an in-scope override-declaration with type T	$c: T$	If pipeline creation specified a value for the constant ID, then the result is that value. This value may be different for different pipeline instances. Otherwise, the result is the value computed for the initializer expression. Pipeline-overridable constants appear at module-scope, so evaluation occurs before the shader begins execution. NOTE: Pipeline creation fails if no initial value was specified in the API call and the <code>let</code>-declaration has no initializer expression.
c is an identifier resolving to an in-scope let-declaration with type T	$c: T$	Result is the value computed for the initializer expression. A let-declaration appears inside a function body, and its initializer is evaluated each time control flow reaches the declaration.

§ 8.16. Enumeration Expressions

Enumeration expressions

Precondition	Conclusion	Description
e is an identifier resolving to a predeclared enumerant belonging to enumeration type E	$e: E$	See § 6.4.1 Predeclared enumerants

§ 8.17. Type Expressions

Type expressions

Precondition	Conclusion	Description
t is an identifier resolving to a predeclared type	$t: \text{AllTypes}$	See § 6.10 Predeclared Types and Type-Generators Summary
a is an identifier resolving to a type alias .	$a: \text{AllTypes}$	Additionally, a denotes the type to which it is aliased.
s is an identifier resolving to the declaration of a structure type.	$s: \text{AllTypes}$	Additionally, s denotes the structure type.
tg is an identifier resolving to a type-generator	tg __template_args_start	Each type-generator has its own requirements on the template parameters it requires and accepts, and defines

Precondition	Conclusion	Description
$e1: T1$	$e1,$	how the template parameters help determine the resulting type.
$\dots$	$\dots,$	
$eN: TN$	eN	The expressions $e1$ through eN are the template parameters for the type-generator.
	_template_args_end	
	$: AllTypes$	For example, the type expression <code>vec2<f32></code> is the vector of two f32 elements.
	tg	
	_template_args_start	See § 6.10 Predeclared Types and Type-Generators Summary for the list of predeclared type-generators.
	$e1,$	
	$\dots,$	
	$eN,$	
	_template_args_end	
	$: AllTypes$	NOTE: The two variants here differ only in whether they have a trailing comma after eN.

§ 8.18. Expression Grammar Summary

When an [identifier](#) is the first [token](#) in a [call_phrase](#), it is one of:

- The name of a [user-defined function](#) or [built-in function](#), as part of a [function call](#).
- The name of a [type](#), [type alias](#), or [type-generator](#), as part of a [value constructor](#) expression.

[Declaration and scope](#) rules ensure those names are always distinct.

```

primary_expression :
    \_template\_elaborated\_ident
| \_call\_expression
| \_literal
| \_paren\_expression

```

```

call_expression :
    \_call\_phrase

```

Note: The [_call_expression](#) rule exists to ensure [type checking](#) applies to the call expression.

```

call_phrase :
    \_template\_elaborated\_ident \_argument\_expression\_list

```

```

paren_expression :
    '(' \_expression ')'

```

```

argument_expression_list :
    '(' \_expression\_comma\_list ? ')'

```

```

expression_comma_list :
    \_expression ( ',' \_expression ) * ',' ?

```

```

component_or_swizzle_specifier :

```

'[' expression ']' component_or_swizzle_specifier ?
 | '.' member_ident component_or_swizzle_specifier ?
 | '.' swizzle_name component_or_swizzle_specifier ?

unary_expression :

singular_expression
 | '-' unary_expression
 | '!' unary_expression
 | '~' unary_expression
 | '*' unary_expression
 | '&' unary_expression

singular_expression :

primary_expression component_or_swizzle_specifier ?

lhs_expression :

core_lhs_expression component_or_swizzle_specifier ?
 | '*' lhs_expression
 | '&' lhs_expression

core_lhs_expression :

ident
 | '(' lhs_expression ')'
 | call_expression

multiplicative_expression :

unary_expression
 | multiplicative_expression multiplicative_operator unary_expression

multiplicative_operator :

'*'
 | '/'
 | '%'

additive_expression :

multiplicative_expression
 | additive_expression additive_operator multiplicative_expression

additive_operator :

'+'
 | '-'

shift_expression :

additive_expression
 | unary_expression _shift_left unary_expression
 | unary_expression _shift_right unary_expression

relational_expression :

shift_expression

- | shift_expression less_than shift_expression
- | shift_expression greater_than shift_expression
- | shift_expression less_than_equal shift_expression
- | shift_expression greater_than_equal shift_expression
- | shift_expression '==' shift_expression
- | shift_expression '!=' shift_expression

short_circuit_and_expression :

- relational_expression
- | short_circuit_and_expression '&&' relational_expression

short_circuit_or_expression :

- relational_expression
- | short_circuit_or_expression '||' relational_expression

binary_or_expression :

- unary_expression
- | binary_or_expression '|' unary_expression

binary_and_expression :

- unary_expression
- | binary_and_expression '&' unary_expression

binary_xor_expression :

- unary_expression
- | binary_xor_expression '^' unary_expression

bitwise_expression :

- binary_and_expression '&' unary_expression
- | binary_or_expression '|' unary_expression
- | binary_xor_expression '^' unary_expression

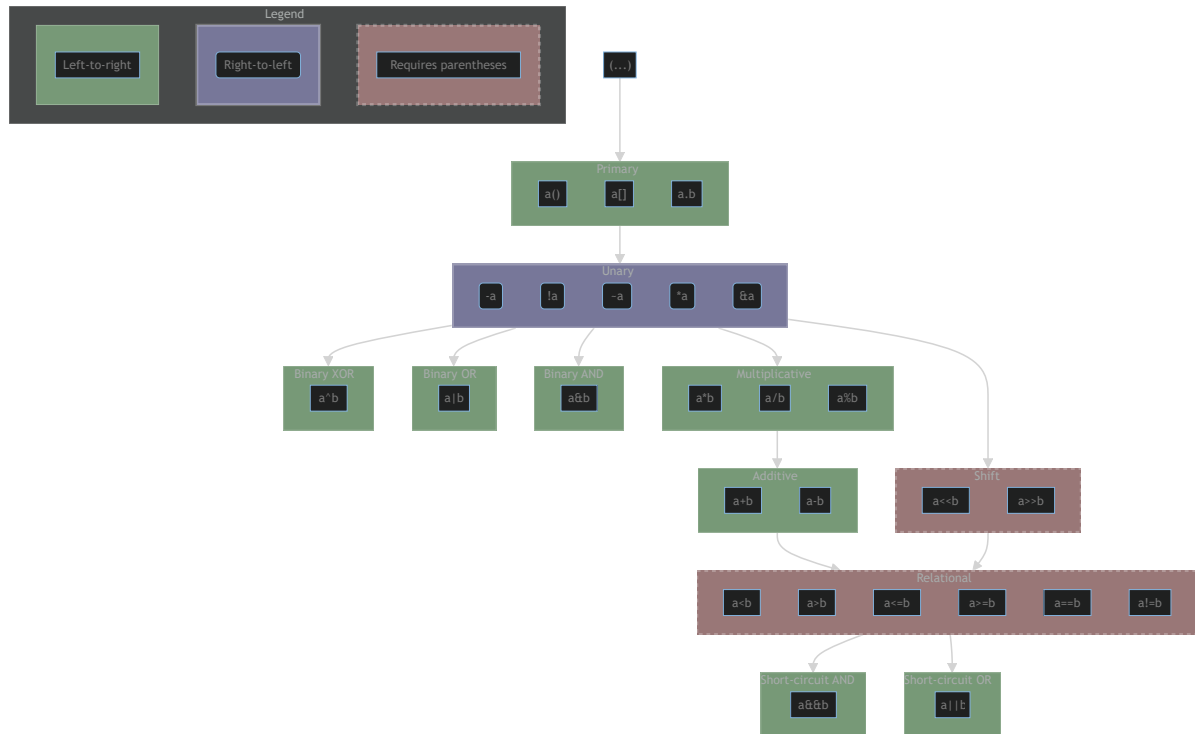
expression :

- relational_expression
- | short_circuit_or_expression '||' relational_expression
- | short_circuit_and_expression '&&' relational_expression
- | bitwise_expression

§ 8.19. Operator Precedence and Associativity

This entire subsection is non-normative.

Operator precedence and associativity in [right-hand side WGSL expressions](#) emerge from their grammar in summary. Right-hand expressions group operators to organize them, as illustrated by the following diagram:

Figure 1 Operator precedence and associativity graph

To promote readability through verbosity, the following groups do not associate with other groups:

- [Short-circuit OR](#) (can associate with [self](#) and [relational](#) weakly),
- [Short-circuit AND](#) (can associate with [self](#) and [relational](#) weakly),
- [Binary OR](#) (can associate with [self](#) and [unary](#) weakly),
- [Binary AND](#) (can associate with [self](#) and [unary](#) weakly),
- [Binary XOR](#) (can associate with [self](#) and [unary](#) weakly).

And the following groups do not associate with themselves:

- [Shift](#) (can associate with [unary](#) weakly),
- [Relational](#) (can associate with [additive](#) and [shift](#) weakly).

Associating both group sections above requires parentheses to set the relationship explicitly. The following exemplifies where these rules render expressions invalid in comments:

EXAMPLE: OPERATOR PRECEDENCE CORNER CASES

```
let a = x & (y ^ (z | w)); // Invalid: x & y ^ z | w
let b = (x + y) << (z >= w); // Invalid: x + y << z >= w
let c = x < (y > z); // Invalid: x < y > z
let d = x && (y || z); // Invalid: x && y || z
```

Emergent precedence controls the implicit parentheses of an expression, where the stronger binding operator will act as if it is surrounded by parentheses when together with operators of weaker precedence. For example, stronger binding multiplicative operators than additive will infer $(a + (b * c))$ from $a + b * c$ expression. Similarly, the emergent associativity controls the direction of these implicit parentheses. For example, a left-to-right association will infer $((a + b) + c)$ from $a + b + c$ expression, whereas a right-to-left association will infer $(* (* a))$ from $* * a$ expression.

The following table summarizes operator precedence, associativity, and binding, sorting by starting with strongest to weakest. The binding column contains the stronger expression of the given operator, meaning, for example, if "All above" is the value, then this operator can include any of the stronger expressions. But, for example, if "Unary" is the value, then anything weaker than unary but stronger than the operator at row would require parentheses to bind with this operator. This column is necessary for linearly listing operators.

Operator precedence, associativity, and binding for right-hand side expressions, sorted from strong to weak

Name	Operators	Associativity	Binding
Parenthesized	(...)		
Primary	a(), a[], a.b	Left-to-right	
Unary	-a, !a, ~a, *a, &a	Right-to-left	All above
Multiplicative	a*b, a/b, a%b	Left-to-right	All above
Additive	a+b, a-b	Left-to-right	All above
Shift	a<<b, a>>b	Requires parentheses	Unary
Relational	a<b, a>b, a<=b, a>=b, a==b, a!=b	Requires parentheses	All above
Binary AND	a&b	Left-to-right	Unary
Binary XOR	a^b	Left-to-right	Unary
Binary OR	a b	Left-to-right	Unary
Short-circuit AND	a&&b	Left-to-right	Relational
Short-circuit OR	a b	Left-to-right	Relational

9. Statements

A **statement** is a program fragment that controls execution. Statements are generally executed in sequential order; however, [control flow statements](#) may cause a program to execute in non-sequential order.

9.1. Compound Statement

A **compound statement** is a brace-enclosed sequence of zero or more statements. When a [declaration](#) is one of those statements, its [identifier](#) is [in scope](#) from the start of the next statement until the end of the compound statement.

```
compound_statement :
    attribute * '{' statement * '}'
```

The [continuing compound statement](#) is a special form of compound statement that forms the body of a [continuing](#) statement, and allows an optional [break-if](#) statement at the end.

§ 9.2. Assignment Statement

An *assignment* evaluates an expression, and optionally stores it in memory (thus updating the contents of a variable).

```
assignment_statement :  
    lhs_expression ( '=' | compound_assignment_operator ) expression  
    | '++' '=' expression
```

The text to the left of the operator token is the *left-hand side*, and the expression to the right of the operator token is the *right-hand side*.

§ 9.2.1. Simple Assignment

An [assignment](#) is a *simple assignment* when the [left-hand side](#) is an expression, and the operator is the equal ('=') token. In this case the value of the [right-hand side](#) is written to the memory referenced by the left-hand side.

Precondition	Statement	Description
$e: T$, T is a concrete constructible type, $r: \text{ref}\langle AS, T, AM \rangle$, AS is a writable address space , access mode AM is write or read_write	$r = e$	Evaluates r , then evaluates e , then writes the value computed for e into the memory locations referenced by r . NOTE: If the reference is an invalid memory reference, the write may not execute, or may write to a different memory location than expected.

In the simplest case, the left hand side is the name of a variable. See [§ 6.5.8 Forming Reference and Pointer Values](#) for other cases.

EXAMPLE: ASSIGNMENTS

```
struct S {
    age: i32,
    weight: f32
}
var<private> person: S;

fn f() {
    var a: i32 = 20;
    a = 30;           // Replace the contents of 'a' with 30.

    person.age = 31;  // Write 31 into the age field of the person variable.

    var uv: vec2<f32>;
    uv.y = 1.25;      // Place 1.25 into the second component of uv.

    let uv_x_ptr: ptr<function,f32> = &uv.x;
    *uv_x_ptr = 2.5;  // Place 2.5 into the first component of uv.

    var sibling: S;
    // Copy the contents of the 'person' variable into the 'sibling' variable.
    sibling = person;
}
```

9.2.2. Phony Assignment

An [assignment](#) is a *phony assignment* when the [left-hand side](#) is the underscore ('_') token. In this case the [right-hand side](#) is evaluated, and then ignored.

Precondition	Statement	Description
$e: T$, T is constructible , a pointer type , a texture type, or a sampler type	<code>_ = e</code>	Evaluates e . NOTE: The resulting value is not stored. The <code>_</code> token is not an identifier, and therefore cannot be used in an expression.

A phony-assignment is useful for:

- Calling a function that returns a value, but clearly expressing that the resulting value is not needed.
- [Statically accessing](#) a variable, thus establishing it as a part of the [shader’s resource interface](#).

NOTE: A buffer variable’s store type may not be constructible, e.g. it contains an atomic type, or a runtime-sized array. In these cases, use a pointer to the variable’s contents instead.

EXAMPLE: USING PHONY-ASSIGNMENT TO THROW AWAY AN UN-NEEDED FUNCTION RESULT

```

var<private> counter: i32;

fn increment_and_yield_previous() -> i32 {
    let previous = counter;
    counter = counter + 1;
    return previous;
}

fn user() {
    // Increment the counter, but don't use the result.
    _ = increment_and_yield_previous();
}

```

EXAMPLE: USING PHONY-ASSIGNMENT TO OCCUPY BINDINGS WITHOUT USING THEM

```

struct BufferContents {
    counter: atomic<u32>,
    data: array<vec4<f32>>
}

@group(0) @binding(0) var<storage> buf: BufferContents;
@group(0) @binding(1) var t: texture_2d<f32>;
@group(0) @binding(2) var s: sampler;

@fragment
fn shade_it() -> @location(0) vec4<f32> {
    // Declare that buf, t, and s are part of the shader interface, without
    // using them for anything.
    _ = &buf;
    _ = t;
    _ = s;
    return vec4<f32>();
}

```

§ 9.2.3. Compound Assignment

An [assignment](#) is a *compound assignment* when the [left-hand side](#) is an expression, and the operator is one of the [compound_assignment_operators](#).

compound_assignment_operator :

```

    '+'=
|   '-'=
|   '*='
|   '/='
|   '%='
|   '&='
|   '|='
|   '^='
|   _shift_right_assign
|   _shift_left_assign

```

The type requirements, semantics, and behavior of each statement is defined as if the compound assignment expands as in the following table, except that:

- the reference expression el is evaluated only once, and
- the [reference type](#) for el [must](#) have a [read_write access mode](#).

Statement	Expansion
$el += e2$	$el = el + (e2)$
$el -= e2$	$el = el - (e2)$
$el *= e2$	$el = el * (e2)$
$el /= e2$	$el = el / (e2)$
$el \% = e2$	$el = el \% (e2)$
$el \&= e2$	$el = el \& (e2)$
$el = e2$	$el = el (e2)$
$el \wedge = e2$	$el = el \wedge (e2)$
$el >> = e2$	$el = el >> (e2)$
$el << = e2$	$el = el << (e2)$

NOTE: The syntax does not allow a [compound assignment](#) to also be a [phony assignment](#).

NOTE: Even though the reference el is evaluated once, its underlying memory is accessed twice: first a [read access](#) gets the old value, and then a [write access](#) stores the updated value.

EXAMPLE: COMPOUND ASSIGNMENT

```

var<private> next_item: i32 = 0;

fn advance_item() -> i32 {
    next_item += 1;    // Adds 1 to next_item.
    return next_item - 1;
}

fn bump_item() {
    var data: array<f32,10>;
    next_item = 0;
    // Adds 5.0 to data[0], calling advance_item() only once.
    data[advance_item()] += 5.0;
    // next_item will be 1 here.
}

fn precedence_example() {
    var value = 1;
    // The right-hand side of a compound assignment is its own expression.
    value *= 2 + 3; // Same as value = value * (2 + 3);
    // 'value' now holds 5.
}

```

NOTE: A compound assignment can be rewritten as different WGSL code that uses a [simple assignment](#) instead. The idea is to use a pointer to hold the result of evaluating the reference once.

For example, when *e1* is *not* a reference to a component inside a vector, then

```
e1 += e2;
```

can be rewritten as

```
{ let p = &(e1); *p = *p + (e2); }
```

where the identifier *p* is chosen to be different from all other identifiers in the program.

When *e1* is a reference to a component inside a vector, the above technique needs to be modified because WGSL does not allow [taking the address](#) in that case. For example, if *ev* is a reference to a vector, the statement

```
ev[c] += e2;
```

can be rewritten as

```
{ let p = &(ev); let c0 = c; (*p)[c0] = (*p)[c0] + (e2); }
```

where identifiers *c0* and *p* are chosen to be different from all other identifiers in the program.

9.3. Increment and Decrement Statements

An **increment statement** adds 1 to the contents of a variable. A **decrement statement** subtracts 1 from the contents of a variable.

```
increment_statement :  
    lhs_expression '++'  
  
decrement_statement :  
    lhs_expression '--'
```

The expression must evaluate to a reference with a [concrete integer scalar store type](#) and [read_write access mode](#).

Precondition	Statement	Description
r : ref<AS,T,read_write>, T is a concrete integer scalar	$r++$	Adds 1 to the contents of memory referenced by r . Same as $r += T(1)$
r : ref<AS,T,read_write>, T is a concrete integer scalar	$r--$	Subtracts 1 from the contents of memory referenced by r . Same as $r -= T(1)$

EXAMPLE: INCREMENT AND DECREMENT

```
fn f() {  
    var a: i32 = 20;  
    a++;  
    // Now a contains 21  
    a--;  
    // Now a contains 20  
}
```

9.4. Control Flow

Control flow statements may cause the program to execute in non-sequential order.

9.4.1. If Statement

An **if** statement conditionally executes at most one [compound statement](#) based on the evaluation of condition expressions.

An if statement has an if clause, followed by zero or more else if clauses, followed by an optional else clause.

```
if_statement :  
    attribute * if_clause else_if_clause * else_clause ?  
  
if_clause :  
    'if' expression compound_statement  
  
else_if_clause :
```



```
'else' 'if' expression compound_statement
```

```
else_clause :
```

```
'else' compound_statement
```

Type rule precondition: The expression in each `if` and `else if` clause [must](#) be of [bool](#) type.

An `if` statement is executed as follows:

- The condition associated with the `if` clause is evaluated. If the result is `true`, control transfers to the first compound statement (immediately after the condition expression).
- Otherwise, the condition of the next `else if` clause in textual order (if one exists) is evaluated and, if the result is `true`, control transfers to the associated compound statement.
 - This behavior is repeated for all `else if` clauses until one of the conditions evaluates to `true`.
- If no condition evaluates to `true`, then control transfers to the compound statement associated with the `else` clause (if it exists).

9.4.2. Switch Statement

A ***switch*** statement transfers control to one of a set of [case clauses](#), or to the [default clause](#), depending on the evaluation of a selector expression.

```
switch_statement :
```

```
attribute * 'switch' expression switch_body
```

```
switch_body :
```

```
attribute * '{' switch_clause + '}'
```

```
switch_clause :
```

```
case_clause
```

```
| default_alone_clause
```

```
case_clause :
```

```
'case' case_selectors ':' ? compound_statement
```

```
default_alone_clause :
```

```
'default' ':' ? compound_statement
```

```
case_selectors :
```

```
case_selector ( ',' case_selector ) * ',' ?
```

```
case_selector :
```

```
'default'
```

```
| expression
```

A ***case clause*** is the `'case'` token followed by a comma-separated list of [case selectors](#) and a body in the form of a [compound statement](#).

A ***default-alone clause*** is the `'default'` token followed by a body in the form of a [compound statement](#).

A *default clause* is either:

- a [case clause](#) where `'default'` appears as one of its selectors, or
- a [default-alone clause](#).

Each switch statement [must](#) have exactly one [default clause](#).

The `'default'` token [must](#) not appear more than once in a single [case_selector](#) list.

Type rule precondition: For a single switch statement, the selector expression and all case selector expressions [must](#) be of the same [concrete integer scalar](#) type.

The expressions in the [case_selectors](#) [must](#) be [const-expressions](#).

Two different case selector expressions in the same switch statement [must not](#) have the same value.

If the selector value equals the value of an expression in a [case_selector](#) list, then control is transferred to the body of that [case clause](#). If the selector value does not equal any of the case selector values, then control is transferred to the body of the [default clause](#).

When control reaches the end of the body of a clause, control transfers to the first statement after the switch statement.

When one of the statements in the body of a clause is a [declaration](#), it follows the normal [scope](#) and [lifetime](#) rules of a declaration in a [compound statement](#). That is, the body is a sequence of statements, and if one of those is a declaration then the scope of that declaration extends from the start of the next statement in the sequence until the end of the body. The declaration executes when it is reached, creating a new instance of the [variable](#) or [value](#), and initializes it.

EXAMPLE: WGSL SWITCH

```
var a : i32;
let x : i32 = generateValue();
switch x {
  case 0: {      // The colon is optional
    a = 1;
  }
  default {     // The default need not appear last
    a = 2;
  }
  case 1, 2, {  // Multiple selector values can be used
    a = 3;
  }
  case 3, {     // The trailing comma is optional
    a = 4;
  }
  case 4 {
    a = 5;
  }
}
```

EXAMPLE: WGS L SWITCH WITH DEFAULT COMBINED

```

const c = 2;
var a : i32;
let x : i32 = generateValue();
switch x {
  case 0: {
    a = 1;
  }
  case 1, c {          // Const-expression can be used in case selectors
    a = 3;
  }
  case 3, default { // The default keyword can be used with other clauses
    a = 4;
  }
}

```

§ 9.4.3. Loop Statement

loop_statement :

attribute * 'loop' *attribute* * '{' *statement* * *continuing_statement* ? '}'

A **loop** statement repeatedly executes a **loop body**; the loop body is specified as a [compound statement](#). Each execution of the loop body is called an **iteration**.

This repetition can be interrupted by a [break](#), or [return](#) statement.

Optionally, the last statement in the loop body may be a [continuing](#) statement.

A [dynamic error](#) occurs if the [loop](#) would execute an unbounded number of [iterations](#). This may result in early termination of the loop, other non-local effects, or even [device loss](#).

When one of the statements in the loop body is a [declaration](#), it follows the normal [scope](#) and [lifetime](#) rules of a declaration in a [compound statement](#). That is, the loop body is a sequence of statements, and if one of those is a declaration then the scope of that declaration extends from the start of the next statement in the sequence until the end of the loop body. The declaration executes each time it is reached, so each new iteration creates a new instance of the [variable](#) or [value](#), and re-initializes it.

NOTE: The loop statement is a specialized construct, you probably want the `for` or `while` statements. The loop statement is one of the biggest differences from other shader languages.

This design directly expresses loop idioms commonly found in compiled code. In particular, placing the loop update statements at the end of the loop body allows them to naturally use values defined in the loop body.

EXAMPLE: FOR LOOP

```

var a: i32 = 2;
for (var i: i32 = 0; i < 4; i++) {
  a *= 2;
}

```

EXAMPLE: LOOP

```
var a: i32 = 2;
var i: i32 = 0;    // <1>
loop {
    if i >= 4 { break; }

    a = a * 2;

    i++;
}
```

- <1> The initialization is listed before the loop.

EXAMPLE: FOR LOOP WITH CONTINUE

```
var a: i32 = 2;
let step: i32 = 1;
for (var i: i32 = 0; i < 4; i += step) {
    if (i % 2 == 0) { continue; }
    a *= 2;
}
```

EXAMPLE: LOOP WITH CONTINUE

```
var a: i32 = 2;
var i: i32 = 0;
loop {
    if i >= 4 { break; }

    let step: i32 = 1;

    i = i + step;
    if i % 2 == 0 { continue; }

    a = a * 2;
}
```

EXAMPLE: LOOP WITH CONTINUE AND CONTINUING

```

var a: i32 = 2;
var i: i32 = 0;
loop {
    if i >= 4 { break; }

    let step: i32 = 1;

    if i % 2 == 0 { continue; }

    a = a * 2;

    continuing { // <2>
        i = i + step;
    }
}

```

- <2> The continue construct is placed at the end of the loop

§ 9.4.4. For Statement

for_statement :

attribute * 'for' '(' *for_header* ')' *compound_statement*

for_header :

for_init ? ';' *expression* ? ';' *for_update* ?

for_init :

variable_or_value_statement

| *variable_updating_statement*

| *func_call_statement*

for_update :

variable_updating_statement

| *func_call_statement*

The **for** statement is syntactic sugar over a [compound statement](#) containing a [loop](#) statement. In general, the **for** statement takes the form

```
for ( initializer ; condition ; update_part ) { body }
```

When the condition expression is present, the **for** statement desugars to a loop of the form:

```

{
  initializer ;
  loop {
    if !(condition) { break; }
    body
    continuing { update_part }
  }
}

```

When the condition expression is absent, the `for` statement desugars to a loop of the form:

```

{
  initializer ;
  loop {
    body
    continuing { update_part }
  }
}

```

Additionally:

- If `initializer` is non-empty, it is executed inside an additional [scope](#) before the first [iteration](#). The scope of a declaration in the initializer extends to the end of the loop body.
- [Type rule precondition](#): If the condition is non-empty, it [must](#) be an expression of [bool](#) type.
 - If present, the condition is evaluated immediately before executing the for-loop body. If the condition is false, then a [§ 9.4.6 Break Statement](#) is executed, finishing execution of the loop. This check is performed at the start of each loop iteration.
- If `update_part` is non-empty, it becomes a [continuing](#) statement at the end of the loop construct.
- Desugaring [will](#) rename identifiers declared in the body as needed to ensure all identifiers in the `update_part` still [resolve](#) to the same declarations they did before desugaring occurred.

The `initializer` of a `for` loop is executed once prior to executing the loop. When a [declaration](#) appears in the initializer, its [identifier](#) is [in scope](#) until the end of the body. Unlike declarations in the body, the declaration is not re-initialized each iteration.

The condition, body and `update_part` execute in that order to form a loop [iteration](#). The body is a special form of [compound statement](#). The identifier of a declaration in the body is [in scope](#) from the start of the next statement until the end of the body. The declaration is executed each time it is reached, so each new iteration creates a new instance of the variable or constant, and re-initializes it.

EXAMPLE: FOR TO LOOP TRANSFORMATION, WITH A CONDITION: BEFORE

```

var a: i32 = 2;
for (var i: i32 = 0; i < 4; i++) {
  if a == 0 {
    continue;
  }
  a = a + 2;
}

```

Converts to:

EXAMPLE: FOR TO LOOP TRANSFORMATION, WITH A CONDITION: AFTER

```

var a: i32 = 2;
{ // Introduce new scope for loop variable i
  var i: i32 = 0;
  loop {
    if !(i < 4) {
      break;
    }

    if a == 0 {
      continue;
    }
    a = a + 2;

    continuing {
      i++;
    }
  }
}

```

EXAMPLE: FOR TO LOOP TRANSFORMATION, WITHOUT A CONDITION: BEFORE

```

var a: i32 = 2;
for (var i: i32 = 0; ; i++) {
  if a == 0 {
    continue;
  }
  if i == 4 { break; }
  a = a + 2;
}

```

Converts to:

EXAMPLE: FOR TO LOOP TRANSFORMATION, WITHOUT A CONDITION: AFTER

```

var a: i32 = 2;
{ // Introduce new scope for loop variable i
  var i: i32 = 0;
  loop {
    // Note: Desugaring does not introduce an if clause here.

    if a == 0 {
      continue;
    }
    if i == 4 { break; }
    a = a + 2;

    continuing {
      i++;
    }
  }
}

```

A [dynamic error](#) occurs if the [for](#) loop would execute an unbounded number of [iterations](#). This may result in early termination of the loop, other non-local effects, or even [device loss](#).

§ 9.4.5. While Statement

```
while_statement :  
    attribute * 'while' expression compound_statement
```

The **while** statement is a kind of loop parameterized by a condition. At the start of each loop [iteration](#), a boolean condition is evaluated. If the condition is false, the while loop ends execution. Otherwise, the rest of the iteration is executed.

[Type rule precondition](#): The condition [must](#) be of [bool](#) type.

A while loop can be viewed as syntactic sugar over either a [loop](#) or [for](#) statement. The following statement forms are equivalent:

- `while condition { body_statements }`
- `loop { if ! condition {break;} body_statements }`
- `for (; condition ;) { body_statements }`

A [dynamic error](#) occurs if the [while](#) loop would execute an unbounded number of [iterations](#). This may result in early termination of the loop, other non-local effects, or even [device loss](#).

§ 9.4.6. Break Statement

```
break_statement :  
    'break'
```

A **break** statement transfers control to immediately after the body of the nearest-enclosing loop or [switch](#) statement, thus ending execution of the loop or switch statement.

A break statement [must](#) only be used within [loop](#), [for](#), [while](#), and [switch](#) statements.

A break statement [must not](#) be placed such that it would exit from a loop's [continuing](#) statement. Use a [break-if](#) statement instead.

EXAMPLE: WGSL INVALID LOOP BREAK FROM A CONTINUING CLAUSE

```

var a: i32 = 2;
var i: i32 = 0;
loop {
    let step: i32 = 1;

    if i % 2 == 0 { continue; }

    a = a * 2;

    continuing {
        i = i + step;
        if i >= 4 { break; } // Invalid. Use break-if instead.
    }
}

```

§ 9.4.7. Break-If Statement

break_if_statement :

```
'break' 'if' expression ';' ;
```

A **break-if** statement evaluates a boolean condition; If the condition is true, control is transferred to immediately after the body of the nearest-enclosing [loop](#) statement, ending execution of that loop.

[Type rule precondition](#): The condition [must](#) be of [bool](#) type.

NOTE: A break-if statement may only appear as the last statement in the body of a [continuing](#) statement.

EXAMPLE: WGSL VALID LOOP BREAK-IF FROM A CONTINUING CLAUSE

```

var a: i32 = 2;
var i: i32 = 0;
loop {
    let step: i32 = 1;

    if i % 2 == 0 { continue; }

    a = a * 2;

    continuing {
        i = i + step;
        break if i >= 4;
    }
}

```

§ 9.4.8. Continue Statement

continue_statement :

'continue'

A **continue** statement transfers control in the nearest-enclosing [loop](#):

- forward to the [continuing](#) statement at the end of the body of that loop, if it exists.
- otherwise backward to the first statement in the loop body, starting the next [iteration](#).

A continue statement **must** only be used in a [loop](#), [for](#) or [while](#) statement. A continue statement **must not** be placed such that it would transfer control to an enclosing [continuing](#) statement. (It is a *forward* branch when branching to a continuing statement.)

A continue statement **must not** be placed such that it would transfer control past a declaration used in the targeted [continuing](#) statement.

NOTE: A continue can only be used in a continuing statement if it is used for transferring control flow within another loop nested in the continuing statement. That is, a continue cannot be used to transfer control to the start of the currently executing continuing statement.

EXAMPLE: INVALID CONTINUE BYPASSES DECLARATION

```
var i: i32 = 0;
loop {
  if i >= 4 { break; }
  if i % 2 == 0 { continue; } // <3>

  let step: i32 = 2;

  continuing {
    i = i + step;
  }
}
```

- <3> The continue is invalid because it bypasses the declaration of step used in the continuing construct

9.4.9. Continuing Statement

continuing_statement :

'continuing' *continuing_compound_statement*

continuing_compound_statement :

attribute * '{' *statement* * *break_if_statement* ? '}'

A **continuing** statement specifies a [compound statement](#) to be executed at the end of a loop [iteration](#). The construct is optional.

The compound statement **must not** contain a [return](#) at any compound statement nesting level.

9.4.10. Return Statement

return_statement :

'**return**' *expression* ?

A **return** statement ends execution of the current function. If the function is an [entry point](#), then the current shader invocation is terminated. Otherwise, evaluation continues with the next expression or statement after the evaluation of the [call site](#) of the current function invocation.

If the function does not have a [return type](#), then the **return** statement is optional. If the return statement is provided for such a function, it [must not](#) supply a value. Otherwise the expression [must](#) be present, and is called the **return value**. In this case the call site of this function invocation evaluates to the return value. The type of the return value [must](#) match the return type of the function.

§ 9.4.11. Discard Statement

A **discard** statement converts the invocation into a [helper invocation](#) and throws away the fragment output. The discard statement [must](#) only be used in a [fragment](#) shader stage.

More precisely, executing a discard statement [will](#):

- convert the current invocation into a [helper invocation](#), and
- prevent the current fragment output from being processed downstream in the [GPURenderPipeline](#).

Only statements executed prior to the discard statement [will](#) have observable effects.

NOTE: A discard statement may be executed by any [function in a fragment stage](#) and the effect is the same: the fragment output will be thrown away.

EXAMPLE: USING THE DISCARD STATEMENT TO THROW AWAY A FRAGMENT

```
@group(0) @binding(0)
var<storage, read_write> will_emit_color : u32;

fn discard_if_shallow(pos: vec4<f32>) {
    if pos.z < 0.001 {
        // If this is executed, then the will_emit_color variable will
        // never be set to 1 because helper invocations will not write
        // to shared memory.
        discard;
    }
    will_emit_color = 1;
}

@fragment
fn main(@builtin(position) coord_in: vec4<f32>)
-> @location(0) vec4<f32>
{
    discard_if_shallow(coord_in);

    // Set the value to 1 and emit red, but only if the helper function
    // did not execute the discard statement.
    will_emit_color = 1;
    return vec4<f32>(1.0, 0.0, 0.0, 1.0);
}
```

§ 9.5. Function Call Statement

```
func_call_statement :  
    call_phrase
```

A function call statement executes a [function call](#).

A [shader-creation error](#) results if the called function has the [must_use](#) attribute.

NOTE: If the function [returns a value](#), and the function does not have the [must_use](#) attribute, that value is ignored.

§ 9.6. Statements Grammar Summary

The [statement](#) rule matches statements that can be used in most places inside a function body.

```
statement :  
    ';' |  
    | return_statement ';' |  
    | if_statement |  
    | switch_statement |  
    | loop_statement |  
    | for_statement |  
    | while_statement |  
    | func_call_statement ';' |  
    | variable_or_value_statement ';' |  
    | break_statement ';' |  
    | continue_statement ';' |  
    | 'discard' ';' |  
    | variable_updating_statement ';' |  
    | compound_statement |  
    | assert_statement ';' |
```

```
variable_updating_statement :  
    assignment_statement |  
    | increment_statement |  
    | decrement_statement
```

Additionally, certain statements may only be used in very specific contexts:

- [break if statement](#)
- [continuing compound statement](#)

§ 9.7. Statements Behavior Analysis

§ 9.7.1. Rules

Some statements affecting control-flow are only valid in some contexts. For example, [continue](#) is invalid outside of a [loop](#), [for](#), or [while](#). Additionally, the uniformity analysis (see [§ 15.2 Uniformity](#)) needs to know when control flow can exit a statement in multiple different ways.

Both goals are achieved by a system for summarizing execution behaviors of statements. Behavior analysis maps each statement to the set of possible ways execution proceeds after evaluation of the statement completes. As with type analysis for values and expressions, behavior analysis proceeds bottom up: first determine behaviors for certain basic statements, and then determine behavior for higher level constructs by applying combining rules.

A *behavior* is a set, whose elements may be:

- Return
- Break
- Continue
- Next

Each of those correspond to a way to exit a compound statement: either through a keyword, or by falling to the next statement ("Next").

We note " $s: B$ " to say that s respects the rules regarding behaviors, and has [behavior](#) B .

For each function:

- Its body [must](#) be a valid statement by these rules.
- If the function has a return type, the [behavior](#) of its body [must](#) be {Return}.
- Otherwise, the [behavior](#) of its body [must](#) be a subset of {Next, Return}.

We assign a [behavior](#) to each function: it is its body's [behavior](#) (treating the body as a regular statement), with any "Return" replaced by "Next". As a consequence of the rules above, a function behavior is always one of {}, or {Next}.

Behavior analysis [must](#) be able to determine a non-empty [behavior](#) for each statement, and function.

Rules for analyzing and validating the behaviors of statements

Statement	Preconditions	Resulting behavior
<i>empty statement</i>		{Next}
{ s }	$s: B$	B
$s1\ s2$	$s1: B1$ Next in $B1$ $s2: B2$	$(B1 \setminus \{\text{Next}\}) \cup B2$
NOTE: $s1$ often ends in a semicolon.	$s1: B1$ Next not in $B1$ $s2: B2$	$B1$
var $x:T$;		{Next}
let $x = e$;		{Next}
var $x = e$;		{Next}
$x = e$;		{Next}
$_ = e$;		{Next}
$f(e1, ..., en)$;	f has behavior B	B

Statement	Preconditions	Resulting behavior
return;		{Return}
return <i>e</i> ;		{Return}
discard;		{Next}
break;		{Break}
break if <i>e</i> ;		{Break, Next}
continue;		{Continue}
const_assert <i>e</i> ;		{Next}
if <i>e</i> <i>s1</i> else <i>s2</i>	<i>s1</i> : <i>B1</i> <i>s2</i> : <i>B2</i>	<i>B1</i> ∪ <i>B2</i>
loop { <i>s1</i> continuing { <i>s2</i> }}	<i>s1</i> : <i>B1</i> <i>s2</i> : <i>B2</i> <i>B1</i> = {Return} None of {Continue, Return} are in <i>B2</i>	{Return}
	<i>s1</i> : <i>B1</i> <i>s2</i> : <i>B2</i> <i>B1</i> ≠ {Return} None of {Continue, Return} are in <i>B2</i> Break is not in (<i>B1</i> ∪ <i>B2</i>)	(<i>B1</i> ∪ <i>B2</i>) \ {Continue, Next}
	<i>s1</i> : <i>B1</i> <i>s2</i> : <i>B2</i> <i>B1</i> ≠ {Return} None of {Continue, Return} are in <i>B2</i> Break is in (<i>B1</i> ∪ <i>B2</i>)	(<i>B1</i> ∪ <i>B2</i> ∪ {Next}) \ {Break, Continue}
	<i>s1</i> : <i>B1</i> ... <i>sn</i> : <i>Bn</i> Break is not in (<i>B1</i> ∪ ... ∪ <i>Bn</i>)	<i>B1</i> ∪ ... ∪ <i>Bn</i>
switch <i>e</i> {case <i>c1</i> : <i>s1</i> ... case <i>cn</i> : <i>sn</i> }	<i>s1</i> : <i>B1</i> ... <i>sn</i> : <i>Bn</i> Break is in (<i>B1</i> ∪ ... ∪ <i>Bn</i>)	(<i>B1</i> ∪ ... ∪ <i>Bn</i> ∪ {Next}) \ Break

NOTE: ∪ is a set union operation and \ is a set difference operation.

NOTE: The empty statement case occurs when a loop has an empty body, or when a for loop lacks an initialization or update statement.

For the purpose of this analysis:

- for loops get desugared (see [§ 9.4.4 For Statement](#))

- while loops get desugared (see § 9.4.5 While Statement)
- loop {s} is treated as loop {s continuing {}}
- if statements without an else branch are treated as if they had an empty else branch, i.e. ended in else {}; this adds Next to their [behavior](#)
- if statements with else if branches are treated as if they were nested simple if/else statements
- a [switch_clause](#) starting with default behaves just like a [switch_clause](#) starting with case _:

Each [built-in function](#) has a [behavior](#) of {Next}. And each operator application not listed in the table above has the same [behavior](#) as if it were a function call with the same operands and with a function's [behavior](#) of {Next}.

The behavior of a function [must](#) satisfy the rules given above.

NOTE: The rules above imply that the behavior of a loop is {Next}, {Return}, or {Next,Return}.

NOTE: It is unnecessary to analyze the behavior of expressions because they will always be {Next} or a previously analyzed function will have produced a error.

§ 9.7.2. Notes

This section is informative, non-normative.

Behavior analysis can cause a program to be rejected in the following ways (restating requirements from above):

- The body of a function (treated as a regular statement) has a behavior not included in {Next, Return}.
- The body of a function with a return type has a behavior which is not {Return}.
- The behavior of a continuing block contains any of Continue, or Return.
- Some obviously infinite loops have an empty behavior set, and are therefore invalid.

This analysis can be run in linear time, by analyzing the call-graph bottom-up (since the behavior of a function call can depend on the function's code).

§ 9.7.3. Examples

Here are some examples showing this analysis in action:

EXAMPLE: TRIVIALY DEAD CODE IS ALLOWED

```
fn simple() -> i32 {
  var a: i32;
  return 0; // Behavior: {Return}
  a = 1;    // Valid, statically unreachable code.
            // Statement behavior: {Next}
            // Overall behavior (due to sequential statements): {Return}
  return 2; // Valid, statically unreachable code. Behavior: {Return}
} // Function behavior: {Return}
```

EXAMPLE: COMPOUND STATEMENTS ARE SUPPORTED

```

fn nested() -> i32 {
    var a: i32;
    {
        // The start of a compound statement.
        a = 2;      // Behavior: {Next}
        return 1;   // Behavior: {Return}
    }             // The compound statement as a whole has behavior {Return}
    a = 1;         // Valid, statically unreachable code.
                  // Statement behavior: {Next}
                  // Overall behavior (due to sequential statements): {Return}
    return 2;      // Valid, statically unreachable code. Behavior: {Return}
}

```

EXAMPLE: IF/THEN BEHAVES AS IF THERE IS AN EMPTY ELSE

```

fn if_example() {
    var a: i32 = 0;
    loop {
        if a == 5 {
            break;      // Behavior: {Break}
        }              // Behavior of the whole if compound statement: {Break, Next},
                        // as the if has an implicit empty else
        a = a + 1;      // Valid, as the previous statement had "Next" in its behavior
    }
}

```

EXAMPLE: IF/THEN/ELSE HAS THE BEHAVIOR OF BOTH SIDES

```

fn if_example() {
    var a: i32 = 0;
    loop {
        if a == 5 {
            break;      // Behavior: {Break}
        } else {
            continue;   // Behavior: {Continue}
        }              // Behavior of the whole if compound statement: {Break, Continue}
        a = a + 1;      // Valid, statically unreachable code.
                        // Statement behavior: {Next}
                        // Overall behavior: {Break, Continue}
    }
}

```


EXAMPLE: IF/ELSE IF/ELSE BEHAVES LIKE A NESTED IF/ELSE

```

fn if_example() {
    var a: i32 = 0;
    loop {
        // if e1 s1 else if e2 s2 else s3
        // is identical to
        // if e1 else { if e2 s2 else s3 }
        if a == 5 {
            break; // Behavior: {Break}
        } else if a == 42 {
            continue; // Behavior: {Continue}
        } else {
            return; // Behavior {Return}
        }
        // Behavior of the whole if compound statement:
        // {Break, Continue, Return}
    }
    // Behavior of the whole loop compound statement {Next, Return}
}
// Behavior of the whole function {Next}

```

EXAMPLE: BREAK IN SWITCH BECOMES NEXT

```

fn switch_example() {
    var a: i32 = 0;
    switch a {
        default: {
            break; // Behavior: {Break}
        }
    }
    // Behavior: {Next}, as switch replaces Break by Next
    a = 5; // Valid, as the previous statement had Next in its behavior
}

```

EXAMPLE: OBVIOUSLY INFINITE LOOPS

```

fn invalid_infinite_loop() {
    loop { } // Behavior: { }. Invalid because it's empty.
}

```

EXAMPLE: DISCARD WILL NOT TERMINATE A LOOP

```

fn invalid_infinite_loop() {
    loop {
        discard; // Behavior { Next }.
    }
    // Invalid, behavior of the whole loop is { }.
}

```

EXAMPLE: A CONDITIONAL CONTINUE WITH CONTINUING STATEMENT

```

fn conditional_continue() {
    var a: i32;
    loop {
        if a == 5 { break; } // Behavior: {Break, Next}
        if a % 2 == 1 {      // Valid, as the previous statement has Next in its behavior
            continue;       // Behavior: {Continue}
        }                  // Behavior: {Continue, Next}
        a = a * 2;          // Valid, as the previous statement has Next in its behavior
        continuing {        // Valid as the continuing statement has behavior {Next}
                            // which does not include any of:
                            // {Break, Continue, Return}

            a = a + 1;
        }
    }                      // The loop as a whole has behavior {Next},
                          // as it absorbs "Continue" and "Next",
                          // then replaces "Break" with "Next"
}

```

EXAMPLE: A REDUNDANT CONTINUE WITH CONTINUING STATEMENT

```

fn redundant_continue_with_continuing() {
    var a: i32;
    loop {
        if a == 5 { break; }
        continue; // Valid. This is redundant, branching to the next statement.
        continuing {
            a = a + 1;
        }
    }
}

```

EXAMPLE: A CONTINUE AT THE END OF A LOOP BODY

```

fn continue_end_of_loop_body() {
    for (var i: i32 = 0; i < 5; i++) {
        continue; // Valid. This is redundant,
                  // branching to the end of the loop body.
    }             // Behavior: {Next},
                  // as loops absorb "Continue",
                  // and "for" loops always add "Next"
}

```

for loops desugar to loop with a conditional break. As shown in a previous example, the conditional break has [behavior](#) {Break, Next}, which leads to adding "Next" to the loop's [behavior](#).

EXAMPLE: RETURN REQUIRED IN FUNCTIONS THAT HAVE A RETURN TYPE

```
fn missing_return () -> i32 {
    var a: i32 = 0;
    if a == 42 {
        return a;           // Behavior: {Return}
    }                       // Behavior: {Next, Return}
}                           // Error: Next is invalid in the body of a
                           // function with a return type
```

EXAMPLE: CONTINUE MUST BE IN A LOOP

```
fn continue_out_of_loop () {
    var a: i32 = 0;
    if a > 0 {
        continue;          // Behavior: {Continue}
    }                       // Behavior: {Next, Continue}
}                           // Error: Continue is invalid in the body of a function
```

The same example would also be invalid for the same reason if `continue` was replaced by `break`.

§ 10. Assertions

An **assertion** is a check that ensures a boolean condition is satisfied.

```
global_assert :
    const_assert ';
```

WGSL defines one kind of [assertion](#): the [const assertion](#).

```
const_assert :
    'const_assert' expression
```

[Type rule precondition](#): The expression [must](#) be of [bool](#) type.

§ 10.1. Const Assertion Statement

A const assertion statement is an [assertion](#) that produces a [shader-creation error](#) if the expression evaluates to false. The expression [must](#) be a [const-expression](#). The statement can satisfy [static access](#) conditions in a shader, but otherwise has no effect on the compiled shader. A const assertion can appear at [module scope](#) or as a [function scope statement](#).

```
assert_statement :
    const_assert
```

EXAMPLE: STATIC ASSERTION EXAMPLES

```

const x = 1;
const y = 2;
const_assert x < y; // valid at module-scope.
const_assert(y != 0); // parentheses are optional.

fn foo() {
  const z = x + y - 2;
  const_assert z > 0; // valid in functions.
  let a = 3;
  const_assert a != 0; // invalid, the expression must be a const-expression.
}

```

§ 11. Functions

A **function** performs computational work when invoked.

A function is invoked in one of the following ways:

- By evaluating a function call expression. See [§ 8.10 Function Call Expression](#).
- By executing a function call statement. See [§ 9.5 Function Call Statement](#).
- An [entry point](#) function is invoked by the WebGPU implementation to perform the work of a [shader stage](#) in a [pipeline](#). See [§ 13 Entry Points](#)

Functions in WGSL may be defined in any order, including later in the source than they are used. Because of this, function prototypes or forward declarations are not needed, and there is no way to do so.

There are two kinds of functions:

- A [built-in function](#) is provided by the WGSL implementation, and is always available to a WGSL module. See [§ 17 Built-in Functions](#).
- A [user-defined function](#) is declared in a WGSL module.

§ 11.1. Declaring a User-defined Function

A **function declaration** creates a **user-defined function**, by specifying:

- An optional set of [attributes](#).
- The name of the function.
- The formal parameter list: an ordered sequence of zero or more [formal parameter](#) declarations, which may have attributes applied, separated by commas, and surrounded by parentheses.
- An optional **return type**, which may have attributes applied.
- The **function body**. This is the set of statements to be executed when the function is [called](#).

A function declaration [must](#) only occur at [module scope](#). A function name is [in scope](#) for the entire program.

NOTE: Each [user-defined function](#) only has one [overload](#).

A **formal parameter declaration** specifies an **identifier** name and a type for a value that **must** be provided when invoking the function. A formal parameter may have attributes. See § 11.2 [Function Calls](#). The **scope** of the identifier is the **function body**. Two formal parameters for a given function **must not** have the same name.

NOTE: Some built-in functions may allow parameters to be [abstract numeric types](#); however, this functionality is not currently supported for user-declared functions.

The **return type**, if specified, **must** be **constructible**.

WGSL defines the following attributes that can be applied to function declarations:

- the **shader stage attributes**: [vertex](#), [fragment](#), and [compute](#)
- [workgroup_size](#)
- [subgroup_size](#)

WGSL defines the following attributes that can be applied to function parameters and return types:

- [builtin](#)
- [location](#)
- [blend_src](#)
- [interpolate](#)
- [invariant](#)

function_decl :

*attribute * function_header compound_statement*

function_header :

*'fn' ident '(' param_list ? ')' (':->' attribute * template_elaborated_ident) ?*

param_list :

*param (',' param) * ',' ?*

param :

*attribute * ident ':' type_specifier*

EXAMPLE: SIMPLE FUNCTIONS

```

// Declare the add_two function.
// It has two formal parameters, i and b.
// It has a return type of i32.
// It has a body with a return statement.
fn add_two(i: i32, b: f32) -> i32 {
    return i + 2; // A formal parameter is available for use in the body.
}

// A compute shader entry point function, 'main'.
// It has no specified return type.
// It invokes the add_two function, and captures
// the resulting value in the named value 'six'.
@compute @workgroup_size(1)
fn main() {
    let six: i32 = add_two(4, 5.0);
}

```

§ 11.2. Function Calls

A **function call** is a statement or expression which invokes a function.

The function containing the function call is the **calling function**, or **caller**. The function being invoked is the **called function**, or **callee**.

The function call:

- Names the [called function](#), and
- Provides a parenthesized, comma-separated list of argument value expressions.

The function call [must](#) supply the same number of argument values as there are [formal parameters](#) in the [called function](#). Each argument value [must](#) evaluate to the same type as the corresponding formal parameter, by position.

In summary, when calling a function:

1. Execution of the [calling function](#) is suspended.
2. The [called function](#) executes until it [returns](#).
3. Execution of the [calling function](#) resumes.

A called function **returns** as follows:

- A [built-in function](#) returns when its work has completed.
- A [user-defined function](#) with a [return type](#) returns when it executes a [return](#) statement.
- A [user-defined function](#) with no [return type](#) returns when it executes a [return](#) statement, or when execution reaches the end of its [function body](#).

In detail, when a function call is executed the following steps occur:

1. Function call argument values are evaluated. The relative order of evaluation is left-to-right.
2. Execution of the [calling function](#) is suspended. All [function scope](#) variables and constants maintain their current values.
3. If the called function is [user-defined](#), memory is allocated for each function scope variable in the called function.

- Initialization occurs as described in [§ 7.3 var Declarations](#).
- 4. Values for the formal parameters of the called function are determined by matching the function call argument values by position. For example, the first formal parameter of the called function will have the value of the first argument at the [call site](#).
- 5. Control is transferred to the called function. If the called function is [user-defined](#), execution proceeds starting from the first statement in the [body](#).
- 6. The called function is executed, until it [returns](#).
- 7. Control is transferred back to the calling function, and the called function's execution is unsuspended. If the called function [returns a value](#), that value is supplied for the value of the function call expression.

The location of a function call is referred to as a *call site*, specifically the location of the first [token](#) in the parsed instance of the [call_phrase](#) grammar rule. Call sites are a [dynamic context](#). As such, the same textual location may represent multiple call sites.

NOTE: It is possible that a function call in a [fragment](#) shader never returns if all of the invocations in a [quad](#) are [discarded](#). In such a case, control will not be transferred back to the calling function.

§ 11.3. const Functions

A function declared with a [const](#) attribute can be evaluated at [shader-creation time](#). These functions are called *const-functions*. Calls to these functions can be part of [const-expressions](#).

It is a [shader-creation error](#) if the function contains any expressions that are not [const-expressions](#), or any declarations that are not [const-declarations](#).

NOTE: The [const](#) attribute cannot be applied to user-declared functions.

EXAMPLE: CONST-FUNCTIONS

```
const first_one = firstLeadingBit(1234 + 4567); // Evaluates to 12
                                                // first_one has the type i32, because
                                                // firstLeadingBit cannot operate on
                                                // AbstractInt

@id(1) override x : i32;
override y = firstLeadingBit(x); // const-expressions can be
                                // used in override-expressions.
                                // firstLeadingBit(x) is not a
                                // const-expression in this context.

fn foo() {
    var a : array<i32, firstLeadingBit(257)>; // const-functions can be used in
                                            // const-expressions if all their
                                            // parameters are const-expressions.
}
```

§ 11.4. Restrictions on Functions

- A [vertex](#) shader [must](#) return the [position built-in output value](#).

- An entry point [must](#) never be the target of a [function call](#).
- If a function has a return type, it [must](#) be a [constructible](#) type.
- A [function parameter must](#) be one the following types:
 - a constructible type
 - a pointer type
 - a texture type
 - a sampler type
- Each function call argument [must](#) evaluate to the type of the corresponding function parameter.
 - In particular, an argument that is a pointer [must](#) agree with the formal parameter on [address space](#), [store type](#), and [access mode](#).

NOTE: Recursion is disallowed because cycles are not permitted among any kinds of declarations.

EXAMPLE: VALID AND INVALID POINTER ARGUMENTS

```

fn bar(p : ptr<function, f32>) {
}

fn baz(p : ptr<private, i32>) {
}

fn bar2(p : ptr<function, f32>) {
    let a = &*&(p);

    bar(p); // Valid
    bar(a); // Valid
}

fn baz2(p : ptr<storage, f32>) {
}

struct S {
    x : i32
}

@group(0) @binding(0)
var<storage> ro_storage : f32;
@group(0) @binding(1)
var<storage, read_write> rw_storage : f32;

var usable_priv : i32;
var unusable_priv : array<i32, 4>;
fn foo() {
    var usable_func : f32;
    var unusable_func : S;
    var i32_func : i32;

    let a_priv = &usable_priv;
    let b_priv = a_priv;
    let c_priv = &*&usable_priv;
    let d_priv = &(unusable_priv.x);
    let e_priv = d_priv;

    let a_func = &usable_func;
    let b_func = &unusable_func;
    let c_func = &(*b_func)[0];
    let d_func = c_func;
    let e_func = &a_func;

    baz(&usable_priv); // Valid, address-of a variable.
    baz(a_priv);       // Valid, effectively address-of a variable.
    baz(b_priv);       // Valid, effectively address-of a variable.
    baz(c_priv);       // Valid, effectively address-of a variable.
    baz(d_priv);       // Valid, memory view has changed.
    baz(e_priv);       // Valid, memory view has changed.
    baz(&i32_func);     // Invalid, address space mismatch.

    bar(&usable_func); // Valid, address-of a variable.
    bar(c_func);       // Valid, memory view has changed.
    bar(d_func);       // Valid, memory view has changed.

```

```

    bar(e_func);          // Valid, effectively address-of a variable.

    baz2(&ro_storage);    // Valid, address-of a variable.
    baz2(&rw_storage);    // Invalid, access mode mismatch.
}

```

11.4.1. Alias Analysis

11.4.1.1. Root Identifier

[Memory locations](#) can be accessed during the execution of a function using [memory views](#). Within a function, each [memory view](#) has a particular **root identifier**, which names the variable or formal parameter that first provides access to that memory in that function.

Locally derived expressions of [reference](#) or [pointer](#) type may introduce new names for a particular root identifier, but each expression has a statically determinable root identifier.

Given an expression E of [pointer](#) or [reference type](#), the **root identifier** is the [originating variable](#) or [formal parameter](#) of [pointer type](#) found as follows:

- If E is an identifier [resolving](#) to a variable, then the root identifier is that variable.
- If E is an identifier [resolving](#) to a formal parameter of pointer type, then the root identifier is that formal parameter.
- If E is an identifier [resolving](#) to a [let-declaration](#) with initializer $E2$, then the root identifier is the root identifier of $E2$.
- If E is of the form $(E2)$, $\&E2$, $*E2$, or $E2[Ei]$ then the root identifier is the root identifier of $E2$.
- If E is a [vector access expression](#) of the form $E2.swiz$, where $swiz$ is a [swizzle](#) name, then the root identifier is the root identifier of $E2$.
- If E is a [structure access expression](#) of the form $E2.member_name$, then the root identifier is the root identifier of $E2$.
- If E is a [bufferView](#) or [bufferArrayView](#) built-in call, then the root identifier is the root identifier of the pointer argument.

11.4.1.2. Aliasing

While the [originating variable](#) of a **root identifier** is a dynamic concept that depends on the [call sites](#) for the function, WGSL modules can be statically analyzed to determine the set of all possible [originating variables](#) for each root identifier.

Two **root identifiers** *alias* when they have the same [originating variable](#). Execution of a WGSL function **must not** potentially access memory through aliased root identifiers, where one access is a write and the other is a read or a write. This is determined by analyzing the program from the leaves of the callgraph upwards (i.e. topological order). For each function the analysis records the following sets:

- [Module-scope](#) variables that are [written](#). This includes any module-scope variables that are written in functions called from this function.
- [Module-scope](#) variables that are [read](#). This includes any module-scope variables that are read in functions called from this function.
- Pointer parameters used as root identifiers of memory views that are [written](#) in this function or in called functions.
- Pointer parameters used as root identifiers of memory views that are [read](#) in this function or in called functions.

At each [call site](#) of a function, it is a [shader-creation error](#) if any of the following occur:

- Two arguments of pointer type have the same root identifier and either corresponding parameter is in the written parameter set.
- An argument of pointer type whose root identifier is a module-scope variable where:
 - the corresponding pointer parameter is in the set of written pointer parameters, and
 - the module-scope variable is in the read set for the called function.
- An argument of pointer type whose root identifier is a module-scope variable where:
 - the corresponding pointer parameter is in the set of written pointer parameters, and
 - the module-scope variable is in the written set for the called function.
- An argument of pointer type whose root identifier is a module-scope variable where:
 - the corresponding pointer parameter is in the set of read pointer parameters, and
 - the module-scope variable is in the written set for the called function.

EXAMPLE: ALIAS ANALYSIS

```

var<private> x : i32 = 0;

fn f1(p1 : ptr<function, i32>, p2 : ptr<function, i32>) {
    *p1 = *p2;
}

fn f2(p1 : ptr<function, i32>, p2 : ptr<function, i32>) {
    f1(p1, p2);
}

fn f3() {
    var a : i32 = 0;
    f2(&a, &a); // Invalid. Cannot pass two pointer parameters
                // with the same root identifier when one or
                // more are written (even by a subfunction).
}

fn f4(p1 : ptr<function, i32>, p2 : ptr<function, i32>) -> i32 {
    return *p1 + *p2;
}

fn f5() {
    var a : i32 = 0;
    let b = f4(&a, &a); // Valid. p1 and p2 in f4 are both only read.
}

fn f6(p : ptr<private, i32>) {
    x = *p;
}

fn f7(p : ptr<private, i32>) -> i32 {
    return x + *p;
}

fn f8() {
    let a = f6(&x); // Invalid. x is written as a global variable and
                  // read as a parameter.
    let b = f7(&x); // Valid. x is only read as both a parameter and
                  // a variable.
}

```

§ 12. Attributes

An *attribute* modifies an object. WGSL provides a unified syntax for applying attributes. Attributes are used for a variety of purposes such as specifying the interface with the API.

Generally speaking, from the language's point-of-view, attributes can be ignored for the purposes of type and semantic checking. Additionally, the attribute name is a [context-dependent name](#), and some attribute parameters are also context-dependent names.

attribute :

'@' *ident* *pattern_token* *argument_expression_list* ?

```

| align_attr
| binding_attr
| blend_src_attr
| builtin_attr
| const_attr
| diagnostic_attr
| group_attr
| id_attr
| interpolate_attr
| invariant_attr
| location_attr
| must_use_attr
| size_attr
| workgroup_size_attr
| subgroup_size_attr
| vertex_attr
| fragment_attr
| compute_attr

```

Unless explicitly permitted in an attribute's description, an attribute [must not](#) be specified more than once per object or type.

§ 12.1. align

```

align_attr :
  '@' 'align' '(' expression ',' ? ')'

```

align Attribute

Description Constrains the placement of a structure member in memory.

[Must](#) only be applied to a member of a [structure](#) type.

This attribute influences how a value of the enclosing structure type can appear in memory: it constrains the byte addresses at which the structure itself and its component members can appear.

If `align(n)` is applied to a member of *S* with type *T*, and *S* can be the [store type](#) for a variable in address space *AS*, where *AS* is not [uniform](#), then *n* [must](#) satisfy:

$$n = k \times \text{RequiredAlignOf}(T, AS) \text{ for some positive integer } k.$$

The rules for alignment and size are mutually recursive. However, the above constraint is well defined because it depends on the required alignment of a *nested* type, and types have bounded [nesting depth](#).

See [§ 14.4 Memory Layout](#).

Parameters [Must](#) be a [const-expression](#) that [resolves](#) to an [i32](#) or [u32](#).
[Must](#) be positive.
[Must](#) be a power of 2.

§ 12.2. binding

```
binding_attr :
    '@' 'binding' '(' expression ',' ? ')'
```

binding Attribute

Description Specifies the binding number of the resource in a bind [group](#). See [§ 13.3.2 Resource Interface](#).
[Must](#) only be applied to a [resource](#) variable.

Parameters [Must](#) be a [const-expression](#) that [resolves](#) to an [i32](#) or [u32](#).
[Must](#) be non-negative.

§ 12.3. blend_src

```
blend_src_attr :
    '@' 'blend_src' '(' expression ',' ? ')'
```

blend_src Attribute

Description Specifies a part of the [fragment](#) output when the feature [dual_source_blending](#) is enabled. See [§ 13.3.1.3 Input-output Locations](#).
[Must](#) only be applied to a member of a [structure](#) type with a [location](#) attribute. [Must](#) only be applied to declarations of objects with [numeric scalar](#) or [numeric vector](#) type. [Must not](#) be included in a [shader stage input](#). [Must not](#) be included in a [shader stage output](#), except for the [fragment](#) shader stage.

Parameters [Must](#) be a [const-expression](#) that [resolves](#) to an [i32](#) or [u32](#) with value of 0 or 1.

§ 12.4. builtin

```
builtin_attr :
    '@' 'builtin' '(' builtin_value_name ',' ? ')'
```

builtin Attribute

Description Specifies that the associated object is a built-in value, as denoted by the specified [token](#). See [§ 13.3.1.1 Built-in Inputs and Outputs](#).
[Must](#) only be applied to an entry point function parameter, entry point return type, or member of a [structure](#).

Parameters [Must](#) be a [built-in value name-token](#) for a [built-in value](#).

§ 12.5. `const`

```
const_attr :  
  '@' 'const'
```

const Attribute

Description Specifies that the function can be used as a [const-function](#). This attribute [must not](#) be applied to a user-defined function.

[Must](#) only be applied to function declarations.

NOTE: This attribute is used as a notational convention to describe which built-in functions can be used in [const-expressions](#).

Parameters *None*

§ 12.6. `diagnostic`

```
diagnostic_attr :  
  '@' 'diagnostic' diagnostic_control
```

```
diagnostic_control :  
  '(' severity_control_name ',' diagnostic_rule_name ',' ? ')''
```

diagnostic Attribute

Description Specifies a [range diagnostic filter](#). See [§ 2.3 Diagnostics](#).

More than one [diagnostic](#) attribute may be specified on a syntactic form, but they [must](#) specify different [triggering rules](#).

Parameters The first parameter is a [severity_control_name](#).

The second parameter is a [diagnostic_rule_name](#) token specifying a [triggering rule](#).

§ 12.7. `group`

```
group_attr :  
  '@' 'group' '(' expression ',' ? ')''
```

group Attribute

Description Specifies the binding group of the resource. See [§ 13.3.2 Resource Interface](#).

[Must](#) only be applied to a [resource](#) variable.

Parameters [Must](#) be a [const-expression](#) that [resolves](#) to an [i32](#) or [u32](#).
[Must](#) be non-negative.

§ 12.8. id

```
id_attr :
    '@' 'id' '(' expression ',' ? ')'
```

id Attribute

Description Specifies a numeric identifier as an alternate name for a [pipeline-overrideable](#) constant.

[Must](#) only be applied to an [override-declaration](#) of [scalar](#) type.

Parameters [Must](#) be a [const-expression](#) that [resolves](#) to an [i32](#) or [u32](#).
[Must](#) be non-negative.

§ 12.9. interpolate

```
interpolate_attr :
    '@' 'interpolate' '(' interpolate_type_name ',' ? ')'
```

```
| '@' 'interpolate' '(' interpolate_type_name ',' interpolate_sampling_name ',' ? ')'
```

```
interpolate_type_name :
    ident_pattern_token
```

interpolate Attribute

Description Specifies how the user-defined IO [must](#) be interpolated. See § 13.3.1.4 Interpolation.

[Must](#) only be applied to a declaration that has a [location](#) attribute applied.

When present, the attribute is only significant on user-defined [vertex](#) outputs and [fragment](#) inputs.

Parameters The first parameter [must](#) be an [interpolation type name-token](#) for an [interpolation type](#).

The second parameter, if present, [must](#) be an [interpolation sampling name-token](#) for the [interpolation sampling](#).

§ 12.10. invariant

```
invariant_attr :
```


@ 'invariant'

invariant Attribute

Description When applied to the [position built-in output value](#) of a vertex shader, the computation of the result is invariant across different programs and different invocations of the same entry point. That is, if the data and control flow match for two position outputs in different entry points, then the result values are guaranteed to be the same. There is no effect on a position [built-in input value](#).

[Must](#) only be applied to the [position](#) built-in value.

NOTE: This attribute maps to the precise qualifier in HLSL, and the invariant qualifier in GLSL.

Parameters *None*

§ 12.11. location

location_attr :

@ 'location' '(' *expression* ',' ? ')'

Location Attribute

Description Specifies a part of the user-defined IO of an entry point. See [§ 13.3.1.3 Input-output Locations](#).

[Must](#) only be applied to an entry point function parameter, entry point return type, or member of a [structure](#) type. [Must](#) only be applied to declarations of objects with [numeric scalar](#) or [numeric vector](#) type. [Must not](#) be included in [compute shader stage inputs](#).

Parameters [Must](#) be a [const-expression](#) that [resolves](#) to an [i32](#) or [u32](#).
[Must](#) be non-negative.

§ 12.12. must_use

must_use_attr :

@ 'must_use'

must_use Attribute

Description Specifies that a [call](#) to this function [must](#) be used as an [expression](#). That is, a call to this function [must](#) not be the entirety of a [function call statement](#).

[Must](#) only be applied to the declaration of a [function](#) with a [return type](#).

NOTE: Many functions return a value and do not have side effects. It is often a programming defect to call such a function as the only thing in a function call statement. Built-in functions with these properties are declared as `@must_use`. User-defined functions can also have the `@must_use` attribute.

NOTE: To deliberately work around the `@must_use` rule, use a [phony assignment](#) or [declare a value](#) using the function call as the initializer.

Parameters *None*

§ 12.13. `size`

```
size_attr :
  '@' 'size' '(' expression ',' ? ')''
```

size Attribute

Description Specifies the number of bytes reserved for a structure member.

This number [must](#) be at least the [byte-size](#) of the type of the member:

If `size(n)` is applied to a member with type *T*, then `SizeOf(T) ≤ n`.

See § 14.4 Memory Layout.

[Must](#) only be applied to a member of a [structure](#) type. The member type [must](#) have [creation-fixed footprint](#).

Parameters [Must](#) be a [const-expression](#) that [resolves](#) to an [i32](#) or [u32](#).
[Must](#) be positive.

§ 12.14. `subgroup_size`

```
subgroup_size_attr :
  '@' 'subgroup_size' '(' expression ',' ? ')''
```

subgroup_size Attribute

Description Specifies the subgroup size for a compute shader invocation.

[Must](#) only be applied to a [compute shader](#) entry point function.

Requires [Must](#) only be used when the [subgroup_size_control](#) extension is enabled.

Parameters [must](#) be a [const-expression](#) or an [override-expression](#) that [resolves](#) to an [i32](#) or [u32](#).

If the value is not a power of two, then:

- It is a [shader-creation error](#) if the expression is a [const-expression](#).
- It is a [pipeline-creation error](#) if the expression is an [override-expression](#).

A [pipeline-creation error](#) results if the x-dimension of the entry point's `workgroup_size` is not a multiple of the `subgroup_size` value.

A [pipeline-creation error](#) results if the value is greater than [subgroupMaxSize](#) or smaller than [subgroupMinSize](#).

Pipeline creation may result in an [uncategorized error](#) due to register pressure or specific hardware limitations even when the value is between [subgroupMinSize](#) and [subgroupMaxSize](#).

§ 12.15. `workgroup_size`

```
workgroup_size_attr :
  '@' 'workgroup_size' '(' expression ',' ? ')'
| '@' 'workgroup_size' '(' expression ',' expression ',' ? ')'
| '@' 'workgroup_size' '(' expression ',' expression ',' expression ',' ? ')'
```

workgroup_size Attribute

Description Specifies the x, y, and z dimensions of the [workgroup grid](#) for the compute shader.

The first parameter specifies the x dimension. The second parameter, if provided, specifies the y dimension, otherwise is assumed to be 1. The third parameter, if provided, specifies the z dimension, otherwise is assumed to be 1.

[Must](#) only be applied to a [compute shader](#) entry point function. [Must not](#) be applied to any other object.

Parameters Takes one, two, or three parameters.

Each parameter [must](#) be a [const-expression](#) or an [override-expression](#). All parameters [must](#) be the same type, either [i32](#) or [u32](#).

A [shader-creation error](#) results if any specified parameter is a [const-expression](#) that evaluates to a non-positive value.

A [pipeline-creation error](#) results if any specified parameter evaluates to a non-positive value or exceeds an upper bound specified by the WebGPU API, or if the product of the parameter values exceeds the upper bound specified by the WebGPU API (see [WebGPU § 3.6.2 Limits](#)).

§ 12.16. Shader Stage Attributes

The *shader stage attributes* below designate a function as an [entry point](#) for a particular [shader stage](#). These attributes [must](#) only be applied to [function declarations](#), and at most one may be present on a given function. They take no parameters.

§ 12.16.1. vertex

```
vertex_attr :
    '@' 'vertex'
```

The **vertex** attribute declares the function to be an [entry point](#) for the [vertex shader stage](#) of a [render pipeline](#).

§ 12.16.2. fragment

```
fragment_attr :
    '@' 'fragment'
```

The **fragment** attribute declares the function to be an [entry point](#) for the [fragment shader stage](#) of a [render pipeline](#).

§ 12.16.3. compute

```
compute_attr :
    '@' 'compute'
```

The **compute** attribute declares the function to be an [entry point](#) for the [compute shader stage](#) of a [compute pipeline](#).

§ 13. Entry Points

An **entry point** is a [user-defined function](#) that performs the work for a particular [shader stage](#).

§ 13.1. Shader Stages

WebGPU issues work to the GPU in the form of [draw](#) or [dispatch commands](#). These commands execute a pipeline in the context of a set of shader stage [inputs](#), [outputs](#), and attached [resources](#).

A **pipeline** describes the work to be performed on the GPU, as a sequence of stages, some of which are programmable. In WebGPU, a pipeline is created before scheduling a draw or dispatch command for execution. There are two kinds of pipelines: GPUComputePipeline, and GPURenderPipeline.

A [dispatch command](#) uses a **GPUComputePipeline** to run a **compute shader stage** over a logical grid of points with a controllable amount of parallelism, while reading and possibly updating buffer and image resources.

A [draw command](#) uses a **GPURenderPipeline** to run a multi-stage process with two programmable stages among other fixed-function stages:

- A **vertex shader stage** maps input attributes for a single vertex into output attributes for the vertex.
- Fixed-function stages map vertices into graphic primitives (such as triangles) which are then rasterized to produce **rasterization fragments**. Each rasterization fragment represents the overlap between the primitive being drawn, and a particular pixel in the [framebuffer](#).
- A **fragment shader stage** processes each fragment, possibly producing a fragment output.
 - Typically, one fragment shader invocation is created for each [rasterization fragment](#). More than one fragment invocation may be created if the rasterization fragment covers multiple samples. See [WebGPU § 23.2.10 Per-Sample Shading](#).

- If the rasterization fragment is near the boundary of the primitive, then additional [helper invocations](#) may also be created. See [§ 15.4 Fragment Shaders and Helper Invocations](#).
- Fixed-function stages consume a fragment output, possibly updating external state such as color attachments and depth and stencil buffers.

The WebGPU specification describes pipelines in greater detail.

WGSL defines three *shader stages*, corresponding to the programmable parts of pipelines:

- *compute*
- *vertex*
- *fragment*

Each shader stage has its own set of features and constraints, described elsewhere.

§ 13.2. Entry Point Declaration

To create an [entry point](#), declare a [user-defined function](#) with a [shader stage attribute](#).

When configuring a [pipeline](#) in the WebGPU API, the entry point's function name maps to the `entryPoint` attribute of the WebGPU `GPUProgrammableStage` object.

The entry point's [formal parameters](#) denote the stage's [shader stage inputs](#). A structure type can be used to group [user-defined inputs](#) with each other and optionally with [built-in inputs](#). Each parameter [must](#) either be a shader stage input, or it [must](#) be declared with a structure type where each structure member is a shader stage input.

If specified, the entry point's [return type](#) and its attributes denotes the stage's [shader stage outputs](#). A structure type can be used to group [user-defined outputs](#) with each other and optionally with [built-in outputs](#). If present, the return type and its attributes [must](#) be a shader stage output, or it [must](#) be a structure type where each structure member is a shader stage output.

NOTE: [Compute](#) entry points never have a return type.

EXAMPLE: ENTRY POINT

```
@vertex
fn vert_main() -> @builtin(position) vec4<f32> {
    return vec4<f32>(0.0, 0.0, 0.0, 1.0);
}

@fragment
fn frag_main(@builtin(position) coord_in: vec4<f32>) -> @location(0) vec4<f32> {
    return vec4<f32>(coord_in.x, coord_in.y, 0.0, 1.0);
}

@compute @workgroup_size(1)
fn comp_main() { }
```

The set of *functions in a shader stage* is the union of:

- The entry point function for the stage.
- The targets of function calls from within the body of a function in the shader stage, whether or not that call is executed.

The union is applied repeatedly until it stabilizes. It will stabilize in a finite number of steps.

§ 13.2.1. Function Attributes for Entry Points

WGSL defines the following attributes that can be applied to entry point declarations:

- the [shader stage attributes](#): [vertex](#), [fragment](#), and [compute](#)
- [workgroup_size](#)
- [subgroup_size](#)

EXAMPLE: WORKGROUP_SIZE ATTRIBUTE

```
@compute @workgroup_size(8,4,1)
fn sorter() { }

@compute @workgroup_size(8u)
fn reverser() { }

// Using an pipeline-overridable constant.
@id(42) override block_width = 12u;
@compute @workgroup_size(block_width)
fn shuffler() { }

// Error: workgroup_size must be specified on compute shader
@compute
fn bad_shader() { }
```

§ 13.3. Shader Interface

The shader interface is the set of objects through which the shader accesses data external to the [shader stage](#), either for reading or writing, and the [pipeline-overridable](#) constants used to configure the shader. The interface includes:

- [Shader stage inputs](#)
- [Shader stage outputs](#)
- [Override-declarations](#)
- [Immediate data](#)
- Attached [resources](#), which include:
 - [Uniform buffers](#)
 - [Storage buffers](#)
 - [Texture resources](#)
 - [Sampler resources](#)

A declaration *D* is **statically accessed** by a shader when:

- An identifier [resolving](#) to *D* appears in the [declaration](#) of any of the [functions in the shader stage](#).
- An identifier [resolving](#) to *D* is used to define a type for a [statically accessed](#) declaration.
- An identifier [resolving](#) to *D* is used in the initializer for a [statically accessed](#) declaration.
- An identifier [resolving](#) to *D* is used by an attribute used by a [statically accessed](#) declaration.

NOTE: Static access is recursively defined, taking into account the following:

- All the parts of a [function declaration](#) including attributes, formal parameters, return type, and function body.
- Any type needed to define the above, including following [type aliases](#).
- As a particular case of helping to define a type, any [override-declaration](#) used in an [override-expression](#) that is the [element count](#) of an [array](#) type for a variable in the [workgroup](#) address space, when that variable itself is statically accessed.
- Any override declarations used to support the evaluation of override-expressions in any of the above.
- Any attributes on any of the above.

We can now precisely define the *interface of a shader* as consisting of:

- The [formal parameters](#) of the [entry point](#). These denote the shader stage inputs.
- The [return value](#) of the entry point. This denotes the shader stage outputs.
- The [uniform buffer](#), [storage buffer](#), [texture resource](#), and [sampler resource](#) variables [statically accessed](#) by the shader.
- The [override-declarations](#) [statically accessed](#) by the shader.
- The [immediate data](#) variables [statically accessed](#) by the shader.

§ 13.3.1. Inter-stage Input and Output Interface

A *shader stage input* is a datum provided to the shader stage from upstream in the pipeline. Each datum is either a [built-in input value](#), or a [user-defined input](#).

A *shader stage output* is a datum the shader provides for further processing downstream in the pipeline. Each datum is either a [built-in output value](#), or a [user-defined output](#).

[IO attributes](#) are used to establish an object as a [shader stage input](#) or a [shader stage output](#), or to further describe the properties of an input or output. The *IO attributes* are:

- [builtin](#)
- [location](#)
- [blend_src](#)
- [interpolate](#)
- [invariant](#)

§ 13.3.1.1. Built-in Inputs and Outputs

A *built-in input value* provides access to system-generated control information. An entry point [must](#) not have two built-in inputs with the same [built-in value name](#).

A built-in input for stage S with name X and type T_X is accessed via a [formal parameter](#) to an [entry point](#) for [shader stage](#) S , in one of two ways:

1. The parameter has attribute `builtin( $X$ )` and is of type T_X .
2. The parameter has structure type, where one of the structure members has attribute `builtin( $X$ )` and is of type T_X .

Conversely, when a parameter or member of a parameter for an entry point has a [builtin](#) attribute, the corresponding builtin [must](#) be an input for the entry point's shader stage.

A **built-in output value** is used by the shader to convey control information to later processing steps in the pipeline. An entry point [must](#) not have two built-in outputs with the same [builtin value name](#).

A built-in output for stage S with name Y and type T_Y is set via the [return value](#) for an [entry point](#) for [shader stage](#) S , in one of two ways:

1. The entry point [return type](#) has attribute `builtin( $Y$ )` and is of type T_Y .
2. The entry point [return type](#) has structure type, where one of the structure members has attribute `builtin( $Y$ )` and is of type T_Y .

Conversely, when the return type or member of a return type for an entry point has a [builtin](#) attribute, the corresponding builtin [must](#) be an output for the entry point's shader stage.

NOTE: The [position](#) builtin is both an output of a vertex shader, and an input to the fragment shader.

Collectively, built-in input and built-in output values are known as **built-in values**.

The following table summarizes the available built-in values. Each is a [builtin value name token](#) for a [builtin value](#). Each is described in detail in subsequent sections.

Built-in input and output values

Name	Stage	Direction	Type	Extension
vertex_index	vertex	input	u32	
instance_index	vertex	input	u32	
clip_distances	vertex	output	array<f32, N> ($N \leq 8$)	clip_distances
position	vertex	output	vec4<f32>	
	fragment	input	vec4<f32>	
front_facing	fragment	input	bool	
frag_depth	fragment	output	f32	
primitive_index	fragment	input	u32	primitive_index
sample_index	fragment	input	u32	
sample_mask	fragment	input	u32	
	fragment	output	u32	
local_invocation_id	compute	input	vec3<u32>	
local_invocation_index	compute	input	u32	
global_invocation_id	compute	input	vec3<u32>	
global_invocation_index	compute	input	u32	linear_indexing

Name	Stage	Direction	Type	Extension
workgroup_id	compute	input	vec3<u32>	
workgroup_index	compute	input	u32	linear_indexing
num_workgroups	compute	input	vec3<u32>	
subgroup_invocation_id	compute fragment	input	u32	subgroups
subgroup_size	compute fragment	input	u32	subgroups
subgroup_id	compute	input	u32	subgroups and subgroup_id
num_subgroups	compute	input	u32	subgroups and subgroup_id

EXAMPLE: DECLARING BUILT-IN VALUES

```

struct VertexOutput {
    @builtin(position) my_pos: vec4<f32>,
    @builtin(clip_distances) my_clip_distances: array<f32, 8>,
}

@vertex
fn vs_main(
    @builtin(vertex_index) my_index: u32,
    @builtin(instance_index) my_inst_index: u32,
) -> VertexOutput {}

struct FragmentOutput {
    @builtin(frag_depth) depth: f32,
    @builtin(sample_mask) mask_out: u32
}

@fragment
fn fs_main(
    @builtin(front_facing) is_front: bool,
    @builtin(position) coord: vec4<f32>,
    @builtin(sample_index) my_sample_index: u32,
    @builtin(sample_mask) mask_in: u32,
) -> FragmentOutput {}

@compute @workgroup_size(64)
fn cs_main(
    @builtin(local_invocation_id) local_id: vec3<u32>,
    @builtin(local_invocation_index) local_index: u32,
    @builtin(global_invocation_id) global_id: vec3<u32>,
) {}

```

§ 13.3.1.1.1. clip_distances

Name	<i>clip_distances</i>
Stage	vertex
Type	array<f32, N>
Direction	Output
Description	Each value in the array represents a distance to a user-defined clip plane. A clip distance of 0 means the vertex is on the plane, a positive distance means the vertex is inside the clip half-space, and a negative distance means the vertex is outside the clip half-space. The array size of clip_distances must be ≤ 8 . See WebGPU § 23.2.4 Primitive Clipping .

§ 13.3.1.1.2. frag_depth

Name	<i>frag_depth</i>
Stage	fragment
Type	f32
Direction	Output
Description	Updated depth of the fragment, in the viewport depth range. See WebGPU § 3.3 Coordinate Systems .

§ 13.3.1.1.3. front_facing

Name	<i>front_facing</i>
Stage	fragment
Type	bool
Direction	Input
Description	True when the current fragment is on a front-facing primitive. False otherwise.

§ 13.3.1.1.4. `global_invocation_id`

Name	<i><code>global_invocation_id</code></i>
Stage	compute
Type	<code>vec3<u32></code>
Direction	Input
Description	The current invocation's global invocation ID , i.e. its position in the compute shader grid . The value of <code>global_invocation_id</code> is equal to <code>workgroup_id * workgroup_size + local_invocation_id</code> .

§ 13.3.1.1.5. `global_invocation_index`

Name	<i><code>global_invocation_index</code></i>
Stage	compute
Type	<code>u32;</code>
Direction	Input

The current invocation's [global invocation index](#), i.e. its linear position in the [compute shader grid](#). The value of `global_invocation_index` is equal to:

$$\begin{aligned} &\text{global_invocation_id.x} + \\ &(\text{global_invocation_id.y} * \text{workgroup_size.x} * \text{num_workgroups.x}) + \\ &(\text{global_invocation_id.z} * \text{workgroup_size.x} * \text{num_workgroups.x} * \text{workgroup_size.y} * \\ &\text{num_workgroups.y}) \end{aligned}$$

Description

NOTE: If the number of dispatched workgroups would cause this value to exceed the range of the `u32` type, then the dispatch will fail:

- For `dispatchWorkgroups`, the [GPUComputePassEncoder](#) will be invalidated.
- For `dispatchWorkgroupsIndirect`, the dispatch will not execute.

§ 13.3.1.1.6. `instance_index`

Name	<i><code>instance_index</code></i>
Stage	vertex
Type	<code>u32</code>

Direction	Input
Description	Instance index of the current vertex within the current API-level draw command. The first instance has an index equal to the <code>firstInstance</code> argument of the draw, whether provided directly or indirectly. The index is incremented by one for each additional instance in the draw.

13.3.1.1.7. `local_invocation_id`

Name	<i><code>local_invocation_id</code></i>
Stage	compute
Type	<code>vec3<u32></code>
Direction	Input
Description	The current invocation's local invocation ID , i.e. its position in the workgroup grid .

13.3.1.1.8. `local_invocation_index`

Name	<i><code>local_invocation_index</code></i>
Stage	compute
Type	<code>u32</code>
Direction	Input
Description	The current invocation's local invocation index , a linearized index of the invocation's position within the workgroup grid .

13.3.1.1.9. `num_workgroups`

Name	<i><code>num_workgroups</code></i>
Stage	compute
Type	<code>vec3<u32></code>
Direction	Input
Description	The dispatch size , <code>vec3<u32>(group_count_x, group_count_y, group_count_z)</code> , of the compute shader dispatched by the API.

§ 13.3.1.1.10. position

Name	<i>position</i>
Stage	vertex
Type	vec4<f32>
Direction	Output
Description	The clip position of the current vertex, in clip space coordinates. An output value (x,y,z,w) will map to $(x/w, y/w, z/w)$ in WebGPU normalized device coordinates. See WebGPU § 3.3 Coordinate Systems and WebGPU § 23.2.4 Primitive Clipping. A dynamic error occurs if the w coordinate is zero.

Name	position
Stage	fragment
Type	vec4<f32>
Direction	Input
Description	Input position of the current fragment rasterization point. Let rp be the RasterizationPoint for the fragment. Let vp be the [[viewport]] in effect for the draw command. Then fp is the computed input position of the fragment rasterization point. Then schematically: $fp.xy = rp.destination.position$$fp.z = rp.depth$$fp.w = rp.perspectiveDivisor$

In more detail:

- $fp.x$ and $fp.y$ are the interpolated x and y coordinates of the position the current fragment rasterization point in the [framebuffer](#).

The framebuffer is a two-dimensional grid of pixels with the top-left at $(0.0,0.0)$ and the bottom right at $(vp.width, vp.height)$. Each pixel has an extent of 1.0 unit in each of the x and y dimensions, and pixel centers are at $(0.5,0.5)$ offset from integer coordinates.
- $fp.z$ is the interpolated depth of the current fragment rasterization point. For example:
 - depth 0 in [normalized device coordinates](#) maps to $fp.z = vp.minDepth$,
 - depth 1 in normalized device coordinates maps to $fp.z = vp.maxDepth$.
- $fp.w$ is the perspective divisor for the fragment rasterization point, which is the interpolation of $1.0 \div vertex_w$, where $vertex_w$ is the w component of the [position](#) output of the vertex shader.

See [WebGPU § 3.3 Coordinate Systems](#) and [WebGPU § 23.2.5 Rasterization](#).

13.3.1.1.11. `primitive_index`

Name	<i><code>primitive_index</code></i>
Stage	fragment
Type	u32
Direction	Input
Description	Per-primitive index based on the number of primitives processed for the current instance since the beginning of the current draw operation. Begins at 0 and increments by 1 after every point, line, or triangle primitive is processed. Resets to 0 between each instance drawn. Restarting a strip primitive with the primitive restart value has no effect on the primitive index. The index is uniform across all fragments of the primitive.

13.3.1.1.12. `sample_index`

Name	<i><code>sample_index</code></i>
Stage	fragment
Type	u32
Direction	Input
Description	Sample index for the current fragment rasterization point. The value is least 0 and at most <code>sampleCount-1</code>, where <code>sampleCount</code> is the sample count specified for the GPU render pipeline. When this attribute is applied, if the effects of the fragment shader would vary based on the value of sample_index, the fragment shader will be invoked once per sample. See WebGPU § 10.3 GPURenderPipeline.

13.3.1.1.13. `sample_mask`

Name	<i><code>sample_mask</code></i>
Stage	fragment
Type	u32
Direction	Input

Description	Sample coverage bitmask for the current fragment.
	Bits are indexed by a sample index in the half-open interval $[0, \text{sampleCount})$, where <code>sampleCount</code> is the sample count specified for the GPU render pipeline.
	A bit will be set to 1 <i>only if</i> the sample is covered by the primitive being rendered. Bits with index <code>sampleCount</code> and above are always set to zero.
	There are two possible values for the bitmask:
	• The bitmask has a 1 bit set for every sample covered by the fragment. In this case the bitmask equals the fragment’s RasterizationPoint coverageMask. • The bitmask has only 1 bit set, corresponding to the sample being processed by the current fragment shader invocation. That is, the bitmask is $(1 \ll \text{sample_index})$, as if using the <code>sample_index</code> built-in input.
	NOTE: These two cases are the same when <code>sampleCount = 1</code>.
	NOTE: This is a known portability hazard when <code>sampleCount > 1</code>. Some devices yield the full coverage mask. Other devices yield the single-bit mask.
	See WebGPU § 23.2.10 Per-Sample Shading and WebGPU § 23.2.11 Sample Masking .

Name	<code>sample_mask</code>
Stage	fragment
Type	<code>u32</code>
Direction	Output
Description	Sample coverage mask control for the current fragment. The last value written to this variable becomes the shader-output mask. Zero bits in the written value will cause corresponding samples in the color attachments to be discarded. See WebGPU § 23.2.11 Sample Masking.

§ 13.3.1.1.14. `vertex_index`

Name	<i><code>vertex_index</code></i>
Stage	vertex
Type	<code>u32</code>
Direction	Input
Description	Index of the current vertex within the current API-level draw command, independent of draw instancing.

For a non-indexed draw, the first vertex has an index equal to the `firstVertex` argument of the draw, whether provided directly or indirectly. The index is incremented by one for each additional vertex in the draw instance.

For an indexed draw, the index is equal to the index buffer entry for the vertex, plus the `baseVertex` argument of the draw, whether provided directly or indirectly.

13.3.1.1.15. `workgroup_id`

Name	<i><code>workgroup_id</code></i>
Stage	compute
Type	<code>vec3<u32></code>
Direction	Input
Description	The current invocation's workgroup ID, i.e. the position of the workgroup in overall compute shader grid. All invocations in the same workgroup have the same workgroup ID. Workgroup IDs span from (0,0,0) to (group_count_x - 1, group_count_y - 1, group_count_z - 1).

13.3.1.1.16. `workgroup_index`

Name	<i><code>workgroup_index</code></i>
Stage	compute
Type	<code>u32</code>
Direction	Input
Description	The current invocation's workgroup index, i.e. the linear position of the workgroup in overall compute shader grid. All invocations in the same workgroup have the same workgroup index. NOTE: If the number of dispatched workgroups would cause this value to exceed the range of the u32 type, then the dispatch will fail: - For <code>dispatchWorkgroups</code>, the GPUComputePassEncoder will be invalidated. - For <code>dispatchWorkgroupsIndirect</code>, the dispatch will not execute.

13.3.1.1.17. subgroup_invocation_id

Name	<i>subgroup_invocation_id</i>
Stage	compute or fragment
Type	u32
Direction	Input
The current invocation's subgroup invocation ID. The ID is within the range [0, subgroup_size - 1]. In a compute shader, IDs begin at zero and are dense. That is, when the compute shader begins execution, then within each subgroup:	
Description	• One invocation will have subgroup_invocation_id = 0, and • If the subgroup has an invocation with subgroup_invocation_id = <i>k</i>, then the subgroup has an invocation with subgroup_invocation_id = <i>j</i>, for all $0 \leq j < k$.
NOTE: Subgroup invocation indexing in fragment shaders may not be dense. The implementation may assign some lower-numbered IDs to helper invocations.	

13.3.1.1.18. subgroup_size

Name	<i>subgroup_size</i>
Stage	compute or fragment
Type	u32
Direction	Input
Description	The subgroup size of current invocation's subgroup. For compute shaders with a subgroup_size attribute, this value is equal to the specified attribute value.

13.3.1.1.19. subgroup_id

Name	<i>subgroup_id</i>
Stage	compute
Type	u32

Direction	Input
Description	The subgroup ID of current invocation's subgroup within the workgroup . The ID is in the range [0, num_subgroups - 1].

§ 13.3.1.1.20. num_subgroups

Name	<i>num_subgroups</i>
Stage	compute
Type	u32
Direction	Input
Description	The number of subgroups in the current invocation's workgroup .

§ 13.3.1.2. User-defined Inputs and Outputs

User-defined data can be passed as input to the start of a pipeline, passed between stages of a pipeline or output from the end of a pipeline.

Each *user-defined input datum* and *user-defined output datum* [must](#):

- be of [numeric scalar](#) type or [numeric vector](#) type.
- be assigned an IO location. See § 13.3.1.3 [Input-output Locations](#).

A [compute](#) shader [must not](#) have user-defined inputs or outputs.

§ 13.3.1.3. Input-output Locations

Each input-output location can store a value up to 16 bytes in size. The byte size of a type is defined using the *SizeOf* column in § 14.4.1 [Alignment and Size](#). For example, a four-component vector of floating-point values occupies a single location.

IO locations are specified via the [location](#) attribute.

Each user-defined [input](#) and [output must](#) have an explicitly specified IO location. Each structure member in the entry point IO [must](#) be one of either a built-in value (see § 13.3.1.1 [Built-in Inputs and Outputs](#)), or assigned a location.

For each entry point defined in a WGSL module, let *inputs* be its set of shader stage inputs (i.e. locations for its formal parameters, or for the members of its formal parameters of structure type).

- *inputs* [must not](#) contain two entries with the same [location](#) value.

For each structure type *S* defined in a WGSL module (not just those used in shader stage inputs or outputs), let *members* be the set of members of *S* that have [location](#) attributes.

- If any entry in *members* specifies a [blend_src](#) attribute, *members* must *use dual source blending*, which means:

- *members* must contain exactly 2 entries, one with `@location(0) @blend_src(0)` and one with `@location(0) @blend_src(1)`.
- All the *members* must have same data type.
- Otherwise, *members* must not contain two entries with the same location value.

NOTE: Location numbering is distinct between inputs and outputs: Location numbers for an entry point's shader stage inputs do not conflict with location numbers for the entry point's shader stage outputs.

NOTE: No additional rule is required to prevent location overlap within an entry point's outputs. When the output is a structure, the first rule above prevents overlap. Otherwise, the output is a scalar or a vector, and can have only a single location assigned to it.

NOTE: The number of available locations for an entry point is defined by the WebGPU API.

EXAMPLE: APPLYING LOCATION ATTRIBUTES

```
struct A {
    @location(0) x: f32,
    // Despite locations being 16-bytes, x and y cannot share a location
    @location(1) y: f32
}

// in1 occupies locations 0 and 1.
// in2 occupies location 2.
// The return value occupies location 0.
@fragment
fn fragShader(in1: A, @location(2) in2: f32) -> @location(0) vec4<f32> {
    // ...
}
```

User-defined IO can be mixed with built-in values in the same structure. For example,

EXAMPLE: MIXING BUILTINS AND USER-DEFINED IO

```
// Mixed builtins and user-defined inputs.
struct MyInputs {
    @location(0) x: vec4<f32>,
    @builtin(front_facing) y: bool,
    @location(1) @interpolate(flat) z: u32
}

struct MyOutputs {
    @builtin(frag_depth) x: f32,
    @location(0) y: vec4<f32>
}

@fragment
fn fragShader(in1: MyInputs) -> MyOutputs {
    // ...
}
```

EXAMPLE: INVALID LOCATION ASSIGNMENTS

```

struct A {
    @location(0) x: f32,
    // Invalid, x and y cannot share a location.
    @location(0) y: f32
}

struct B {
    @location(0) x: f32
}

struct C {
    // Invalid, structures with user-defined IO cannot be nested.
    b: B
}

struct D {
    x: vec4<f32>
}

@fragment
// Invalid, location cannot be applied to a structure type.
fn fragShader1(@location(0) in1: D) {
    // ...
}

@fragment
// Invalid, in1 and in2 cannot share a location.
fn fragShader2(@location(0) in1: f32, @location(0) in2: f32) {
    // ...
}

@fragment
// Invalid, location cannot be applied to a structure.
fn fragShader3(@location(0) in1: vec4<f32>) -> @location(0) D {
    // ...
}

```

13.3.1.4. Interpolation

Authors can control how user-defined IO data is interpolated through the use of the [interpolate](#) attribute. WGSL offers two aspects of interpolation to control: the type of interpolation, and the sampling of the interpolation.

The *interpolation type* [must](#) be one of the following [interpolation type name tokens](#):

perspective

Values are interpolated in a perspective correct manner.

linear

Values are interpolated in a linear, non-perspective correct manner.

flat

Values are not interpolated.

The *interpolation sampling* [must](#) be one of the following [interpolation sampling name tokens](#):

center

Interpolation is performed at the center of the pixel.

centroid

Interpolation is performed at a point that lies within all the samples covered by the fragment within the current primitive. This value is the same for all samples in the primitive.

sample

Interpolation is performed per sample. The [fragment](#) shader is invoked once per sample when this attribute is applied.

first

The value is provided by the first vertex of the primitive.

either

The value is provided by the first vertex or last vertex of the primitive. Whether the value comes from the first or last vertex is implementation dependent.

For user-defined IO of scalar or vector floating-point type:

- If the interpolation attribute is not specified, then `@interpolate(perspective, center)` is assumed.
- If the interpolation attribute is specified with an interpolation type:
 - If the interpolation type is `flat`, then:
 - If interpolation sampling is specified, it [must](#) be `first` or `either`.
 - Otherwise, `first` is assumed.
 - If the interpolation type is `perspective` or `linear`, then:
 - If interpolation sampling is specified, it [must](#) be `center`, `centroid`, or `sample`.
 - Otherwise, `center` is assumed.

User-defined [vertex](#) outputs and [fragment](#) inputs of scalar or vector integer type [must](#) always be specified with interpolation type `flat`.

[Interstage interface validation](#) checks that, within a [render pipeline](#), the interpolation properties of each user-defined [fragment](#) input match the interpolation properties of a vertex output with the same [location](#) assignment. If not, a [pipeline-creation error will](#) result.

§ 13.3.2. Resource Interface

A **resource** is an object which provides access to data external to a [shader stage](#), and which is not an [override-declaration](#), not an [immediate data](#) variable, and not a [shader stage input or output](#). Resources are shared by all invocations of the shader.

There are four kinds of resources:

- [Uniform buffers](#)
- [Storage buffers](#)
- [Texture resources](#)
- [Sampler resources](#)

The **resource interface of a shader** is the set of module-scope resource variables [statically accessed](#) by [functions in the shader stage](#).

Each resource variable [must](#) be declared with both [group](#) and [binding](#) attributes. Together with the shader's stage, these identify the binding address of the resource on the shader's pipeline. See [WebGPU § 8.3 GPUPipelineLayout](#).

[Immediate data](#) variables do not use [group](#) or [binding](#) attributes.

Two different resource variables in a [shader must not](#) have the same [group](#) and [binding](#) values, when considered as a pair.

§ 13.3.3. Resource Layout Compatibility

WebGPU requires that a shader’s resource interface match the [layout of the pipeline](#) using the shader.

It is a [pipeline-creation error](#) if a WGSL variable in a resource interface is bound to an incompatible WebGPU [binding member](#) or [binding type](#), where compatibility is defined by the following table.

WebGPU binding type compatibility

WGSL resource	WebGPU binding member	WebGPU binding type
uniform buffer		"uniform"
storage buffer with read_write access	buffer	GPUBufferBindingType "storage"
storage buffer with read access		"read-only-storage"
sampler	sampler	GPUSamplerBindingType "filtering" "non-filtering"
sampler_comparison		"comparison"
sampled texture , depth texture , or multisampled texture	texture	GPUTextureSampleType "float" "unfilterable-float" "sint" "uint" "depth"
write-only storage texture		"write-only"
read-write storage texture	storageTexture	GPUStorageTextureAccess "read-write"
read-only storage texture		"read-only"
external texture	externalTexture	(not applicable)

See the [WebGPU API](#) specification for interface validation requirements.

§ 13.3.4. Buffer Binding Determines Runtime-Sized Array Element Count

When a [storage buffer](#) variable contains a [runtime-sized](#) array, then the number of elements in that array is determined from the size of the corresponding [resource](#):

- Let T be the [store type](#) for a [storage buffer](#) variable, where T is a [runtime-sized](#) array type or contains a runtime-sized array type.
- Let $bufferBinding$ be [get as buffer binding\(resource\)](#).
- Let EBS be the [effective buffer binding size](#) for the $bufferBinding$ bound to the pipeline binding address corresponding to the storage buffer variable.
- Then $NRuntime$, i.e. the number of elements in the runtime-sized array, is the largest integer such that $SizeOf(T) \leq EBS$.

In more detail, the [NRuntime](#) for a runtime-size array of type RAT is:

[truncate](#)(($EBBS - array_offset$) $\div$ $array_stride$), where:

- $EBBS$ is the [effective buffer binding size](#) associated with the variable,
- $array_offset$ is the byte offset of the runtime-sized array within the store type of the variable.
 - It is zero if the store type is RAT , the runtime-sized array type itself.
 - Otherwise the store type is a structure, and its last member is the runtime-sized array. In this case $array_offset$ is the [byte offset of that member](#) within the structure.
- $array_stride$ is the stride of the array type, i.e. [StrideOf](#)(RAT).

A shader can compute [NRuntime](#) via the [arrayLength](#) builtin function.

NOTE: This algorithm is unambiguous: When a runtime-sized array is part of a larger type, it may only appear as the last element of a structure, which itself cannot be part of an enclosing array or structure.

[NRuntime](#) is determined by the size of the corresponding buffer binding, and that can be different for each [draw](#) or [dispatch command](#).

WebGPU validation rules ensure that $1 \leq NRuntime$.

In the following code sample:

- The `weights` variable is a [storage buffer](#).
- Its store type is the runtime-sized array type `array<f32>`.
- The array offset is 0.
- The array stride is [StrideOf](#)(`array<f32>`), which is 4.

EXAMPLE: NUMBER OF ELEMENTS IN A SIMPLE RUNTIME SIZED ARRAY

```
@group(0) @binding(1) var<storage> weights: array<f32>;
```

The following table shows examples of [NRuntime](#) for the `weights` variable, based on the corresponding effective buffer binding size.

Example number of elements for simple runtime-sized array

Effective buffer binding size	NRuntime for weights variable	Calculation
1024	256	truncate (1024 ÷ 4)
1025	256	truncate (1025 ÷ 4)
1026	256	truncate (1026 ÷ 4)
1027	256	truncate (1027 ÷ 4)
1028	257	truncate (1028 ÷ 4)

In the following code sample:

- The `lights` variable is a [storage buffer](#).
- Its store type is `LightStorage`.
- The `point` member of `LightStorage` is a runtime-sized array of type `array<PointLight>`.
 - The member is at byte offset 16 in the variable’s storage.
 - The array stride is `StrideOf(array<PointLight>) = roundUp(AlignOf(PointLight),SizeOf(PointLight)) = roundUp(16,32) = 32`

EXAMPLE: NUMBER OF ELEMENTS IN A COMPLEX RUNTIME SIZED ARRAY

```
struct PointLight {
    position : vec3f,
    // -- implicit member alignment padding --
    color : vec3f,
    // -- implicit struct size padding --
}

struct LightStorage {
    pointCount : u32,
    // -- implicit member alignment padding --
    point : array<PointLight>,
}

@group(0) @binding(1) var<storage> lights : LightStorage;
```

The following table shows examples of [NRuntime](#) for the `point` member of the `lights` variable.

Example number of elements for complex runtime-sized array

Effective buffer binding size	NRuntime for point member of lights variable	Calculation
1024	31	<code>truncate((1024 - 16) ÷ 32)</code>
1025	31	<code>truncate((1025 - 16) ÷ 32)</code>
1039	31	<code>truncate((1039 - 16) ÷ 32)</code>
1040	32	<code>truncate((1040 - 16) ÷ 32)</code>

§ 14. Memory

In WGSL, a value of [storable](#) type may be stored in memory, for later retrieval. This section describes the structure of memory, and the semantics of operations accessing memory. See [§ 6.5 Memory Views](#) for the types of values that can be placed in memory, and the types used to perform memory accesses.

§ 14.1. Memory Locations

Memory consists of a set of distinct *memory locations*. Each memory location is 8-bits in size. An operation affecting memory interacts with a set of one or more memory locations. Memory operations on [composites will not](#) access padding memory locations. Therefore, the set of memory locations accessed by an operation may not be contiguous.

Two sets of memory locations *overlap* if the intersection of their sets of memory locations is non-empty.

§ 14.2. Memory Access Mode

A *memory access* is an operation that acts on [memory locations](#).

- A *read access* observes the contents of memory locations.
- A *write access* sets the contents of memory locations.

A single operation can read, write, or both read and write.

Particular memory locations may support only certain kinds of accesses, expressed as the memory's *access mode*.

Access Modes

Access mode	Supported accesses
<i>read</i>	Supports read accesses, but not writes.
<i>write</i>	Supports write accesses, but not reads.
<i>read_write</i>	Supports both read and write accesses.

WGSL [predeclares](#) the [enumerants](#) `read`, `write`, and `read_write`.

§ 14.3. Address Spaces

Memory locations are partitioned into *address spaces*. Each address space has unique properties determining mutability, visibility, the values it may contain, and how to use variables with it. See [§ 7 Variable and Value Declarations](#) for more details.

The access mode of a given [memory view](#) is often determined by context:

The [storage](#) address spaces supports both [read](#) and [read_write](#) access modes. Each other address space supports only one access mode. The default access mode for each address space is described in the following table.

Address Spaces

Address space	Sharing among invocations	Default access mode	Notes
<i>function</i>	Same invocation only	read_write	
<i>private</i>	Same invocation only	read_write	
<i>workgroup</i>	Invocations in the same compute shader workgroup	read_write	The element count of an outermost array may be a pipeline-overrideable constant.

Address space	Sharing among invocations	Default access mode	Notes
<i>uniform</i>	Invocations in the same shader stage	read	For uniform buffer variables
<i>storage</i>	Invocations in the same shader stage	read	For storage buffer variables
<i>immediate</i>	Invocations in the same shader stage	read	For immediate data variables. Each entry point can statically access at most one immediate variable.
<i>handle</i>	Invocations in the same shader stage	read	For sampler and texture variables.

WGSL [predeclares](#) an [enumerant](#) for each address space, except for the `handle` address space.

[Variables](#) in the [workgroup](#) address space [must](#) only be [statically accessed](#) in a [compute shader stage](#).

[Variables](#) in the [storage](#) address space ([storage buffers](#)) can only be [statically accessed](#) by a [vertex shader stage](#) if the access mode is [read](#). Variables whose [store type](#) is a [storage texture](#) with a [write](#) or [read write access mode](#) cannot be [statically accessed](#) by a [vertex shader stage](#). See WebGPU [createBindGroupLayout\(\)](#).

NOTE: Each address space may have different performance characteristics.

When writing a [variable declaration](#) or a [pointer type](#) in WGSL source:

- For the [storage](#) address space, the access mode is optional, and defaults to [read](#).
- For other address spaces, the access mode [must not](#) be written.

§ 14.4. Memory Layout

The layout of types in WGSL is independent of [address space](#). Strictly speaking, however, that layout can only be observed by [host-shareable](#) buffers. [Uniform buffer](#) and [storage buffer](#) variables are used to share bulk data organized as a sequence of bytes in memory. Buffers are shared between the CPU and the GPU, or between different shader stages in a pipeline, or between different pipelines.

Because buffer data are shared without reformatting or translation, it is a [dynamic error](#) if buffer producers and consumers do not agree on the **memory layout**, which is the description of how the bytes in a buffer are organized into typed WGSL values. These bytes are [memory locations](#) of a value relative to a common base location.

The [store type](#) of a buffer variable [must](#) be [host-shareable](#), with fully elaborated memory layout, as described below.

Each buffer variable [must](#) be declared in either the [uniform](#) or [storage](#) address spaces.

The memory layout of a type is significant only when evaluating an expression with:

- a variable in the [uniform](#) or [storage](#) address space, or
- a pointer into the [uniform](#) or [storage](#) address space.

An 8-bit byte is the most basic unit of [host-shareable](#) memory. The terms defined in this section express counts of 8-bit bytes.

We will use the following notation, where T is a [host-shareable](#) or [fixed footprint](#) type, S is a host-shareable or fixed footprint structure type, and A is a host-shareable or fixed footprint array or runtime-sized array:

- $\text{AlignOf}(T)$ is the [alignment](#) of T , when T is not a [buffer](#).
- $\text{AlignOfMember}(S, i)$ is the alignment of the i 'th member of S .
- $\text{SizeOf}(T)$ is the [byte-size](#) of T .
- $\text{SizeOfMember}(S, i)$ is the size of the i 'th member of S .
- $\text{OffsetOfMember}(S, i)$ is the offset of the i 'th member from the start of S .
- $\text{StrideOf}(A)$ is the **element stride** of A , defined as the number of bytes from the start of one array element to the start of the next element. It equals the size of the array's element type, rounded up to the alignment of the element type:

$\text{StrideOf}(\text{array}\langle E, N \rangle) = \text{roundUp}(\text{AlignOf}(E), \text{SizeOf}(E))$

$\text{StrideOf}(\text{array}\langle E \rangle) = \text{roundUp}(\text{AlignOf}(E), \text{SizeOf}(E))$

- $\text{AccessibleBytes}(T)$ is the set of byte offsets within an instance of type T that contain data.
 - If T is a scalar or vector, then $\text{AccessibleBytes}(T)$ is the set of integers k such that $0 \leq k < \text{SizeOf}(T)$.
 - If T is a matrix with C columns and R rows, then $\text{AccessibleBytes}(T)$ is the set calculated as:
 - Let Stride be $\text{roundUp}(\text{AlignOf}(\text{vec}R), \text{SizeOf}(\text{vec}R))$.
 - For each i in $0..C-1$:
 - For each k in $\text{AccessibleBytes}(\text{vec}R)$:
 - The set contains $k + i * \text{Stride}$.
 - If T is a structure S , then $\text{AccessibleBytes}(T)$ is the set calculated as:
 - For each member M_i at byte offset $\text{Offset} = \text{OffsetOfMember}(S, i)$:
 - For each k in $\text{AccessibleBytes}(M_i)$:
 - The set contains $k + \text{Offset}$.
- $\text{AccessibleSlots}(T)$ is the set of integers i such that the intersection of the interval $[4*i, 4*i+4)$ and $\text{AccessibleBytes}(T)$ is not empty.

§ 14.4.1. Alignment and Size

Each [host-shareable](#) or [fixed footprint](#) data type T has a size. Except for [buffer](#) types, each host-shareable or fixed footprint type has an alignment.

The **alignment** of a type is a constraint on where values of that type may be placed in memory, expressed as an integer: a type's alignment **must** evenly divide the byte address of the starting [memory location](#) of a value of that type. Alignments enable use of more efficient hardware instructions for accessing the values, or satisfy more restrictive hardware requirements on certain address spaces. (See [address space layout constraints](#)).

NOTE: Each alignment value is always a power of two, by construction.

The **byte-size** of a type or structure member is the number of contiguous bytes reserved in host-shareable memory for the purpose of storing a value of the type or structure member. The size may include non-addressable padding at the end of the type. Consequently, loads and stores of a value might access fewer memory locations than the value's size.

Alignment and size of [host-shareable](#) and [fixed footprint](#) types are defined recursively in the following table:

Alignment and size for host-shareable and fixed footprint types

Host-shareable or fixed footprint type T	AlignOf(T)	SizeOf(T)
bool See Note .	4	4
i32 , u32 , or f32	4	4
f16	2	2
atomic<T>	4	4
vec2<T> , T is bool , i32 , u32 , or f32	8	8
vec2<f16>	4	4
vec3<T> , T is bool , i32 , u32 , or f32	16	12
vec3<f16>	8	6
vec4<T> , T is bool , i32 , u32 , or f32	16	16
vec4<f16>	8	8
matCxR (col-major) (General form)	AlignOf (vecR)	SizeOf (array < vecR , C >)
mat2x2<f32>	8	16
mat2x2<f16>	4	8
mat3x2<f32>	8	24
mat3x2<f16>	4	12
mat4x2<f32>	8	32
mat4x2<f16>	4	16
mat2x3<f32>	16	32
mat2x3<f16>	8	16
mat3x3<f32>	16	48
mat3x3<f16>	8	24
mat4x3<f32>	16	64
mat4x3<f16>	8	32
mat2x4<f32>	16	32

Host-shareable or fixed footprint type T	AlignOf(T)	SizeOf(T)
<code>mat2x4<f16></code>	8	16
<code>mat3x4<f32></code>	16	48
<code>mat3x4<f16></code>	8	24
<code>mat4x4<f32></code>	16	64
<code>mat4x4<f16></code>	8	32
struct S with members $M_1 \dots M_N$	$\max(\text{AlignOfMember}(S, 1), \dots, \text{AlignOfMember}(S, N))$	$\text{roundUp}(\text{AlignOf}(S), \text{justPastLastMember})$ where $\text{justPastLastMember} = \text{OffsetOfMember}(S, N) + \text{SizeOfMember}(S, N)$
<code>array<E, N></code>	AlignOf(E)	$N \times \text{roundUp}(\text{AlignOf}(E), \text{SizeOf}(E))$
<code>array<E></code>	AlignOf(E)	$\text{NRuntime} \times \text{roundUp}(\text{AlignOf}(E), \text{SizeOf}(E))$ where NRuntime is the runtime-determined number of elements of T
<code>buffer<N></code>	N/A	N
<code>buffer</code>	N/A	The size of the attached WebGPU GPUBuffer

NOTE: The alignment of [buffer](#) types is irrelevant since they cannot be contained in any other type.

NOTE: Many GPUs cannot implement single-byte writes without introducing potential data races. By specifying that a `bool` value occupies 4 bytes with 4 byte alignment, implementations can support adjacent boolean values in memory without introducing data races.

§ 14.4.2. Structure Member Layout

The internal layout of a [structure](#) is computed from the sizes and alignments of its members. By default, the members are arranged tightly, in order, without overlap, while satisfying member alignment requirements.

This default internal layout can be overridden by using *layout attributes*, which are:

- [size](#)
- [align](#)

The i 'th member of structure type S has a size and alignment, denoted by [SizeOfMember\(\$S, i\$ \)](#) and [AlignOfMember\(\$S, i\$ \)](#), respectively. The member sizes and alignments are used to calculate each member's byte offset from the start of the structure, as described in § 14.4.4 Internal Layout of Values.

[SizeOfMember\(\$S, i\$ \)](#) is k if the i 'th member of S has attribute [size\(\$k\$ \)](#). Otherwise, it is [SizeOf\(\$T\$ \)](#) where T is the type of the member.

$\text{AlignOfMember}(S, i)$ is k if the i 'th member of S has attribute `align( $k$ )`. Otherwise, it is $\text{AlignOf}(T)$ where T is the type of the member.

If a structure member has the `size` attribute applied, the value `must` be at least as large as the size of the member's type:

$\text{SizeOfMember}(S, i) \geq \text{SizeOf}(T)$

Where T is the type of the i 'th member of S .

The first structure member always has a zero byte offset from the start of the structure:

$\text{OffsetOfMember}(S, 1) = 0$

Each subsequent member is placed at the lowest offset that satisfies the member type alignment, and which avoids overlap with the previous member. For each member index $i > 1$:

$\text{OffsetOfMember}(S, i) = \text{roundUp}(\text{AlignOfMember}(S, i), \text{OffsetOfMember}(S, i-1) + \text{SizeOfMember}(S, i-1))$

EXAMPLE: LAYOUT OF STRUCTURES USING IMPLICIT MEMBER SIZES AND ALIGNMENTS

```

struct A {
    u: f32,           // offset(0)   align(4) size(4)
    v: f32,           // offset(4)   align(4) size(4)
    w: vec2<f32>,     // offset(8)   align(8) size(8)
    x: f32            // offset(16)  align(4) size(4)
    // -- implicit struct size padding -- // offset(20)           size(4)
}

struct B {
    a: vec2<f32>,     // offset(0)   align(8) size(8)
    // -- implicit member alignment padding -- // offset(8)           size(8)
    b: vec3<f32>,     // offset(16)  align(16) size(12)
    c: f32,           // offset(28)  align(4) size(4)
    d: f32,           // offset(32)  align(4) size(4)
    // -- implicit member alignment padding -- // offset(36)           size(4)
    e: A,             // offset(40)  align(8) size(24)
    f: vec3<f32>,     // offset(64)  align(16) size(12)
    // -- implicit member alignment padding -- // offset(76)           size(4)
    g: array<A, 3>,    // element stride 24 offset(80) align(8) size(72)
    h: i32            // offset(152) align(4) size(4)
    // -- implicit struct size padding -- // offset(156)          size(4)
}

@group(0) @binding(0)
var<storage,read_write> storage_buffer: B;

```

EXAMPLE: LAYOUT OF STRUCTURES WITH EXPLICIT MEMBER SIZES AND ALIGNMENTS

```

struct A {
    u: f32,
    v: f32,
    w: vec2<f32>,
    @size(16) x: f32
}

struct B {
    a: vec2<f32>,
    // -- implicit member alignment padding --
    b: vec3<f32>,
    c: f32,
    d: f32,
    // -- implicit member alignment padding --
    @align(16) e: A,
    f: vec3<f32>,
    // -- implicit member alignment padding --
    g: array<A, 3>, // element stride 32
    h: i32
    // -- implicit struct size padding --
}

@group(0) @binding(0)
var<uniform> uniform_buffer: B;

```

§ 14.4.3. Array Layout Examples

EXAMPLE: FIXED-SIZE ARRAY LAYOUT EXAMPLES

```

// Array where:
// - alignment is 4 = AlignOf(f32)
// - element stride is 4 = roundUp(AlignOf(f32), SizeOf(f32)) = roundUp(4,4)
// - size is 32 = stride * number_of_elements = 4 * 8
var small_stride: array<f32, 8>;

// Array where:
// - alignment is 16 = AlignOf(vec3<f32>) = 16
// - element stride is 16 = roundUp(AlignOf(vec3<f32>), SizeOf(vec3<f32>))
// = roundUp(16,12)
// - size is 128 = stride * number_of_elements = 16 * 8
var bigger_stride: array<vec3<f32>, 8>;

```


EXAMPLE: RUNTIME-SIZED ARRAY LAYOUT EXAMPLES

```

// Array where:
//   - alignment is 4 = AlignOf(f32)
//   - element stride is 4 = roundUp(AlignOf(f32), SizeOf(f32)) = 4
// If B is the effective buffer binding size for the binding on the
// draw or dispatch command, the number of elements is:
//   N_runtime = floor(B / element stride) = floor(B / 4)
@group(0) @binding(0)
var<storage> weights: array<f32>;

// Array where:
//   - alignment is 16 = AlignOf(vec3<f32>) = 16
//   - element stride is 16 = roundUp(AlignOf(vec3<f32>), SizeOf(vec3<f32>))
//                                     = roundUp(16, 12)
// If B is the effective buffer binding size for the binding on the
// draw or dispatch command, the number of elements is:
//   N_runtime = floor(B / element stride) = floor(B / 16)
var<storage> directions: array<vec3<f32>>;

```

§ 14.4.4. Internal Layout of Values

This section describes how the internals of a [host-shareable](#) value are placed in the byte locations of a buffer, given an assumed placement of the overall value. These layouts depend on the value's type, and the [align](#) and [size](#) attributes on structure members. These rules apply to [non-opaque](#) types as described below; values of [buffer](#) type do not have an internal structure, and are not constrained by these rules.

The buffer byte offset at which a value is placed [must](#) satisfy the type alignment requirement: If a value of type T is placed at buffer offset k , then $k = c \times \text{AlignOf}(T)$, for some non-negative integer c .

The data [will](#) appear identically regardless of the address space.

NOTE: The [bool](#) type is not [host-shareable](#). WGSL specifies that a [bool](#) value has a size and alignment of 4 bytes, but does not specify the internal layout of a bool value.

When a value V of type [u32](#) or [i32](#) is placed at byte offset k of a host-shared buffer, then:

- Byte k contains bits 0 through 7 of V
- Byte $k+1$ contains bits 8 through 15 of V
- Byte $k+2$ contains bits 16 through 23 of V
- Byte $k+3$ contains bits 24 through 31 of V

NOTE: Recall that [i32](#) uses twos-complement representation, so the sign bit is in bit position 31.

64-bit integer layout: Some features of the WebGPU API write 64-bit unsigned integer values into buffers. When such a value V appears at byte offset k of a host-shared buffer, then:

- Byte k contains bits 0 through 7 of V
- Byte $k+1$ contains bits 8 through 15 of V
- Byte $k+2$ contains bits 16 through 23 of V
- Byte $k+3$ contains bits 24 through 31 of V

- Byte $k+4$ contains bits 32 through 39 of V
- Byte $k+5$ contains bits 40 through 47 of V
- Byte $k+6$ contains bits 48 through 55 of V
- Byte $k+7$ contains bits 56 through 63 of V

NOTE: WGSL does not have a [concrete 64-bit integer](#) type.

A value V of type [f32](#) is represented in [IEEE-754 binary32](#) format. It has one sign bit, 8 exponent bits, and 23 fraction bits. When V is placed at byte offset k of host-shared buffer, then:

- Byte k contains bits 0 through 7 of the fraction.
- Byte $k+1$ contains bits 8 through 15 of the fraction.
- Bits 0 through 6 of byte $k+2$ contain bits 16 through 22 of the fraction.
- Bit 7 of byte $k+2$ contains bit 0 of the exponent.
- Bits 0 through 6 of byte $k+3$ contain bits 1 through 7 of the exponent.
- Bit 7 of byte $k+3$ contains the sign bit.

A value V of type [f16](#) is represented in [IEEE-754 binary16](#) format. It has one sign bit, 5 exponent bits, and 10 fraction bits. When V is placed at byte offset k of host-shared buffer, then:

- Byte k contains bits 0 through 7 of the fraction.
- Bits 0 through 1 of byte $k+1$ contain bits 8 through 9 of the fraction.
- Bits 2 through 6 of byte $k+1$ contain bits 0 through 4 of the exponent.
- Bit 7 of byte $k+1$ contains the sign bit.

NOTE: The above rules imply that numeric values in host-shared buffers are stored in little-endian format.

When a value V of [atomic type](#) `atomic< $T$ >` is placed in a host-shared buffer, it has the same internal layout as a value of the underlying type T .

When a value V of [vector type](#) `vecN< $T$ >` is placed at byte offset k of a host-shared buffer, then:

- $V.x$ is placed at byte offset k
- $V.y$ is placed at byte offset $k + \text{SizeOf}(T)$
- If $N \geq 3$, then $V.z$ is placed at byte offset $k + 2 \times \text{SizeOf}(T)$
- If $N \geq 4$, then $V.w$ is placed at byte offset $k + 3 \times \text{SizeOf}(T)$

When a value V of [matrix type](#) `matCxR< $T$ >` is placed at byte offset k of a host-shared buffer, then:

- Column vector i of V is placed at byte offset $k + i \times \text{AlignOf}(\text{vecR}<\mathcal{T}>)$

When a value of [array type](#) A is placed at byte offset k of a host-shared memory buffer, then:

- Element i of the array is placed at byte offset $k + i \times \text{StrideOf}(A)$

When a value of [structure type](#) S is placed at byte offset k of a host-shared memory buffer, then:

- The i^{th} member of the structure value is placed at byte offset $k + \text{OffsetOfMember}(S, i)$. See [§ 14.4.2 Structure Member Layout](#).

§ 14.4.5. Address Space Layout Constraints

The [storage](#) and [uniform](#) address spaces have different layout constraints which are described in this section.

All [address spaces](#) except [uniform](#) have the same constraints as the [storage](#) address space.

All structure and array types directly or indirectly referenced by a variable [must](#) obey the constraints of the variable's address space. Violations of an address space constraint results in a [shader-creation error](#).

In this section we define $\text{RequiredAlignOf}(S, C)$ as the byte offset [alignment](#) requirement of values of host-shareable or fixed-footprint type S when used in address space C .

Alignment requirements of host-shareable or fixed footprint types in address space C

Host-shareable or fixed footprint type S , assuming S can appear in C	RequiredAlignOf (S, C), uniform_buffer_standard_layout is supported or C is not uniform	RequiredAlignOf (S, C), uniform_buffer_standard_layout is not supported and C is uniform
bool , i32 , u32 , f32 , or f16	AlignOf (S)	AlignOf (S)
atomic < T >	AlignOf (S)	AlignOf (S)
vec N < T >	AlignOf (S)	AlignOf (S)
mat $C \times R$ < T >	AlignOf (S)	AlignOf (S)
array < T, N >	AlignOf (S)	roundUp (16, AlignOf (S))
array < T >	AlignOf (S) Disallowed if C is uniform except as the result of a buffer_view built-in function	roundUp (16, AlignOf (S)) Disallowed except as the result of a buffer_view built-in function
struct S	AlignOf (S)	roundUp (16, AlignOf (S))

Structure members of type T [must](#) have a byte offset from the start of the structure that is a multiple of the [RequiredAlignOf](#)(T, C) for the address space C :

$$\text{OffsetOfMember}(S, i) = k \times \text{RequiredAlignOf}(T, C)$$

Where k is a non-negative integer and the i 'th member of structure S has type T

Arrays of element type T [must](#) have an [element stride](#) that is a multiple of the [RequiredAlignOf](#)(T, C) for the address space C :

$$\text{StrideOf}(\text{array}\langle T, N \rangle) = k \times \text{RequiredAlignOf}(T, C)$$

$$\text{StrideOf}(\text{array}\langle T \rangle) = k \times \text{RequiredAlignOf}(T, C)$$

Where k is a positive integer

When [uniform_buffer_standard_layout](#) is **not** supported, the [uniform](#) address space requires that:

- Array elements are aligned to 16 byte boundaries. That is, $\text{StrideOf}(\text{array}\langle T, N \rangle) = 16 \times k'$ for some positive integer k' .
- If a structure member itself has a structure type S , then the number of bytes between the start of that member and the start of any following member [must](#) be at least [roundUp](#)(16, [SizeOf](#)(S)).

NOTE: The following examples show how to use [align](#) and [size](#) attributes on structure members to satisfy layout requirements for uniform buffers. In particular, these techniques can be used mechanically transform a GLSL buffer with std140 layout to WGSL.

EXAMPLE: SATISFYING OFFSET REQUIREMENTS FOR UNIFORM ADDRESS SPACE

```
struct S {
    x: f32
}
struct Invalid {
    a: S,
    b: f32 // invalid: offset between a and b is 4 bytes, but must be at least 16
}
@group(0) @binding(0) var<uniform> invalid: Invalid;

struct Valid {
    a: S,
    @align(16) b: f32 // valid: offset between a and b is 16 bytes
}
@group(0) @binding(1) var<uniform> valid: Valid;
```

EXAMPLE: SATISFYING STRIDE REQUIREMENTS FOR UNIFORM ADDRESS SPACE

```
struct small_stride {
    a: array<f32,8> // stride 4
}
// Invalid, stride must be a multiple of 16
@group(0) @binding(0) var<uniform> invalid: small_stride;

struct wrapped_f32 {
    @size(16) elem: f32
}
struct big_stride {
    a: array<wrapped_f32,8> // stride 16
}
@group(0) @binding(1) var<uniform> valid: big_stride; // Valid
```

§ 14.5. Memory Model

In general, WGSL follows the [Vulkan Memory Model](#). The remainder of this section describes how WGSL programs map to the Vulkan Memory Model.

NOTE: The Vulkan Memory Model is a textual version of a [formal Alloy model](#).

§ 14.5.1. Memory Operation

In WGSL, a [read access](#) is equivalent to a memory read operation in the Vulkan Memory Model. In WGSL, a [write access](#) is equivalent to a memory write operation in the Vulkan Memory Model.

A [read access](#) occurs when an invocation executes one of the following:

- An evaluation of the [Load Rule](#)
- Any [texture builtin function](#) except:
 - [textureDimensions](#)
 - [textureStore](#)
 - [textureNumLayers](#)
 - [textureNumLevels](#)
 - [textureNumSamples](#)
- Any atomic built-in function except [atomicStore](#)
- A [workgroupUniformLoad](#) built-in function
- A [compound assignment](#) statement (for the [left-hand side](#) expression)

A [write access](#) occurs when an invocation executes one of the following:

- An [assignment](#) statement ([simple](#) or [compound](#) for the [left-hand side](#) expression)
- A [textureStore](#) built-in function
- Any atomic built-in function except [atomicLoad](#)
 - [atomicCompareExchangeWeak](#) only performs a write if the exchanged member of the returned result is true

[Atomic read-modify-write](#) built-in functions perform a single memory operation that is both a [read access](#) and a [write access](#).

Read and write accesses do not occur under any other circumstances. Read and write accesses are collectively known as [memory operations](#) in the Vulkan Memory Model.

A memory operation accesses exactly the set of [locations](#) associated with the particular [memory view](#) used in the operation. For example, a memory read that accesses a [u32](#) from a struct containing multiple members, only reads the memory locations associated with that u32 member.

NOTE: A write access to a component of a vector **may** access all memory locations associated with that vector.

EXAMPLE: ACCESSING MEMORY LOCATIONS

```
struct S {
    a : f32,
    b : u32,
    c : f32
}

@group(0) @binding(0)
var<storage> v : S;

fn foo() {
    let x = v.b; // Does not access memory locations for v.a or v.c.
}
```

§ 14.5.2. Memory Model Reference

Each module-scope [resource](#) variable forms a [memory model reference](#) for the unique [group](#) and [binding](#) pair. Each other variable (i.e. variables in the [function](#), [private](#), and [workgroup](#) address spaces) forms a unique [memory model reference](#) for the lifetime of the variable.

§ 14.5.3. Scoped Operations

When an invocation performs a scoped operation, it will affect one or two sets of invocations. These sets are the memory scope and the execution scope. The *memory scope* specifies the set of invocations that will see any updates to memory contents affected by the operation. For [synchronization built-in functions](#), this also means that all affected memory operations program ordered before the function are visible to affected operations program ordered after the function. The *execution scope* specifies the set of invocations which may participate in an operation (see [§ 15.6 Collective Operations](#)).

[Atomic built-in functions](#) map to [atomic operations](#) whose memory [scope](#) is:

- Workgroup if the atomic pointer is in the [workgroup](#) address space
- QueueFamily if the atomic pointer is in the [storage](#) address space

[Synchronization built-in functions](#) map to control barriers whose execution and memory [scopes](#) are Workgroup.

Implicit and explicit [derivatives](#) have an implicit [quad](#) execution scope.

NOTE: If the Vulkan memory model is not enabled in generated shaders, Device scope should be used instead of QueueFamily.

§ 14.5.4. Memory Semantics

All [Atomic built-in functions](#) use Relaxed [memory semantics](#) and, thus, no storage class semantics.

NOTE: Address space in WGSL is equivalent to storage class in SPIR-V.

[workgroupBarrier](#) uses AcquireRelease [memory semantics](#) and WorkgroupMemory semantics. [storageBarrier](#) uses AcquireRelease [memory semantics](#) and UniformMemory semantics. [textureBarrier](#) uses AcquireRelease [memory semantics](#) and ImageMemory semantics.

NOTE: A combined [workgroupBarrier](#) and [storageBarrier](#) uses AcquireRelease ordering semantics and both WorkgroupMemory and UniformMemory memory semantics.

NOTE: No atomic or synchronization built-in functions use MakeAvailable or MakeVisible semantics.

§ 14.5.5. Private vs Non-private

All non-atomic [read accesses](#) in the [storage](#) or [workgroup](#) address spaces are considered [non-private](#) and correspond to read operations with NonPrivatePointer | MakePointerVisible memory operands with the Workgroup scope.

All non-atomic [write accesses](#) in the [storage](#) or [workgroup](#) address spaces are considered [non-private](#) and correspond to write operations with NonPrivatePointer | MakePointerAvailable memory operands with the Workgroup scope.

All non-atomic [read accesses](#) in the [handle](#) address space are considered [non-private](#) and correspond to read operations with `NonPrivateTexel | MakeTexelVisible` memory operands with the Workgroup scope.

All non-atomic [write accesses](#) in the [handle](#) address space are considered [non-private](#) and correspond to write operations with `NonPrivateTexel | MakeTexelAvailable` memory operands with the Workgroup scope.

§ 15. Execution

[§ 1.1 Overview](#) describes how a shader is invoked and partitioned into [invocations](#). This section describes further constraints on how invocations execute, individually and collectively.

§ 15.1. Program Order Within an Invocation

Each statement in a WGSL module may be executed zero or more times during execution. For a given invocation, each execution of a given statement represents a unique *dynamic statement instance*.

When a statement includes an expression, the statement's semantics determines:

- Whether the expression is evaluated as part of statement execution.
- The relative ordering of evaluation between independent expressions in the statement.

Expression nesting defines data dependencies which must be satisfied to complete evaluation. That is, a nested expression must be evaluated before the enclosing expression can be evaluated. The order of evaluation for operands of an expression is left-to-right in WGSL. For example, `foo() + bar()` must evaluate `foo()` before `bar()`. See [§ 8 Expressions](#).

Statements in a WGSL module are executed in control flow order. See [§ 9 Statements](#) and [§ 11.2 Function Calls](#).

§ 15.2. Uniformity

A [collective operation](#) (e.g. barrier, derivative, or a texture operation relying on an implicitly computed derivative) requires coordination among different invocations running concurrently on the GPU. The operation executes correctly and portably when all invocations execute it concurrently, i.e. in [uniform control flow](#).

Conversely, incorrect or non-portable behavior occurs when a strict subset of invocations execute the operation, i.e. in non-uniform control flow. Informally, some invocations reach the collective operation, but others do not, or not at the same time, as a result of non-uniform control dependencies. Non-uniform control dependencies arise from [control flow](#) statements whose behavior depends on [non-uniform values](#).

For example, a non-uniform control dependency arises when different invocations compute different values for the condition of an [if](#), [break-if](#), [while](#), or [for](#), different values for the selector of a [switch](#), or the left-hand operand of a short-circuiting binary operator (`&&` or `|`).

These non-uniform values can often be traced back to certain sources that are not statically proven to be uniform. These sources include, but are not limited to:

- Mutable [module-scope variables](#)
- Most [built-in values](#)
- [User-defined inputs](#)
- Certain [built-in functions](#) (see [§ 15.2.7 Uniformity Rules for Function Calls](#))

To ensure correct and portable behavior, a WGSL implementation [will](#) perform a static ***uniformity analysis***, attempting to prove that each collective operation executes in [uniform control flow](#). Subsequent subsections describe the analysis.

A ***uniformity failure*** [will](#) be triggered when [uniformity analysis](#) cannot prove that a particular [collective operation](#) executes in [uniform control flow](#).

- If a uniformity failure is triggered for a [builtin function that computes a derivative](#), then a [derivative_uniformity diagnostic](#) is [triggered](#).
 - The diagnostic's [triggering location](#) is the location of the [call site](#) of that builtin.
 - The diagnostic's [severity](#) defaults to an [error](#) but can be controlled with a [diagnostic filter](#).
- If a uniformity failure is triggered for a [synchronization builtin](#), an [error diagnostic](#) is [triggered](#), which results in a [shader-creation error](#).
- If a uniformity failure is triggered for a [subgroup](#) or [quad](#) builtin, then a [subgroup_uniformity diagnostic](#) is [triggered](#).
 - The diagnostic's [triggering location](#) is the location of the [call site](#) of that builtin or the location of the parameter required to be uniform in the case of [subgroupShuffleUp](#), [subgroupShuffleDown](#), or [subgroupShuffleXor](#).
 - The diagnostic's [severity](#) defaults to an [error](#), but can be controlled with a [diagnostic filter](#).

§ 15.2.1. Terminology and Concepts

The following definitions are merely informative, trying to give an intuition for what the analysis in the next subsection is computing. The analysis is what actually defines these concepts, and when a program is valid or breaks the uniformity rules.

For a given group of invocations:

- If all invocations in a given scope execute as if they are executing in lockstep at a given point in the program, that point is said to have ***uniform control flow*** for the given ***uniformity scope***.
 - ***Workgroup uniformity scope***: For a [compute shader stage](#), the uniformity scope is all invocations in the same [workgroup](#).
 - ***Draw uniformity scope***: For other shader stages, the uniformity scope is all invocations for that [entry point](#) in the same [draw command](#).
 - ***Subgroup uniformity scope***: If the [subgroup_uniformity](#) feature is supported, the uniformity scope for [subgroup](#) and [quad](#) built-in functions instead is all invocations in the same [subgroup](#).
- If an expression is executed in uniform control flow, and all invocations compute the same value, it is said to be a ***uniform value***.
- If invocations hold the same value for a local variable at every point where it is live, it is said to be a ***uniform variable***.

§ 15.2.2. Uniformity Analysis Overview

The remaining subsections specify a static analysis that verifies that [collective operations](#) are only executed in [uniform control flow](#). If the [subgroup_uniformity](#) feature is supported then there are multiple [uniformity scopes](#). This analysis is performed once for each scope.

NOTE: The analysis is described as being executed once per scope, but implementations may perform a single analysis that includes each scope. [Workgroup](#) and [draw](#) uniformity scopes are effectively equivalent since they operate on different shader stages and represent the largest uniformity scope in a shader stage.

The analysis assumes [dynamic errors](#) do not occur. A shader stage with a [dynamic error](#) is already non-portable, no matter the outcome of uniformity analysis.

NOTE: This analysis has the following desirable properties:

- Sound, meaning that a [uniformity failure will](#) be triggered for a program that would break the uniformity requirements of builtins.
- Refactoring a piece of code into a function, or inlining a function, cannot make a shader invalid if it was valid before the transformation.
- If the analysis refuses a program, it provides a straightforward chain of implications that can be used by the user agent to craft a good error message.

Each function is analyzed, trying to ensure two things:

- Uniformity requirements are satisfied when it calls other functions, and
- Uniformity requirements are satisfied whenever it is called.

A [uniformity failure](#) is triggered if either of these two checks fail.

As part of this work, the analysis computes metadata about the function to help analyze its callers in turn. This means that the call graph must first be built, and functions must be analyzed from the leaves upwards, i.e. from functions that call no function outside the standard library toward the entry point. This way, whenever a function is analyzed, the metadata for all of its callees has already been computed. There is no risk of being trapped in a cycle, as recurrence is forbidden in the language.

NOTE: Another way of saying the same thing is that we do a topological sort of functions ordered by the "is a (possibly indirect) callee of" partial order, and analyze them in that order.

Additionally, for each function call, the analysis computes and propagates the set of [triggering rules](#), if any, that would be triggered if that call cannot be proven to be in uniform control flow. We call this the *potential-trigger-set* for the call. The elements of this set are drawn from the following possibilities:

- [derivative_uniformity](#), for functions relying on computing a derivative,
- [subgroup_uniformity](#), for functions in [§ 17.12 Subgroup Built-in Functions](#) or [§ 17.13 Quad Operations](#), or
- an unnamed triggering rule, for the uniformity requirements that cannot be filtered.
 - This is used for compute shader functions relying on [synchronization functions](#).

§ 15.2.3. Analyzing the Uniformity Requirements of a Function

Each function is analyzed in two phases.

The first phase walks over the syntax of the function, building a directed graph along the way based on the rules in the following subsections. The second phase explores that graph, computing the constraints on calling this function, and potentially triggering a [uniformity failure](#).

NOTE: Apart from four special nodes [RequiredToBeUniform.error](#), [RequiredToBeUniform.warning](#), [RequiredToBeUniform.info](#), and [MaybeNonUniform](#), each node can be understood as capturing the truth-value one of the following statements:

- A specific point of the program must be executed in [uniform control flow](#).
- An expression must be a [uniform value](#).
- A variable must be a [uniform variable](#).
- A value stored in memory, that could be loaded via a pointer, must be a [uniform value](#).

An edge can be understood as an implication from the statement corresponding to its source node to the statement corresponding to its target node.

For example, one uniformity requirement is that the `workgroupBarrier` builtin function must only be called within uniform control flow. To express this, we add an edge from [RequiredToBeUniform.error](#) to the node corresponding to the `workgroupBarrier` [call site](#). One way to understand this is that [RequiredToBeUniform.error](#) corresponds to the proposition True, so that [RequiredToBeUniform.error](#) $\rightarrow$ X is the same as saying that X is true.

Reciprocally, to express that we cannot ensure the uniformity of something (e.g. a variable which holds the thread id), we add an edge from the corresponding node to [MaybeNonUniform](#). One way to understand this, is that [MaybeNonUniform](#) corresponds to the proposition False, so that $X \rightarrow$ [MaybeNonUniform](#) is the same as saying that X is false.

A consequence of this interpretation is that every node reachable from [RequiredToBeUniform.error](#) corresponds to something which is required to be uniform for the program to be valid, and every node from which [MaybeNonUniform](#) is reachable corresponds to something whose uniformity we cannot guarantee. It follows that we have a uniformity violation, triggering a [uniformity failure](#), if there is any path from [RequiredToBeUniform.error](#) to [MaybeNonUniform](#).

The nodes [RequiredToBeUniform.warning](#) and [RequiredToBeUniform.info](#) are used in a similar way, but instead help determine when [warning](#) or [info diagnostics](#) should be triggered:

- If there is a path from [RequiredToBeUniform.warning](#) to [MaybeNonUniform](#), then a [warning diagnostic](#) will be triggered.
- If there is a path from [RequiredToBeUniform.info](#) to [MaybeNonUniform](#), then an [info diagnostic](#) will be triggered.

As described in [§ 2.3 Diagnostics](#), lower severity diagnostics may be discarded if higher severity diagnostics have also been generated.

For each function, two tags are computed:

- A **call site tag** describing the control flow uniformity requirements on the [call sites](#) of the function, and
- A **function tag** describing the function's effects on uniformity.

For each [formal parameter](#) of a function, one or two tags are computed:

- A **parameter tag** describes the uniformity requirement of the parameter value.
- A **parameter return tag** describes how the uniformity of the parameter influences that of the function's [return value](#).
- A **pointer parameter tag**, when the parameter type is a pointer into the [function](#) address space. The tag describes whether the value in the memory pointed to by the parameter may become [non-uniform](#) during the execution of the function call.

[Call site tag values](#)

Call Site Tag	Description
<i>CallSiteRequiredToBeUniform.S</i> , where <i>S</i> is one of the severities: error , warning , or info .	The function must only be called from uniform control flow . Otherwise a diagnostic with severity <i>S</i> will be triggered. Associated with a potential-trigger-set .
<i>CallSiteNoRestriction</i>	The function may be called from non-uniform control flow . Function tag values
Function Tag	Description
<i>ReturnValueMayBeNonUniform</i>	The return value of the function may be non-uniform.
<i>NoRestriction</i>	The function does not introduce non-uniformity. Parameter tag values
Parameter Tag	Description
<i>ParameterRequiredToBeUniform.S</i> , where <i>S</i> is one of the severities: error , warning , or info .	The parameter must be a uniform value . If the parameter type is a pointer, the memory view, but not necessarily its contents, must be uniform. Otherwise a diagnostic with severity <i>S</i> will be triggered. Associated with a potential-trigger-set .
<i>ParameterContentsRequiredToBeUniform.S</i> , where <i>S</i> is one of the severities: error , warning , or info .	The value stored in the memory pointed to by the pointer parameter must be a uniform value . Otherwise a diagnostic with severity <i>S</i> will be triggered. Associated with a potential-trigger-set .
<i>ParameterNoRestriction</i>	The parameter value has no uniformity requirement. Parameter return tag values
Parameter Return Tag	Description
<i>ParameterReturnContentsRequiredToBeUniform</i>	The parameter must be a uniform value in order for the return value to be a uniform value. If the parameter is a pointer, then the values stored in the memory pointed to by the pointer must also be uniform .
<i>ParameterReturnNoRestriction</i>	The parameter value has no uniformity requirement. Pointer parameter tag values
Pointer Parameter Tag	Description

Pointer Parameter Tag	Description
<i>PointerParameterMayBeNonUniform</i>	The value stored in the memory pointed to by the pointer parameter may be non-uniform after the function call.
<i>PointerParameterNoRestriction</i>	The uniformity of the value stored in the memory pointed to by the pointer parameter is analyzed by the function call.

The following algorithm describes how to compute these tags for a given function:

- Create the following nodes:
 - RequiredToBeUniform.error***, ***RequiredToBeUniform.warning***, and ***RequiredToBeUniform.info***. Collectively these are called the ***RequiredToBeUniform.S*** nodes.
 - Each such node is associated with a [potential-trigger-set](#), which is initially empty.
 - MayBeNonUniform***
 - CF_start***, representing the uniformity requirement for control flow when the function begins executing.
 - param_i***, where *i* ranges over the function's [formal parameters](#).
 - If the function has a [return type](#), create a node called ***Value_return***.
- Desugar pointers as described in [§ 15.2.4 Pointer Desugaring](#).
 - For each formal parameter that is a pointer in the [function](#) address space, create the following nodes:
 - param_i_contents***: this represents the uniformity of the contents of the memory view.
 - Value_return_i_contents***: this represents the function's effects on the uniformity of the contents of the memory view.
- Walk over the syntax of the function, adding nodes and edges to the graph following the rules of the next sections ([§ 15.2.5 Function-scope Variable Value Analysis](#), [§ 15.2.6 Uniformity Rules for Statements](#), [§ 15.2.7 Uniformity Rules for Function Calls](#), [§ 15.2.8 Uniformity Rules for Expressions](#)), using ***CF_start*** as the starting control-flow for the function's body.
 - The nodes added in this step are called ***interior nodes***.
- Initialize as follows:
 - The [function tag](#) is initialized to [NoRestriction](#).
 - The [call site tag](#) is initialized to [CallSiteNoRestriction](#).
 - The [parameter tag](#) for each [param_i](#) is initialized to [ParameterNoRestriction](#).
 - The [parameter return tag](#) for each [param_i](#) is initialized to [ParameterReturnNoRestriction](#).
 - The [pointer parameter tag](#) for each [param_i](#), if it exists, is initialized to [PointerParameterNoRestriction](#).
- For each severity *S* in the order {[error](#), [warning](#), [info](#)}, perform the following:
 - Let *R.S* be the set of unvisited nodes reachable from [RequiredToBeUniform.S](#).
 - Mark the [interior nodes](#) in *R.S* as visited.
 - Let *PTS* be the [potential-trigger-set](#) associated with [RequiredToBeUniform.S](#).
 - If *R.S* includes the node [MayBeNonUniform](#), then trigger a [uniformity failure](#):
 - [Trigger](#) a diagnostic with severity *S* and triggering rule *t*, for each *t* in *PTS*.
 - Otherwise:

- If $R.S$ includes [CF_start](#), and the [call site tag](#) has not been updated since initialization, then set the [call site tag](#) to [CallSiteRequiredToBeUniform.S](#), and set its [potential-trigger-set](#) to PTS .
- For each [param_i](#) in $R.S$, if its corresponding [parameter tag](#) has not been updated since initialization, then set that tag to [ParameterRequiredToBeUniform.S](#), and set its [potential-trigger-set](#) to PTS .
- For each [param_i_contents](#) in $R.S$, if its corresponding [parameter tag](#) has not been updated since initialization, then set that tag to [ParameterContentsRequiredToBeUniform.S](#), and set its [potential-trigger-set](#) to PTS .
- Mark all the [interior nodes](#) as unvisited.
- If [Value_return](#) exists, let VR be the set of nodes reachable from [Value_return](#).
 - If VR includes [MayBeNonUniform](#), then set the [function tag](#) to [ReturnValueMayBeNonUniform](#).
 - For each [param_i](#) in VR , set the corresponding [parameter return tag](#) to [ParameterReturnContentsRequiredToBeUniform](#).
- For each [Value_return_i_contents](#) node, let VRi be the set of nodes reachable from [Value_return_i_contents](#).
 - If VRi includes [MayBeNonUniform](#), set the corresponding [pointer parameter tag](#) to [PointerParameterMayBeNonUniform](#).

NOTE: The entire graph can be destroyed at this point. The tags described above are all that we need to remember to analyze callers of this function. However, the graph contains information that can be used to provide more informative diagnostics. For example a value in a one function may not be provably [uniform](#), which then contributes to triggering a uniformity failure in another function. An informative diagnostic would describe the non-uniform value, as well as the function call at the diagnostic's [triggered location](#).

§ 15.2.4. Pointer Desugaring

Each [parameter](#) of [pointer type](#) in the [function](#) address space is desugared as a local variable declaration whose initial value is equivalent to dereferencing the parameter. That is, function address space pointers are viewed as aliases to a local variable declaration. The initial value assignment produces an edge to [param_i_contents](#) for i^{th} parameter (i.e. $V(e)$ is [param_i_contents](#)).

Each [let-declaration](#), L , with an [effective-value-type](#) that is a [pointer type](#) is desugared as follows:

- Visit each subexpression, SE , of the initializer expression of L in a postorder depth-first traversal:
 - If SE invokes the [load rule](#) during [type checking](#) and the [root identifier](#) is a mutable variable then:
 - Create a new let-declaration, LSE , immediately prior to L initialized with SE .
 - Replace SE in L with a [value identifier expression](#) composed of LSE .
- Record the, possibly updated, initializer expression of L .
- Substitute each [identifier](#) that [resolves](#) to L with the recorded initializer expression (wrapped in a [parenthesized expression](#)).

This desugaring simplifies the subsequent analyses by exposing the [root identifier](#) of the pointer directly at each of its uses.

NOTE: For the purposes of uniformity analysis [type checking](#) is described to occur both before and after this desugaring has occurred.

EXAMPLE: POINTERS IN THE UNIFORMITY ANALYSIS

```

fn foo(p : ptr<function, array<f32, 4>>, i : i32) -> f32 {
    let p1 = p;
    var x = i;
    let p2 = &(*p1)[x];
    x = 0;
    *p2 = 5;
    return (*p1)[x];
}

// This is the equivalent version of foo for the analysis.
fn foo_for_analysis(p : ptr<function, array<f32, 4>>, i : i32) -> f32 {
    var p_var = *p;           // Introduce variable for p.
    let p1 = &p_var;          // Use the variable for p1
    var x = i;
    let x_tmp1 = x;           // Capture value of x
    let p2 = &(p_var[x_tmp1]); // Substitute p1's initializer
    x = 0;
    *(&(p_var[x_tmp1])) = 5;   // Substitute p2's initializer
    return (*(&p_var))[x];     // Substitute p1's initializer
}

```

§ 15.2.5. Function-scope Variable Value Analysis

The value of each [function-scope variable](#) at a particular statement can be analyzed in terms of the assignments that reach it and, potentially, its initial value.

An assignment is a *full assignment* if:

- The variable's [effective-value-type](#) is a [scalar](#) type, or
- the variable's [effective-value-type](#) is a [composite](#) type and each [component](#) of the composite is assigned a value.

Otherwise, an assignment is a *partial assignment*.

A *full reference* is an expression of [reference type](#) that is one of:

- an identifier x that [resolves](#) to a variable, or
- (r) where r is a [full reference](#), or
- $*p$ where p is a [full pointer](#).

A *full pointer* is an expression of [pointer type](#) that is one of:

- $\&r$ where r is a [full reference](#), or
- an identifier p that [resolves](#) to a [let-declaration](#) initialized to a [full pointer](#), or
- (p) where p is a [full pointer](#).

NOTE: For the purposes of this analysis, we don't need the case where a formal parameter of pointer type may be a full pointer.

A [full reference](#), and similarly a [full pointer](#), is a [memory view](#) for *all* the memory locations for the corresponding [originating variable](#) x .

A reference that is not a [full reference](#) is a *partial reference*. As such, a partial reference is either:

- a memory view for a strict subset of the memory locations for the corresponding [originating variable](#), or
- a memory view the with same set of locations as the corresponding originating variable, but with a different [store type](#).

NOTE: A [partial reference](#) can still cover all the same memory locations as a full reference, i.e. all the locations used by a variable declaration. This can occur when the store type is a structure type having only one member, or when the store type is an array type with one element.

Consider a structure type with a single member, and a variable storing that type:

```
struct S { member: i32; }
fn foo () {
    var v: S;
}
```

Then `v` is a full reference and `v.member` is a partial reference. Their memory views cover the same memory locations, but the store type for `v` is `S` and the store type of `v.s` is `i32`.

A similar situation occurs with arrays having a single element:

```
fn foo () {
    var arr: array<i32,1>;
}
```

Then `arr` is a full reference and `arr[0]` is a partial reference. Their memory views cover the same memory locations, but the store type for `arr` is `array<i32,1>` and the store type of `arr[0]` is `i32`.

To simplify analysis, an assignment via *any* kind of [partial reference](#) is treated as if it does *not* modify every memory location in the associated [originating variable](#). This causes the analysis to be conservative, potentially triggering a [uniformity failure](#) for more programs than strictly necessary.

An assignment through a [full reference](#) is a [full assignment](#).

An assignment through a [partial reference](#) is a [partial assignment](#).

When the uniformity rules in subsequent sections refer to the value for a function-scope variable used as an [RHSValue](#), it means the value of the variable prior to evaluation of the RHSValue expression. When the uniformity rules in subsequent sections refer to the value for a function-scope variable used as an [LHSValue](#), it means the value of the variable after execution of the statement the expression appears in.

Multiple assignments to a variable might reach a use of that variable due to [control-flow statements](#) or partial assignments. The analysis joins multiple assignments reaching out of control-flow statements by unioning the set of assignments that reach each control-flow exit.

The following table describes the rules for joining assignments. In the uniformity graph, each join is an edge from the result node to node representing the source of the value. It is written in terms of an arbitrary variable `x`. It uses the following notations:

- $V_{in}(S)$ is the value of `x` prior the execution of the statement `S`.
- $V_{out}(S)$ is the value of `x` after the execution of the statement `S`.
- $V_{out}(prev)$ is the value of `x` prior to the execution of the current statement.
- $V_{in}(next)$ is the value of `x` prior to the execution of the next statement.

- $V(e)$ is a value node for an expression as in the subsequent sections.
- $V(0)$ is the zero value of x 's [effective-value-type](#).

Rules for joining multiple assignments to a function-scope variable.

Statement	Result	Edges from the Result
<code>var x;</code>	$Vin(next)$	$V(0)$
<code>var x = e;</code>		
<code>x = e;</code>		$V(e)$
<code>r = e;</code> where r is a full reference to variable x	$Vin(next)$	NOTE: This is a full assignment to x .
<code>r = e;</code> where r is a partial reference to variable x	$Vout(S)$	$V(e), Vout(prev)$ NOTE: This is a partial assignment to x . NOTE: Partial assignments include the previous value. The assignment either writes only a subset of the stored components, or the type of the written value differs from the store type of the originating variable .
<code>s1 s2</code> where $Next$ is in behavior of $s1$.	$Vin(s2)$	$Vout(s1)$ NOTE: $s1$ often ends in a semicolon.
<code>if e s1 else s2</code> where $Next$ is in the behaviors of both $s1$ and $s2$	$Vin(next)$	$Vout(s1), Vout(s2)$
<code>if e s1 else s2</code> where $Next$ is in the behavior of $s1$, but not $s2$	$Vin(next)$	$Vout(s1)$
<code>if e s1 else s2</code> where $Next$ is in the behavior of $s2$, but not $s1$	$Vin(next)$	$Vout(s2)$
<code>loop { s1 continuing { s2 } }</code>	$Vin(s1)$	$Vout(prev),$ $Vout(s2)$ if the behavior of $s1$ intersects $\{Next, Continue\}$

Statement	Result	Edges from the Result
loop { <i>s1</i> continuing { <i>s2</i> } }	$Vin(s_2)$	$Vout(s_1)$ if Next is in the behavior of <i>s1</i> , $Vout(s_i)$ for all s_i in <i>s1</i> whose behavior is {Continue} and transfer control to <i>s2</i>
loop { <i>s1</i> continuing { <i>s2</i> } }	$Vin(next)$	$Vout(s_2)$ if Break is in the behavior of <i>s1</i> , $Vout(s_i)$ for all s_i in <i>s1</i> whose behavior is {Break} and transfer control to <i>next</i>
switch <i>e</i> { case _: <i>s1</i> case _: <i>s2</i> ... case _: <i>s3</i> }	$Vin(s_i)$	$Vout(prev)$
switch <i>e</i> { case _: <i>s1</i> case _: <i>s2</i> ... case _: <i>s3</i> }	$Vin(next)$	$Vout(s_i)$, for all s_i whose behavior includes Next or Break, and $Vout(s_j)$ for all statements inside s_j whose behavior is {Break} and transfer control to <i>next</i>

For all other statements (except function calls), $Vin(next)$ is equivalent to $Vout(prev)$.

NOTE: The same desugarings apply as in [statement behavior analysis](#).

§ 15.2.6. Uniformity Rules for Statements

The rules for analyzing statements take as argument both the statement itself and the node corresponding to control flow at the beginning of it (which we'll note "CF" below) and return both of the following:

- A node corresponding to control flow at the exit of it
- A set of new nodes and edges to add to the graph

In the table below, (CF1, *S*) => CF2 means "run the analysis on *S* starting with control flow CF1, apply the required changes to the graph, and name the resulting control flow CF2". Similarly, (CF1, *E*) => *V* means "run the analysis on expression *E*, starting with control flow CF1, apply the required changes to the graph, and name the resulting value node *V*" (see next section for the analysis of expressions). This evaluation of expressions is used for any expression that is not part of the [left-hand side](#) of an [assignment](#) and is called an **RHSValue**.

There is a similar set of rules for expressions that are part of the [left-hand side](#) of an [assignment](#), the **LHSValue**, that we denote by LHSValue: (CF, *E*) => *L*. Instead of computing the node which corresponds to the uniformity of the value, it computes the node which corresponds to the uniformity of the variable we are addressing.

NOTE: [LHSValues](#) include expressions in [increment](#) and [decrement](#) statements.

NOTE: [RHSValues](#) include expression that are part of the [right-hand side](#) of an [assignment](#) statement or expressions that are not part of an assignment, [increment](#), or [decrement](#) statement.

When several edges have to be created we use $X \rightarrow \{Y, Z\}$ as a short-hand for $X \rightarrow Y, X \rightarrow Z$.

In analyzing loops, we use the following patterns:

- Node CF' models control flow uniformity at the start of each [iteration](#).
- Node $CF1$ models control flow uniformity at the end of the loop body $s1$.
- Node $CF2$ models control flow uniformity at the end of the continuing block statements $s2$.
- The edges $CF' \rightarrow CF1$ and $CF' \rightarrow CF2$ model the fact control flow uniformity at the end of a loop affects the uniformity at the start of the next iteration. They are absent when loop body statements $s1$ only break or return, i.e. the behavior of $s1$ is a subset of {Break, Return}.
- The $CF' \rightarrow CF$ edge models the fact that uniformity at the start of at least some iterations depends on the uniformity when control flow reaches the whole loop statement.
- When the whole loop has behavior {Next}, we assume any control flow divergence is resolved by the time the loop ends. The loop's result control flow node is therefore set to CF , to model the fact that uniformity upon leaving the loop matches uniformity when first arriving at the loop.
- **NOTE:** [Statement behavior analysis](#) implies that the behavior of a loop is one of {Next}, {Return}, or {Next,Return}.

Uniformity rules for statements

Statement	New nodes	Recursive analyses	Resulting control flow node	New edges
<i>empty statement</i>			CF	
$\{s\}$		$(CF, s) \Rightarrow CF'$	CF'	
$s1\ s2,$ with Next in behavior of $s1$		$(CF, s1) \Rightarrow CF1$ $(CF1, s2) \Rightarrow CF2$	$CF2$	
NOTE: $s1$ often ends in a semicolon.				
$s1\ s2,$ without Next in behavior of $s1$		$(CF, s1) \Rightarrow CF1$	$CF1$	
NOTE: $s1$ often ends in a semicolon.				
		NOTE: $s2$ is statically unreachable and not recursively analyzed. $s2$ does not contribute to the uniformity analysis.		
$\text{if } e\ s1\ \text{else } s2$ with behavior {Next}		$(CF, e) \Rightarrow V$ $(V, s1) \Rightarrow CF1$	CF	
$\text{if } e\ s1\ \text{else } s2$ with another behavior	$CFend$	$(V, s2) \Rightarrow CF2$	$CFend$	$CFend \rightarrow \{CF1, CF2\}$

Statement	New nodes	Recursive analyses	Resulting control flow node	New edges
loop { <i>s</i> ₁ } Return not in behavior of <i>s</i> ₁	<i>CF'</i>	$(CF', s_1) \Rightarrow CF_1$	<i>CF</i>	$CF' \rightarrow \{CF_1, CF\}$, if behavior of <i>s</i> ₁ intersects {Next,Continue} $CF' \rightarrow CF$, if behavior of <i>s</i> ₁ does not intersect {Next,Continue}
loop { <i>s</i> ₁ } Return in behavior of <i>s</i> ₁	<i>CF'</i>	$(CF', s_1) \Rightarrow CF_1$	<i>CF</i> ₁	$CF' \rightarrow \{CF_1, CF\}$ if behavior of <i>s</i> ₁ intersects {Next,Continue} $CF' \rightarrow CF$ if behavior of <i>s</i> ₁ does not intersect {Next,Continue}
loop { <i>s</i> ₁ continuing { <i>s</i> ₂ } } <i>s</i> ₁ has behavior {Break}		$(CF, s_1) \Rightarrow CF_1$	<i>CF</i>	NOTE: The loop executes only a single iteration, so no additional edges are required.
loop { <i>s</i> ₁ continuing { <i>s</i> ₂ } } <i>s</i> ₁ has behavior {Return} or {Break,Return}		$(CF, s_1) \Rightarrow CF_1$	<i>CF</i> ₁	
loop { <i>s</i> ₁ continuing { <i>s</i> ₂ } } <i>s</i> ₁ 's behaviour intersects {Next,Continue}	<i>CF'</i>	$(CF', s_1) \Rightarrow CF_1$ $(CF_1, s_2) \Rightarrow CF_2$	<i>CF</i> if Return not in behavior of <i>s</i> ₁ <i>CF'</i> if Return in behavior of <i>s</i> ₁	$CF' \rightarrow \{CF_2, CF\}$
switch <i>e</i> case _ : <i>s</i> ₁ .. case _ : <i>s</i> _{<i>n</i>} with behavior {Next}		$(CF, e) \Rightarrow V$ $(V, s_1) \Rightarrow CF_1$	<i>CF</i>	
switch <i>e</i> case _ : <i>s</i> ₁ .. case _ : <i>s</i> _{<i>n</i>} with another behavior	<i>CF</i> _{end}	... $(V, s_n) \Rightarrow CF_n$	<i>CF</i> _{end}	$CF_{end} \rightarrow \{CF_1, \dots, CF_n\}$
var x: T; break; continue;			<i>CF</i>	NOTE: If x is a function address space variable, <i>CF</i> is used as the zero value initializer in the value analysis.

Statement	New nodes	Recursive analyses	Resulting control flow node	New edges
				$CFend \rightarrow V$
break if e ;	$CFend$	$(CF, e) \Rightarrow V$	$CFend$	NOTE: The edge from $CFend$ to V captures the fact that when the condition value is non-uniform, then control flow resulting from this break if statement is also non-uniform.
return;			CF	For each function address space pointer parameter i , Value_return_i_contents $\rightarrow Vin(prev)$ (see § 15.2.5 Function-scope Variable Value Analysis)
				Value_return $\rightarrow V$
return e ;		$(CF, e) \Rightarrow V$	CF	For each function address space pointer parameter i , Value_return_i_contents $\rightarrow Vin(prev)$ (see § 15.2.5 Function-scope Variable Value Analysis)
				$LV \rightarrow RV$
$e1 = e2$;		LHSValue : $(CF, e1) \Rightarrow LV$ $(CF, e2) \Rightarrow RV$	CF	NOTE: LV is the result value from the value analysis.
$_ = e$		$(CF, e) \Rightarrow V$	CF	
let x = e ;		$(CF, e) \Rightarrow V$	CF	

Statement	New nodes	Recursive analyses	Resulting control flow node	New edges
<code>var x = e;</code>		$(CF, e) \Rightarrow V$	<i>CF</i>	NOTE: If <i>x</i> is a function address space variable, <i>V</i> is used as the result value in the value analysis.
$f()$ Function call statement without arguments		Invoke function call analysis : $(CF, f()) \Rightarrow$ Result	<i>CF</i>	
$f(e1, \dots, eN)$ Function call statement with arguments For the purpose of this analysis:		Invoke function call analysis : $(CF, f(e1, \dots, eN)) \Rightarrow$ Result	<i>CF</i>	
- for loops get desugared (see § 9.4.4 For Statement) - while loops get desugared (see § 9.4.5 While Statement) - loop {s} is treated as loop {s continuing {}} - if statements without an else branch are treated as if they had an empty else branch, i.e. ended in else {} - if statements with else if branches are treated as if they were nested simple if/else statements - a switch_clause starting with default behaves just like a switch_clause starting with case _: To maximize performance, implementations often try to minimize the amount of non-uniform control flow. However, the points at which invocations can be said to be uniform varies depending on a number of factors. WGSL's static analysis conservatively assumes a return to uniform control flow occurring at the end of if, switch, and loop statements if the behavior for the statement is {Next}. This is modeled in the preceding table as the resulting control flow node being the same as input control flow node.				

§ 15.2.7. Uniformity Rules for Function Calls

The most complex rule is for function calls:

- Let *CF* be the control flow at the beginning of the function call expression.
- For each argument, apply the corresponding expression rule with *CF*. Name the corresponding value nodes *arg_i*
- Create a new node named **Result**
 - Add an edge from [Result](#) to *CF*
- If the [call site tag](#) of the function is [CallSiteRequiredToBeUniform.S](#), then:
 - Add an edge from [RequiredToBeUniform.S](#) to *CF*
 - Add the members of the [potential-trigger-set](#) of the [call site tag](#) to the potential-trigger-set associated with [RequiredToBeUniform.S](#).

- If the [function tag](#) is [ReturnValueMaybeNonUniform](#), then add an edge from [Result](#) to [MaybeNonUniform](#)
- For each argument *i*:
 - If the corresponding [parameter tag](#) is [ParameterRequiredToBeUniform.S](#), then:
 - Add an edge from [RequiredToBeUniform.S](#) to [arg_i](#).
 - Add the members of the [potential-trigger-set](#) of the [parameter tag](#) to the potential-trigger-set associated with [RequiredToBeUniform.S](#).
 - If the [parameter return tag](#) is [ParameterReturnContentsRequiredToBeUniform](#), then add an edge from [Result](#) to [arg_i](#)
 - If the corresponding parameter has a [pointer parameter tag](#) of [PointerParameterMaybeNonUniform](#), then add an edge from *Vout(call)* to [MaybeNonUniform](#)
 - If the parameter is a pointer in the [function](#) address space, add an edge from *Vout(call)* to each corresponding [arg_i](#) for the reachable parameters recorded previously
 - If the [parameter tag](#) is [ParameterContentsRequiredToBeUniform.S](#), add an edge from [RequiredToBeUniform.S](#) to *Vout(call)*

NOTE: Refer to [§ 15.2.5 Function-scope Variable Value Analysis](#) for the definition of *Vout(call)*.

Most built-in functions have tags of:

- A [call site tag](#) of [CallSiteNoRestriction](#).
- A [function tag](#) of [NoRestriction](#).
- For each parameter:
 - a [parameter tag](#) of [ParameterNoRestriction](#).
 - a [parameter return tag](#) of [ParameterReturnContentsRequiredToBeUniform](#).

Here is the list of exceptions:

- A call to a function in [§ 17.11 Synchronization Built-in Functions](#):
 - Has a [function tag](#) of [NoRestriction](#).
 - Has a [call site tag](#) of:
 - [CallSiteRequiredToBeUniform.error](#), with [potential-trigger-set](#) consisting of an unnamed [triggering rule](#) if [subgroup_uniformity](#) is not supported or the [uniformity scope](#) is not subgroup.

NOTE: The triggering rule has no name, and so it cannot be filtered.
 - [CallSiteNoRestriction](#) otherwise.
 - Additionally if [subgroup_uniformity](#) is not supported or the [uniformity scope](#) is not subgroup, for a call to [workgroupUniformLoad](#), the parameter *p* has a [parameter tag](#) of [ParameterRequiredToBeUniform.error](#), with [potential-trigger-set](#) consisting of an unnamed [triggering rule](#).
- A call to a function in [§ 17.6 Derivative Built-in Functions](#), [§ 17.7.8 textureSample](#), [§ 17.7.9 textureSampleBias](#), and [§ 17.7.10 textureSampleCompare](#):
 - Has a [function tag](#) of [ReturnValueMaybeNonUniform](#).
 - Has a [call site tag](#) as follows:
 - If [subgroup_uniformity](#) is not supported or the [uniformity scope](#) is not subgroup

- Let DF be the [nearest enclosing diagnostic filter](#) for the call site location and triggering rule [derivative_uniformity](#)
 - If DF exists, then let S be the DF 's new severity parameter.
 - If S is the severity [off](#), the call site tag is [CallSiteNoRestriction](#).
 - Otherwise the call site tag is [CallSiteRequiredToBeUniform.S](#), with [potential-trigger-set](#) consisting of a [derivative_uniformity](#) element.
 - If there is no such DF , the call site tag is [CallSiteRequiredToBeUniform.error](#), with [potential-trigger-set](#) consisting of a [derivative_uniformity](#) element.
 - [CallSiteNoRestriction](#) otherwise.
- A call to [§ 17.7.4 textureLoad](#):
 - Has a [call site tag](#) of [CallSiteNoRestriction](#)
 - Has a [function tag](#) as follows:
 - [ReturnValueMaybeNonUniform](#) if the argument corresponding to the t parameter is a [read-write storage texture](#)
 - [NoRestriction](#) otherwise
- A call to a function in [§ 17.12 Subgroup Built-in Functions](#) or [§ 17.13 Quad Operations](#):
 - Has a [function tag](#) of:
 - [NoRestriction](#) if the [uniformity scope](#) is subgroup and the function is one of the following:
 - [subgroupAdd](#)
 - [subgroupAll](#)
 - [subgroupAnd](#)
 - [subgroupAny](#)
 - [subgroupBallot](#)
 - [subgroupBroadcast](#)
 - [subgroupBroadcastFirst](#)
 - [subgroupMax](#)
 - [subgroupMin](#)
 - [subgroupMul](#)
 - [subgroupOr](#)
 - [subgroupXor](#)
 - [ReturnValueMaybeNonUniform](#) otherwise
 - Let DF be the [nearest enclosing diagnostic filter](#) for the call site location and triggering rule [subgroup_uniformity](#)
 - Has a [call site tag](#) as follows:
 - If [subgroup_uniformity](#) is not supported or the [uniformity scope](#) is subgroup then:
 - If DF exists, then let S be the DF 's new severity parameter.
 - If S is the severity [off](#), the call site tag is [CallSiteNoRestriction](#).
 - Otherwise, the call site tag is [CallSiteRequiredToBeUniform.S](#), with [potential-trigger-set](#) consisting of a [subgroup_uniformity](#) element.

- If there is no such DF , the call site tag is [CallSiteRequiredToBeUniform.error](#), with [potential-trigger-set](#) consisting of a [subgroup_uniformity](#) element.
- [CallSiteNoRestriction](#) otherwise.
- Additionally for the case of a call to [subgroupShuffleUp](#) or [subgroupShuffleDown](#), the parameter `delta` has a [parameter tag](#) of:
 - If [subgroup_uniformity](#) is not supported or the [uniformity scope](#) is subgroup then:
 - If DF exists, then let S be DF 's new severity parameter.
 - If S is the severity [off](#), the parameter tag is [NoRestriction](#).
 - Otherwise, the parameter tag is [ParameterRequiredToBeUniform.S](#) with [potential-trigger-set](#) consisting of a [subgroup_uniformity](#) element.
 - If there is no such DF , the parameter tag is [ParameterRequiredToBeUniform.error](#), with [potential-trigger-set](#) consisting of a [subgroup_uniformity](#) element.
 - [NoRestriction](#) otherwise.
- Additionally for the case of a call to [subgroupShuffleXor](#), the parameter `mask` has a [parameter tag](#) of:
 - If [subgroup_uniformity](#) is not supported or the [uniformity scope](#) is subgroup then:
 - If DF exists, then let S be DF 's new severity parameter.
 - If S is the severity [off](#), the parameter tag is [NoRestriction](#).
 - Otherwise, the parameter tag is [ParameterRequiredToBeUniform.S](#) with [potential-trigger-set](#) consisting of a [subgroup_uniformity](#) element.
 - If there is no such DF , the parameter tag is [ParameterRequiredToBeUniform.error](#), with [potential-trigger-set](#) consisting of a [subgroup_uniformity](#) element.
 - [NoRestriction](#) otherwise.

NOTE: A WGSL implementation will ensure that if control flow prior to a function call is [uniform](#) for a particular scope, it will also be uniform after the function call.

§ 15.2.8. Uniformity Rules for Expressions

The rules for analyzing expressions take as argument both the expression itself and the node corresponding to control flow at the beginning of it (which we'll note "CF" below) and return the following:

- A node corresponding to its value
- A set of new nodes and edges to add to the graph

Uniformity rules for [RHSValue](#) expressions

Expression	New nodes	Recursive analyses	Resulting value node	New edges
$e1 \parallel e2$		$(CF, e1) \Rightarrow V1$	$V2$	
$e1 \&\& e2$		$(V1, e2) \Rightarrow V2$		
Literal			CF	

Expression	New nodes	Recursive analyses	Resulting value node	New edges
identifier resolving to function-scope variable "x", where the identifier appears as the root identifier of a memory view expression, <i>MVE</i>, and the load rule is invoked on <i>MVE</i> during type checking	<i>Result</i>	<i>X</i> is the node corresponding to the value of "x" at the input to the statement containing this expression	<i>Result</i>	<i>Result</i> -> {<i>CF</i>, <i>X</i>} NOTE: <i>X</i> is equivalent to <i>Vout(prev)</i> for "x" (see § 15.2.5 Function-scope Variable Value Analysis)
identifier resolving to function-scope variable "x", where "x" is the desugared pointer parameter <i>i</i>, and where the identifier appears as the root identifier of a memory view expression, <i>MVE</i>, and the load rule is not invoked on <i>MVE</i> during type checking			param_i	
identifier resolving to function-scope variable "x", where the identifier appears as the root identifier of a memory view expression, <i>MVE</i>, and the load rule is not invoked on <i>MVE</i> during type checking			<i>CF</i>	
identifier resolving to const-declaration, override-declaration, let-declaration, or non-built-in formal parameter of non-pointer type "x"	<i>Result</i>	<i>X</i> is the node corresponding to "x"	<i>Result</i>	<i>Result</i> -> { <i>CF</i> , <i>X</i> }
identifier resolving to a formal parameter of pointer type in the storage, workgroup, or private address spaces with a non-read-only access mode where the identifier appears as the root identifier of a memory view expression, <i>MVE</i>, and the load rule is invoked on <i>MVE</i> during type checking			MaybeNonUniform	
identifier resolving to a formal parameter of pointer type in the storage, workgroup, or private address spaces with a non-read-			<i>CF</i>	

Expression	New nodes	Recursive analyses	Resulting value node	New edges
only access mode where the identifier appears as the root identifier of a memory view expression, <i>MVE</i> , and the load rule is not invoked on <i>MVE</i> during type checking				
identifier resolving to a formal parameter of pointer type in an address space other than function with a read-only access mode			<i>CF</i>	
identifier resolving to uniform built-in value "x"			<i>CF</i>	
identifier resolving to non-uniform built-in value "x"			MaybeNonUniform	
identifier resolving to read-only module-scope variable "x"			<i>CF</i>	
identifier resolving to non-read-only module-scope variable "x" where the identifier appears as the root identifier of a memory view expression, <i>MVE</i> , and the load rule is invoked on <i>MVE</i> during type checking			MaybeNonUniform	
identifier resolving to non-read-only module-scope variable "x" where the identifier appears as the root identifier of a memory view expression, <i>MVE</i> , and the load rule is not invoked on <i>MVE</i> during type checking			<i>CF</i>	
(<i>e</i>)				
<i>op e</i> , where <i>op</i> is a unary operator		(<i>CF</i> , <i>e</i>) => <i>V</i>	<i>V</i>	
<i>e</i> .field				
<i>e1 op e2</i> , where <i>op</i> is a non-short-circuiting binary operator	<i>Result</i>	(<i>CF</i> , <i>e1</i>) => <i>V1</i> (<i>CF</i> , <i>e2</i>) => <i>V2</i>	<i>Result</i>	<i>Result</i> -> { <i>V1</i> , <i>V2</i> }
<i>e2</i> [<i>e1</i>]				

Expression	New nodes	Recursive analyses	Resulting value node	New edges
$f()$ Function call expression without arguments		Invoke function call analysis : $(CF, f()) \Rightarrow$ Result	<i>Result</i>	
$f(e1, \dots, eN)$ Function call expression with arguments		Invoke function call analysis : $(CF, f(e1, \dots, eN)) \Rightarrow$ Result	<i>Result</i>	

The following built-in input variables are considered uniform:

- [workgroup_id](#)
- [num_workgroups](#)
- [subgroup_size](#) when used in a [compute shader stage](#)
- [num_subgroups](#)

At [subgroup uniformity scope](#), the following built-in input variables are also considered uniform:

- [subgroup_id](#)

All other ones (see [built-in values](#)) are considered non-uniform.

NOTE: An author should avoid grouping the uniform built-in values together with other non-uniform inputs because the analysis does not analyze the [components](#) of a [composite](#) type separately.

Uniformity rules for [LHSValue](#) expressions

Expression	New nodes	Recursive analyses	Resulting variable node	New edges
identifier resolving to function-scope variable "x"	<i>Result</i>	X is the node corresponding to the value of "x" at the output of the statement containing this expression.	<i>Result</i>	<i>Result</i> $\rightarrow \{CF, X\}$ NOTE: X is equivalent to $Vin(next)$ for "x" (see § 15.2.5 Function-scope Variable Value Analysis)
identifier resolving to const-declaration , override-		X is the node corresponding to "x"	X	

Expression	New nodes	Recursive analyses	Resulting variable node	New edges
declaration , let-declaration , or formal parameter "x"				
identifier resolving to module-scope variable "x"			MaybeNonUniform	
<i>e</i> .field				
* <i>e</i>		LHSValue : (CF, <i>e</i>) => <i>L1</i>	<i>L1</i>	
& <i>e</i>				
<i>e2</i> [<i>e1</i>]		(CF, <i>e1</i>) => <i>V1</i> LHSValue : (CF, <i>e2</i>) => <i>L2</i>	<i>L2</i>	<i>L2</i> -> <i>V1</i>
<i>f</i> () Function call expression without arguments		Invoke function call analysis : LHSValue : (CF, <i>f</i> ()) => Result	<i>Result</i>	
<i>f</i> (<i>e1</i> ,..., <i>eN</i>) Function call expression with arguments		Invoke function call analysis : LHSValue : (CF, <i>f</i> (<i>e1</i> ,..., <i>eN</i>)) => Result	<i>Result</i>	

15.2.9. Annotating the Uniformity of Every Point in the Control-flow

This entire subsection is non-normative.

If implementers want to provide developers with a diagnostic mode that shows for each point in the control-flow of the entire shader whether it is uniform or not (and thus whether it would be valid to call a function that requires uniformity there), we suggest the following:

- Run the (mandatory, normative) analysis described in the previous subsections, keeping the graph for every function.
- Reverse all edges in all of those graphs
- Go through each function, starting with the entry point and never visiting a function before having visited all of its callers:
 - Add an edge from [MaybeNonUniform](#) to every argument that was non-uniform in at least one caller.
 - Add an edge from [MaybeNonUniform](#) to [CF_start](#) if the function was called in non-uniform control-flow in at least one caller.
 - Look at which nodes are reachable from [MaybeNonUniform](#). Every node visited is an expression or point in the control-flow whose uniformity cannot be proven by the analysis.

Any node which is not visited by these reachability analyses can be proven to be uniform by the analysis (and so it would be safe to call a derivative or similar function there).

NOTE: The bottom-up analysis is still required, as it lets us know what edges to add to the graphs when encountering calls.

§ 15.2.10. Examples

The graphs in the subsequent example use the following conventions for nodes:

- Rectangles represent value nodes.
- Rounded rectangles represent control flow nodes.

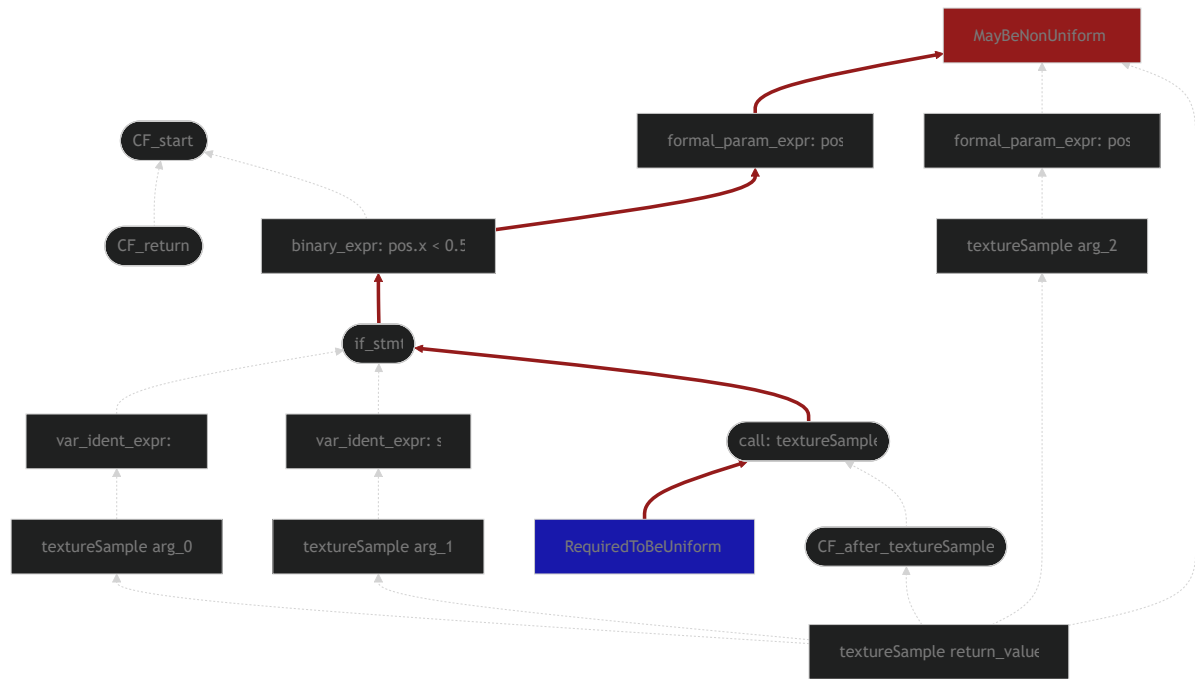
§ 15.2.10.1. Invalid `textureSample` Function Call

This example shows an invalid use of a [textureSample](#) built-in function call. The function call is made within an if statement whose condition depends on a non-uniform value (i.e. the built-in value `position`). The invalid dependency chain is highlighted in red.

EXAMPLE: WGSL INVALID TEXTURESAMPLE

```
@group(0) @binding(0) var t : texture_2d<f32>;
@group(0) @binding(1) var s : sampler;

@fragment
fn main(@builtin(position) pos : vec4<f32>) {
  if (pos.x < 0.5) {
    // Invalid textureSample function call.
    _ = textureSample(t, s, pos.xy);
  }
}
```

Figure 2 Uniformity graph

The example also shows that uniformity of the control flow after the if statement is the same as the uniformity prior to the if statement (CF_return being connected to [CF_start](#)). That is, the control flow is once again uniform after the if statement (because it is guaranteed to start as uniform control flow at the beginning of the entry point). If the textureSample function call had been moved outside the if statement the program would have been valid. Likewise, if the condition of the if statement were a uniform value (e.g. each invocation read the same value from a [uniform buffer](#)), the program would also have been valid.

15.2.10.2. Function-scope Variable Uniformity

This example shows both a valid and an invalid [barrier](#) function call that depend on the value of a function-scope variable. The `workgroupBarrier` is invalid because the value of `x` is derived from the mutable module-scope variable `a`. The `storageBarrier` is valid because the value of `x` is derived from the immutable module-scope variable `b`. This example highlights the [value analysis](#)' ability to separate different periods of uniformity in a function-scope variable's lifetime. This example also clearly shows that control flow becomes uniform again after the end of the first [if](#) statement. We know this because that section of the graph is independent from the second if statement.

EXAMPLE: WGS L USING FUNCTION VARIABLE

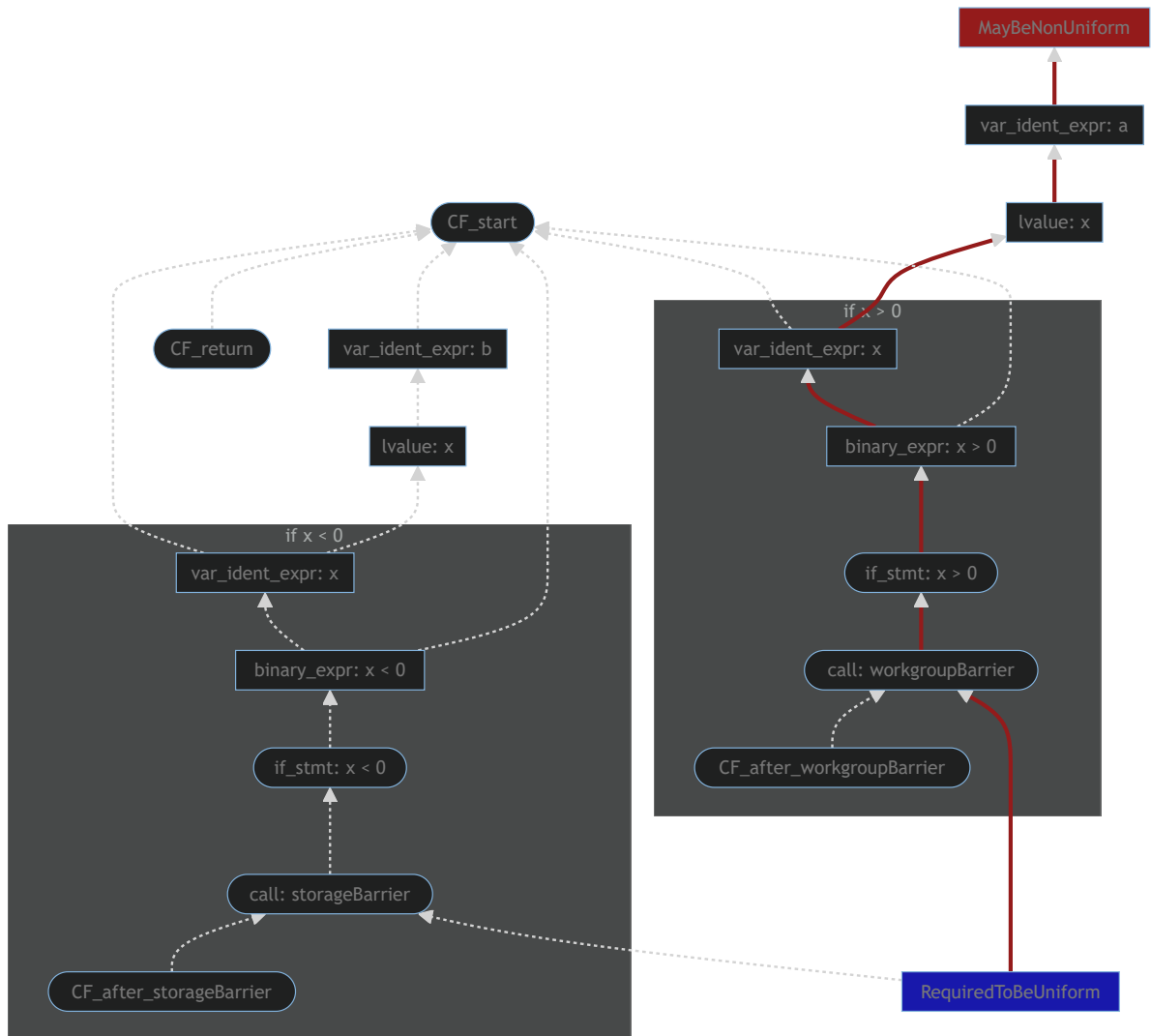
```

@group(0) @binding(0) var<storage, read_write> a : i32;
@group(0) @binding(1) var<uniform> b : i32;

@compute @workgroup_size(16,1,1)
fn main() {
    var x : i32;
    x = a;
    if x > 0 {
        // Invalid barrier function call.
        workgroupBarrier();
    }
    x = b;
    if x < 0 {
        // Valid barrier function call.
        storageBarrier();
    }
}

```

Figure 3 Uniformity graph



NOTE: The subgraphs are only included in the example for ease of understanding.

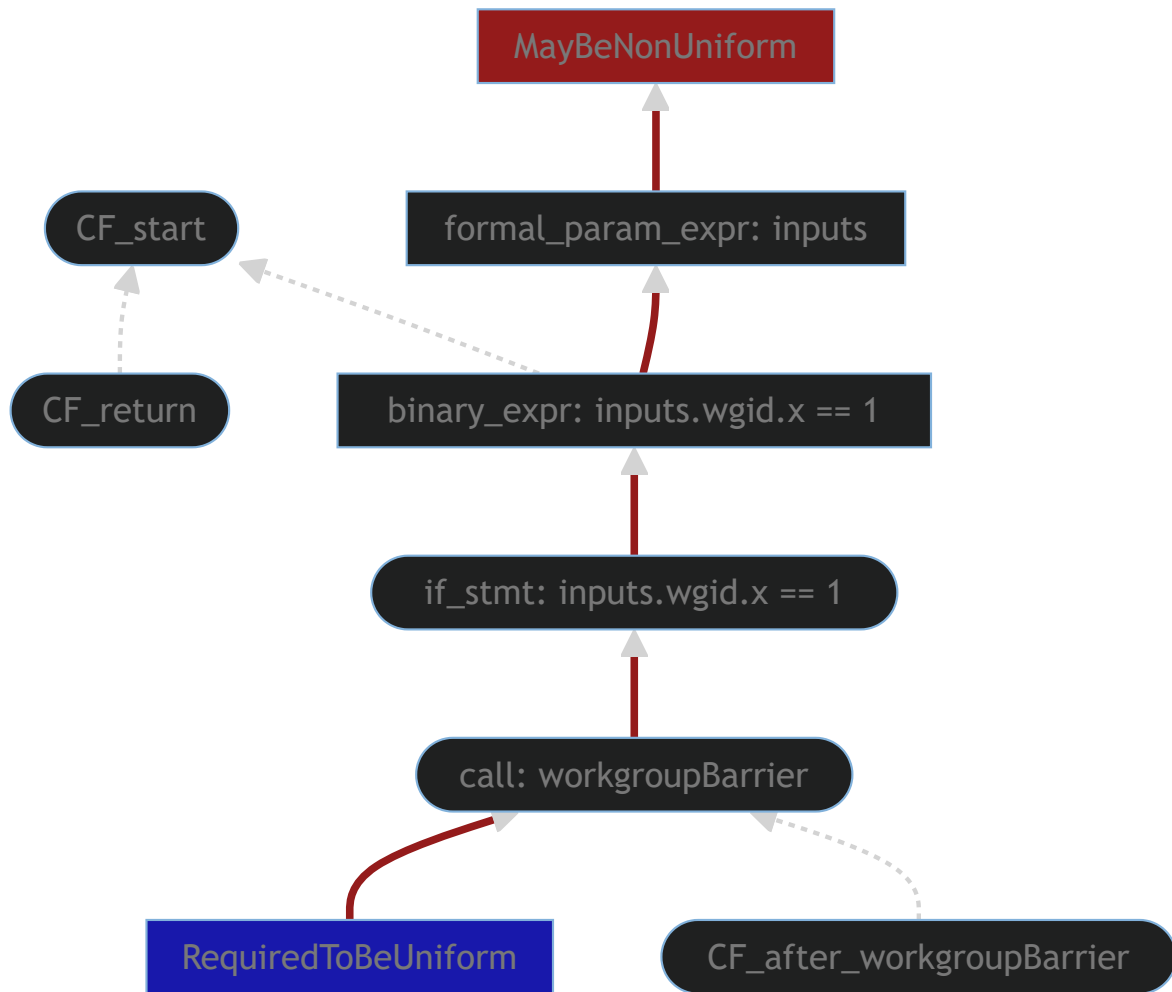
15.2.10.3. Composite Value Analysis Limitations

One limitation of the uniformity analysis is that it does not track the [components](#) of a [composite](#) value independently. That is, any non-uniform component value [will](#) cause the analysis to treat the entire composite value as non-uniform. This example illustrates this issue and a potential workaround that shader authors can employ to avoid this limitation.

EXAMPLE: INVALID COMPOSITE VALUE WGSL

```
struct Inputs {
    // workgroup_id is a uniform built-in value.
    @builtin(workgroup_id) wgid : vec3<u32>,
    // local_invocation_index is a non-uniform built-in value.
    @builtin(local_invocation_index) lid : u32
}

@compute @workgroup_size(16,1,1)
fn main(inputs : Inputs) {
    // This comparison is always uniform,
    // but the analysis cannot determine that.
    if inputs.wgid.x == 1 {
        workgroupBarrier();
    }
}
```


Figure 4 Invalid uniformity graph

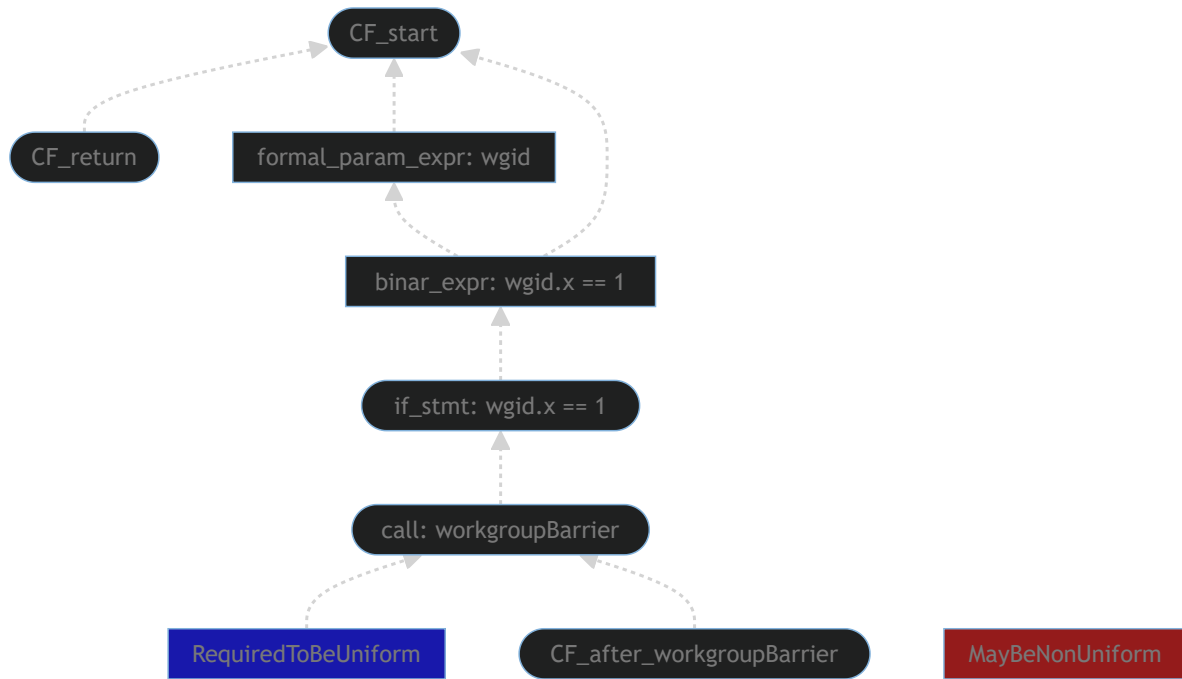
The easiest way to work around this limitation of the analysis is to split the composite up so that values that are known to be uniform are separate from value that are known to be non-uniform. In the alternative WGSL below, splitting the two built-in values into separate parameters satisfies the uniformity analysis. This can be seen by the lack of a path from [RequiredToBeUniform.S](#) to [MaybeNonUniform](#) in the graph.

EXAMPLE: VALID ALTERNATIVE WGSL

```

@compute @workgroup_size(16,1,1)
fn main(@builtin(workgroup_id) wgid : vec3<u32>,
       @builtin(local_invocation_index) lid : u32) {
    // The uniformity analysis can now correctly determine this comparison is
    // always uniform.
    if wgid.x == 1 {
        // Valid barrier function call.
        workgroupBarrier();
    }
}

```

Figure 5 Valid alternative uniformity graph

15.2.10.4. Uniformity in a Loop

In this example, there is an invalid `workgroupBarrier` function call in a loop. The non-uniform built-in value `local_invocation_index` is the ultimate cause despite the fact that it appears after the barrier in the loop. This occurs, because on later iterations some of the invocations in the workgroup will have exited the loop prematurely while others attempt to execute the barrier. The analysis models the inter-iteration dependencies as an edge, where the control at the start of the loop body (`CF_loop_body`) depends on the control flow at the end of the loop body (`CF_after_if`).

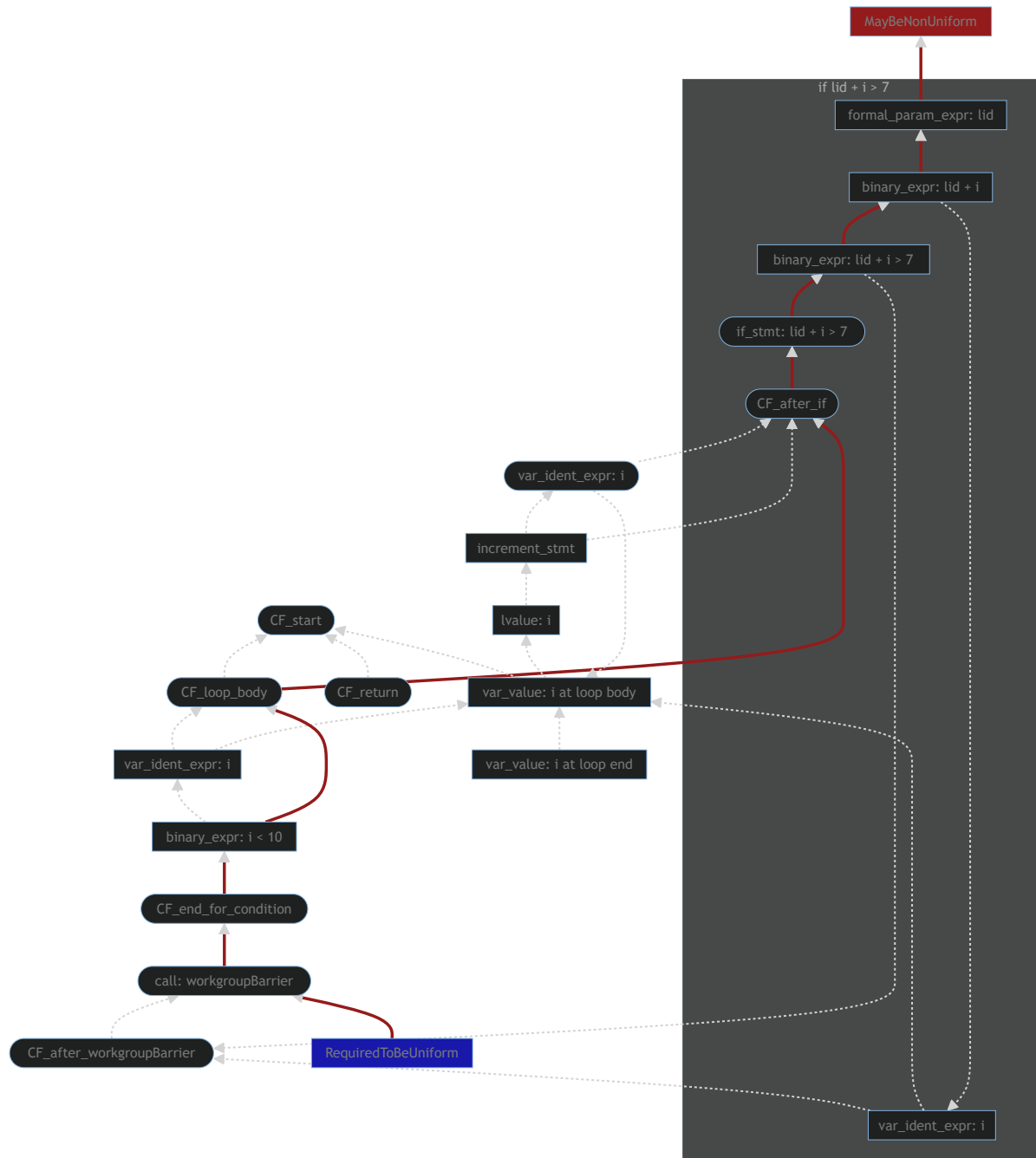
EXAMPLE: LOOP UNIFORMITY WGSL

```

@compute @workgroup_size(16,1,1)
fn main(@builtin(local_invocation_index) lid : u32) {
  for (var i = 0u; i < 10; i++) {
    workgroupBarrier();
    if (lid + i) > 7 {
      break;
    }
  }
}

```

Figure 6 Uniformity graph



15.2.10.5. User-defined Function Calls

This example is modification of the [first example](#), but uses a user-defined function call. The analysis sets the [parameter return tag](#) of both parameters of `scale` as [ParameterReturnContentsRequiredToBeUniform](#). This leads to the path in `main` between the return value of the `scale` function call and the `position` built-in value. That path is a subpath of the overall invalid path from [RequiredToBeUniform.S](#) to [MayBeNonUniform](#).

EXAMPLE: USER-DEFINED FUNCTION CALL UNIFORMITY WGS

```

fn scale(in1 : f32, in2 : f32) -> f32 {
    let v = in1 / in2;
    return v;
}

@group(0) @binding(0) var t : texture_2d<f32>;
@group(0) @binding(1) var s : sampler;

@fragment
fn main(@builtin(position) pos : vec4<f32>) {
    let tmp = scale(pos.x, 0.5);
    if tmp > 1.0 {
        _ = textureSample(t, s, pos.xy);
    }
}

```

Figure 7 Uniformity graph for scale

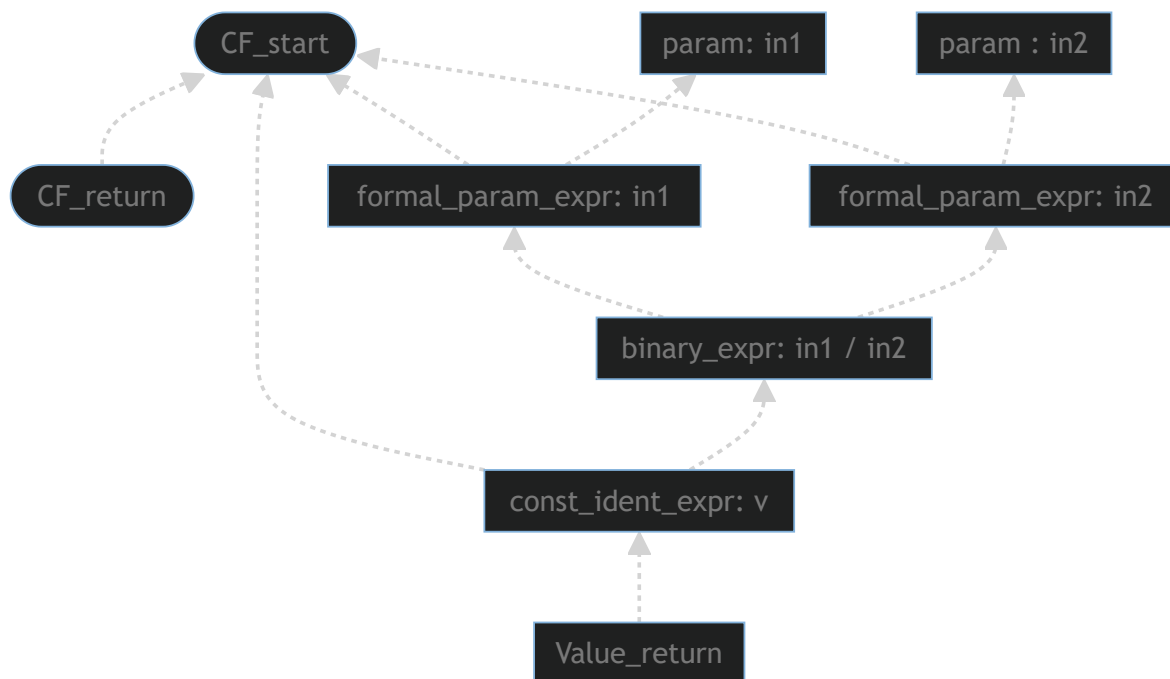
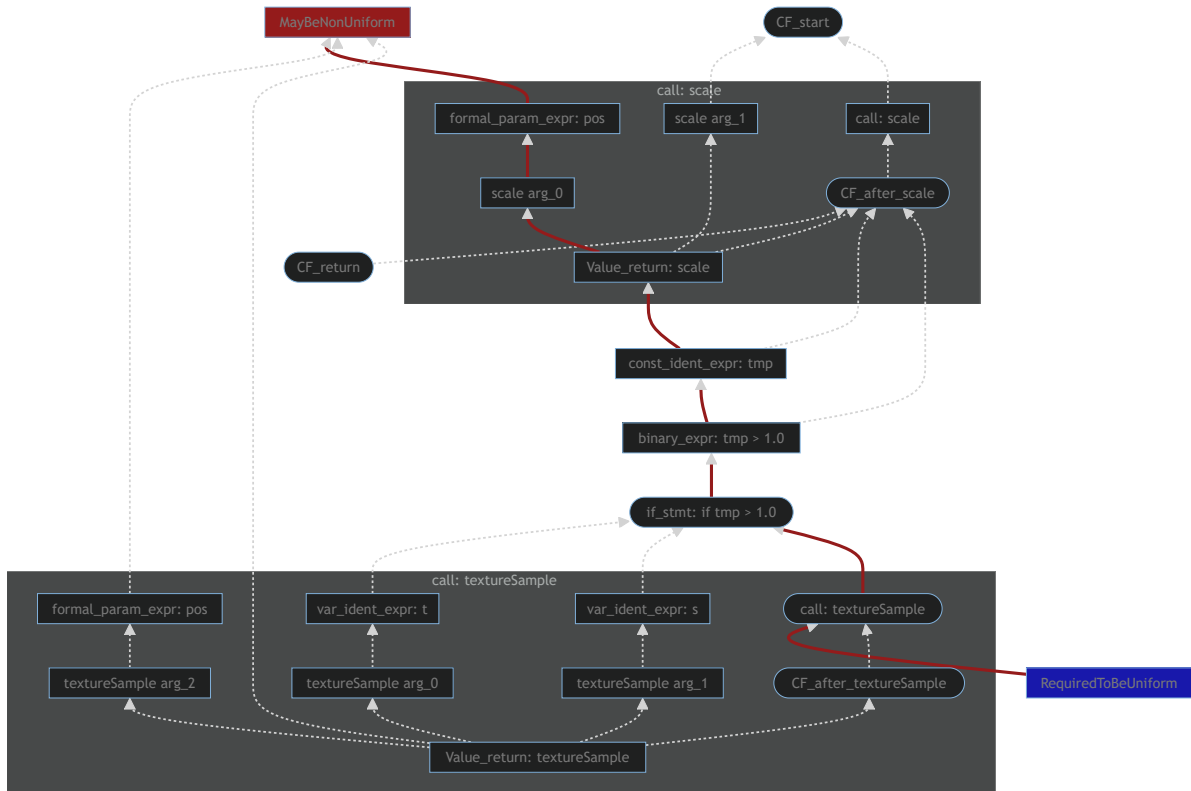


Figure 8 Uniformity graph for main



NOTE: The subgraphs are only included in the example for ease of understanding.

15.3. Compute Shaders and Workgroups

A **workgroup** is a set of invocations which concurrently execute a [compute shader stage entry point](#), and share access to shader variables in the [workgroup](#) address space.

The **workgroup grid** for a compute shader is the set of points with integer coordinates (i,j,k) with:

- $0 \leq i < \text{workgroup_size_x}$
- $0 \leq j < \text{workgroup_size_y}$
- $0 \leq k < \text{workgroup_size_z}$

where $(\text{workgroup_size_x}, \text{workgroup_size_y}, \text{workgroup_size_z})$ is the value specified for the [workgroup_size](#) attribute of the entry point.

There is exactly one invocation in a workgroup for each point in the workgroup grid.

An invocation's **local invocation ID** is the coordinate triple (i,j,k) for the invocation's corresponding workgroup grid point.

When an invocation has [local invocation ID](#), then its **local invocation index** is:

$$i + (j \times \text{workgroup_size_x}) + (k \times \text{workgroup_size_x} \times \text{workgroup_size_y})$$

Note that if a workgroup has W invocations, then each invocation I the workgroup has a unique local invocation index $L(I)$ such that $0 \leq L(I) < W$, and that entire range is covered.

A compute shader begins execution when a WebGPU implementation removes a dispatch command from a queue and begins the specified work on the GPU. The dispatch command specifies a *dispatch size*, which is an integer triple (*group_count_x*, *group_count_y*, *group_count_z*) indicating the number of workgroups to be executed, as described in the following.

The *compute shader grid* for a particular dispatch is the set of points with integer coordinates (*CSi*, *CSj*, *CSk*) with:

- $0 \leq CSi < \text{workgroup_size_x} \times \text{group_count_x}$
- $0 \leq CSj < \text{workgroup_size_y} \times \text{group_count_y}$
- $0 \leq CSk < \text{workgroup_size_z} \times \text{group_count_z}$

where *workgroup_size_x*, *workgroup_size_y*, and *workgroup_size_z* are as above for the compute shader entry point.

The work to be performed by a compute shader dispatch is to execute exactly one invocation of the entry point for each point in the compute shader grid.

An invocation's *global invocation ID* is the coordinate triple for the invocation's corresponding compute shader grid point.

The invocations are organized into workgroups, so that each invocation's *global invocation ID* (*CSi*, *CSj*, *CSk*) maps to a single workgroup, identified by *workgroup ID*:

$$(\lfloor CSi \div \text{workgroup_size_x} \rfloor, \lfloor CSj \div \text{workgroup_size_y} \rfloor, \lfloor CSk \div \text{workgroup_size_z} \rfloor)$$

and a single invocation within that workgroup, identified by *local invocation ID*:

$$(CSi \bmod \text{workgroup_size_x}, CSj \bmod \text{workgroup_size_y}, CSk \bmod \text{workgroup_size_z}).$$

NOTE: Workgroup IDs span from (0,0,0) to (*group_count_x* - 1, *group_count_y* - 1, *group_count_z* - 1).

If the *linear indexing* feature is supported, then when a workgroup has a *workgroup ID* (*WGi*, *WGj*, *WGk*), then its *workgroup index* is:

$$WGi + (WGj \times \text{group_count_x}) + (WGk \times \text{group_count_x} \times \text{group_count_y})$$

If the *linear indexing* feature is supported, then when an invocation has *global invocation ID* (*CSi*, *CSj*, *CSk*), then its *global invocation index* is:

$$CSi + (CSj \times \text{workgroup_size_x} \times \text{group_count_x}) + (CSk \times \text{workgroup_size_x} \times \text{group_count_x} \times \text{workgroup_size_y} \times \text{group_count_y})$$

WebGPU provides no guarantees about:

- Whether invocations from different workgroups execute concurrently. That is, you cannot assume more than one workgroup executes at a time.
- Whether, once invocations from a workgroup begin executing, that other workgroups are blocked from execution. That is, you cannot assume that only one workgroup executes at a time. While a workgroup is executing, the implementation may choose to concurrently execute other workgroups as well, or other queued but unblocked work.
- Whether invocations from one particular workgroup begin executing before the invocations of another workgroup. That is, you cannot assume that workgroups are launched in a particular order.

§ 15.4. Fragment Shaders and Helper Invocations

Invocations in the *fragment shader stage* are divided into 2x2 grids of invocations with neighbouring *positions* in the X and Y dimensions. Each of these grids is referred to as a *quad*. Quads can collaborate in some collective operations (see § 15.6.2

[Derivatives](#)). An invocation's *quad invocation ID* is the unique ID within the quad, where:

- ID 0 is the upper-left invocation.
- ID 1 is the upper-right invocation.
- ID 2 is the lower-left invocation.
- ID 3 is the lower-right invocation.

NOTE: There is no built-in value accessor for the quad ID.

Ordinarily, [fragment processing](#) creates one invocation of a fragment shader for each [RasterizationPoint](#) produced by [rasterization](#). Sometimes there may be insufficient RasterizationPoints to fully populate a quad, for example at the edge of a graphics primitive. When a quad has only 1, 2, or 3 invocations corresponding to RasterizationPoints, fragment processing [will](#) create a *helper invocation* for each unpopulated position in the quad.

Helper invocations have limited observable effects. They help compute [derivatives](#) and may participate in [subgroup operations](#). As such, helper invocations are subject to the following restrictions:

- No [write accesses](#) (see also [§ 14.5.1 Memory Operation](#)) [will](#) be performed on the [storage](#) or [handle](#) address spaces.
- [Atomic built-in functions](#) [will](#) return [indeterminate](#) results.
- The [Entry point return value](#) [will](#) not be further processed downstream in the [GPURenderPipeline](#).

If all of the invocations in a quad become helper invocations (e.g. due to executing a [discard](#) statement), execution of the quad may be terminated; however, such termination is not considered to produce [non-uniform control flow](#).

§ 15.5. Subgroups

A *subgroup* is a set of invocations which concurrently execute a [compute](#) or [fragment](#) shader stage [entry point](#), and which can efficiently share data and [collectively](#) compute results. Each invocation in a compute or fragment shader belongs to exactly one subgroup. In a compute shader, each subgroup is a subset of a particular [workgroup](#). In a fragment shader, a subgroup might contain invocations from multiple [draw commands](#). Each [quad](#) will be contained in a single subgroup.

The *subgroup size* is the maximum number of invocations in a subgroup. Within a shader, the value is accessible via the [subgroup_size](#) built-in value. The subgroup size is a [uniform value](#) within a [dispatch command](#) and hence within a [workgroup](#), but it may not be a uniform value within a [draw command](#). All subgroup sizes are powers of two within the range [4, 128], and the value for a shader compiled for a specific device will be within the range [[subgroupMinSize](#), [subgroupMaxSize](#)] for the [WebGPU § 4.3 GPUAdapter](#). The actual size depends on the shader, device properties, and the device compiler. Each device supports a subset of the possible range of subgroup sizes (possibly a single value). The device compiler selects a size from the supported sizes using a variety of heuristics. Each subgroup may contain fewer invocations than the reported subgroup size (e.g. if fewer invocations than the subgroup size are launched).

An invocation's *subgroup invocation ID* is a unique ID within the subgroup. This ID is accessible via the [subgroup_invocation_id](#) built-in value and is in the range [0, [subgroup_size](#) - 1].

If the [subgroup_id](#) feature is supported, within a compute shader the *subgroup ID* is the unique ID for the subgroup within the workgroup. This ID is accessible via the [subgroup_id](#) built-in value and is in the range [0, [num_subgroups](#) - 1].

There is no defined relationship between subgroup values (i.e. [subgroup_invocation_id](#) and [subgroup_id](#)) and [local_invocation_index](#). To avoid non-portable code, shader authors should not assume a particular mapping between these two values.

When invocations in the same subgroup execute different control flow paths, we say subgroup execution has diverged. This is a special case of [non-uniform control flow](#). Divergence affects the semantics of subgroup operations. The invocations in a

subgroup that concurrently execute a subgroup operation are *active* for that operation. Other invocations in the subgroup are *inactive* for that operation. When the subgroup size exceeds the number of invocations in a subgroup, the extra hypothetical invocations are considered inactive. [Helper invocations](#) may be active or inactive in an operation. That is, on some devices helper invocations may participate in subgroup operations, while on other devices they may not.

NOTE: There is considerable non-portability among underlying devices when operating in non-uniform control flow, and device compilers often aggressively optimize such code. The result is that the subgroup may contain a different set of active invocations than the shader author expects.

§ 15.6. Collective Operations

§ 15.6.1. Barriers

A barrier is a [synchronization built-in function](#) that orders memory operations in a program. A *control barrier* is executed by all invocations in the same [workgroup](#) as if it were executed concurrently. As such, control barriers [must](#) only be executed in [uniform control flow](#) in a [compute](#) shader.

§ 15.6.2. Derivatives

A *partial derivative* is the rate of change of a value along an axis. Fragment shader invocations within the same [quad](#) collaborate to compute approximate partial derivatives.

The *builtin functions that compute a derivative* are:

- [textureSample](#), [textureSampleBias](#), and [textureSampleCompare](#)
- [dpx](#), [dpxCoarse](#), and [dpxFine](#)
- [dpy](#), [dpyCoarse](#), and [dpyFine](#)
- [fwidth](#), [fwidthCoarse](#), and [fwidthFine](#)

Partial derivatives of the *fragment coordinate* are computed implicitly as part of operation of the following built-in functions:

- [textureSample](#),
- [textureSampleBias](#), and
- [textureSampleCompare](#).

For these, the derivatives help determine the mip levels of texels to be sampled, or in the case of `textureSampleCompare`, sampled and compared against a reference value.

Partial derivatives of *invocation-specified* values are computed by the built-in functions described in [§ 17.6 Derivative Built-in Functions](#):

- [dpx](#), [dpxCoarse](#), and [dpxFine](#) compute partial derivatives along the x axis.
- [dpy](#), [dpyCoarse](#), and [dpyFine](#) compute partial derivatives along the y axis.
- [fwidth](#), [fwidthCoarse](#), and [fwidthFine](#) compute the Manhattan metric over the associated x and y partial derivatives.

Because neighbouring invocations collaborate to compute derivatives, these functions should only be invoked in [uniform control flow](#) in a fragment shader. For each call to one of these functions, a [derivative_uniformity_diagnostic](#) is triggered if [uniformity analysis](#) cannot prove the call occurs in uniform control flow.

If one of these functions is called in non-uniform control flow, then the result is an [indeterminate value](#).

NOTE: Derivatives are an implicit type of [quad operation](#). Their use does not require the [subgroups](#) extension.

§ 15.6.3. Subgroup Operations

The [subgroup built-in functions](#) allow efficient communication and computation between the invocations in a [subgroup](#). Subgroup operations are single-instruction multiple-thread (SIMT) operations.

The [active](#) invocations in a subgroup communicate to determine results. Therefore, portability is maximized when these functions are invoked when all invocations are active (i.e. in [uniform control flow](#) at the subgroup level).

§ 15.6.4. Quad Operations

The [quad built-in functions](#) operate on a [quad](#) of invocations. They are useful for data communication among the [quad](#).

The [active](#) invocations in a quad communicate to determine results. Therefore, portability is maximized when these functions are invoked when all invocations are active (i.e. in [uniform control flow](#) at the quad level).

§ 15.7. Floating Point Evaluation

WGSL floating point features are based on the [IEEE-754](#) standard for floating point, but with reduced functionality reflecting the compromises made by GPUs, and with some additional guardrails for portability.

§ 15.7.1. Overview of IEEE-754

WGSL floating point types are based on [IEEE-754](#) binary floating point types.

An [IEEE-754](#) binary floating point type approximates the [extended real](#) number line as follows:

- The type has a finite set of values, including distinct categories for:
 - Positive and negative rational numbers.
 - Each of these is finite, and is either [normal](#) or [subnormal](#) as defined below.
 - [Positive infinity](#) and [negative infinity](#).
 - NaN values. A *NaN*, short for "Not a Number", represents the result of an [invalid operation](#). IEEE-754 requires both signalling and quiet NaNs, for distinguishing cases related to error reporting. WGSL does not require such error reporting, and may yield [indeterminate values](#) instead of NaNs. See [§ 15.7.2 Differences from IEEE-754](#).
- The type supports operations including:
 - Basic arithmetic such as: addition (+), subtraction (-), multiplication (*), and division (/).
 - [Scalar conversion](#) to and from other numeric types.
 - Built-in functions such as: [max](#), [sqrt](#), [cos](#)
 - Note: Infinities are ordinary participants in most operations. For example adding $+\infty$ to 5 produces $+\infty$.
- The type has a bit representation characterized by:
 - A fixed bit width, where each value's bit representation has three contiguous bit fields, ordered from most significant bit to least:

- A 1-bit *sign field*.
- A fixed-width *exponent field*.
- A fixed-width *trailing significand field*.
- An integer-valued *exponent bias* related to interpretation of the [exponent field](#).

The *finite range* of a floating point type is the [interval](#) $[low, high]$, where *low* is the lowest finite value in the type, and *high* is the highest finite value in the type.

The IEEE-754 floating point types of interest are:

- **binary16:**
 - [exponent field](#) width 5
 - [trailing significand field](#) width 10
 - [exponent bias](#) 15
 - [finite range](#): $[-65504, 65504]$
- **binary32:**
 - [exponent field](#) width 8
 - [trailing significand field](#) width 23
 - [exponent bias](#) 127
 - [finite range](#): $[-(2 - 2^{-23}) \times 2^{127}, (2 - 2^{-23}) \times 2^{127}]$, or approximately $[-3.4028235 \times 10^{38}, 3.4028235 \times 10^{38}]$.
- **binary64:**
 - [exponent field](#) width 11
 - [trailing significand field](#) width 52
 - [exponent bias](#) 1023
 - [finite range](#): $[-(2 - 2^{-52}) \times 2^{1023}, (2 - 2^{-52}) \times 2^{1023}]$, or approximately $[-1.798 \times 10^{308}, 1.798 \times 10^{308}]$.

The following algorithm maps a bit representation of a floating point value to its corresponding [extended real](#) value, or NaN:

Algorithm: Floating point interpretation of bits

Input: *Bits*, a bit representation for a value in a binary floating point type.

Output: *F*, the floating point value represented by *Bits*.

Procedure:

- Let *bias* be the [exponent bias](#) for the type.
- Let *tsw* be the bit width of the [trailing significand field](#) for the type.
- Partition *Bits* into the [sign field](#), [exponent field](#), and the [trailing significand field](#).
- Let *Sign*, *E*, and *T* be the interpretations of those respective fields as unsigned integers.
- If the [exponent field](#) is all 1 bits, then:
 - The result $F = +\infty$ when $Sign = 0$ and $T = 0$.
 - The result $F = -\infty$ when $Sign = 1$ and $T = 0$.
 - The result F is a [NaN](#) when $T \neq 0$.
- Otherwise, if the [exponent field](#) is all 0 bits, then:
 - The result $F = (-1)^{Sign} \times 2^{-bias} \times T \times 2^{-tsw+1}$.
 - If $T = 0$, then the value is a zero.
 - Each floating point type has both a positive zero and a negative zero. A **negative zero** is a zero value with 1 for its [sign](#) bit. Negative zero and positive zero compare as equal. IEEE-754 uses negative zero to indicate certain edge cases unimportant to WGSL.
 - If $T \neq 0$, then the value F is **subnormal**. (**Denormalized** is a synonym for subnormal.)
- Otherwise, the [exponent field](#) is neither all 1 bits, nor all 0 bits:
 - The result $F = (-1)^{Sign} \times 2^{(E-bias)} \times (1 + T \times 2^{-tsw})$.
 - The value F is **normal**.

The **domain** of a floating point operation is the set of [extended real](#) number inputs for which the operation is well defined.

- For example, the domain of the mathematical function $\sqrt{}$ is the interval $[0, +\infty]$: $\sqrt{}$ is not well defined for inputs less than zero.
- When evaluated *inside* its [domain](#), an operation is defined in terms of an infinitely precise [extended real intermediate result](#), which is then converted to a floating point result, via [rounding](#).
- When evaluated *outside* its [domain](#), the default exception handling rules of IEEE-754 require an implementation to generate an [exception](#) and yield a [NaN](#) value. In contrast, WGSL does not mandate floating point exceptions, and may instead yield an [indeterminate value](#). See [§ 15.7.2 Differences from IEEE-754](#).

Rounding maps an [extended real](#) value x to a value x' in the floating point type. When x is in the floating point type, then rounding maps x to itself: $x = x'$. Rounding may [overflow](#) when x is outside the [finite range](#) of the type. Otherwise x' is the either the lowest floating point value above x , or the highest floating point value below x ; a **rounding mode** determines which one is chosen.

Generally, an operation with a [NaN](#) input will yield a [NaN](#) output. Exceptions include:

- A NaN is never equal to, less than, or greater than any other floating point value. Such comparisons yield false.

IEEE-754 defines five kinds of **exceptions**:

- **Invalid operation.** This occurs when an operation is evaluated on [extended real](#) inputs outside its [domain](#). Such operations yield a NaN. Examples of invalid operations are $0 \times +\infty$, and $\text{sqrt}(-1)$.
- **Division by zero.** This occurs when an operation on finite operands is defined as having an exact infinite result. Examples are $1 \div 0$, and $\log(0)$.
- **Overflow.** This occurs when an [intermediate result](#) exceeds the [finite range](#) of the type. See [§ 15.7.3 Floating Point Rounding and Overflow](#).
- **Underflow.** This occurs when the [intermediate result](#) or the rounded result is [subnormal](#).
- **Inexact.** This occurs when the rounded result is different from the [intermediate result](#), or when overflow occurs.

§ 15.7.2. Differences from IEEE-754

WGSL follows the [IEEE-754](#) standard, but with the following differences:

- No [rounding mode](#) is specified. An implementation may round an [intermediate result](#) up or down.
- When [converting](#) a floating point value x to an integer type, x is first clamped to the value range of the target type. See [§ 15.7.6 Floating Point Conversion](#).
- No floating point [exceptions](#) are generated.
 - A floating point operation in WGSL [will](#) produce an [intermediate result](#) according to IEEE-754 rules, but exceptions mandated by IEEE-754 will map to different behaviors depending on whether the expression is a [const-expression](#), an [override-expression](#), or a [runtime expression](#).
 - Consider an operation on finite operands. The [intermediate result](#) for an operation produces overflow, infinity, or a NaN if and only if IEEE-754 would require the operation to signal an [overflow](#), [invalid operation](#), or [division by zero](#) exception. Behavior is further modified by the [Finite Math Assumption](#).
- Signaling NaNs may not be generated. In an intermediate calculation, any signaling NaN may be converted to a quiet NaN.
- **Finite Math Assumption:**
 - [Overflow](#), infinities, and NaNs generated before [shader execution will](#) generate errors.
 - [Const-expressions](#) and [override-expressions](#) over finite values [will](#) generate overflow, infinities, and NaNs as [intermediate result](#) values, following IEEE-754 rules.

▪ **NOTE:** This rule requires implementations to reliably detect overflow, infinities, and NaNs to within accuracy limits for these kinds of expressions, so that errors can be generated consistently.
 - A [shader-creation error](#) results if any [const-expression](#) of floating-point type overflows or evaluates to NaN or infinity.
 - A [pipeline-creation error](#) results if any [override-expression](#) of floating-point type overflows or evaluates to NaN or infinity.
 - Implementations may assume that overflow, infinities, and NaNs are not present during [shader execution](#).
 - In such an implementation, if the [intermediate result](#) of evaluating a [runtime expression](#) overflows, or yields an infinity or a NaN, the final result [will](#) be an [indeterminate value](#) of the target type.

▪ **NOTE:** This means some functions (e.g. `min` and `max`) may not return the expected result due to optimizations about the presence of NaNs and infinities.
- Implementations may ignore the [sign field](#) of a floating point zero value. That is, a zero with a positive sign may behave like a zero with a negative sign, and vice versa.

- To *flush to zero* is to replace a [subnormal](#) value for a floating point type with a zero value of that type.
 - Any inputs or outputs of operations listed in [§ 15.7.4 Floating Point Accuracy](#) may be flushed to zero.
 - Additionally, [intermediate result](#) values of operations listed in [§ 17.2 Bit Reinterpretation Built-in Functions](#), [§ 17.9 Data Packing Built-in Functions](#), or [§ 17.10 Data Unpacking Built-in Functions](#) may be flushed to zero.
 - Other operations are required to preserve [subnormal](#) numbers.
- The accuracy of operations is given in [§ 15.7.4 Floating Point Accuracy](#).
- Some built-in functions in WGSL have different semantics from the corresponding IEEE-754 operation. Such cases are listed as needed at the definition of the WGSL built-in function.

For example the WGSL [§ 17.5.32 fma](#) function may expand to an ordinary multiply (including a rounding step) and an add (and another rounding step), while the IEEE-754 fusedMultiplyAdd operation requires that only final rounding step occurs.

§ 15.7.3. Floating Point Rounding and Overflow

Overflowing computations can round to infinity or to the nearest finite value. The outcome depends on the magnitude of the overflowing [intermediate result](#) value and on whether evaluation occurs during [shader module creation](#), [pipeline creation](#), or during [shader execution](#).

For a floating point type T , define $MAX(T)$ as the largest positive finite value of T , and $2^{EMAX(T)}$ as the largest power of 2 representable by T . In particular, $EMAX(\text{f32}) = 127$, and $EMAX(\text{f16}) = 15$.

Let X be an infinitely precise [intermediate result](#) from a floating point computation. The final value of the expression is determined in two stages, via [intermediate result](#) values X' and X'' as follows:

From X , compute X' in T by rounding:

- If X is in the [finite range](#) of T then X' is the result of rounding X up or down.
- If X is NaN, then X' is NaN.
- If $MAX(T) < X < 2^{EMAX(T)+1}$, then either rounding direction is used: X' is $MAX(T)$ or $+\infty$.
- If $2^{EMAX(T)+1} \leq X$, then $X' = +\infty$.
- **NOTE:** This clause matches the [IEEE-754](#) rule.
- If $-MAX(T) > X > -2^{EMAX(T)+1}$, then either rounding direction is used: X' is $-MAX(T)$ or $-\infty$.
- If $-2^{EMAX(T)+1} \geq X$, then $X' = -\infty$.

- **NOTE:** This clause matches the IEEE-754 rule.

From X' , compute the final value of the expression, X'' , or detect a program error:

- If X' is an infinity or NaN, then by the [Finite Math Assumption](#):
 - If the expression is a [const-expression](#), generate a [shader-creation error](#).
 - If the expression is a [override-expression](#), generate a [pipeline-creation error](#).
 - Otherwise the expression is a [runtime expression](#) and X'' is an [indeterminate value](#).
- Otherwise $X'' = X'$.

§ 15.7.4. Floating Point Accuracy

Let x be the exact real-valued or infinite result of an operation when computed with unbounded precision. The **correctly rounded** result of the operation for floating point type T is:

- x , when x is in T ,
- Otherwise:
 - the smallest value in T greater than x , or
 - the largest value in T less than x .

That is, the result may be [rounded](#) up or down: WGSL does not specify a [rounding mode](#).

NOTE: Floating point types include positive and negative infinity, so the correctly rounded result may be finite or infinite.

NOTE: The result of an operation when computed with unbounded precision may require precision which exceeds that of a double. An example of such a case is in $x - y$ where $x=1.0$ and $y=1.17e-38$ (the smallest positive normal single precision float). These numbers have exponents that are 126 units apart. The [IEEE-754 binary64](#) (double precision) format only has 52 bits in its significand, so in doing the subtraction, all the significant bits of y are lost. Depending on rounding modes, for this and many other cases where y is small but non-zero, the WGSL expression $x - y$ may yield the same value as x . Note that [ECMAScript](#) uses the equivalent of the [IEEE-754](#) roundTiesToEven rounding mode.

The units in the last place, **ULP**, for a floating point number x is defined as follows [\[Muller2005\]](#):

- If x is in the [finite range](#) of the floating point type, then $ULP(x)$ is the minimum distance between two non-equal, finite floating point numbers a and b such that $a \leq x \leq b$ (i.e. $ulp(x) = \min_{a,b} |b - a|$).
- Otherwise, $ULP(x)$ is $|b - a|$ where b and a are the largest and second-largest representable finite floating point values.

The accuracy of an operation is provided among five possibilities:

- Correct result (for non-floating point result values).
- [Correctly rounded](#).
- An absolute error bound.
- A relative error bound expressed as [ULP](#).
- An expression that the accuracy is **inherited from**. That is, the accuracy of the operation is defined as the accuracy of evaluating the given WGSL expression. The given expression is only one valid implementation of the function.

When evaluating the inherited-from expression, subexpression evaluations are subject to other rules for floating-point evaluation, including those regarding [rounding](#), [overflow](#), [reassociation](#), [fusion](#), and [flush-to-zero](#).

A WebGPU implementation may implement the operation differently, with better accuracy or with greater tolerance for extreme inputs.

When the accuracy for an operation is specified over an input range, the accuracy is undefined for input values outside that range.

If an allowed result is outside the [finite range](#) of the result type, then the rules in [§ 15.7.3 Floating Point Rounding and Overflow](#) apply.

15.7.4.1. Accuracy of Concrete Floating Point Expressions

Accuracy of concrete floating point operations

Expression	Accuracy for f32	Accuracy for f16
$x + y$	Correctly rounded	
$x - y$	Correctly rounded	
$x * y$	Correctly rounded	
x / y	2.5 ULP for $ y $ in the range $[2^{-126}, 2^{126}]$	2.5 ULP for $ y $ in the range $[2^{-14}, 2^{14}]$
$x \% y$	Inherited from $x - y * \text{trunc}(x/y)$	
$-x$	Correctly rounded	
$x == y$	Correct result	
$x != y$	Correct result	
$x < y$	Correct result	
$x <= y$	Correct result	
$x > y$	Correct result	
$x >= y$	Correct result	

Accuracy of concrete floating point built-in functions

Built-in Function	Accuracy for f32	Accuracy for f16
$\text{abs}(x)$	Correctly rounded	
$\text{acos}(x)$	The worse of: - Absolute error 6.77×10^{-5} - Inherited from $\text{atan2}(\text{sqrt}(1.0 - x * x), x)$	The worse of: - Absolute error 3.91×10^{-3} - Inherited from $\text{atan2}(\text{sqrt}(1.0 - x * x), x)$
$\text{acosh}(x)$	Inherited from $\log(x + \text{sqrt}(x * x - 1.0))$	
$\text{asin}(x)$	The worse of: - Absolute error 6.81×10^{-5} - Inherited from $\text{atan2}(x, \text{sqrt}(1.0 - x * x))$	The worse of: - Absolute error 3.91×10^{-3} - Inherited from $\text{atan2}(x, \text{sqrt}(1.0 - x * x))$
$\text{asinh}(x)$	Inherited from $\log(x + \text{sqrt}(x * x + 1.0))$	
$\text{atan}(x)$	4096 ULP	5 ULP

Built-in Function	Accuracy for f32	Accuracy for f16
<code>atan2(y, x)</code>	4096 ULP for $ x $ in the range $[2^{-126}, 2^{126}]$, and y is finite and normal	5 ULP for $ x $ in the range $[2^{-14}, 2^{14}]$, and y is finite and normal
<code>atanh(x)</code>	Inherited from $\log((1.0 + x) / (1.0 - x)) * 0.5$	
<code>ceil(x)</code>	Correctly rounded	
<code>clamp(x, low, high)</code>	Correctly rounded. The infinitely precise result is computed as either $\min(\max(x, \text{low}), \text{high})$, or with a median-of-3-values formulation. These may differ when $\text{low} > \text{high}$. If x and either low or high are subnormal, the result may be any of the subnormal values. This follows from the possible results from the <code>min</code> and <code>max</code> functions on subnormal inputs.	
<code>cos(x)</code>	Absolute error at most 2^{-11} when x is in the interval $[-\pi, \pi]$	Absolute error at most 2^{-7} when x is in the interval $[-\pi, \pi]$
<code>cosh(x)</code>	Inherited from $(\exp(x) + \exp(-x)) * 0.5$	
<code>cross(x, x)</code>	Inherited from $(x[i] * y[j] - x[j] * y[i])$ where $i \neq j$	
<code>degrees(x)</code>	Inherited from $x * 57.295779513082322865$	
<code>determinant(m:mat2x2<T>)</code> <code>determinant(m:mat3x3<T>)</code> <code>determinant(m:mat4x4<T>)</code>	Infinite ULP. NOTE: WebGPU implementations should provide a pragmatically useful determinant function. In ideal math, determinants are computed with add, subtract, and multiply operations. However, GPUs use floating point math, and GPU implementations of determinant favour speed and simplicity over robustness against overflow and error. For example, the naive computation of even a 2x2 determinant ($m[0][0] * m[1][1] - m[1][0] * m[0][1]$) fails to guard against catastrophic cancellation. Providing tighter error bounds for 2x2 determinants is the subject of relatively recent research [Jeannerod2013]. The challenges compound quickly as matrix sizes increase. The lack of a finite error bound for determinants in WGSL reflects the same lack in underlying implementations.	
<code>distance(x, y)</code>	Inherited from <code>length(x - y)</code>	
<code>dot(x, y)</code>	Inherited from $\text{sum of } x[i] * y[i]$	
<code>dpx(x)</code> <code>dpxCoarse(x)</code> <code>dpxFine(x)</code>	Infinite ULP.	

Built-in Function	Accuracy for f32	Accuracy for f16
dpdy(x) dpdyCoarse(x) dpdyFine(x) fwidth(x) fwidthCoarse(x) fwidthFine(x)	NOTE: WebGPU implementations should provide a pragmatically useful derivative functions. Derivatives are implemented as differences (or difference of absolutes in the case of fwidth) between values in different invocations on GPUs. The lack of a finite error bound for derivatives in WGSL reflects the same lack in underlying implementations.	
exp(x)	$3 + 2 * x \text{ ULP}$	$1 + 2 * x \text{ ULP}$
exp2(x)	$3 + 2 * x \text{ ULP}$	$1 + 2 * x \text{ ULP}$
faceForward(x, y, z)	Inherited from <code>select(-x, x, dot(z, y) < 0.0)</code>	
floor(x)	Correctly rounded	
fma(x, y, z)	Inherited from $x * y + z$	
fract(x)	Inherited from $x - \text{floor}(x)$	
frexp(x)	Correctly rounded, when x is zero or normal.	
inverseSqrt(x)	2 ULP	
ldexp(x, y)	Correctly rounded	
length(x)	Inherited from <code>sqrt(dot(x, x))</code> in the vector case, and <code>sqrt(x*x)</code> in the scalar case.	
log(x)	Absolute error at most 2^{-21} when x is in the interval [0.5, 2.0]. 3 ULP when x is outside the interval [0.5, 2.0].	Absolute error at most 2^{-7} when x is in the interval [0.5, 2.0]. 3 ULP when x is outside the interval [0.5, 2.0].
log2(x)	Absolute error at most 2^{-21} when x is in the interval [0.5, 2.0]. 3 ULP when x is outside the interval [0.5, 2.0].	Absolute error at most 2^{-7} when x is in the interval [0.5, 2.0]. 3 ULP when x is outside the interval [0.5, 2.0].
max(x, y)	Correctly rounded If both x and y are subnormal , the result may be either input.	
min(x, y)	Correctly rounded. If both x and y are subnormal , the result may be either input.	
mix(x, y, z)	Inherited from $x * (1.0 - z) + y * z$	
modf(x)	Correctly rounded	
normalize(x)	Inherited from $x / \text{length}(x)$	

Built-in Function	Accuracy for f32	Accuracy for f16
<code>pack4x8snorm(x)</code>	Correctly rounded intermediate result value. Correct result.	
<code>pack4x8unorm(x)</code>	Correctly rounded intermediate result value. Correct result.	
<code>pack2x16snorm(x)</code>	Correctly rounded intermediate result value. Correct result.	
<code>pack2x16unorm(x)</code>	Correctly rounded intermediate result value. Correct result.	
<code>pack2x16float(x)</code>	Correctly rounded intermediate result value. Correct result.	
<code>pow(x, y)</code>	Inherited from $\exp_2(y * \log_2(x))$	
<code>quantizeToF16(x)</code>	Correctly rounded	
<code>radians(x)</code>	Inherited from $x * 0.017453292519943295474$	
<code>reflect(x, y)</code>	Inherited from $x - 2.0 * \text{dot}(x, y) * y$	
<code>refract(x, y, z)</code>	Inherited from $z * x - (z * \text{dot}(y, x) + \text{sqrt}(k)) * y$, where $k = 1.0 - z * z * (1.0 - \text{dot}(y, x) * \text{dot}(y, x))$ If $k < 0.0$ the result is precisely 0.0	
<code>round(x)</code>	Correctly rounded	
<code>sign(x)</code>	Correctly rounded	
<code>sin(x)</code>	Absolute error at most 2^{-11} when x is in the interval $[-\pi, \pi]$	Absolute error at most 2^{-7} when x is in the interval $[-\pi, \pi]$
<code>sinh(x)</code>	Inherited from $(\exp(x) - \exp(-x)) * 0.5$	
<code>saturate(x)</code>	Correctly rounded	
<code>smoothstep(edge0, edge1, x)</code>	Inherited from $t * t * (3.0 - 2.0 * t)$, where $t = \text{clamp}((x - \text{edge0}) / (\text{edge1} - \text{edge0}), 0.0, 1.0)$	
<code>sqrt(x)</code>	Inherited from $1.0 / \text{inverseSqrt}(x)$	
<code>step(edge, x)</code>	Correctly rounded	
<code>tan(x)</code>	Inherited from $\sin(x) / \cos(x)$	
<code>tanh(x)</code>	The worse of: - Absolute error 1.0×10^{-5} - Inherited from $\sinh(x) / \cosh(x)$	
<code>transpose(x)</code>	Correctly rounded	
<code>trunc(x)</code>	Correctly rounded	
<code>unpack4x8snorm(x)</code>	3 ULP	N/A
<code>unpack4x8unorm(x)</code>	3 ULP	N/A
<code>unpack2x16snorm(x)</code>	3 ULP	N/A

Built-in Function	Accuracy for f32	Accuracy for f16
<code>unpack2x16unorm(x)</code>	3 ULP	N/A
<code>unpack2x16float(x)</code>	Correctly rounded	N/A
<code>subgroupBroadcast(x, i)</code>		Correctly rounded
<code>subgroupBroadcastFirst(x)</code>		Correctly rounded
<code>subgroupAdd(x)</code>	Inherited from sum of x for all active invocations in the subgroup	
<code>subgroupExclusiveAdd(x)</code>	Inherited from the sum of x for all active invocations in the subgroup whose subgroup invocation IDs are less than the current invocation's ID.	
<code>subgroupInclusiveAdd(x)</code>	Inherited from the sum of x for all active invocations in the subgroup whose subgroup invocation IDs are less than or equal to current invocation's ID.	
<code>subgroupMul(x)</code>	Inherited from the product of x for all active invocations in the subgroup	
<code>subgroupExclusiveMul(x)</code>	Inherited from the product of x_i for all active invocations in the subgroup whose subgroup invocation ID is less than ID of the i'th invocation	
<code>subgroupInclusiveMul(x)</code>	Inherited from the product of x_i for all active invocations in the subgroup whose subgroup invocation ID is less than or equal to ID of the i'th invocation	
<code>subgroupMax(x)</code>	Inherited from max(x) for all active invocations in the subgroup	
<code>subgroupMin(x)</code>	Inherited from min(x) for all active invocations in the subgroup	
<code>subgroupShuffle(x, id)</code>		Correctly rounded
<code>subgroupShuffleDown(x, delta)</code>		Correctly rounded
<code>subgroupShuffleUp(x, delta)</code>		Correctly rounded
<code>subgroupShuffleXor(x, mask)</code>		Correctly rounded
<code>quadBroadcast(x, id)</code>		Correctly rounded
<code>quadSwapDiagonal(x)</code>		Correctly rounded
<code>quadSwapX(x)</code>		Correctly rounded
<code>quadSwapY(x)</code>		Correctly rounded

15.7.4.2. Accuracy of AbstractFloat Expressions

The accuracy of an [AbstractFloat](#) operation is as follows:

- A correct result is required when the corresponding [f32](#) operation requires a correct result.
- A [correctly rounded](#) result is required when the corresponding [f32](#) operation requires a correctly rounded result.

- $\text{fract}(x)$'s error is inherited from $x - \text{floor}(x)$, where the intermediate calculations are performed as [AbstractFloat](#) operations.
- Otherwise, the error of the corresponding [f32](#) operation is an absolute error, a relative error, an error inherited from a potential implementation, or a combination of these. In this case the error of the [AbstractFloat](#) is unbounded.
 - However, the error of the [AbstractFloat](#) operation *should* be at most the error of the corresponding operation in [f32](#), in absolute terms.
 - This recommendation is meant to avoid surprises: the accuracy of an expression should not be *reduced* when changing its type from f32 to AbstractFloat.
 - The operation may be evaluated in a WebAssembly [\[WASM-CORE-2\]](#) or an ECMAScript [\[ECMAScript\]](#) environment, and those specifications do not specify error bounds on many of the corresponding numeric computations. For example, ECMAScript specifies many floating point operations as being [implementation-approximated](#). Implementations are encouraged to strive to approximate the ideal, but no strict requirement is specified.

NOTE: A given absolute error bound, when quantified as ULP, depends critically on the underlying floating point type.

The [ULP](#) for an [AbstractFloat](#) value assumes AbstractFloat is identical to the [IEEE-754 binary64](#) type.

One [ULP](#) for an f32 value is 2^{29} times larger than 1 ULP for an IEEE-754 binary64 value, since the significand in the binary64 format is 29 bits longer than the significand in the f32 type.

For example, suppose the true result value of an operation is x , but it is computed as x' . If its error $x-x'$ is 3 ULP in f32, then the same absolute error, $x-x'$, is $3 \cdot 2^{29}$ ULP in AbstractFloat.

§ 15.7.5. Reassociation and Fusion

Reassociation is the reordering of operations in an expression such that the answer is the same if computed exactly. For example:

- $(a + b) + c$ reassociates to $a + (b + c)$
- $(a - b) + c$ reassociates to $(a + c) - b$
- $(a * b) / c$ reassociates to $(a / c) * b$

However, the result may not be the same when computed in floating point. The reassociated result may be inaccurate due to approximation, or may trigger an overflow or NaN when computing [intermediate results](#).

An implementation may reassociate operations.

An implementation may fuse operations if the transformed expression is at least as accurate as the original formulation. For example, some fused multiply-add implementations can be more accurate than performing a multiply followed by an addition.

§ 15.7.6. Floating Point Conversion

This section describes the details of a [scalar conversion](#) where either the source or destination is a floating point type.

In this section, a floating point type may be any of:

- The [f32](#), [f16](#), and [AbstractFloat](#) types in WGSL.

- A hypothetical type corresponding to a binary format defined by the [IEEE-754](#) floating point standard.

NOTE: Recall that the [f32](#) WGSL type corresponds to the IEEE-754 [binary32](#) format, and the [f16](#) WGSL type corresponds to the IEEE-754 [binary16](#) format.

The *scalar floating point to integral conversion* algorithm is:

To convert a floating point scalar value X to an [integer scalar](#) type T :

- If X is a [NaN](#), the result is an [indeterminate value](#) in T .
- If X is exactly representable in the target type T , then the result is that value.
- Otherwise, the result is the value in T closest to [truncate](#)(X) and also exactly representable in the original floating point type.

NOTE: In other words, for non-NaN cases, floating point to integer conversion clamps the value to be within the range of the target type, then rounds toward zero. This clamping requirement is one place where WGSL mandates a meaningful result, but which would yield undefined behavior in C and C++, and where [IEEE-754](#) mandates an invalid operation exception and a NaN result.

NOTE: For example:

- 3.9f converted to [u32](#) is 3u
- -1f converted to [u32](#) is 0u
- 1e20f converted to [u32](#) is the largest float value representable in u32, 4294967040u
- Note this is smaller than the maximum u32 value 4294967295u
- -3.9f converted to [i32](#) is -3i
- 1e20f converted to [i32](#) is the maximum i32 value, 2147483520i
- Note this is smaller than the maximum i32 value 2147483647i
- -1e20f converted to [i32](#) is the minimum i32 value, i32(-2147483648)

The *numeric scalar conversion to floating point* algorithm is:

Algorithm: Numeric [scalar conversion](#) to floating point

Inputs:

- X , a [numeric scalar](#) value of type S
- T , a destination floating point type.

Output: $XOut$, the result of converting X to type T , or generate an error.

Procedure:

- If X is a NaN for the source type S , then $XOut$ is a NaN in type T .
- If X is exactly representable in the destination type T , then $XOut$ is the value in T equal to X .
 - Additionally, if X is zero and of [integer scalar](#) type, then $XOut$ has a zero [sign bit](#).
- Otherwise, X is not exactly representable in T :
 - If X lies between two adjacent finite values in T , then $XOut$ is one of those two values. WGSL does not specify whether the higher or lower representable value is chosen, and different instances of such a conversion may choose differently.
 - Otherwise, X lies outside the [finite range](#) of the destination type:
 - A [shader-creation error](#) results if the expression for X is a [const-expression](#).
 - A [pipeline-creation error](#) results if the expression for X is an [override-expression](#).
 - Otherwise the conversion proceeds as follows:
 1. Set X' to the original value X .
 2. If source type S is a floating point type with more significand bits than the destination type T , the extra significand bits of the source value X may be discarded (i.e. treated as if they are 0). Update X' accordingly.
 3. If X' is the most-positive or most-negative finite value of the destination type T , then set $XOut = X'$.
 4. Otherwise, set $XOut$ to the infinity value of destination type T , with the same sign as X' .

NOTE: An integer value may lie between two adjacent representable floating point values. In particular, the [f32](#) type uses 23 explicit fractional bits. Additionally, when the floating point value is in the normal range (the exponent is neither extreme value), then the significand is the set of fractional bits together with an extra 1-bit at the most significant position at bit position 23. Then, for example, integers 2^{28} and $1+2^{28}$ both map to the same floating point value: the difference in the least significant 1 bit is not representable by the floating point format. This kind of collision occurs for pairs of adjacent integers with a magnitude of at least 2^{25} .

NOTE: The original value is always within range of the destination type when the original type is one of [i32](#) or [u32](#) and the destination type is [f32](#).

NOTE: The original value is always within range of the destination type when the source type is a floating point type with fewer exponent and significand bits than the target floating point type.

15.7.7. Domains of Floating Point Expressions and Built-in Functions

Previous sections describe the expected behavior when a floating point expression is evaluated outside its [domain](#).

Sections [§ 8.7 Arithmetic Expressions](#) and [§ 17.5 Numeric Built-in Functions](#) define the [domains](#) for floating point expressions and built-in functions, respectively. If no restriction is listed for a given operation, then the domain is total: the domain includes all finite and infinite inputs. Otherwise an explicit domain is listed.

In many cases where a WGSL operation corresponds to an operation defined by [IEEE-754](#), they will have the same domain. For example the both the WGSL and IEEE-754 `acos` operations have a domain of $[-1, 1]$.

For a [component-wise](#) WGSL operation with an explicitly listed domain, only the scalar case is described. The vector case is inferred from the component-wise semantics.

Some WGSL operations may be implemented in terms of other WGSL expressions. Section [§ 15.7.4 Floating Point Accuracy](#) lists these as having [inherited from](#) accuracy. When listing the domain for one of these operations, either:

- The domain is explicitly described, or
- The domain is stated as *implied from linear terms*, meaning the domain is derived by:
 - Assuming the original operation was replaced by the "inherited from" expression, which is a combination of floating point addition, subtraction, and multiplication operations.
 - Applying and combining the domain restrictions for those remaining operations over the given parameters.

For example: The `dot(a, b)` function over 2-element vectors a and b has its accuracy [inherited from](#) the expression $a[0] * b[0] + a[1] * b[1]$. This uses two floating point multiplications, and one floating point addition.

- Floating point multiplication is well defined over the extended reals except when one operand is zero and the other is infinite.
- Floating point addition is well defined except when the two operands are infinities of opposite sign.
- Therefore, the domain is all pairs of extended real two-element vectors a and b except:
 - Implied from the multiplications:
 - $a[i]$ is zero and $b[i]$ is infinite.
 - $a[i]$ is infinite and $b[i]$ is zero.
 - Implied from the addition:
 - $a[0] \times b[0]$ is $+\infty$ and $a[1] \times b[1]$ is $+\infty$
 - $a[0] \times b[0]$ is $-\infty$ and $a[1] \times b[1]$ is $-\infty$

§ 16. Keyword and Token Summary

§ 16.1. Keyword Summary

- `alias`
- `break`
- `case`
- `const`
- `const_assert`
- `continue`
- `continuing`
- `default`

- **diagnostic**
- **discard**
- **else**
- **enable**
- **false**
- **fn**
- **for**
- **if**
- **let**
- **loop**
- **override**
- **requires**
- **return**
- **struct**
- **switch**
- **true**
- **var**
- **while**

§ 16.2. Reserved Words

A *reserved word* is a [token](#) which is reserved for future use. A WGSL module [must not](#) contain a reserved word.

The following are reserved words:

```
_reserved :
| 'NULL'
| 'Self'
| 'abstract'
| 'active'
| 'alignas'
| 'alignof'
| 'as'
| 'asm'
| 'asm_fragment'
| 'async'
| 'attribute'
| 'auto'
| 'await'
| 'become'
| 'cast'
| 'catch'
| 'class'
| 'co_await'
| 'co_return'
| 'co_yield'
```


- | 'coherent'
- | 'column_major'
- | 'common'
- | 'compile'
- | 'compile_fragment'
- | 'concept'
- | 'const_cast'
- | 'consteval'
- | 'constexpr'
- | 'constinit'
- | 'crate'
- | 'debugger'
- | 'decltype'
- | 'delete'
- | 'demote'
- | 'demote_to_helper'
- | 'do'
- | 'dynamic_cast'
- | 'enum'
- | 'explicit'
- | 'export'
- | 'extends'
- | 'extern'
- | 'external'
- | 'fallthrough'
- | 'filter'
- | 'final'
- | 'finally'
- | 'friend'
- | 'from'
- | 'fxgroup'
- | 'get'
- | 'goto'
- | 'groupshared'
- | 'highp'
- | 'impl'
- | 'implements'
- | 'import'
- | 'inline'
- | 'instanceof'
- | 'interface'
- | 'layout'
- | 'lowp'
- | 'macro'
- | 'macro_rules'
- | 'match'
- | 'mediump'
- | 'meta'
- | 'mod'
- | 'module'
- | 'move'
- | 'mut'

```
| 'mutable'  
| 'namespace'  
| 'new'  
| 'nil'  
| 'noexcept'  
| 'noinline'  
| 'nointerpolation'  
| 'non_coherent'  
| 'noncoherent'  
| 'noperspective'  
| 'null'  
| 'nullptr'  
| 'of'  
| 'operator'  
| 'package'  
| 'packoffset'  
| 'partition'  
| 'pass'  
| 'patch'  
| 'pixelfragment'  
| 'precise'  
| 'precision'  
| 'premerge'  
| 'priv'  
| 'protected'  
| 'pub'  
| 'public'  
| 'readonly'  
| 'ref'  
| 'regardless'  
| 'register'  
| 'reinterpret_cast'  
| 'require'  
| 'resource'  
| 'restrict'  
| 'self'  
| 'set'  
| 'shared'  
| 'sizeof'  
| 'smooth'  
| 'snorm'  
| 'static'  
| 'static_assert'  
| 'static_cast'  
| 'std'  
| 'subroutine'  
| 'super'  
| 'target'  
| 'template'  
| 'this'  
| 'thread_local'  
| 'throw'
```

```
| 'trait'
| 'try'
| 'type'
| 'typedef'
| 'typeid'
| 'typename'
| 'typeof'
| 'union'
| 'unless'
| 'unorm'
| 'unsafe'
| 'unsized'
| 'use'
| 'using'
| 'varying'
| 'virtual'
| 'volatile'
| 'wgs1'
| 'where'
| 'with'
| 'writeonly'
| 'yield'
```

16.3. Syntactic Tokens

A *syntactic token* is a sequence of special code points, used:

- to spell an expression operator, or
- as punctuation: to group, sequence, or separate other grammar elements.

The [syntactic tokens](#) are:

- '&' (Code point: U+0026)
- '&&' (Code points: U+0026 U+0026)
- '->' (Code points: U+002D U+003E)
- '@' (Code point: U+0040)
- '/' (Code point: U+002F)
- '!' (Code point: U+0021)
- '[' (Code point: U+005B)
- ']' (Code point: U+005D)
- '{' (Code point: U+007B)
- '}' (Code point: U+007D)
- ':' (Code point: U+003A)
- ',' (Code point: U+002C)
- '=' (Code point: U+003D)
- '==' (Code points: U+003D U+003D)
- '!=' (Code points: U+0021 U+003D)

- '>' (Code point: U+003E) (also `_greater_than` for template disambiguation)
- '>=' (Code points: U+003E U+003D) (also `_greater_than_equal` for template disambiguation)
- '>>' (Code point: U+003E U+003E) (also `_shift_right` for template disambiguation)
- '<' (Code point: U+003C) (also `_less_than` for template disambiguation)
- '<=' (Code points: U+003C U+003D) (also `_less_than_equal` for template disambiguation)
- '<<' (Code points: U+003C U+003C) (also `_shift_left` for template disambiguation)
- '%' (Code point: U+0025)
- '-' (Code point: U+002D)
- '--' (Code points: U+002D U+002D)
- '.' (Code point: U+002E)
- '+' (Code point: U+002B)
- '++' (Code points: U+002B U+002B)
- '|' (Code point: U+007C)
- '||' (Code points: U+007C U+007C)
- '(' (Code point: U+0028)
- ')' (Code point: U+0029)
- ';' (Code point: U+003B)
- '*' (Code point: U+002A)
- '~' (Code point: U+007E)
- '_' (Code point: U+005F)
- '^' (Code point: U+005E)
- '+=' (Code points: U+002B U+003D)
- '-=' (Code points: U+002D U+003D)
- '*=' (Code points: U+002A U+003D)
- '/=' (Code points: U+002F U+003D)
- '%=' (Code points: U+0025 U+003D)
- '&=' (Code points: U+0026 U+003D)
- '|=' (Code points: U+007C U+003D)
- '^=' (Code points: U+005E U+003D)
- '>>=' (Code point: U+003E U+003E U+003D) (also `_shift_right_assign` for template disambiguation)
- '<<=' (Code points: U+003C U+003C U+003D) (also `_shift_left_assign` for template disambiguation)
- `_template_args_end`
 - Text: '>' (Code point: U+003E)
 - This token is textually the same as the [greater_than](#) syntactic token.
 - It is generated by template list disambiguation, and is used as the last token in a template list.
- `_template_args_start`
 - Text: '<' (Code point: U+003C)
 - This token is textually the same as the [less_than](#) syntactic token.
 - It is generated by template list disambiguation, and is used as the first token in a template list.

- `_disambiguate_template`
 - Text: None
 - In the standard parser formulation given in § 3.1 Parsing, this token serves no purpose, and is effectively ignored. To ignore it, the scanner always succeeds in matching it to an empty string.
 - In the alternative parsing formulation given in § 3.1 Parsing, this token triggers the scanner to perform template list discovery in an on-demand fashion, as described in that section.

§ 17. Built-in Functions

Certain functions are [predeclared](#), provided by the implementation, and therefore always available for use in a WGSL module. These are called ***built-in functions***.

A built-in function is a family of functions, all with the same name, but distinguished by the number, order, and types of their [formal parameters](#). Each of these distinct function variations is an [overload](#).

NOTE: Each [user-defined function](#) only has one [overload](#).

Each [overload](#) is described below via:

- Type parameterizations, if any.
- The built-in function name, a parenthesized list of [formal parameters](#), and optionally a [return type](#).
- The behavior of this overload of the function.

When calling a built-in function, all arguments to the function are evaluated before function evaluation begins. See § 11.2 [Function Calls](#).

§ 17.1. Constructor Built-in Functions

A ***value constructor*** built-in function explicitly creates a value of a given type.

WGSL provides value constructors for all [predeclared](#) types and all [constructible structure](#) types. Such a constructor built-in function has the same spelling as the type, or a [type alias](#) for the type. Wherever such a built-in function is used, the [identifier must](#) be [in scope](#) of the type or the type alias, and the [identifier must not resolve](#) to another declaration.

NOTE: The structure types returned by [fexp](#), [modf](#), and [atomicCompareExchangeWeak](#) cannot be written in WGSL modules.

NOTE: A value declaration of the type needs to be valid at that statement of the WGSL text.

WGSL provides two kinds of value constructors:

- [zero value constructors](#)
- [value constructors](#) (which also provide conversion)

§ 17.1.1. Zero Value Built-in Functions

Each [concrete](#), [constructible](#) T has a unique ***zero value***, and a corresponding built-in function written in WGSL as the type followed by an empty pair of parentheses: $T()$. [Abstract numeric types](#) also have zero values, but there are no built-in

functions to access them.

The zero values are as follows:

- `bool()` is `false`
- `i32()` is `0i`
- `u32()` is `0u`
- `f32()` is `0.0f`
- `f16()` is `0.0h`
- The zero value for an N -component vector of type T is the N -component vector of the zero value for T .
- The zero value for an C -column R -row matrix of type T is the matrix of those dimensions filled with the zero value for T .
- The zero value for a [constructible](#) N -element array with element type E is an array of N elements of the zero value for E .
- The zero value for a [constructible](#) structure type S is the structure value S with zero-valued members.
- The zero value of an [AbstractInt](#) is `0`.
- The zero value of an [AbstractFloat](#) is `0.0`.

NOTE: WGSL does not have zero built-in functions for [atomic types](#), [runtime-sized](#) arrays, buffers, or other types that are not [constructible](#).

Overload

```
@const @must_use fn T() -> T
```

Parameterization T is a [concrete constructible](#) type.

Description Construct [zero value](#) of the type T .

NOTE: Zero-filled vectors of `AbstractInt` can be written as `vec2()`, `vec3()`, and `vec4()`.

EXAMPLE: ZERO-VALUED VECTORS

```
vec2<f32>()           // The zero-valued vector of two f32 components.
vec2<f32>(0.0, 0.0)   // The same value, written explicitly.

vec3<i32>()           // The zero-valued vector of three i32 components.
vec3<i32>(0, 0, 0)    // The same value, written explicitly.
```

EXAMPLE: ZERO-VALUED ARRAYS

```
array<bool, 2>()      // The zero-valued array of two booleans.
array<bool, 2>(false, false) // The same value, written explicitly.
```

EXAMPLE: ZERO-VALUED STRUCTURES

```
struct Student {  
    grade: i32,  
    GPA: f32,  
    attendance: array<bool,4>  
}  
  
fn func() {  
    var s: Student;  
  
    // The zero value for Student  
    s = Student();  
  
    // The same value, written explicitly.  
    s = Student(0, 0.0, array<bool,4>(false, false, false, false));  
  
    // The same value, written with zero-valued members.  
    s = Student(i32(), f32(), array<bool,4>());  
}
```

§ 17.1.2. Value Constructor Built-in Functions

The built-in functions defined in following subsections create a [constructible](#) value by:

- Copying an existing value of the same type (i.e. the identity function), or
- Creating a composite value from an explicit list of components.
- Converting from another value type.

The vector and matrix forms construct vector and matrix values from various combinations of components and subvectors with matching component types. There are [overloads](#) for constructing vectors and matrices that specify the dimensions of the target type without having to specify the component type; the component type is inferred from the constructor arguments.

§ 17.1.2.1. *array*

Overload

```
@const @must_use fn array<T, N>(e1 : T, ..., eN : T) -> array<T, N>
```

Parameterization T is [concrete](#) and [constructible](#)

Description Construction of an [array](#) from elements.

NOTE: `array<T,N>` is [constructible](#) because its [element count](#) is equal to the number of arguments to the constructor, and hence fully determined at [shader-creation](#) time.

Overload

```
@const @must_use fn array(e1 : T, ..., eN : T) -> array<T, N>
```

Parameterization T is [constructible](#)

Description Construction of an [array](#) from elements.

The component type is inferred from the elements' type. The size of the array is determined by the number of elements.

§ 17.1.2.2. *bool*

Overload

```
@const @must_use fn bool(e : T) -> bool
```

Parameterization T is a [scalar](#) type.

Description Construct a [bool](#) value.

If T is [bool](#), this is an identity operation.

Otherwise this is a boolean coercion. The result is `false` if e is a [zero value](#) (or -0.0 for floating point types) and `true` otherwise.

§ 17.1.2.3. *f16*

Overload

```
@const @must_use fn f16(e : T) -> f16
```

Parameterization T is a [scalar](#) type

Description	Construct an f16 value. If T is f16 , this is an identity operation. If T is a numeric scalar (other than f16), e is converted to f16 (including invalid conversions). If T is bool , the result is <code>1.0h</code> if e is true and <code>0.0h</code> otherwise.
-------------	---

§ 17.1.2.4. *f32*

Overload	<pre>@const @must_use fn f32(e : T) -> f32</pre>
Parameterization	T is a concrete scalar type
Description	Construct an f32 value. If T is f32 , this is an identity operation. If T is a numeric scalar (other than f32), e is converted to f32 (including invalid conversions). If T is bool , the result is <code>1.0f</code> if e is true and <code>0.0f</code> otherwise.

§ 17.1.2.5. *i32*

Overload	<pre>@const @must_use fn i32(e : T) -> i32</pre>
Parameterization	T is a scalar type
Description	Construct an i32 value. If T is i32 , this is an identity operation. If T is u32 , this is a reinterpretation of bits (i.e. the result is the unique value in i32 that has the same bit pattern as e). If T is a floating point type , e is converted to i32 , rounding towards zero. If T is bool , the result is <code>1i</code> if e is true and <code>0i</code> otherwise. If T is an AbstractInt , this is an identity operation if e can be represented in i32 , otherwise it produces a shader-creation error .

§ 17.1.2.6. *mat2x2*

Overload	<pre>@const @must_use fn mat2x2<T>(e : mat2x2<S>) -> mat2x2<T> @const @must_use fn mat2x2(e : mat2x2<S>) -> mat2x2<S></pre>
----------	---

Parameterization	T is f16 or f32 S is AbstractFloat, f16, or f32
Description	Constructor for a 2x2 column-major matrix. If T does not match S, a conversion occurs.
Overload	<pre>@const @must_use fn mat2x2<T>(v1 : vec2<T>, v2 : vec2<T>) -> mat2x2<T> @const @must_use fn mat2x2(v1 : vec2<T>, v2 : vec2<T>) -> mat2x2<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 2x2 column-major matrix from column vectors.
Overload	<pre>@const @must_use fn mat2x2<T>(e1 : T, e2 : T, e3 : T, e4 : T) -> mat2x2<T> @const @must_use fn mat2x2(e1 : T, e2 : T, e3 : T, e4 : T) -> mat2x2<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 2x2 column-major matrix from elements. Same as <code>mat2x2(vec2(e1,e2), vec2(e3,e4))</code>.

§ 17.1.2.7. *mat2x3*

Overload

```
@const @must_use fn mat2x3<T>(e : mat2x3<S>) -> mat2x3<T>
@const @must_use fn mat2x3(e : mat2x3<S>) -> mat2x3<S>
```

Parameterization	T is f16 or f32 S is AbstractFloat , f16 , or f32
------------------	--

Description

Constructor for a 2x3 column-major [matrix](#).

If T does not match S, a [conversion](#) occurs.

Overload

```
@const @must_use fn mat2x3<T>(v1 : vec3<T>, v2 : vec3<T>) -> mat2x3<T>
@const @must_use fn mat2x3(v1 : vec3<T>, v2 : vec3<T>) -> mat2x3<T>
```

Parameterization	T is AbstractFloat , f16 , or f32
------------------	---

Description

Construct a 2x3 column-major [matrix](#) from column vectors.

Overload

```
@const @must_use fn mat2x3<T>(e1 : T, ..., e6 : T) -> mat2x3<T>
@const @must_use fn mat2x3(e1 : T, ..., e6 : T) -> mat2x3<T>
```

Parameterization	T is AbstractFloat , f16 , or f32
------------------	---

Description

Construct a 2x3 column-major [matrix](#) from elements.

Same as `mat2x3(vec3(e1,e2,e3), vec3(e4,e5,e6))`.

§ 17.1.2.8. *mat2x4*

Overload	<pre>@const @must_use fn mat2x4<T>(e : mat2x4<S>) -> mat2x4<T> @const @must_use fn mat2x4(e : mat2x4<S>) -> mat2x4<S></pre>
Parameterization	T is f16 or f32 S is AbstractFloat , f16 , or f32
Description	Constructor for a 2x4 column-major matrix . If T does not match S, a conversion occurs.
Overload	<pre>@const @must_use fn mat2x4<T>(v1 : vec4<T>, v2 : vec4<T>) -> mat2x4<T> @const @must_use fn mat2x4(v1 : vec4<T>, v2 : vec4<T>) -> mat2x4<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 2x4 column-major matrix from column vectors.
Overload	<pre>@const @must_use fn mat2x4<T>(e1 : T, ..., e8 : T) -> mat2x4<T> @const @must_use fn mat2x4(e1 : T, ..., e8 : T) -> mat2x4<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 2x4 column-major matrix from elements. Same as mat2x4(vec4(e1,e2,e3,e4), vec4(e5,e6,e7,e8)).

17.1.2.9. *mat3x2*

Overload	<pre>@const @must_use fn mat3x2<T>(e : mat3x2<S>) -> mat3x2<T> @const @must_use fn mat3x2(e : mat3x2<S>) -> mat3x2<S></pre>
Parameterization	T is f16 or f32 S is AbstractFloat , f16 , or f32
Description	Constructor for a 3x2 column-major matrix . If T does not match S, a conversion occurs.
Overload	<pre>@const @must_use fn mat3x2<T>(v1 : vec2<T>, v2 : vec2<T>, v3 : vec2<T>) -> mat3x2<T> @const @must_use fn mat3x2(v1 : vec2<T>, v2 : vec2<T>, v3 : vec2<T>) -> mat3x2<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 3x2 column-major matrix from column vectors.
Overload	<pre>@const @must_use fn mat3x2<T>(e1 : T, ..., e6 : T) -> mat3x2<T> @const @must_use fn mat3x2(e1 : T, ..., e6 : T) -> mat3x2<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 3x2 column-major matrix from elements. Same as <code>mat3x2(vec2(e1,e2), vec2(e3,e4), vec2(e5,e6))</code> .

17.1.2.10. *mat3x3*

Overload	<pre>@const @must_use fn mat3x3<T>(e : mat3x3<S>) -> mat3x3<T> @const @must_use fn mat3x3(e : mat3x3<S>) -> mat3x3<S></pre>
Parameterization	T is f16 or f32 S is AbstractFloat , f16 , or f32
Description	Constructor for a 3x3 column-major matrix .

If T does not match S, a [conversion](#) occurs.

Overload	<pre>@const @must_use fn mat3x3<T>(v1 : vec3<T>, v2 : vec3<T>, v3 : vec3<T>) -> mat3x3<T> @const @must_use fn mat3x3(v1 : vec3<T>, v2 : vec3<T>, v3 : vec3<T>) -> mat3x3<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 3x3 column-major matrix from column vectors.
Overload	<pre>@const @must_use fn mat3x3<T>(e1 : T, ..., e9 : T) -> mat3x3<T> @const @must_use fn mat3x3(e1 : T, ..., e9 : T) -> mat3x3<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 3x3 column-major matrix from elements. Same as mat3x3(vec3(e1,e2,e3), vec3(e4,e5,e6), vec3(e7,e8,e9)).

§ 17.1.2.11. mat3x4

Overload	<pre>@const @must_use fn mat3x4<T>(e : mat3x4<S>) -> mat3x4<T> @const @must_use fn mat3x4(e : mat3x4<S>) -> mat3x4<S></pre>
Parameterization	T is f16 or f32 S is AbstractFloat , f16 , or f32
Description	Constructor for a 3x4 column-major matrix . If T does not match S, a conversion occurs.
Overload	<pre>@const @must_use fn mat3x4<T>(v1 : vec4<T>, v2 : vec4<T>, v3 : vec4<T>) -> mat3x4<T> @const @must_use fn mat3x4(v1 : vec4<T>, v2 : vec4<T>, v3 : vec4<T>) -> mat3x4<T></pre>

Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 3x4 column-major matrix from column vectors.
Overload	<pre>@const @must_use fn mat3x4<T>(e1 : T, ..., e12 : T) -> mat3x4<T> @const @must_use fn mat3x4(e1 : T, ..., e12 : T) -> mat3x4<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 3x4 column-major matrix from elements. Same as mat3x4(vec4(e1,e2,e3,e4), vec4(e5,e6,e7,e8), vec4(e9,e10,e11,e12)).
17.1.2.12. mat4x2	
Overload	<pre>@const @must_use fn mat4x2<T>(e : mat4x2<S>) -> mat4x2<T> @const @must_use fn mat4x2(e : mat4x2<S>) -> mat4x2<S></pre>
Parameterization	T is f16 or f32 S is AbstractFloat , f16 , or f32
Description	Constructor for a 4x2 column-major matrix . If T does not match S, a conversion occurs.
Overload	<pre>@const @must_use fn mat4x2<T>(v1 : vec2<T>, v2 : vec2<T>, v3 : vec2<T>, v4: vec2<T>) -> mat4x2<T> @const @must_use fn mat4x2(v1 : vec2<T>, v2 : vec2<T>, v3 : vec2<T>, v4: vec2<T>) -> mat4x2<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32
Description	Construct a 4x2 column-major matrix from column vectors.
Overload	<pre>@const @must_use fn mat4x2<T>(e1 : T, ..., e8 : T) -> mat4x2<T> @const @must_use fn mat4x2(e1 : T, ..., e8 : T) -> mat4x2<T></pre>
Parameterization	T is AbstractFloat , f16 , or f32

Description	Construct a 4x2 column-major matrix from elements. Same as <code>mat4x2(vec2(e1,e2), vec2(e3,e4), vec2(e5,e6), vec2(e7,e8))</code> .
-------------	---

17.1.2.13. `mat4x3`

Overload	<pre>@const @must_use fn mat4x3<T>(e : mat4x3<S>) -> mat4x3<T> @const @must_use fn mat4x3(e : mat4x3<S>) -> mat4x3<S></pre>
----------	---

Parameterization	T is f16 or f32 S is AbstractFloat , f16 , or f32
------------------	--

Description	Constructor for a 4x3 column-major matrix . If T does not match S, a conversion occurs.
-------------	--

Overload	<pre>@const @must_use fn mat4x3<T>(v1 : vec3<T>, v2 : vec3<T>, v3 : vec3<T>, v4 : vec3<T>) -> mat4x3<T> @const @must_use fn mat4x3(v1 : vec3<T>, v2 : vec3<T>, v3 : vec3<T>, v4 : vec3<T>) -> mat4x3<T></pre>
----------	---

Parameterization	T is AbstractFloat , f16 , or f32
------------------	---

Description	Construct a 4x3 column-major matrix from column vectors.
-------------	--

Overload	<pre>@const @must_use fn mat4x3<T>(e1 : T, ..., e12 : T) -> mat4x3<T> @const @must_use fn mat4x3(e1 : T, ..., e12 : T) -> mat4x3<T></pre>
----------	---

Parameterization	T is AbstractFloat , f16 , or f32
------------------	---

Description	Construct a 4x3 column-major matrix from elements. Same as <code>mat4x3(vec3(e1,e2,e3), vec3(e4,e5,e6), vec3(e7,e8,e9), vec3(e10,e11,e12))</code> .
-------------	--

§ 17.1.2.14. *mat4x4*

Overload

```
@const @must_use fn mat4x4<T>(e : mat4x4<S>) -> mat4x4<T>
@const @must_use fn mat4x4(e : mat4x4<S>) -> mat4x4<S>
```

Parameterization

T is [f16](#) or [f32](#)
 S is [AbstractFloat](#), [f16](#), or [f32](#)

Description

Constructor for a 4x4 column-major [matrix](#).
 If T does not match S, a [conversion](#) occurs.

Overload

```
@const @must_use fn mat4x4<T>(v1 : vec4<T>,
                               v2 : vec4<T>,
                               v3 : vec4<T>,
                               v4 : vec4<T>) -> mat4x4<T>
@const @must_use fn mat4x4(v1 : vec4<T>,
                             v2 : vec4<T>,
                             v3 : vec4<T>,
                             v4 : vec4<T>) -> mat4x4<T>
```

Parameterization

T is [AbstractFloat](#), [f16](#), or [f32](#)

Description

Construct a 4x4 column-major [matrix](#) from column vectors.

Overload

```
@const @must_use fn mat4x4<T>(e1 : T, ..., e16 : T) -> mat4x4<T>
@const @must_use fn mat4x4(e1 : T, ..., e16 : T) -> mat4x4<T>
```

Parameterization

T is [AbstractFloat](#), [f16](#), or [f32](#)

Description

Construct a 4x4 column-major [matrix](#) from elements.
 Same as `mat4x4(vec4(e1,e2,e3,e4), vec4(e5,e6,e7,e8), vec4(e9,e10,e11,e12), vec4(e13,e14,e15,e16))`.

§ 17.1.2.15. *Structures*

Overload

```
@const @must_use fn S(e1 : T1, ..., eN : TN) -> S
```

Parameterization

S is a [constructible](#) structure type with members having types T1 ... TN.

Description Construct a [structure](#) of type S from members.

§ 17.1.2.16. [u32](#)

Overload

```
@const @must_use fn u32(e : T) -> u32
```

Parameterization T is a [scalar](#) type

Description Construct a [u32](#) value.

If T is [u32](#), this is an identity operation.

If T is [i32](#), this is a reinterpretation of bits (i.e. the result is the unique value in [u32](#) that has the same bit pattern as e).

If T is a [floating point type](#), e is [converted](#) to [u32](#), [rounding](#) towards zero.

If T is [bool](#), the result is 1u if e is true and 0u otherwise.

If T is [AbstractInt](#), this is an identity operation if the e can be represented in [u32](#), otherwise it produces a [shader-creation error](#).

NOTE: The overload from [AbstractInt](#) exists so expressions such as `u32(4*1000*1000*1000)` can create a `u32` value that would otherwise overflow the `i32` type. If this overload did not exist, [overload resolution](#) would select the `u32(i32)` overload, the `AbstractInt` expression would automatically convert to `i32`, and this would cause a `shader-creation error` due to overflow.

§ 17.1.2.17. [vec2](#)

Overload

```
@const @must_use fn vec2<T>(e : T) -> vec2<T>
@const @must_use fn vec2(e : S) -> vec2<S>
```

Parameterization T is a [concrete scalar](#)
S is [scalar](#)

Description Construction of a two-component [vector](#) with e as both components.

Overload

```
@const @must_use fn vec2<T>(e : vec2<S>) -> vec2<T>
@const @must_use fn vec2(e : vec2<S>) -> vec2<S>
```

Parameterization T is a [concrete scalar](#)
S is [scalar](#)

Description	Component-wise construction of a two-component vector with <code>e.x</code> and <code>e.y</code> as components. If <code>T</code> does not match <code>S</code> a conversion is used and the components are <code>T(e.x)</code> and <code>T(e.y)</code>.
Overload	<pre>@const @must_use fn vec2<T>(e1 : T, e2 : T) -> vec2<T> @const @must_use fn vec2(e1 : T, e2 : T) -> vec2<T></pre>
Parameterization	<code>T</code> is scalar
Description	Component-wise construction of a two-component vector with <code>e1</code> and <code>e2</code> as components.
Overload	<pre>@const @must_use fn vec2() -> vec2<T></pre>
Parameterization	<code>T</code> is <code>AbstractInt</code>
Description	Returns the value <code>vec2(0,0)</code> .

§ 17.1.2.18. `vec3`

Overload	<pre>@const @must_use fn vec3<T>(e : T) -> vec3<T> @const @must_use fn vec3(e : S) -> vec3<S></pre>
Parameterization	<code>T</code> is a concrete scalar <code>S</code> is scalar
Description	Construction of a three-component vector with <code>e</code> as all components.
Overload	<pre>@const @must_use fn vec3<T>(e : vec3<S>) -> vec3<T> @const @must_use fn vec3(e : vec3<S>) -> vec3<S></pre>
Parameterization	<code>T</code> is a concrete scalar <code>S</code> is scalar
Description	Component-wise construction of a three-component vector with <code>e.x</code>, <code>e.y</code>, and <code>e.z</code> as components. If <code>T</code> does not match <code>S</code> a conversion is used and the components are <code>T(e.x)</code>, <code>T(e.y)</code>, and <code>T(e.z)</code>.
Overload	<pre>@const @must_use fn vec3<T>(e1 : T, e2 : T, e3 : T) -> vec3<T></pre>

```
@const @must_use fn vec3(e1 : T, e2 : T, e3 : T) -> vec3<T>
```

Parameterization	T is scalar
Description	Component-wise construction of a three-component vector with e1, e2, and e3 as components.
Overload	<pre>@const @must_use fn vec3<T>(v1 : vec2<T>, e1 : T) -> vec3<T> @const @must_use fn vec3(v1 : vec2<T>, e1 : T) -> vec3<T></pre>
Parameterization	T is scalar
Description	Component-wise construction of a three-component vector with v1.x, v1.y, and e1 as components.
Overload	<pre>@const @must_use fn vec3<T>(e1 : T, v1 : vec2<T>) -> vec3<T> @const @must_use fn vec3(e1 : T, v1 : vec2<T>) -> vec3<T></pre>
Parameterization	T is scalar
Description	Component-wise construction of a three-component vector with e1, v1.x, and v1.y as components.
Overload	<pre>@const @must_use fn vec3() -> vec3<T></pre>
Parameterization	T is <code>AbstractInt</code>
Description	Returns the value <code>vec3(0,0,0)</code> .

§ 17.1.2.19. *vec4*

Overload	<pre>@const @must_use fn vec4<T>(e : T) -> vec4<T> @const @must_use fn vec4(e : S) -> vec4<S></pre>
Parameterization	T is a concrete scalar S is scalar
Description	Construction of a four-component vector with e as all components.
Overload	<pre>@const @must_use fn vec4<T>(e : vec4<S>) -> vec4<T> @const @must_use fn vec4(e : vec4<S>) -> vec4<S></pre>
Parameterization	T is a concrete scalar S is scalar
Description	Component-wise construction of a four-component vector with e.x, e.y, e.z, and e.w as components. If T does not match S a conversion is used and the components are T(e.x), T(e.y), T(e.z) and T(e.w).
Overload	<pre>@const @must_use fn vec4<T>(e1 : T, e2 : T, e3 : T, e4 : T) -> vec4<T> @const @must_use fn vec4(e1 : T, e2 : T, e3 : T, e4 : T) -> vec4<T></pre>
Parameterization	T is scalar
Description	Component-wise construction of a four-component vector with e1, e2, e3, and e4 as components.
Overload	<pre>@const @must_use fn vec4<T>(e1 : T, v1 : vec2<T>, e2 : T) -> vec4<T> @const @must_use fn vec4(e1 : T, v1 : vec2<T>, e2 : T) -> vec4<T></pre>
Parameterization	T is scalar
Description	Component-wise construction of a four-component vector with e1, v1.x, v1.y, and e2 as components.
Overload	<pre>@const @must_use fn vec4<T>(e1 : T, e2 : T, v1 : vec2<T>) -> vec4<T> @const @must_use fn vec4(e1 : T, e2 : T, v1 : vec2<T>) -> vec4<T></pre>
Parameterization	T is scalar

Description	Component-wise construction of a four-component vector with e1, e2, v1.x, and v1.y as components.
Overload	<pre>@const @must_use fn vec4<T>(v1 : vec2<T>, v2 : vec2<T>) -> vec4<T> @const @must_use fn vec4(v1 : vec2<T>, v2 : vec2<T>) -> vec4<T></pre>
Parameterization	T is scalar

Description	Component-wise construction of a four-component vector with v1.x, v1.y, v2.x, and v2.y as components.
Overload	<pre>@const @must_use fn vec4<T>(v1 : vec2<T>, e1 : T, e2 : T) -> vec4<T> @const @must_use fn vec4(v1 : vec2<T>, e1 : T, e2 : T) -> vec4<T></pre>
Parameterization	T is scalar

Description	Component-wise construction of a four-component vector with v1.x, v1.y, e1, and e2 as components.
Overload	<pre>@const @must_use fn vec4<T>(v1 : vec3<T>, e1 : T) -> vec4<T> @const @must_use fn vec4(v1 : vec3<T>, e1 : T) -> vec4<T></pre>
Parameterization	T is scalar

Description	Component-wise construction of a four-component vector with v1.x, v1.y, v1.z, and e1 as components.
Overload	<pre>@const @must_use fn vec4<T>(e1 : T, v1 : vec3<T>) -> vec4<T> @const @must_use fn vec4(e1 : T, v1 : vec3<T>) -> vec4<T></pre>
Parameterization	T is scalar

Description	Component-wise construction of a four-component vector with e1, v1.x, v1.y, and v1.z as components.
Overload	<pre>@const @must_use fn vec4() -> vec4<T></pre>
Parameterization	T is AbstractInt

Description	Returns the value vec4(0,0,0,0).
-------------	----------------------------------

§ 17.2. Bit Reinterpretation Built-in Functions

§ 17.2.1. **bitcast**

A **bitcast** built-in function is used to reinterpret the bit representation of a value in one type as a value in another type.

The internal layout rules are described in [§ 14.4.4 Internal Layout of Values](#).

Overload	<code>@const @must_use fn bitcast<T>(e : T) -> T</code>
Parameterization	T is a concrete numeric scalar or concrete numeric vector
Description	Identity transform. Component-wise when T is a vector . The result is e.
Overload	<code>@const @must_use fn bitcast<T>(e : S) -> T</code>
Parameterization	S is i32, u32, or f32 T is not S and is i32, u32, or f32
Description	Reinterpretation of bits as T. The result is the reinterpretation of bits in e as a T value.
Overload	<code>@const @must_use fn bitcast<vecN<T>>(e : vecN<S>) -> vecN<T></code>
Parameterization	S is i32, u32, or f32 T is not S and is i32, u32, or f32
Description	Component-wise reinterpretation of bits as T. The result is the reinterpretation of bits in e as a vecN<T> value.
Overload	<code>@const @must_use fn bitcast<u32>(e : AbstractInt) -> u32</code> <code>@const @must_use fn bitcast<vecN<u32>>(e : vecN<AbstractInt>) -> vecN<u32></code>
Parameterization	
Description	The identity operation if e can be represented as u32 , otherwise it produces a shader-creation error . That is, produces the same result as u32(e). Component-wise when e is a vector.
Overload	<code>@const @must_use fn bitcast<T>(e : vec2<f16>) -> T</code>
Parameterization	T is i32, u32, or f32

Description	Component-wise reinterpretation of bits as T. The result is the reinterpetation of the 32 bits in e as a T value, following the internal layout rules.
Overload	<code>@const @must_use fn bitcast<vec2<T>>(e : vec4<f16>) -> vec2<T></code>
Parameterization	T is i32, u32, or f32
Description	Component-wise reinterpretation of bits as T. The result is the reinterpetation of the 64 bits in e as a T value, following the internal layout rules.
Overload	<code>@const @must_use fn bitcast<vec2<f16>>(e : T) -> vec2<f16></code>
Parameterization	T is i32, u32, or f32
Description	Component-wise reinterpretation of bits as f16. The result is the reinterpetation of the 32 bits in e as an f16 value, following the internal layout rules.
Overload	<code>@const @must_use fn bitcast<vec4<f16>>(e : vec2<T>) -> vec4<f16></code>
Parameterization	T is i32, u32, or f32
Description	Component-wise reinterpretation of bits as vec2<f16>. The result is the reinterpetation of the 64 bits in e as an f16 value, following the internal layout rules.

17.3. Logical Built-in Functions

17.3.1. all

Overload	<code>@const @must_use fn all(e: vecN<bool>) -> bool</code>
Description	Returns true if each component of e is true.
Overload	<code>@const @must_use fn all(e: bool) -> bool</code>
Description	Returns e.

§ 17.3.2. any

Overload

```
@const @must_use fn any(e: vecN<bool>) -> bool
```

Description Returns true if any component of e is true.

Overload

```
@const @must_use fn any(e: bool) -> bool
```

Description Returns e.

§ 17.3.3. select

Overload

```
@const @must_use fn select(f: T,
                           t: T,
                           cond: bool) -> T
```

Parameterization T is [scalar](#) or [vector](#)

Description Returns t when cond is true, and f otherwise.

Overload

```
@const @must_use fn select(f: vecN<T>,
                           t: vecN<T>,
                           cond: vecN<bool>) -> vecN<T>
```

Parameterization T is [scalar](#)

Description [Component-wise](#) selection. Result component i is evaluated as select(f[i], t[i], cond[i]).

§ 17.4. Array Built-in Functions

§ 17.4.1. arrayLength

Overload

```
@must_use fn arrayLength(p: ptr<AS, array<E>, AM>) -> u32
```

Parameterization E is an element type for a [runtime-sized](#) array.
AS is one of [storage](#), [uniform](#), or [workgroup](#).
[access mode](#) AM is [read](#) or [read_write](#) and [must](#) be valid for AS.

NOTE: AS can only be [uniform](#) or [workgroup](#) when p is derived from the result of a [bufferView](#) or [bufferArrayView](#) built-in function.

Description	Returns NRuntime , the number of elements in the runtime-sized array. See § 13.3.4 Buffer Binding Determines Runtime-Sized Array Element Count When p is derived from a bufferView or bufferArrayView built-in function, NRuntime is based on the size of the buffer as determined by the analysis in § 17.14 Buffer View Built-in Functions .
-------------	--

EXAMPLE: GETTING THE NUMBER OF ELEMENTS IN A RUNTIME-SIZED ARRAY

```
struct PointLight {
    position : vec3f,
    color : vec3f,
}

struct LightStorage {
    pointCount : u32,
    point : array<PointLight>,
}

@group(0) @binding(1) var<storage> lights : LightStorage;

fn num_point_lights() -> u32 {
    return arrayLength( &lights.point );
}
```

§ 17.5. Numeric Built-in Functions

§ 17.5.1. abs

Overload	<code>@const @must_use fn abs(e: T) -> T</code>
Parameterization	S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S, or vecN<S>
Description	The absolute value of e. Component-wise when T is a vector. If e is a floating-point type, then the result is e with a positive sign bit. If e is an unsigned integer scalar type, then the result is e. If e is a signed integer scalar type and evaluates to the largest negative value, then the result is e.

§ 17.5.2. **acos**

Overload

`@const @must_use fn acos(e: T) -> T`

Parameterization

S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description

Returns the principal value, in radians, of the inverse cosine ($\cos^{-1}$) of e.
 That is, approximates x with $0 \leq x \leq \pi$, such that $\cos(x) = e$.

[Component-wise](#) when T is a vector.

Scalar [domain](#)Interval $[-1, 1]$ § 17.5.3. **acosh**

Overload

`@const @must_use fn acosh(x: T) -> T`

Parameterization

S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description

Returns the inverse hyperbolic cosine ($\cosh^{-1}$) of x, as a [hyperbolic angle](#).
 That is, approximates a with $0 \leq a \leq +\infty$, such that $\cosh(a) = x$.

[Component-wise](#) when T is a vector.

Scalar [domain](#)Interval $[1, +\infty]$ § 17.5.4. **asin**

Overload

`@const @must_use fn asin(e: T) -> T`

Parameterization

S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description

Returns the principal value, in radians, of the inverse sine ($\sin^{-1}$) of e.
 That is, approximates x with $-\pi/2 \leq x \leq \pi/2$, such that $\sin(x) = e$.

[Component-wise](#) when T is a vector.

Scalar [domain](#)Interval $[-1, 1]$

§ 17.5.5. asinh

Overload

```
@const @must_use fn asinh(y: T) -> T
```

Parameterization

S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description

Returns the inverse hyperbolic sine ($\sinh^{-1}$) of y , as a [hyperbolic angle](#).
That is, approximates a such that $\sinh(y) = a$.

[Component-wise](#) when T is a vector.

§ 17.5.6. atan

Overload

```
@const @must_use fn atan(e: T) -> T
```

Parameterization

S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description

Returns the principal value, in radians, of the inverse tangent ($\tan^{-1}$) of e .
That is, approximates x with $-\pi/2 \leq x \leq \pi/2$, such that $\tan(x) = e$.

[Component-wise](#) when T is a vector.

§ 17.5.7. atanh

Overload

```
@const @must_use fn atanh(t: T) -> T
```

Parameterization

S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description

Returns the inverse hyperbolic tangent ($\tanh^{-1}$) of t , as a [hyperbolic angle](#).
That is, approximates a such that $\tanh(a) = t$.

[Component-wise](#) when T is a vector.

Scalar [domain](#)

Interval $[-1, 1]$

§ 17.5.8. atan2

Overload

```
@const @must_use fn atan2(y: T,
                           x: T) -> T
```

Parameterization

S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description

Returns an [angle](#), in radians, in the interval $[-\pi, \pi]$ whose tangent is $y \div x$.

The quadrant selected by the result depends on the signs of y and x . For example, the function may be implemented as:

- $\text{atan}(y/x)$ when $x > 0$
- $\text{atan}(y/x) + \pi$ when $(x < 0)$ and $(y > 0)$
- $\text{atan}(y/x) - \pi$ when $(x < 0)$ and $(y < 0)$

NOTE: The error in the result is unbounded:

- When $\text{abs}(x)$ is very small, e.g. [subnormal](#) for its type,
- At the origin $(x,y) = (0,0)$, or
- When y is subnormal or infinite.

[Component-wise](#) when T is a vector.

§ 17.5.9. ceil

Overload

```
@const @must_use fn ceil(e: T) -> T
```

Parameterization

S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description

Returns the [ceiling](#) of e . [Component-wise](#) when T is a vector.

§ 17.5.10. clamp

Overload

```
@const @must_use fn clamp(e: T,
                           low: T,
                           high: T) -> T
```

Parameterization	S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S, or vecN<S>
Description	Restricts the value of e within a range. If T is an integer type, then the result is $\min(\max(e, \text{low}), \text{high})$. If T is a floating-point type, then the result is either $\min(\max(e, \text{low}), \text{high})$, or the median of the three values e, low, high. Component-wise when T is a vector. If low is greater than high, then: - It is a shader-creation error if low and high are const-expressions. - It is a pipeline-creation error if low and high are override-expressions.

§ 17.5.11. cos

Overload	<code>@const @must_use fn cos(e: T) -> T</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the cosine of e, where e is in radians. Component-wise when T is a vector.
Scalar domain	Interval $(-\infty, +\infty)$

§ 17.5.12. cosh

Overload	<code>@const @must_use fn cosh(a: T) -> T</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the hyperbolic cosine of a, where a is a hyperbolic angle. Approximates the pure mathematical function $(e^a + e^{-a}) \div 2$, but not necessarily computed that way. Component-wise when T is a vector

§ 17.5.13. countLeadingZeros

Overload

```
@const @must_use fn countLeadingZeros(e: T) -> T
```

Parameterization T is i32, u32, vecN<i32>, or vecN<u32>

Description The number of consecutive 0 bits starting from the most significant bit of e, when T is a scalar type.
 [Component-wise](#) when T is a vector.
 Also known as "clz" in some languages.

§ 17.5.14. countOneBits

Overload

```
@const @must_use fn countOneBits(e: T) -> T
```

Parameterization T is i32, u32, vecN<i32>, or vecN<u32>

Description The number of 1 bits in the representation of e.
 Also known as "population count".
 [Component-wise](#) when T is a vector.

§ 17.5.15. countTrailingZeros

Overload

```
@const @must_use fn countTrailingZeros(e: T) -> T
```

Parameterization T is i32, u32, vecN<i32>, or vecN<u32>

Description The number of consecutive 0 bits starting from the least significant bit of e, when T is a scalar type.
 [Component-wise](#) when T is a vector.
 Also known as "ctz" in some languages.

§ 17.5.16. cross

Overload

```
@const @must_use fn cross(a: vec3<T>,
                           b: vec3<T>) -> vec3<T>
```

Parameterization T is AbstractFloat, f32, or f16

Description	Returns the cross product of e1 and e2 .
Domain	Implied from linear terms given by a possible implementation: $a[1] \times b[2] - a[2] \times b[1]$ $a[2] \times b[0] - a[0] \times b[2]$ $a[0] \times b[1] - a[1] \times b[0]$

§ 17.5.17. degrees

Overload	<code>@const @must_use fn degrees(e1: T) -> T</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Converts radians to degrees, approximating $e1 \times 180 \div \pi$. Component-wise when T is a vector

§ 17.5.18. determinant

Overload	<code>@const @must_use fn determinant(e: matCxC<T>) -> T</code>
Parameterization	T is AbstractFloat, f32, or f16
Description	Returns the determinant of e .
Domain	Implied from linear terms in a standard mathematical definition of the determinant.

§ 17.5.19. distance

Overload	<code>@const @must_use fn distance(e1: T, e2: T) -> S</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the distance between e1 and e2 (e.g. <code>length(e1 - e2)</code>). The domain is all vectors (<i>e1</i> , <i>e2</i>) where the subtraction <i>e1</i> − <i>e2</i> is valid. That is, the set of all vectors except where e1 [<i>i</i>] and e2 [<i>i</i>] are the same infinite value, for some component <i>i</i> .

§ 17.5.20. dot

Overload

```
@const @must_use fn dot(e1: vecN<T>,
                        e2: vecN<T>) -> T
```

Parameterization T is AbstractInt, AbstractFloat, i32, u32, f32, or f16

Description Returns the dot product of e1 and e2.

Domain [Implied from linear terms](#) of the sum over terms $e1[i] \times e2[i]$.

§ 17.5.21. dot4U8Packed

Overload

```
@const @must_use fn dot4U8Packed(e1: u32,
                                   e2: u32) -> u32
```

Description e1 and e2 are interpreted as vectors with four 8-bit unsigned integer components. Return the unsigned integer dot product of these two vectors.

§ 17.5.22. dot4I8Packed

Overload

```
@const @must_use fn dot4I8Packed(e1: u32,
                                   e2: u32) -> i32
```

Description e1 and e2 are interpreted as vectors with four 8-bit signed integer components. Return the signed integer dot product of these two vectors. Each component is sign-extended to i32 before performing the multiply, and then the add operations are done in WGSL i32 (the addition cannot overflow since the result is mathematically guaranteed to be in the range from -65024 to 65536, which lies within the range of numbers that can be represented by i32).

§ 17.5.23. exp

Overload

```
@const @must_use fn exp(e1: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description Returns the natural exponentiation of e1 (e.g. e^{e1}). [Component-wise](#) when T is a vector.

§ 17.5.24. **exp2**

Overload

```
@const @must_use fn exp2(e: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description Returns 2 raised to the power e (e.g. 2^e). [Component-wise](#) when T is a vector.

§ 17.5.25. **extractBits (signed)**

Overload

```
@const @must_use fn extractBits(e: T,
                                offset: u32,
                                count: u32) -> T
```

Parameterization T is i32 or vecN<i32>

Description Reads bits from an integer, with sign extension.

When T is a scalar type, then:

- w is the bit width of T
- o = min(offset, w)
- c = min(count, w - o)
- The result is 0 if c is 0.
- Otherwise, bits 0..c - 1 of the result are copied from bits o..o + c - 1 of e. Other bits of the result are the same as bit c - 1 of the result.

[Component-wise](#) when T is a vector.

If count + offset is greater than w, then:

- It is a [shader-creation error](#) if count and offset are [const-expressions](#).
- It is a [pipeline-creation error](#) if count and offset are [override-expressions](#).

§ 17.5.26. **extractBits (unsigned)**

Overload

```
@const @must_use fn extractBits(e: T,
                                offset: u32,
                                count: u32) -> T
```

Parameterization	T is u32 or vecN<u32>
Description	Reads bits from an integer, without sign extension. When T is a scalar type, then: • w is the bit width of T • $o = \min(\text{offset}, w)$ • $c = \min(\text{count}, w - o)$ • The result is 0 if c is 0. • Otherwise, bits $0..c - 1$ of the result are copied from bits $o..o + c - 1$ of e. Other bits of the result are 0. Component-wise when T is a vector. If $\text{count} + \text{offset}$ is greater than w, then: • It is a shader-creation error if count and offset are const-expressions. • It is a pipeline-creation error if count and offset are override-expressions.

§ 17.5.27. faceForward

Overload

```
@const @must_use fn faceForward(e1: T,
                                e2: T,
                                e3: T) -> T
```

Parameterization	T is vecN<AbstractFloat>, vecN<f32>, or vecN<f16>
Description	Returns e1 if dot(e2, e3) is negative, and -e1 otherwise.
Domain	The domain restrictions arise from the dot(e2, e3) operation: they are implied from linear terms of the sum over terms $e2[i] \times e3[i]$.

§ 17.5.28. firstLeadingBit (signed)

Overload

```
@const @must_use fn firstLeadingBit(e: T) -> T
```

Parameterization	T is i32 or vecN<i32>
------------------	-----------------------

Description	For scalar T, the result is: • -1 if e is 0 or -1. • Otherwise the position of the most significant bit in e that is different from e's sign bit. Component-wise when T is a vector.
-------------	---

NOTE: Since signed integers use twos-complement representation, the sign bit appears in the most significant bit position.

§ 17.5.29. firstLeadingBit (unsigned)

Overload

```
@const @must_use fn firstLeadingBit(e: T) -> T
```

Parameterization	T is u32 or vecN<u32>
------------------	-----------------------

Description	For scalar T, the result is: • T(-1) if e is zero. • Otherwise the position of the most significant 1 bit in e. Component-wise when T is a vector.
-------------	---

§ 17.5.30. firstTrailingBit

Overload

```
@const @must_use fn firstTrailingBit(e: T) -> T
```

Parameterization	T is i32, u32, vecN<i32>, or vecN<u32>
------------------	--

Description	For scalar T, the result is: • T(-1) if e is zero. • Otherwise the position of the least significant 1 bit in e. Component-wise when T is a vector.
-------------	--

§ 17.5.31. floor

Overload

```
@const @must_use fn floor(e: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description Returns the [floor](#) of e. [Component-wise](#) when T is a vector.

§ 17.5.32. fma

Overload

```
@const @must_use fn fma(e1: T,  
                          e2: T,  
                          e3: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description Returns $e1 * e2 + e3$. [Component-wise](#) when T is a vector.

NOTE: The name fma is short for "fused multiply add".

NOTE: The [IEEE-754](#) fusedMultiplyAdd operation computes the [intermediate results](#) as if with unbounded range and precision, and only the final result is [rounded](#) to a value in the destination type. However, the [§ 15.7.4 Floating Point Accuracy](#) rule for fma allows an implementation which performs an ordinary multiply to the target type followed by an ordinary addition. In this case the [intermediate result](#) values may overflow or lose accuracy, and the overall operation is not "fused" at all.

Domain [Implied from linear terms](#) of the expressions $e2 \times e2 + e3$.

§ 17.5.33. fract

Overload

```
@const @must_use fn fract(e: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description Returns the fractional part of e, computed as $e - \text{floor}(e)$.
[Component-wise](#) when T is a vector.

NOTE: Valid results are in the closed interval [0, 1.0]. For example, if `e` is a very small negative number, then `fract(e)` may be 1.0.

§ 17.5.34. frexp

Overload

```
@const @must_use fn frexp(e: T) -> __frexp_result_f32
```

Parameterization

`T` is `f32`

Description

Splits `e` into a fraction and an exponent.

- When `e` is zero, the fraction is zero.
- When `e` is non-zero and normal, $e = \text{fraction} * 2^{\text{exponent}}$, where the fraction is in the range [0.5, 1.0) or (-1.0, -0.5].
- Otherwise, `e` is [subnormal](#), NaN, or infinite. The result fraction and exponent are [indeterminate values](#).

Returns the `__frexp_result_f32` built-in structure, defined as follows:

```
struct __frexp_result_f32 {
    fract : f32, // fraction part
    exp   : i32  // exponent part
}
```

NOTE: A mnemonic for the name `frexp` is "**f**raction and **e**xponent".

EXAMPLE: FREXP USAGE

```
// Infers result type
let fraction_and_exponent = frexp(1.5);
// Sets fraction_only to 0.75
let fraction_only = frexp(1.5).fract;
```

NOTE: A value cannot be explicitly declared with the type `__frexp_result_f32`, but a value may infer the type.

Overload

```
@const @must_use fn frexp(e: T) -> __frexp_result_f16
```

Parameterization	T is f16
Description	Splits e into a fraction and an exponent. - When e is zero, the fraction is zero. - When e is non-zero and normal, $e = \text{fraction} * 2^{\text{exponent}}$, where the fraction is in the range [0.5, 1.0) or (-1.0, -0.5]. - Otherwise, e is subnormal, NaN, or infinite. The result fraction and exponent are indeterminate values. Returns the <code>__frexp_result_f16</code> built-in structure, defined as if as follows: <pre>struct __frexp_result_f16 { fract : f16, // fraction part exp : i32 // exponent part }</pre> NOTE: A mnemonic for the name <code>frexp</code> is "fraction and exponent".
Overload	<pre>@const @must_use fn frexp(e: T) -> __frexp_result_abstract</pre>
Parameterization	T is AbstractFloat
Description	Splits e into a fraction and an exponent. - When e is zero, the fraction is zero. - When e is non-zero and normal, $e = \text{fraction} * 2^{\text{exponent}}$, where the fraction is in the range [0.5, 1.0) or (-1.0, -0.5]. - When e is subnormal, the fraction and exponent are have unbounded error. The fraction may be any AbstractFloat value, and the exponent may be any AbstractInt value. NOTE: AbstractFloat expressions resulting in infinity or NaN cause a shader-creation error. Returns the <code>__frexp_result_abstract</code> built-in structure, defined as follows: <pre>struct __frexp_result_abstract { fract : AbstractFloat, // fraction part exp : AbstractInt // exponent part }</pre> NOTE: A mnemonic for the name <code>frexp</code> is "fraction and exponent".

EXAMPLE: ABSTRACT FREXP USAGE

```
// Infers result type
const fraction_and_exponent = frexp(1.5);
// Sets fraction_only to 0.75
const fraction_only = frexp(1.5).fract;
```

NOTE: A value cannot be explicitly declared with the type `__frexp_result_abstract`, but a value may infer the type.

Overload

```
@const @must_use fn frexp(e: T) -> __frexp_result_vecN_f32
```

Parameterization T is `vecN<f32>`

Description

Splits components `ei` of `e` into a fraction and an exponent.

- When `ei` is zero, the fraction is zero.
- When `ei` is non-zero and normal, $ei = \text{fraction} * 2^{\text{exponent}}$, where the fraction is in the range `[0.5, 1.0)` or `(-1.0, -0.5]`.
- Otherwise, `ei` is NaN or infinite. The result fraction and exponent are [indeterminate values](#).

Returns the `__frexp_result_vecN_f32` built-in structure, defined as follows:

```
struct __frexp_result_vecN_f32 {
    fract : vecN<f32>, // fraction part
    exp : vecN<i32>    // exponent part
}
```

NOTE: A mnemonic for the name `frexp` is "**f**raction and **e**xponent".

NOTE: A value cannot be explicitly declared with the type `__frexp_result_vecN_f32`, but a value may infer the type.

Overload

```
@const @must_use fn frexp(e: T) -> __frexp_result_vecN_f16
```

Parameterization T is `vecN<f16>`

Description

Splits components `ei` of `e` into a fraction and an exponent.

- When `ei` is zero, the fraction is zero.

- When e_i is non-zero and normal, $e_i = \text{fraction} * 2^{\text{exponent}}$, where the fraction is in the range $[0.5, 1.0)$ or $(-1.0, -0.5]$.
- Otherwise, e_i is NaN or infinite. The result fraction and exponent are [indeterminate values](#).

Returns the `__frexp_result_vecN_f16` built-in structure, defined as if as follows:

```
struct __frexp_result_vecN_f16 {
    fract : vecN<f16>, // fraction part
    exp   : vecN<i32>   // exponent part
}
```

NOTE: A mnemonic for the name `frexp` is "**f**raction and **e**xponent".

NOTE: A value cannot be explicitly declared with the type `__frexp_result_vecN_f16`, but a value may infer the type.

Overload

```
@const @must_use fn frexp(e: T) -> __frexp_result_vecN_abstract
```

Parameterization T is `vecN<AbstractFloat>`

Description Splits components e_i of e into a fraction and an exponent.

- When e_i is zero, the fraction is zero.
- When e_i is non-zero and normal, $e_i = \text{fraction} * 2^{\text{exponent}}$, where the fraction is in the range $[0.5, 1.0)$ or $(-1.0, -0.5]$.
- When e_i is [subnormal](#), the fraction and exponent are have unbounded error. The fraction may be any `AbstractFloat` value, and the exponent may be any `AbstractInt` value.

NOTE: `AbstractFloat` expressions resulting in infinity or NaN cause a [shader-creation error](#).

Returns the `__frexp_result_vecN_abstract` built-in structure, defined as follows:

```
struct __frexp_result_vecN_abstract {
    fract : vecN<AbstractFloat>, // fraction part
    exp   : vecN<AbstractInt>    // exponent part
}
```

NOTE: A mnemonic for the name `frexp` is "**f**raction and **e**xponent".

NOTE: A value cannot be explicitly declared with the type `__frexp_result_vecN_abstract`, but a value may infer the type.

§ 17.5.35. insertBits

Overload

```
@const @must_use fn insertBits(e: T,
                                newbits: T,
                                offset: u32,
                                count: u32) -> T
```

Parameterization	T is i32, u32, vecN<i32>, or vecN<u32>
Description	Sets bits in an integer. When T is a scalar type, then: • w is the bit width of T • o = min(offset, w) • c = min(count, w - o) • The result is e if c is 0. • Otherwise, bits o..o + c - 1 of the result are copied from bits 0..c - 1 of newbits. Other bits of the result are copied from e. Component-wise when T is a vector. If count + offset is greater than w, then: • It is a shader-creation error if count and offset are const-expressions. • It is a pipeline-creation error if count and offset are override-expressions.

§ 17.5.36. inverseSqrt

Overload

```
@const @must_use fn inverseSqrt(e: T) -> T
```

Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the reciprocal of sqrt(e). Component-wise when T is a vector.
Scalar domain	Interval [0, +∞]

§ 17.5.37. ldexp

Overload

```
@const @must_use fn ldexp(e1: T,
                           e2: I) -> T
```

Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S> I is AbstractInt, i32, vecN<AbstractInt>, or vecN<i32> I is a vector if and only if T is a vector T can only be abstract if I is also abstract and vice versa NOTE: If either parameter is concrete then the other parameter will undergo automatic conversion to a concrete type (if applicable) and the result will be a concrete type.
Description	Returns $e1 * 2^{e2}$, except: - The result may be zero if $e2 + bias \leq 0$. - If $e2 > bias + 1$ - It is a shader-creation error if $e2$ is a const-expression. - It is a pipeline-creation error if $e2$ is an override-expression. - Otherwise the result is an indeterminate value for T. Here, <i>bias</i> is the exponent bias of the floating point format: 15 for f16 127 for f32 1023 for AbstractFloat, when AbstractFloat is IEEE-754 binary64 If x is zero or a finite normal value for its type, then: $x = ldexp(frexp(x).fract, frexp(x).exp)$ Component-wise when T is a vector. NOTE: A mnemonic for the name ldexp is "load exponent". The name may have been taken from the corresponding instruction in the floating point unit of the PDP-11.

§ 17.5.38. length

Overload	<pre>@const @must_use fn length(e: T) -> S</pre>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the length of e. Evaluates to the absolute value of e if T is scalar. Evaluates to $\sqrt{e[0]^2 + e[1]^2 + \dots}$ if T is a vector type.

NOTE: The scalar case may be evaluated as `sqrt(e * e)`, which may unnecessarily overflow or lose accuracy.

§ 17.5.39. log

Overload	<code>@const @must_use fn log(e: T) -> T</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the natural logarithm of e. Component-wise when T is a vector.
Scalar domain	Interval [0, +∞]

§ 17.5.40. log2

Overload	<code>@const @must_use fn log2(e: T) -> T</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the base-2 logarithm of e. Component-wise when T is a vector.
Scalar domain	Interval [0, +∞]

§ 17.5.41. max

Overload	<code>@const @must_use fn max(e1: T, e2: T) -> T</code>
Parameterization	S is AbstractInt, AbstractFloat, i32, u32, f32, or f16 T is S, or vecN<S>
Description	Returns e2 if e1 is less than e2, and e1 otherwise. Component-wise when T is a vector. If e1 and e2 are floating-point values, then: • If both e1 and e2 are subnormal, then the result may be <i>either</i> value.

§ 17.5.42. min

Overload

```
@const @must_use fn min(e1: T,
                          e2: T) -> T
```

Parameterization

S is AbstractInt, AbstractFloat, i32, u32, f32, or f16
T is S, or vecN<S>

Description

Returns e2 if e2 is less than e1, and e1 otherwise. [Component-wise](#) when T is a vector.

If e1 and e2 are floating-point values, then:

- If both e1 and e2 are [subnormal](#), then the result may be *either* value.

§ 17.5.43. mix

Overload

```
@const @must_use fn mix(e1: T,
                          e2: T,
                          e3: T) -> T
```

Parameterization

S is AbstractFloat, f32, or f16
T is S or vecN<S>

Description

Returns the linear blend of e1 and e2 (e.g. $e1 * (T(1) - e3) + e2 * e3$). [Component-wise](#) when T is a vector.

Domain

[Implied from linear terms](#) of the expressions: $e1[i] \times (1 - e3[i]) + e2[i] \times e3[i]$. $e2[i] \times e2[i] + e3[i]$.

Overload

```
@const @must_use fn mix(e1: T2,
                          e2: T2,
                          e3: T) -> T2
```

Parameterization

T is AbstractFloat, f32, or f16
T2 is vecN<T>

Description

Returns the component-wise linear blend of e1 and e2, using scalar blending factor e3 for each component.
Same as `mix(e1, e2, T2(e3))`.

Domain

[Implied from linear terms](#) of the expressions: $e1[i] \times (1 - e3) + e2[i] \times e3$.

§ 17.5.44. modf

Overload	<pre>@const @must_use fn modf(e: T) -> __modf_result_f32</pre>
Parameterization	T is f32
Description	Splits e into fractional and whole number parts. The whole part is trunc(e), and the fractional part is e - trunc(e). Returns the __modf_result_f32 built-in structure, defined as follows: <pre>struct __modf_result_f32 { fract : f32, // fractional part whole : f32 // whole part }</pre>
EXAMPLE: MODF USAGE <pre>// Infers result type let fract_and_whole = modf(1.5); // Sets fract_only to 0.5 let fract_only = modf(1.5).fract; // Sets whole_only to 1.0 let whole_only = modf(1.5).whole;</pre>	
NOTE: A value cannot be explicitly declared with the type __modf_result_f32, but a value may infer the type.	
Overload	<pre>@const @must_use fn modf(e: T) -> __modf_result_f16</pre>
Parameterization	T is f16
Description	Splits e into fractional and whole number parts. The whole part is trunc(e), and the fractional part is e - trunc(e). Returns the __modf_result_f16 built-in structure, defined as if as follows: <pre>struct __modf_result_f16 { fract : f16, // fractional part whole : f16 // whole part }</pre>

NOTE: A value cannot be explicitly declared with the type `__modf_result_f16`, but a value may infer the type.

Overload

```
@const @must_use fn modf(e: T) -> __modf_result_abstract
```

Parameterization T is AbstractFloat

Description Splits `e` into fractional and whole number parts.

The whole part is `trunc(e)`, and the fractional part is `e - trunc(e)`.

Returns the `__modf_result_abstract` built-in structure, defined as follows:

```
struct __modf_result_abstract {
    fract : AbstractFloat, // fractional part
    whole : AbstractFloat  // whole part
}
```

EXAMPLE: MODF ABSTRACT USAGE

```
// Infers result type
const fract_and_whole = modf(1.5);
// Sets fract_only to 0.5
const fract_only = modf(1.5).fract;
// Sets whole_only to 1.0
const whole_only = modf(1.5).whole;
```

NOTE: A value cannot be explicitly declared with the type `__modf_result_abstract`, but a value may infer the type.

Overload

```
@const @must_use fn modf(e: T) -> __modf_result_vecN_f32
```

Parameterization T is vecN<f32>

Description Splits the components of `e` into fractional and whole number parts.

The `i`'th component of the whole and fractional parts equal the whole and fractional parts of `modf(e[i])`.

Returns the `__modf_result_vecN_f32` built-in structure, defined as follows:

```
struct __modf_result_vecN_f32 {
    fract : vecN<f32>, // fractional part
    whole : vecN<f32>  // whole part
}
```

NOTE: A value cannot be explicitly declared with the type `__modf_result_vecN_f32`, but a value may infer the type.

Overload

```
@const @must_use fn modf(e: T) -> __modf_result_vecN_f16
```

Parameterization	T is <code>vecN<f16></code>
Description	Splits the components of <code>e</code> into fractional and whole number parts. The <code>i</code>'th component of the whole and fractional parts equal the whole and fractional parts of <code>modf(e[i])</code>. Returns the <code>__modf_result_vecN_f16</code> built-in structure, defined as if as follows: <pre>struct __modf_result_vecN_f16 { fract : vecN<f16>, // fractional part whole : vecN<f16> // whole part }</pre>
	NOTE: A value cannot be explicitly declared with the type <code>__modf_result_vecN_f16</code>, but a value may infer the type.
Overload	<pre>@const @must_use fn modf(e: T) -> __modf_result_vecN_abstract</pre>
Parameterization	T is <code>vecN<AbstractFloat></code>

Description	Splits the components of <code>e</code> into fractional and whole number parts. The <code>i</code>'th component of the whole and fractional parts equal the whole and fractional parts of <code>modf(e[i])</code>. Returns the <code>__modf_result_vecN_abstract</code> built-in structure, defined as follows: <pre>struct __modf_result_vecN_abstract { fract : vecN<AbstractFloat>, // fractional part whole : vecN<AbstractFloat> // whole part }</pre>
-------------	---

§ 17.5.45. `normalize`

Overload	NOTE: A value cannot be explicitly declared with the type <code>__modf_result_vecN_abstract</code>, but a value may infer the type. <pre>@const @must_use fn normalize(e: vecN<T>) -> vecN<T></pre>
Parameterization	<code>T</code> is <code>AbstractFloat</code> , <code>f32</code> , or <code>f16</code>
Description	Returns a unit vector in the same direction as <code>e</code>. The domain is all vectors except the zero vector.

§ 17.5.46. `pow`

Overload	<pre>@const @must_use fn pow(e1: T, e2: T) -> T</pre>
Parameterization	<code>S</code> is <code>AbstractFloat</code>, <code>f32</code>, or <code>f16</code> <code>T</code> is <code>S</code> or <code>vecN<S></code>
Description	Returns <code>e1</code> raised to the power <code>e2</code> . Component-wise when <code>T</code> is a vector.
Scalar domain	The set of all pairs of extended reals (x,y) except: • $x < 0$. • x is 1 and y is infinite. • x is infinite and y is 0. This rule arises from the fact the result may be computed as $\exp_2(y * \log_2(x))$.

§ 17.5.47. `quantizeToF16`

Overload

```
@const @must_use fn quantizeToF16(e: T) -> T
```

Parameterization	T is f32 or vecN<f32>
Description	Quantizes a 32-bit floating point value e as if e were converted to a IEEE-754 binary16 value, and then converted back to a IEEE-754 binary32 value. If e is outside the finite range of binary16, then: - It is a shader-creation error if e is a const-expression. - It is a pipeline-creation error if e is an override-expression. - Otherwise the result is an indeterminate value for T. The intermediate binary16 value may be flushed to zero, i.e. the final result may be zero if the intermediate binary16 value is subnormal. See § 15.7.6 Floating Point Conversion. Component-wise when T is a vector.

NOTE: The vec2<f32> case is the same as `unpack2x16float(pack2x16float(e))`.

§ 17.5.48. `radians`

Overload

```
@const @must_use fn radians(e1: T) -> T
```

Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Converts degrees to radians, approximating $e1 \times \pi \div 180$. Component-wise when T is a vector

§ 17.5.49. `reflect`

Overload

```
@const @must_use fn reflect(e1: T,
                             e2: T) -> T
```

Parameterization	T is vecN<AbstractFloat>, vecN<f32>, or vecN<f16>
Description	For the incident vector e1 and surface orientation e2 , returns the reflection direction $\mathbf{e1} - 2 * \text{dot}(\mathbf{e2}, \mathbf{e1}) * \mathbf{e2}$.

§ 17.5.50. **refract**

Overload

```
@const @must_use fn refract(e1: T,
                             e2: T,
                             e3: I) -> T
```

Parameterization	T is vecN<I> I is AbstractFloat, f32, or f16
Description	For the incident vector e1 and surface normal e2 , and the ratio of indices of refraction e3 , let $k = 1.0 - e3 * e3 * (1.0 - \text{dot}(\mathbf{e2}, \mathbf{e1}) * \text{dot}(\mathbf{e2}, \mathbf{e1}))$. If $k < 0.0$, returns the refraction vector 0.0, otherwise return the refraction vector $e3 * \mathbf{e1} - (e3 * \text{dot}(\mathbf{e2}, \mathbf{e1}) + \text{sqrt}(k)) * \mathbf{e2}$. The incident vector e1 and the normal e2 should be normalized for desired results according to Snell's Law; otherwise, the results may not conform to expected physical behavior.

§ 17.5.51. **reverseBits**

Overload

```
@const @must_use fn reverseBits(e: T) -> T
```

Parameterization	T is i32, u32, vecN<i32>, or vecN<u32>
Description	Reverses the bits in e : The bit at position k of the result equals the bit at position $31 - k$ of e . Component-wise when T is a vector.

§ 17.5.52. **round**

Overload

```
@const @must_use fn round(e: T) -> T
```

Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Result is the integer k nearest to e , as a floating point value. When e lies halfway between integers k and $k + 1$, the result is k when k is even, and $k + 1$ when k is odd. Component-wise when T is a vector.

§ 17.5.53. saturate

Overload	<code>@const @must_use fn saturate(e: T) -> T</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns clamp(e, 0.0, 1.0). Component-wise when T is a vector.

§ 17.5.54. sign

Overload	<code>@const @must_use fn sign(e: T) -> T</code>
Parameterization	S is AbstractInt, AbstractFloat, i32, f32, or f16 T is S, or vecN<S>
Description	Result is: • 1 when e > 0 • 0 when e = 0 • -1 when e < 0 Component-wise when T is a vector.

§ 17.5.55. sin

Overload	<code>@const @must_use fn sin(e: T) -> T</code>
Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
Description	Returns the sine of e, where e is in radians. Component-wise when T is a vector.
Scalar domain	Interval $(-\infty, +\infty)$

§ 17.5.56. sinh

Overload

```
@const @must_use fn sinh(a: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description Returns the hyperbolic sine of *a*, where *a* is a [hyperbolic angle](#). Approximates the pure mathematical function $(e^a - e^{-a})/2$, but not necessarily computed that way.

[Component-wise](#) when T is a vector.

§ 17.5.57. smoothstep

Overload

```
@const @must_use fn smoothstep(edge0: T,
                                edge1: T,
                                x: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description Returns the smooth Hermite interpolation between 0 and 1. [Component-wise](#) when T is a vector.

For scalar T, the result is $t * t * (3.0 - 2.0 * t)$,
 where $t = \text{clamp}((x - \text{edge0}) / (\text{edge1} - \text{edge0}), 0.0, 1.0)$.

Qualitatively:

- When $\text{edge0} < \text{edge1}$, the function is 0 for *x* below *edge0*, then smoothly rises until *x* reaches *edge1*, and remains at 1 afterward.
- When $\text{edge0} > \text{edge1}$, the function is 1 for *x* below *edge1*, then smoothly descends until *x* reaches *edge0*, and remains at 0 afterward.

If $\text{edge0} = \text{edge1}$:

- It is a [shader-creation error](#) if *edge0* and *edge1* are [const-expressions](#).
- It is a [pipeline-creation error](#) if *edge0* and *edge1* are [override-expressions](#).
- Otherwise, the result is an [indeterminate value](#) for T. In this case the computation performs a floating point [division by zero](#), and the [Finite Math Assumption](#) applies.

§ 17.5.58. **sqrt**

Overload

```
@const @must_use fn sqrt(e: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description Returns the square root of e. [Component-wise](#) when T is a vector.

Scalar [domain](#) Interval $[0, +\infty]$

§ 17.5.59. **step**

Overload

```
@const @must_use fn step(edge: T,
                           x: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description Returns 1.0 if $\text{edge} \leq x$, and 0.0 otherwise. [Component-wise](#) when T is a vector.

§ 17.5.60. **tan**

Overload

```
@const @must_use fn tan(e: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description Returns the tangent of e, where e is in radians. [Component-wise](#) when T is a vector.

Scalar [domain](#) Interval $(-\infty, +\infty)$

§ 17.5.61. **tanh**

Overload

```
@const @must_use fn tanh(a: T) -> T
```

Parameterization S is AbstractFloat, f32, or f16
 T is S or vecN<S>

Description	Returns the hyperbolic tangent of a, where a is a hyperbolic angle . Approximates the pure mathematical function $(e^a - e^{-a}) \div (e^a + e^{-a})$ but not necessarily computed that way. Component-wise when T is a vector.
-------------	--

§ 17.5.62. transpose

Overload	<code>@const @must_use fn transpose(e: matRxC<T>) -> matCxR<T></code>
----------	--

Parameterization	T is AbstractFloat, f32, or f16
------------------	---------------------------------

Description	Returns the transpose of e.
-------------	-----------------------------

§ 17.5.63. trunc

Overload	<code>@const @must_use fn trunc(e: T) -> T</code>
----------	--

Parameterization	S is AbstractFloat, f32, or f16 T is S or vecN<S>
------------------	--

Description	Returns truncate (e), the nearest whole number whose absolute value is less than or equal to the absolute value of e. Component-wise when T is a vector.
-------------	--

§ 17.6. Derivative Built-in Functions

See [§ 15.6.2 Derivatives](#).

Calls to these functions:

- [Must](#) only be used in a [fragment](#) shader stage.
- [Trigger](#) a [derivative_uniformity diagnostic](#) if [uniformity analysis](#) cannot prove the call is in [uniform control flow](#).

§ 17.6.1. dpdx

Overload	<code>@must_use fn dpdx(e: T) -> T</code>
----------	--

Parameterization	T is f32 or vecN<f32>
------------------	-----------------------

Description	Partial derivative of e with respect to window x coordinates. The result is the same as either dpdxFine (e) or dpdxCoarse (e).
-------------	--

Returns an [indeterminate value](#) if called in [non-uniform control flow](#).

§ 17.6.2. dpdxCoarse

Overload

```
@must_use fn dpdxCoarse(e: T) -> T
```

Parameterization	T is f32 or vecN<f32>
Description	Returns the partial derivative of e with respect to window x coordinates using local differences. This may result in fewer unique positions than dpdxFine(e). Returns an indeterminate value if called in non-uniform control flow.

§ 17.6.3. dpdxFine

Overload

```
@must_use fn dpdxFine(e: T) -> T
```

Parameterization	T is f32 or vecN<f32>
Description	Returns the partial derivative of e with respect to window x coordinates. Returns an indeterminate value if called in non-uniform control flow.

§ 17.6.4. dpdy

Overload

```
@must_use fn dpdy(e: T) -> T
```

Parameterization	T is f32 or vecN<f32>
Description	Partial derivative of e with respect to window y coordinates. The result is the same as either dpdyFine(e) or dpdyCoarse(e). Returns an indeterminate value if called in non-uniform control flow.

§ 17.6.5. dpdyCoarse

Overload

```
@must_use fn dpdyCoarse(e: T) -> T
```

Parameterization T is f32 or vecN<f32>

Description Returns the partial derivative of e with respect to window y coordinates using local differences. This may result in fewer unique positions than dpdyFine(e).

Returns an [indeterminate value](#) if called in [non-uniform control flow](#).

§ 17.6.6. dpdyFine

Overload

```
@must_use fn dpdyFine(e: T) -> T
```

Parameterization T is f32 or vecN<f32>

Description Returns the partial derivative of e with respect to window y coordinates.

Returns an [indeterminate value](#) if called in [non-uniform control flow](#).

§ 17.6.7. fwidth

Overload

```
@must_use fn fwidth(e: T) -> T
```

Parameterization T is f32 or vecN<f32>

Description Returns $\text{abs}(\text{dpdx}(e)) + \text{abs}(\text{dpdy}(e))$.

Returns an [indeterminate value](#) if called in [non-uniform control flow](#).

§ 17.6.8. fwidthCoarse

Overload

```
@must_use fn fwidthCoarse(e: T) -> T
```

Parameterization T is f32 or vecN<f32>

Description	Returns <code>abs(dpdxCourse(e)) + abs(dpdyCourse(e))</code> . Returns an indeterminate value if called in non-uniform control flow .
-------------	--

§ 17.6.9. fwidthFine

Overload	<code>@must_use fn fwidthFine(e: T) -> T</code>
Parameterization	<code>T</code> is <code>f32</code> or <code>vecN<f32></code>
Description	Returns <code>abs(dpdxFine(e)) + abs(dpdyFine(e))</code> . Returns an indeterminate value if called in non-uniform control flow .

§ 17.7. Texture Built-in Functions

Parameter values [must](#) be valid for the respective texture types.

§ 17.7.1. textureDimensions

Returns the dimensions of a texture, or texture’s mip level in texels.

Parameterization	Overload
<code>ST</code> is i32 , u32 , or f32 <code>F</code> is a texel format <code>A</code> is an access mode	<code>@must_use fn textureDimensions(t: T) -> u32</code>
<code>T</code> is <code>texture_1d<ST></code> or <code>texture_storage_1d<F,A></code>	
<code>ST</code> is i32 , u32 , or f32 <code>T</code> is <code>texture_1d<ST></code> <code>L</code> is i32 , or u32	<code>@must_use fn textureDimensions(t: T, level: L) -> u32</code>
<code>ST</code> is i32 , u32 , or f32 <code>F</code> is a texel format <code>A</code> is an access mode	<code>@must_use fn textureDimensions(t: T) -> vec2<u32></code>
<code>T</code> is <code>texture_2d<ST></code> , <code>texture_2d_array<ST></code> , <code>texture_cube<ST></code> , <code>texture_cube_array<ST></code> ,	

Parameterization	Overload
texture_multisampled_2d<ST>, texture_depth_2d, texture_depth_2d_array, texture_depth_cube, texture_depth_cube_array, texture_depth_multisampled_2d, texture_storage_2d<F,A>, texture_storage_2d_array<F,A>, or texture_external	
<i>ST</i> is i32 , u32 , or f32 <i>T</i> is texture_2d<ST>, texture_2d_array<ST>, texture_cube<ST>, texture_cube_array<ST>, texture_depth_2d, texture_depth_2d_array, texture_depth_cube, or texture_depth_cube_array <i>L</i> is i32 , or u32	<pre>@must_use fn textureDimensions(t: T, level: L) -> vec2<u32></pre>
<i>ST</i> is i32 , u32 , or f32 <i>F</i> is a texel format <i>A</i> is an access mode	<pre>@must_use fn textureDimensions(t: T) -> vec3<u32></pre>
<i>T</i> is texture_3d<ST> or texture_storage_3d<F,A>	
<i>ST</i> is i32 , u32 , or f32 <i>T</i> is texture_3d<ST> <i>L</i> is i32 , or u32	<pre>@must_use fn textureDimensions(t: T, level: L) -> vec3<u32></pre>

Parameters:

t	The sampled , multisampled , depth , storage , or external texture.
level	The mip level , with level 0 containing a full size version of the texture. If omitted, the dimensions of level 0 are returned.

Returns:

The coordinate dimensions of the texture.

That is, the result provides the integer bounds on the coordinates of the [logical texel address](#), excluding the [mip level count](#), [array size](#), and [sample count](#).

For textures based on cubes, the results are the dimensions of each face of the cube. Cube faces are square, so the x and y components of the result are equal.

If `level` is outside the range `[0, textureNumLevels(t))` then an [indeterminate value](#) for the return type may be returned.

§ 17.7.2. `textureGather`

A *texture gather* operation reads from a 2D, 2D array, cube, or cube array texture, computing a four-component vector as follows:

- Find the four texels that would be used in a sampling operation with linear filtering, from [mip level](#) 0:
 - Use the specified coordinate, array index (when present), and offset (when present).
 - The texels are adjacent, forming a square, when considering their texture space coordinates (u,v) .
 - Selected texels at the texture edge, cube face edge, or cube corners are handled as in ordinary texture sampling.
- For each texel, read one channel and convert it into a scalar value.
 - For non-depth textures, a zero-based component parameter specifies the channel to use.
 - If the texture format supports the specified channel, i.e. has more than component channels:
 - Yield scalar value `v[component]` when the texel value is `v`.
 - Otherwise:
 - Yield 0.0 when component is 1 or 2.
 - Yield 1.0 when component is 3 (the alpha channel).
 - For [depth textures](#), yield the texel value. (Depth textures only have one channel.)
- Yield the four-component vector, arranging scalars produced by the previous step into components according to the relative coordinates of the texels, as follows:

◦ **Result component Relative texel coordinate**

x	$(u_{\min}, v_{\max})$
y	$(u_{\max}, v_{\max})$
z	$(u_{\max}, v_{\min})$
w	$(u_{\min}, v_{\min})$

The four texels form the sampled area as described in the [WebGPU sampler descriptor](#).

Parameterization	Overload
C is i32 , or u32 ST is i32 , u32 , or f32	<pre>@must_use fn textureGather(component: C, t: texture_2d<ST>, s: sampler, coords: vec2<f32>) -> vec4<ST></pre>
C is i32 , or u32 ST is i32 , u32 , or f32	<pre>@must_use fn textureGather(component: C, t: texture_2d<ST>, s: sampler,</pre>

Parameterization	Overload
	<pre>coords: vec2<f32>, offset: vec2<i32>) -> vec4<ST></pre>
<i>C</i> is i32, or u32 <i>A</i> is i32, or u32 <i>ST</i> is i32, u32, or f32	<pre>@must_use fn textureGather(component: C, t: texture_2d_array<ST>, s: sampler, coords: vec2<f32>, array_index: A) -> vec4<ST></pre>
<i>C</i> is i32, or u32 <i>A</i> is i32, or u32 <i>ST</i> is i32, u32, or f32	<pre>@must_use fn textureGather(component: C, t: texture_2d_array<ST>, s: sampler, coords: vec2<f32>, array_index: A, offset: vec2<i32>) -> vec4<ST></pre>
<i>C</i> is i32, or u32 <i>ST</i> is i32, u32, or f32	<pre>@must_use fn textureGather(component: C, t: texture_cube<ST>, s: sampler, coords: vec3<f32>) -> vec4<ST></pre>
<i>C</i> is i32, or u32 <i>A</i> is i32, or u32 <i>ST</i> is i32, u32, or f32	<pre>@must_use fn textureGather(component: C, t: texture_cube_array<ST>, s: sampler, coords: vec3<f32>, array_index: A) -> vec4<ST></pre>
	<pre>@must_use fn textureGather(t: texture_depth_2d, s: sampler, coords: vec2<f32>) -> vec4<f32></pre>
	<pre>@must_use fn textureGather(t: texture_depth_2d, s: sampler, coords: vec2<f32>, offset: vec2<i32>) -> vec4<f32></pre>
	<pre>@must_use fn textureGather(t: texture_depth_cube, s: sampler, coords: vec3<f32>) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureGather(t: texture_depth_2d_array, s: sampler,</pre>

Parameterization	Overload
	<pre>coords: vec2<f32>, array_index: A) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureGather(t: texture_depth_2d_array, s: sampler, coords: vec2<f32>, array_index: A, offset: vec2<i32>) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureGather(t: texture_depth_cube_array, s: sampler, coords: vec3<f32>, array_index: A) -> vec4<f32></pre>

Parameters:

component	Only applies to non-depth textures. The index of the channel to read from the selected texels. When provided, the component expression must be a const-expression (e.g. 1). Its value must be at least 0 and at most 3. Values outside of this range will result in a shader-creation error.
t	The sampled or depth texture to read from.
s	The sampler type .
coords	The texture coordinates.
array_index	The 0-based texture array index. This value will be clamped to the range [0, textureNumLayers(t) - 1].
offset	The optional texel offset applied to the unnormalized texture coordinate before sampling the texture. This offset is applied before applying any texture wrapping modes. The offset expression must be a const-expression (e.g. vec2<i32>(1, 2)). Each offset component must be at least -8 and at most 7. Values outside of this range will result in a shader-creation error.

Returns:

A four component vector with components extracted from the specified channel from the selected texels, as described above.

EXAMPLE: GATHER COMPONENTS FROM TEXELS IN 2D TEXTURE

```

@group(0) @binding(0) var t: texture_2d<f32>;
@group(0) @binding(1) var dt: texture_depth_2d;
@group(0) @binding(2) var s: sampler;

fn gather_x_components(c: vec2<f32>) -> vec4<f32> {
    return textureGather(0,t,s,c);
}
fn gather_y_components(c: vec2<f32>) -> vec4<f32> {
    return textureGather(1,t,s,c);
}
fn gather_z_components(c: vec2<f32>) -> vec4<f32> {
    return textureGather(2,t,s,c);
}
fn gather_depth_components(c: vec2<f32>) -> vec4<f32> {
    return textureGather(dt,s,c);
}

```

§ 17.7.3. textureGatherCompare

A *texture gather compare* operation performs a depth comparison on four texels in a depth texture and collects the results into a single vector, as follows:

- Find the four texels that would be used in a depth sampling operation with linear filtering, from [mip level 0](#):
 - Use the specified coordinate, array index (when present), and offset (when present).
 - The texels are adjacent, forming a square, when considering their texture space coordinates (u,v) .
 - Selected texels at the texture edge, cube face edge, or cube corners are handled as in ordinary texture sampling.
- For each texel, perform a comparison against the depth reference value, yielding a 0.0 or 1.0 value, as controlled by the comparison sampler parameters.
- Yield the four-component vector where the components are the comparison results with the texels with relative texel coordinates as follows:

- Result component Relative texel coordinate

x	$(u_{\min}, v_{\max})$
y	$(u_{\max}, v_{\max})$
z	$(u_{\max}, v_{\min})$
w	$(u_{\min}, v_{\min})$

Parameterization	Overload
	<pre>@must_use fn textureGatherCompare(t: texture_depth_2d, s: sampler_comparison, coords: vec2<f32>, depth_ref: f32) -> vec4<f32></pre>
	<pre>@must_use fn textureGatherCompare(t: texture_depth_2d, s: sampler_comparison, coords: vec2<f32>, depth_ref: f32, offset: vec2<i32>) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureGatherCompare(t: texture_depth_2d_array, s: sampler_comparison, coords: vec2<f32>, array_index: A, depth_ref: f32) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureGatherCompare(t: texture_depth_2d_array, s: sampler_comparison, coords: vec2<f32>, array_index: A, depth_ref: f32, offset: vec2<i32>) -> vec4<f32></pre>
	<pre>@must_use fn textureGatherCompare(t: texture_depth_cube, s: sampler_comparison, coords: vec3<f32>, depth_ref: f32) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureGatherCompare(t: texture_depth_cube_array, s: sampler_comparison, coords: vec3<f32>, array_index: A, depth_ref: f32) -> vec4<f32></pre>
Parameters:	
t	The depth texture to read from.
s	The sampler comparison .
coords	The texture coordinates.

A four component vector with comparison result for the selected texels, as described above.

```
@group(0) @binding(0) var dt: texture_depth_2d;
@group(0) @binding(1) var s: sampler;

fn gather_depth_compare(c: vec2<f32>, depth_ref: f32) -> vec4<f32> {
    return textureGatherCompare(dt,s,c,depth_ref);
}
```

Reads a single texel from a texture without sampling or filtering.

334/398

Parameterization	Overload
	<code>level: L) -> vec4<ST></code>
<i>C</i> is i32, or u32 <i>S</i> is i32, or u32 <i>ST</i> is i32, u32, or f32	<pre>@must_use fn textureLoad(t: texture_multisampled_2d<ST>, coords: vec2<C>, sample_index: S)-> vec4<ST></pre>
<i>C</i> is i32, or u32 <i>L</i> is i32, or u32	<pre>@must_use fn textureLoad(t: texture_depth_2d, coords: vec2<C>, level: L) -> f32</pre>
<i>C</i> is i32, or u32 <i>A</i> is i32, or u32 <i>L</i> is i32, or u32	<pre>@must_use fn textureLoad(t: texture_depth_2d_array, coords: vec2<C>, array_index: A, level: L) -> f32</pre>
<i>C</i> is i32, or u32 <i>S</i> is i32, or u32	<pre>@must_use fn textureLoad(t: texture_depth_multisampled_2d, coords: vec2<C>, sample_index: S)-> f32</pre>
<i>C</i> is i32, or u32	<pre>@must_use fn textureLoad(t: texture_external, coords: vec2<C>) -> vec4<f32></pre>
<i>C</i> is i32, or u32 <i>AM</i> is read or read_write <i>CF</i> depends on the storage texel format <i>F</i>. See the texel format table for the mapping of texel format to channel format.	<pre>@must_use fn textureLoad(t : texture_storage_1d<F, AM>, coords : C) -> vec4<CF></pre>
<i>C</i> is i32, or u32 <i>AM</i> is read or read_write <i>CF</i> depends on the storage texel format <i>F</i>. See the texel format table for the mapping of texel format to channel format.	<pre>@must_use fn textureLoad(t : texture_storage_2d<F, AM>, coords : vec2<C>) -> vec4<CF></pre>
<i>C</i> is i32, or u32 <i>AM</i> is read or read_write <i>A</i> is i32 or u32 <i>CF</i> depends on the storage texel format <i>F</i>. See the texel format table	<pre>@must_use fn textureLoad(t : texture_storage_2d_array<F, AM>, coords : vec2<C>, array_index : A) -> vec4<CF></pre>

Parameterization	Overload
for the mapping of texel format to channel format.	
<i>C</i> is i32 , or u32 <i>AM</i> is read or read_write <i>CF</i> depends on the storage texel format <i>F</i> . See the texel format table for the mapping of texel format to channel format.	<pre>@must_use fn textureLoad(t : texture_storage_3d<F, AM>, coords : vec3<C>) -> vec4<CF></pre>

Parameters:

t	The sampled , multisampled , depth , storage , or external texture
coords	The 0-based texel coordinate.
array_index	The 0-based texture array index.
level	The mip level , with level 0 containing a full size version of the texture.
sample_index	The 0-based sample index of the multisampled texture .

Returns:

The unfiltered texel data.

The [logical texel address](#) is invalid if:

- any element of `coords` is outside the range `[0, textureDimensions(t, level))` for the corresponding element, or
- `array_index` is outside the range `[0, textureNumLayers(t))`, or
- `level` is outside the range `[0, textureNumLevels(t))`, or
- `sample_index` is outside the range `[0, textureNumSamples(s))`

If the logical texel address is invalid, the built-in function returns one of:

- The data for some texel within bounds of the texture.
- For non-depth textures, either:
 - The vector (0,0,0,0) or (0,0,0,1) of the appropriate type, with the [texture view swizzle](#) applied, or
 - The vector (0,0,0,0) or (0,0,0,1) of the appropriate type, without the [texture view swizzle](#) applied.
- 0.0 for depth textures.

§ 17.7.5. textureNumLayers

Returns the number of layers (elements) of an [arrayed](#) texture.

Parameterization	Overload
F is a texel format A is an access mode ST is i32, u32, or f32	
T is texture_2d_array<ST>, texture_cube_array<ST>, texture_depth_2d_array, texture_depth_cube_array, or texture_storage_2d_array<F,A>	<pre>@must_use fn textureNumLayers(t: T) -> u32</pre>

Parameters:

t The [sampled](#), [depth](#), or [storage texture](#) array texture.

Returns:

If the texture is based on cubes, returns the number of cubes in the cube arrayed texture.

Otherwise returns the number of layers (homogeneous grids of texels) in the arrayed texture.

§ 17.7.6. textureNumLevels

Returns the number of mip levels of a texture.

Parameterization	Overload
ST is i32, u32, or f32	
T is texture_1d<ST>, texture_2d<ST>, texture_2d_array<ST>, texture_3d<ST>, texture_cube<ST>, texture_cube_array<ST>, texture_depth_2d, texture_depth_2d_array, texture_depth_cube, or texture_depth_cube_array	<pre>@must_use fn textureNumLevels(t: T) -> u32</pre>

Parameters:

t The [sampled](#) or [depth](#) texture.

Returns:

The [mip level count](#) for the texture.

§ 17.7.7. textureNumSamples

Returns the number samples per texel in a [multisampled texture](#).

Parameterization	Overload
<i>ST</i> is i32 , u32 , or f32	
<i>T</i> is texture_multisampled_2d< <i>ST</i> > or texture_depth_multisampled_2d	<code>@must_use fn textureNumSamples(t: T) -> u32</code>

Parameters:

t The [multisampled texture](#).

Returns:

The [sample count](#) for the [multisampled texture](#).

§ 17.7.8. textureSample

Samples a texture.

[Must](#) only be used in a [fragment](#) shader stage.

If [uniformity analysis](#) cannot prove a call to this function is in [uniform control flow](#), then a [derivative_uniformity diagnostic](#) is [triggered](#).

Parameterization	Overload
	<pre>@must_use fn textureSample(t: texture_1d<f32>, s: sampler, coords: f32) -> vec4<f32></pre>
	<pre>@must_use fn textureSample(t: texture_2d<f32>, s: sampler, coords: vec2<f32>) -> vec4<f32></pre>
	<pre>@must_use fn textureSample(t: texture_2d<f32>, s: sampler, coords: vec2<f32>, offset: vec2<i32>) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSample(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSample(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A, offset: vec2<i32>) -> vec4<f32></pre>
<i>T</i> is texture_3d<f32>, or texture_cube<f32>	<pre>@must_use fn textureSample(t: T, s: sampler, coords: vec3<f32>) -> vec4<f32></pre>
	<pre>@must_use fn textureSample(t: texture_3d<f32>, s: sampler, coords: vec3<f32>, offset: vec3<i32>) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSample(t: texture_cube_array<f32>, s: sampler, coords: vec3<f32>, array_index: A) -> vec4<f32></pre>
	<pre>@must_use fn textureSample(t: texture_depth_2d,</pre>

Parameterization	Overload
	<pre>s: sampler, coords: vec2<f32>) -> f32</pre>
	<pre>@must_use fn textureSample(t: texture_depth_2d, s: sampler, coords: vec2<f32>, offset: vec2<i32>) -> f32</pre>
A is i32 , or u32	<pre>@must_use fn textureSample(t: texture_depth_2d_array, s: sampler, coords: vec2<f32>, array_index: A) -> f32</pre>
A is i32 , or u32	<pre>@must_use fn textureSample(t: texture_depth_2d_array, s: sampler, coords: vec2<f32>, array_index: A, offset: vec2<i32>) -> f32</pre>
	<pre>@must_use fn textureSample(t: texture_depth_cube, s: sampler, coords: vec3<f32>) -> f32</pre>
A is i32 , or u32	<pre>@must_use fn textureSample(t: texture_depth_cube_array, s: sampler, coords: vec3<f32>, array_index: A) -> f32</pre>

Parameters:

t	The sampled or depth texture to sample.
s	The sampler type.
coords	The texture coordinates used for sampling.
array_index	The 0-based texture array index to sample. This value will be clamped to the range [0, textureNumLayers(t) - 1].
offset	The optional texel offset applied to the unnormalized texture coordinate before sampling the texture. This offset is applied before applying any texture wrapping modes. The offset expression must be a const-expression (e.g. vec2<i32>(1, 2)). Each offset component must be at least -8 and at most 7. Values outside of this range will result in a shader-creation error .

Returns:

The sampled value.

An [indeterminate value](#) results if called in [non-uniform control flow](#).

§ 17.7.9. textureSampleBias

Samples a texture with a bias to the mip level.

[Must](#) only be used in a [fragment](#) shader stage.

If [uniformity analysis](#) cannot prove a call to this function is in [uniform control flow](#), then a [derivative_uniformity diagnostic](#) is [triggered](#).

Parameterization	Overload
	<pre> @must_use fn textureSampleBias(t: texture_2d<f32>, s: sampler, coords: vec2<f32>, bias: f32) -> vec4<f32> </pre>
	<pre> @must_use fn textureSampleBias(t: texture_2d<f32>, s: sampler, coords: vec2<f32>, bias: f32, offset: vec2<i32>) -> vec4<f32> </pre>
<i>A</i> is i32 , or u32	<pre> @must_use fn textureSampleBias(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A, bias: f32) -> vec4<f32> </pre>
<i>A</i> is i32 , or u32	<pre> @must_use fn textureSampleBias(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A, bias: f32, offset: vec2<i32>) -> vec4<f32> </pre>
<i>T</i> is texture_3d<f32>, or texture_cube<f32>	<pre> @must_use fn textureSampleBias(t: T, s: sampler, coords: vec3<f32>, bias: f32) -> vec4<f32> </pre>
	<pre> @must_use fn textureSampleBias(t: texture_3d<f32>, s: sampler, coords: vec3<f32>, bias: f32, offset: vec3<i32>) -> vec4<f32> </pre>
<i>A</i> is i32 , or u32	<pre> @must_use fn textureSampleBias(t: texture_cube_array<f32>, s: sampler, coords: vec3<f32>, array_index: A, bias: f32) -> vec4<f32> </pre>

Parameters:

t	The sampled texture to sample.
s	The sampler type.
coords	The texture coordinates used for sampling.
array_index	The 0-based texture array index to sample. This value will be clamped to the range <code>[0, textureNumLayers(t) - 1]</code> .
bias	The bias to apply to the mip level before sampling. This value will be clamped in the range <code>[-16.0, 15.99]</code> .
offset	The optional texel offset applied to the unnormalized texture coordinate before sampling the texture. This offset is applied before applying any texture wrapping modes. The offset expression must be a const-expression (e.g. <code>vec2<i32>(1, 2)</code>). Each offset component must be at least -8 and at most 7. Values outside of this range will result in a shader-creation error .

Returns:

The sampled value.

§ 17.7.10. `textureSampleCompare`

Samples a [depth texture](#) and compares the sampled depth values against a reference value.

[Must](#) only be used in a [fragment](#) shader stage.

If [uniformity analysis](#) cannot prove a call to this function is in [uniform control flow](#), then a [derivative_uniformity diagnostic](#) is [triggered](#).

Parameterization	Overload
	<pre>@must_use fn textureSampleCompare(t: texture_depth_2d, s: sampler_comparison, coords: vec2<f32>, depth_ref: f32) -> f32</pre>
	<pre>@must_use fn textureSampleCompare(t: texture_depth_2d, s: sampler_comparison, coords: vec2<f32>, depth_ref: f32, offset: vec2<i32>) -> f32</pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleCompare(t: texture_depth_2d_array, s: sampler_comparison, coords: vec2<f32>,</pre>

Parameterization	Overload
	<pre>array_index: A, depth_ref: f32) -> f32</pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleCompare(t: texture_depth_2d_array, s: sampler_comparison, coords: vec2<f32>, array_index: A, depth_ref: f32, offset: vec2<i32>) -> f32</pre>
	<pre>@must_use fn textureSampleCompare(t: texture_depth_cube, s: sampler_comparison, coords: vec3<f32>, depth_ref: f32) -> f32</pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleCompare(t: texture_depth_cube_array, s: sampler_comparison, coords: vec3<f32>, array_index: A, depth_ref: f32) -> f32</pre>

Parameters:

t	The depth texture to sample.
s	The sampler_comparison type.
coords	The texture coordinates used for sampling.
array_index	The 0-based texture array index to sample. This value will be clamped to the range [0, textureNumLayers(t) - 1].
depth_ref	The reference value to compare the sampled depth value against.
offset	The optional texel offset applied to the unnormalized texture coordinate before sampling the texture. This offset is applied before applying any texture wrapping modes. The offset expression must be a const-expression (e.g. vec2<i32>(1, 2)). Each offset component must be at least -8 and at most 7. Values outside of this range will result in a shader-creation error .

Returns:

A value in the range [0.0..1.0].

Each sampled texel is compared against the reference value using the comparison operator defined by the `sampler_comparison`, resulting in either a 0 or 1 value for each texel.

If the sampler uses bilinear filtering then the returned value is the filtered average of these values, otherwise the comparison result of a single texel is returned.

An [indeterminate value](#) results if called in [non-uniform control flow](#).

§ 17.7.11. `textureSampleCompareLevel`

Samples a [depth texture](#) and compares the sampled depth values against a reference value.

Parameterization	Overload
	<pre>@must_use fn textureSampleCompareLevel(t: texture_depth_2d, s: sampler_comparison, coords: vec2<f32>, depth_ref: f32) -> f32</pre>
	<pre>@must_use fn textureSampleCompareLevel(t: texture_depth_2d, s: sampler_comparison, coords: vec2<f32>, depth_ref: f32, offset: vec2<i32>) -> f32</pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleCompareLevel(t: texture_depth_2d_array, s: sampler_comparison, coords: vec2<f32>, array_index: A, depth_ref: f32) -> f32</pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleCompareLevel(t: texture_depth_2d_array, s: sampler_comparison, coords: vec2<f32>, array_index: A, depth_ref: f32, offset: vec2<i32>) -> f32</pre>
	<pre>@must_use fn textureSampleCompareLevel(t: texture_depth_cube, s: sampler_comparison, coords: vec3<f32>, depth_ref: f32) -> f32</pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleCompareLevel(t: texture_depth_cube_array, s: sampler_comparison, coords: vec3<f32>, array_index: A, depth_ref: f32) -> f32</pre>
Parameters:	
t	The depth texture to sample.
s	The sampler_comparison type.
coords	The texture coordinates used for sampling.

array_index	The 0-based texture array index to sample. This value will be clamped to the range <code>[0, textureNumLayers(t) - 1]</code>.
depth_ref	The reference value to compare the sampled depth value against.
offset	The optional texel offset applied to the unnormalized texture coordinate before sampling the texture. This offset is applied before applying any texture wrapping modes. The offset expression must be a const-expression (e.g. <code>vec2<i32>(1, 2)</code>). Each offset component must be at least -8 and at most 7. Values outside of this range will result in a shader-creation error.

Returns:

A value in the range `[0.0..1.0]`.

The `textureSampleCompareLevel` function is the same as `textureSampleCompare`, except that:

- `textureSampleCompareLevel` always samples texels from mip level 0.
 - The function does not compute derivatives.
 - There is no requirement for `textureSampleCompareLevel` to be invoked in [uniform control flow](#).
- `textureSampleCompareLevel` may be invoked in any shader stage.

§ 17.7.12. `textureSampleGrad`

Samples a texture using explicit gradients.

Parameterization	Overload
	<pre> @must_use fn textureSampleGrad(t: texture_2d<f32>, s: sampler, coords: vec2<f32>, ddx: vec2<f32>, ddy: vec2<f32>) -> vec4<f32> </pre>
	<pre> @must_use fn textureSampleGrad(t: texture_2d<f32>, s: sampler, coords: vec2<f32>, ddx: vec2<f32>, ddy: vec2<f32>, offset: vec2<i32>) -> vec4<f32> </pre>
<i>A</i> is i32 , or u32	<pre> @must_use fn textureSampleGrad(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A, ddx: vec2<f32>, ddy: vec2<f32>) -> vec4<f32> </pre>
<i>A</i> is i32 , or u32	<pre> @must_use fn textureSampleGrad(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A, ddx: vec2<f32>, ddy: vec2<f32>, offset: vec2<i32>) -> vec4<f32> </pre>
<i>T</i> is texture_3d<f32>, or texture_cube<f32>	<pre> @must_use fn textureSampleGrad(t: T, s: sampler, coords: vec3<f32>, ddx: vec3<f32>, ddy: vec3<f32>) -> vec4<f32> </pre>
	<pre> @must_use fn textureSampleGrad(t: texture_3d<f32>, s: sampler, coords: vec3<f32>, ddx: vec3<f32>, ddy: vec3<f32>, offset: vec3<i32>) -> vec4<f32> </pre>
<i>A</i> is i32 , or u32	<pre> @must_use fn textureSampleGrad(t: texture_cube_array<f32>, s: sampler, </pre>

Parameterization	Overload
	<code>coords: vec3<f32>, array_index: A, ddx: vec3<f32>, ddy: vec3<f32>) -> vec4<f32></code>

Parameters:

t	The sampled texture to sample.
s	The sampler .
coords	The texture coordinates used for sampling.
array_index	The 0-based texture array index to sample. This value will be clamped to the range [0, textureNumLayers(t) - 1].
ddx	The x direction derivative vector used to compute the sampling locations.
ddy	The y direction derivative vector used to compute the sampling locations.
offset	The optional texel offset applied to the unnormalized texture coordinate before sampling the texture. This offset is applied before applying any texture wrapping modes. The offset expression must be a const-expression (e.g. vec2<i32>(1, 2)). Each offset component must be at least -8 and at most 7. Values outside of this range will result in a shader-creation error .

Returns:

The sampled value.

§ 17.7.13. textureSampleLevel

Samples a texture using an explicit mip level.

Parameterization	Overload
	<pre>@must_use fn textureSampleLevel(t: texture_1d<f32>, s: sampler, coords: f32, level: f32) -> vec4<f32></pre>
	<pre>@must_use fn textureSampleLevel(t: texture_2d<f32>, s: sampler, coords: vec2<f32>, level: f32) -> vec4<f32></pre>
	<pre>@must_use fn textureSampleLevel(t: texture_2d<f32>, s: sampler, coords: vec2<f32>, level: f32, offset: vec2<i32>) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A, level: f32) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_2d_array<f32>, s: sampler, coords: vec2<f32>, array_index: A, level: f32, offset: vec2<i32>) -> vec4<f32></pre>
<i>T</i> is texture_3d<f32>, or texture_cube<f32>	<pre>@must_use fn textureSampleLevel(t: T, s: sampler, coords: vec3<f32>, level: f32) -> vec4<f32></pre>
	<pre>@must_use fn textureSampleLevel(t: texture_3d<f32>, s: sampler, coords: vec3<f32>, level: f32, offset: vec3<i32>) -> vec4<f32></pre>
<i>A</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_cube_array<f32>,</pre>

Parameterization	Overload
	<pre>s: sampler, coords: vec3<f32>, array_index: A, level: f32) -> vec4<f32></pre>
<i>L</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_depth_2d, s: sampler, coords: vec2<f32>, level: L) -> f32</pre>
<i>L</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_depth_2d, s: sampler, coords: vec2<f32>, level: L, offset: vec2<i32>) -> f32</pre>
<i>A</i> is i32 , or u32 <i>L</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_depth_2d_array, s: sampler, coords: vec2<f32>, array_index: A, level: L) -> f32</pre>
<i>A</i> is i32 , or u32 <i>L</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_depth_2d_array, s: sampler, coords: vec2<f32>, array_index: A, level: L, offset: vec2<i32>) -> f32</pre>
<i>L</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_depth_cube, s: sampler, coords: vec3<f32>, level: L) -> f32</pre>
<i>A</i> is i32 , or u32 <i>L</i> is i32 , or u32	<pre>@must_use fn textureSampleLevel(t: texture_depth_cube_array, s: sampler, coords: vec3<f32>, array_index: A, level: L) -> f32</pre>

Parameters:

t	The sampled or depth texture to sample.
s	The sampler type.
coords	The texture coordinates used for sampling.
array_index	The 0-based texture array index to sample. This value will be clamped to the range <code>[0, textureNumLayers(t) - 1]</code> .
level	The mip level, with level 0 containing a full size version of the texture. For the functions where level is a f32, fractional values may interpolate between two levels if the format is filterable according to the Texture Format Capabilities .
offset	The optional texel offset applied to the unnormalized texture coordinate before sampling the texture. This offset is applied before applying any texture wrapping modes. The offset expression must be a const-expression (e.g. <code>vec2<i32>(1, 2)</code>). Each offset component must be at least -8 and at most 7. Values outside of this range will result in a shader-creation error .

Returns:

The sampled value.

§ 17.7.14. `textureSampleBaseClampToEdge`

Samples a texture view at its base level, with texture coordinates clamped to the edge as described below.

Parameterization	Overload
<i>T</i> is <code>texture_2d<f32></code> or <code>texture_external</code>	<pre>@must_use fn textureSampleBaseClampToEdge(t: T, s: sampler, coords: vec2<f32>) -> vec4<f32></pre>

Parameters:

t	The sampled or external texture to sample.
s	The sampler type.
coords	The texture coordinates used for sampling.

Before sampling, the given coordinates [will](#) be clamped to the rectangle

[*half_texel*, 1 - *half_texel*]

where

half_texel = `vec2(0.5) / vec2<f32>(textureDimensions(t))`

NOTE: The half-texel adjustment ensures that, independent of the sampler’s [addressing](#) and [filter](#) modes, wrapping will not occur. That is, when sampling near an edge, the sampled texels will be at or adjacent to that edge, and not selected from the opposite edge.

Returns:

The sampled value.

 17.7.15. textureStore

Writes a single texel to a texture.

Parameterization	Overload
<i>F</i> is a texel format <i>C</i> is i32, or u32 <i>AM</i> is write or read_write <i>CF</i> depends on the storage texel format <i>F</i>. See the texel format table for the mapping of texel format to channel format.	<pre>fn textureStore(t: texture_storage_1d<F,AM>, coords: C, value: vec4<CF>)</pre>
<i>F</i> is a texel format <i>C</i> is i32, or u32 <i>AM</i> is write or read_write <i>CF</i> depends on the storage texel format <i>F</i>. See the texel format table for the mapping of texel format to channel format.	<pre>fn textureStore(t: texture_storage_2d<F,AM>, coords: vec2<C>, value: vec4<CF>)</pre>
<i>F</i> is a texel format <i>C</i> is i32, or u32 <i>AM</i> is write or read_write <i>A</i> is i32, or u32 <i>CF</i> depends on the storage texel format <i>F</i>. See the texel format table for the mapping of texel format to channel format.	<pre>fn textureStore(t: texture_storage_2d_array<F,AM>, coords: vec2<C>, array_index: A, value: vec4<CF>)</pre>
<i>F</i> is a texel format <i>C</i> is i32, or u32 <i>AM</i> is write or read_write <i>CF</i> depends on the storage texel format <i>F</i>. See the texel format table for the mapping of texel format to channel format.	<pre>fn textureStore(t: texture_storage_3d<F,AM>, coords: vec3<C>, value: vec4<CF>)</pre>

Parameters:

t	The write-only storage texture or read-write storage texture
coords	The 0-based texel coordinate.
array_index	The 0-based texture array index.
value	The new texel value. value is converted using the inverse channel transfer function .

Note:

The [logical texel address](#) is invalid if:

- any element of coords is outside the range [0, textureDimensions(t)) for the corresponding element, or
- array_index is outside the range of [0, textureNumLayers(t))

If the logical texel address is invalid, the built-in function [will](#) not be executed.

§ 17.8. Atomic Built-in Functions

Atomic built-in functions can be used to read/write/read-modify-write atomic objects. They are the only operations allowed on [§ 6.2.8 Atomic Types](#).

All atomic built-in functions use a relaxed [memory ordering](#). This means synchronization and ordering guarantees only apply among atomic operations acting on the same [memory locations](#). No synchronization or ordering guarantees apply between atomic and non-atomic memory accesses, or between atomic accesses acting on different memory locations.

Atomic built-in functions [must not](#) be used in a [vertex](#) shader stage.

The address space AS of the atomic_ptr parameter in all atomic built-in functions [must](#) be either [storage](#) or [workgroup](#).

T [must](#) be either [u32](#) or [i32](#)

§ 17.8.1. atomicLoad

```
fn atomicLoad(atomic_ptr: ptr<AS, atomic<T>, read_write>) -> T
```

Returns the atomically loaded the value pointed to by atomic_ptr. It does not [modify](#) the object.

§ 17.8.2. atomicStore

```
fn atomicStore(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T)
```

Atomically stores the value v in the atomic object pointed to by atomic_ptr.

§ 17.8.3. Atomic Read-modify-write Arithmetic and Logical Functions

Each function performs the following steps atomically:

1. Load the original value pointed to by `atomic_ptr`.
2. Obtains a new value by performing the operation (e.g. max) from the function name with the value `v`.
3. Store the new value using `atomic_ptr`.

Each function returns the original value stored in the atomic object before the operation.

§ 17.8.3.1. *atomicAdd*

```
fn atomicAdd(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically performs an addition operation on the atomic object pointed to by `atomic_ptr` with the value `v`, and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC ADDITION AS A FUNCTION

```
// All operations are performed atomically
fn atomicAdd(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = old + v;
    return old;
}
```

§ 17.8.3.2. *atomicSub*

```
fn atomicSub(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically performs a subtraction operation on the atomic object pointed to by `atomic_ptr` with the value `v`, and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC SUBTRACTION AS A FUNCTION

```
// All operations are performed atomically
fn atomicSub(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = old - v;
    return old;
}
```

§ 17.8.3.3. *atomicMax*

```
fn atomicMax(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically performs a maximum operation on the atomic object pointed to by `atomic_ptr` with the value `v`, and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC MAXIMUM AS A FUNCTION

```
// All operations are performed atomically
fn atomicMax(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = max(old, v);
    return old;
}
```

17.8.3.4. *atomicMin*

```
fn atomicMin(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically performs a minimum operation on the atomic object pointed to by `atomic_ptr` with the value `v`, and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC MINIMUM AS A FUNCTION

```
// All operations are performed atomically
fn atomicMin(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = min(old, v);
    return old;
}
```

17.8.3.5. *atomicAnd*

```
fn atomicAnd(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically performs a bitwise AND operation on the atomic object pointed to by `atomic_ptr` with the value `v`, and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC BITWISE AND AS A FUNCTION

```
// All operations are performed atomically
fn atomicAnd(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = old & v;
    return old;
}
```


§ 17.8.3.6. *atomicOr*

```
fn atomicOr(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically performs a bitwise OR operation on the atomic object pointed to by `atomic_ptr` with the value `v`, and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC BITWISE OR AS A FUNCTION

```
// All operations are performed atomically
fn atomicOr(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = old | v;
    return old;
}
```

§ 17.8.3.7. *atomicXor*

```
fn atomicXor(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically performs a bitwise XOR operation on the atomic object pointed to by `atomic_ptr` with the value `v`, and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC BITWISE XOR AS A FUNCTION

```
// All operations are performed atomically
fn atomicXor(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = old ^ v;
    return old;
}
```

§ 17.8.4. *atomicExchange*

```
fn atomicExchange(atomic_ptr: ptr<AS, atomic<T>, read_write>, v: T) -> T
```

Atomically stores the value `v` in the atomic object pointed to by `atomic_ptr` and returns the original value stored in the atomic object before the operation.

EXAMPLE: OPERATION OF ATOMIC EXCHANGE AS A FUNCTION

```
// All operations are performed atomically
fn atomicExchange(atomic_ptr: ptr<AS, atomic<T>, read_write>, v : T) -> T {
    let old = *atomic_ptr;
    *atomic_ptr = v;
    return old;
}
```

§ 17.8.5. atomicCompareExchangeWeak

```
fn atomicCompareExchangeWeak(
    atomic_ptr: ptr<AS, atomic<T>, read_write>,
    cmp: T,
    v: T) -> __atomic_compare_exchange_result<T>

struct __atomic_compare_exchange_result<T> {
    old_value : T, // old value stored in the atomic
    exchanged : bool // true if the exchange was done
}
```

NOTE: A value cannot be explicitly declared with the type `__atomic_compare_exchange_result`, but a value may infer the type.

Performs the following steps atomically:

1. Load the original value pointed to by `atomic_ptr`.
2. Compare the original value to the value `cmp` using an equality operation.
3. Store the value `v` only if the result of the equality comparison was true.

Returns a two member structure, where the first member, `old_value`, is the original value of the atomic object before the operation and the second member, `exchanged`, is whether or not the comparison succeeded.

EXAMPLE: OPERATION OF ATOMIC COMPARE EXCHANGE AS A FUNCTION

```
// All operations are performed atomically
fn atomicCompareExchangeWeak(atomic_ptr: ptr<AS, atomic<T>, read_write>, cmp : T, v : T) ->
    __atomic_compare_exchange_result<T> {
    let old = *atomic_ptr;
    // This comparison may spuriously fail.
    let comparison = old == cmp;
    if comparison {
        *atomic_ptr = v;
    }
    return __atomic_compare_exchange_result<T>(old, comparison);
}
```

NOTE: The equality comparison may spuriously fail on some implementations. That is, the second component of the result vector may be false even if the first component of the result vector equals `cmp`.

§ 17.9. Data Packing Built-in Functions

Data packing builtin functions can be used to encode values using data formats that do not correspond directly to types in WGSL. This enables a program to write many densely packed values to memory, which can reduce a shader's memory bandwidth demand.

Each builtin applies the *inverse* of a [channel transfer function](#) to several input values, then combines their results into a single output value.

NOTE: For packing unorm values, the normalized floating point values are in the interval [0.0, 1.0].

NOTE: For packing snorm values, the normalized floating point values are in the interval [-1.0, 1.0].

§ 17.9.1. pack4x8snorm

Overload

```
@const @must_use fn pack4x8snorm(e: vec4<f32>) -> u32
```

Description Converts four normalized floating point values to 8-bit signed integers, and then combines them into one u32 value.

Component $e[i]$ of the input is converted to an 8-bit twos complement integer value $\lfloor 0.5 + 127 \times \min(1, \max(-1, e[i])) \rfloor$ which is then placed in bits $8 \times i$ through $8 \times i + 7$ of the result.

§ 17.9.2. pack4x8unorm

Overload

```
@const @must_use fn pack4x8unorm(e: vec4<f32>) -> u32
```

Description Converts four normalized floating point values to 8-bit unsigned integers, and then combines them into one u32 value.

Component $e[i]$ of the input is converted to an 8-bit unsigned integer value $\lfloor 0.5 + 255 \times \min(1, \max(0, e[i])) \rfloor$ which is then placed in bits $8 \times i$ through $8 \times i + 7$ of the result.

§ 17.9.3. pack4xI8

Overload

```
@const @must_use fn pack4xI8(e: vec4<i32>) -> u32
```

Description Pack the lower 8 bits of each component of e into a [u32](#) value and drop all the unused bits.

Component $e[i]$ of the input is mapped to bits $8 \times i$ through $8 \times i + 7$ of the result.

§ 17.9.4. `pack4xU8`

Overload

```
@const @must_use fn pack4xU8(e: vec4<u32>) -> u32
```

Description Pack the lower 8 bits of each component of e into a [u32](#) value and drop all the unused bits.

Component $e[i]$ of the input is mapped to bits $8 \times i$ through $8 \times i + 7$ of the result.

§ 17.9.5. `pack4xI8Clamp`

Overload

```
@const @must_use fn pack4xI8Clamp(e: vec4<i32>) -> u32
```

Description Clamp each component of e in the range $[-128, 127]$ and then pack the lower 8 bits of each component into a [u32](#) value.

Component $e[i]$ of the input is mapped to bits $8 \times i$ through $8 \times i + 7$ of the result.

§ 17.9.6. `pack4xU8Clamp`

Overload

```
@const @must_use fn pack4xU8Clamp(e: vec4<u32>) -> u32
```

Description Clamp each component of e in the range of $[0, 255]$ and then pack the lower 8 bits of each component into a [u32](#) value.

Component $e[i]$ of the input is mapped to bits $8 \times i$ through $8 \times i + 7$ of the result.

§ 17.9.7. `pack2x16snorm`

Overload

```
@const @must_use fn pack2x16snorm(e: vec2<f32>) -> u32
```

Description	Converts two normalized floating point values to 16-bit signed integers, and then combines them into one u32 value. Component $e[i]$ of the input is converted to a 16-bit two's complement integer value $\lfloor 0.5 + 32767 \times \min(1, \max(-1, e[i])) \rfloor$ which is then placed in bits $16 \times i$ through $16 \times i + 15$ of the result.
-------------	---

§ 17.9.8. `pack2x16unorm`

Overload

```
@const @must_use fn pack2x16unorm(e: vec2<f32>) -> u32
```

Description	Converts two normalized floating point values to 16-bit unsigned integers, and then combines them into one u32 value. Component $e[i]$ of the input is converted to a 16-bit unsigned integer value $\lfloor 0.5 + 65535 \times \min(1, \max(0, e[i])) \rfloor$ which is then placed in bits $16 \times i$ through $16 \times i + 15$ of the result.
-------------	--

§ 17.9.9. `pack2x16float`

Overload

```
@const @must_use fn pack2x16float(e: vec2<f32>) -> u32
```

Description	Converts two floating point values to half-precision floating point numbers, and then combines them into one u32 value. Component $e[i]$ of the input is converted to a IEEE-754 binary16 value, which is then placed in bits $16 \times i$ through $16 \times i + 15$ of the result. See § 15.7.6 Floating Point Conversion.
-------------	--

If either $e[0]$ or $e[1]$ is outside the [finite range](#) of binary16 then:

- It is a [shader-creation error](#) if e is a [const-expression](#).
- It is a [pipeline-creation error](#) if e is an [override-expression](#).
- Otherwise the result is an [indeterminate value](#) for u32.

§ 17.10. Data Unpacking Built-in Functions

Data unpacking builtin functions can be used to decode values in data formats that do not correspond directly to types in WGSL. This enables a program to read many densely packed values from memory, which can reduce a shader's memory bandwidth demand.

Each builtin breaks up an input value into channels, then applies a [channel transfer function](#) to each.

NOTE: For unpacking unorm values, the normalized floating point result is in the interval $[0.0, 1.0]$.

NOTE: For unpacking snorm values, the normalized floating point result is in the interval $[-1.0, 1.0]$.

§ 17.10.1. `unpack4x8snorm`

Overload

```
@const @must_use fn unpack4x8snorm(e: u32) -> vec4<f32>
```

Description Decomposes a 32-bit value into four 8-bit chunks, then reinterprets each chunk as a signed normalized floating point value.

Component i of the result is $\max(v \div 127, -1)$, where v is the interpretation of bits $8 \times i$ through $8 \times i + 7$ of e as a two's-complement signed integer.

§ 17.10.2. `unpack4x8unorm`

Overload

```
@const @must_use fn unpack4x8unorm(e: u32) -> vec4<f32>
```

Description Decomposes a 32-bit value into four 8-bit chunks, then reinterprets each chunk as an unsigned normalized floating point value.

Component i of the result is $v \div 255$, where v is the interpretation of bits $8 \times i$ through $8 \times i + 7$ of e as an unsigned integer.

§ 17.10.3. `unpack4xI8`

Overload

```
@const @must_use fn unpack4xI8(e: u32) -> vec4<i32>
```

Description e is interpreted as a vector with four 8-bit signed integer components. Unpack e into a `vec4<i32>` with sign extension.

§ 17.10.4. `unpack4xU8`

Overload

```
@const @must_use fn unpack4xU8(e: u32) -> vec4<u32>
```

Description e is interpreted as a vector with four 8-bit unsigned integer components. Unpack e into a `vec4<u32>` with zero extension.

§ 17.10.5. `unpack2x16snorm`

Overload

```
@const @must_use fn unpack2x16snorm(e: u32) -> vec2<f32>
```

Description	Decomposes a 32-bit value into two 16-bit chunks, then reinterprets each chunk as a signed normalized floating point value. Component i of the result is $\max(v \div 32767, -1)$, where v is the interpretation of bits $16 \times i$ through $16 \times i + 15$ of e as a two's-complement signed integer.
-------------	--

§ 17.10.6. `unpack2x16unorm`

Overload

```
@const @must_use fn unpack2x16unorm(e: u32) -> vec2<f32>
```

Description	Decomposes a 32-bit value into two 16-bit chunks, then reinterprets each chunk as an unsigned normalized floating point value. Component i of the result is $v \div 65535$, where v is the interpretation of bits $16 \times i$ through $16 \times i + 15$ of e as an unsigned integer.
-------------	---

§ 17.10.7. `unpack2x16float`

Overload

```
@const @must_use fn unpack2x16float(e: u32) -> vec2<f32>
```

Description	Decomposes a 32-bit value into two 16-bit chunks, and reinterprets each chunk as a floating point value. Component i of the result is the f32 representation of v , where v is the interpretation of bits $16 \times i$ through $16 \times i + 15$ of e as an IEEE-754 binary16 value. See § 15.7.6 Floating Point Conversion .
-------------	--

§ 17.11. Synchronization Built-in Functions

All synchronization functions execute a [control barrier](#) with Acquire/Release [memory ordering](#). That is, all synchronization functions, and affected memory and atomic operations are ordered in [program order](#) relative to the synchronization function. Additionally, the affected memory and atomic operations program-ordered before the synchronization function must be visible to all other threads in the workgroup before any affected memory or atomic operation program-ordered after the synchronization function is executed by a member of the workgroup.

All synchronization functions use the Workgroup [memory scope](#).

All synchronization functions have a Workgroup [execution scope](#).

All synchronization functions [must](#) only be used in the [compute](#) shader stage. All synchronization functions [must](#) only be invoked in [uniform control flow](#).

§ 17.11.1. storageBarrier

Overload

```
fn storageBarrier()
```

Description	Executes a control barrier synchronization function that affects memory and atomic operations in the storage address space.
-------------	---

§ 17.11.2. textureBarrier

Overload

```
fn textureBarrier()
```

Description	Executes a control barrier synchronization function that affects memory operations in the handle address space.
-------------	---

§ 17.11.3. workgroupBarrier

Overload

```
fn workgroupBarrier()
```

Description	Executes a control barrier synchronization function that affects memory and atomic operations in the workgroup address space.
-------------	---

§ 17.11.4. workgroupUniformLoad

Overload

```
@must_use fn workgroupUniformLoad(p : ptr<workgroup, T>) -> T
```

Parameterization	T is a concrete constructible type.
------------------	---

Description	Returns the value pointed to by p to all invocations in the workgroup. The return value is uniform. p must be a uniform value. Executes a control barrier synchronization function that affects memory and atomic operations in the workgroup address space.
-------------	--

Overload

```
@must_use fn workgroupUniformLoad(p : ptr<workgroup, atomic<T>, read_write>) -> T
```

Description	Atomically loads the value pointed to by p and returns it to all invocations in the workgroup. The return value is uniform . p must be a uniform value . Executes a control barrier synchronization function that affects memory and atomic operations in the workgroup address space.
-------------	---

§ 17.12. Subgroup Built-in Functions

See [§ 15.6.3 Subgroup Operations](#).

Calls to these functions:

- [Must](#) only be used in a [fragment](#) or [compute](#) shader stage.
- [Trigger](#) a [subgroup_uniformity diagnostic](#) if [uniformity analysis](#) cannot prove the call is in [uniform control flow](#).

NOTE: For the [compute shader stage](#), the scope of uniform control flow is the [workgroup](#). For the [fragment shader stage](#), the scope of uniform control flow is the [draw command](#). Both of these scopes are larger than [subgroup](#).

§ 17.12.1. subgroupAdd

Overload	<code>@must_use fn subgroupAdd(e : T) -> T</code>
Preconditions	T is concrete numeric scalar or numeric vector
Description	Reduction operation. Returns the sum of e among all active invocations in the subgroup .

§ 17.12.1.1. subgroupExclusiveAdd

Overload	<code>@must_use fn subgroupExclusiveAdd(e : T) -> T</code>
Preconditions	T is concrete numeric scalar or numeric vector
Description	Exclusive prefix scan operation. Returns the sum of e among all active invocations in the subgroup whose subgroup invocation IDs are less than the current invocation's id. The value returned for the invocation with the lowest id among active invocations is T(0).

§ 17.12.1.2. *subgroupInclusiveAdd*

Overload

```
@must_use fn subgroupInclusiveAdd(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Inclusive prefix scan operation.

Returns the sum of e among all [active](#) invocations in the [subgroup](#) whose [subgroup invocation IDs](#) are less than or equal to the current invocation's id.

NOTE: equivalent to `subgroupExclusiveAdd(x) + x`.

§ 17.12.2. *subgroupAll*

Overload

```
@must_use fn subgroupAll(e : bool) -> bool
```

Description Returns true if e is true for all [active](#) invocations in the [subgroup](#).

§ 17.12.3. *subgroupAnd*

Overload

```
@must_use fn subgroupAnd(e : T) -> T
```

Preconditions T is `i32`, `u32`, `vecN<i32>`, or `vecN<u32>`

Description Reduction operation.

Returns the bitwise and (&) of e among all [active](#) invocations in the [subgroup](#).

§ 17.12.4. *subgroupAny*

Overload

```
@must_use fn subgroupAny(e : bool) -> bool
```

Description Returns true if e is true for any [active](#) invocations in the [subgroup](#).

§ 17.12.5. subgroupBallot

Overload

```
@must_use fn subgroupBallot(pred : bool) -> vec4<u32>
```

Description Returns a bitmask of the [active](#) invocations in the [subgroup](#) for whom pred is true.

The x component of the return value contains invocations 0 through 31.

The y component of the return value contains invocations 32 through 63.

The z component of the return value contains invocations 64 through 95.

The w component of the return value contains invocations 96 through 127.

Within each component, the IDs are in ascending order by bit position (e.g. ID 32 is at bit position 0 in the y component).

§ 17.12.6. subgroupBroadcast

Overload

```
@must_use fn subgroupBroadcast(e : T, id : I) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

I is [u32](#) or [i32](#)

Description Returns the value of e from the invocation whose [subgroup invocation ID](#) matches id in the subgroup to all [active](#) invocations in the [subgroup](#).

id [must](#) be a [const-expression](#) in the range [0, 128).

It is a [dynamic error](#) if id does not select an [active](#) invocation.

NOTE: If a non-constant version of id is required, use [subgroupShuffle](#) instead.

§ 17.12.6.1. subgroupBroadcastFirst

Overload

```
@must_use fn subgroupBroadcastFirst(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Returns the value of e from the invocation that has the lowest [subgroup invocation ID](#) among [active](#) invocations in the [subgroup](#) to all active invocations in the subgroup.

§ 17.12.7. subgroupElect

Overload

```
@must_use fn subgroupElect() -> bool
```

Description Returns true if the current invocation has the lowest [subgroup invocation ID](#) among [active](#) invocations in the [subgroup](#).

§ 17.12.8. subgroupMax

Overload

```
@must_use fn subgroupMax(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Reduction operation.

Returns the maximum value of e among all [active](#) invocations in the [subgroup](#).

§ 17.12.9. subgroupMin

Overload

```
@must_use fn subgroupMin(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Reduction operation.

Returns the minimum value of e among all [active](#) invocations in the [subgroup](#).

§ 17.12.10. subgroupMul

Overload

```
@must_use fn subgroupMul(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Reduction operation.

Returns the product of e among all [active](#) invocations in the [subgroup](#).

§ 17.12.10.1. *subgroupExclusiveMul*

Overload

```
@must_use fn subgroupExclusiveMul(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Exclusive prefix scan operation.

Returns the product of e among all [active](#) invocations in the [subgroup](#) whose [subgroup invocation IDs](#) are less than the current invocation's id.

The value returned for the invocation with the lowest id among active invocations is T(1).

§ 17.12.10.2. *subgroupInclusiveMul*

Overload

```
@must_use fn subgroupInclusiveMul(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Inclusive prefix scan operation.

Returns the product of e among all [active](#) invocations in the [subgroup](#) whose [subgroup invocation IDs](#) are less than or equal to the current invocation's id.

NOTE: equivalent to `subgroupExclusiveMul(x) * x`.

§ 17.12.11. *subgroupOr*

Overload

```
@must_use fn subgroupOr(e : T) -> T
```

Preconditions T is i32, u32, vecN<i32>, or vecN<u32>

Description Reduction operation.

Returns the bitwise or (|) of e among all [active](#) invocations in the [subgroup](#).

§ 17.12.12. subgroupShuffle

Overload

```
@must_use fn subgroupShuffle(e : T, id : I) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)
 I is [u32](#) or [i32](#)

Description Returns e from the invocation whose [subgroup invocation ID](#) matches id.

 If id is outside the range [0, 128), then:

- It is a [shader-creation error](#) if id is a [const-expression](#).
- It is a [pipeline-creation error](#) if id is an [override-expression](#).

 An [indeterminate value](#) is returned if id does not select an [active](#) invocation.

§ 17.12.12.1. subgroupShuffleDown

Overload

```
@must_use fn subgroupShuffleDown(e : T, delta : u32) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Returns e from the invocation whose [subgroup invocation ID](#) matches subgroup_invocation_id + delta for the current invocation.

 If delta is greater than 127, then:

- It is a [shader-creation error](#) if delta is a [const-expression](#).
- It is a [pipeline-creation error](#) if delta is an [override-expression](#).

 A [subgroup_uniformity diagnostic](#) is [triggered](#) if delta is not a [uniform value](#). An [indeterminate value](#) is returned if subgroup_invocation_id + delta does not select an [active](#) invocation or if delta is not a uniform value within the subgroup.

§ 17.12.12.2. subgroupShuffleUp

Overload

```
@must_use fn subgroupShuffleUp(e : T, delta : u32) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Returns e from the invocation whose [subgroup invocation ID](#) matches subgroup_invocation_id - delta for the current invocation.

If `delta` is greater than 127, then:

- It is a [shader-creation error](#) if `delta` is a [const-expression](#).
- It is a [pipeline-creation error](#) if `delta` is an [override-expression](#).

A [subgroup_uniformity diagnostic](#) is [triggered](#) if `delta` is not a [uniform value](#). An [indeterminate value](#) is returned if `subgroup_invocation_id - delta` does not select an [active](#) invocation or if `delta` is not a uniform value within the subgroup.

§ 17.12.12.3. *subgroupShuffleXor*

Overload

```
@must_use fn subgroupShuffleXor(e : T, mask : u32) -> T
```

Preconditions `T` is [concrete numeric scalar](#) or [numeric vector](#)

Description Returns `e` from the invocation whose [subgroup invocation ID](#) matches `subgroup_invocation_id ^ mask` for the current invocation.

If `mask` is greater than 127, then:

- It is a [shader-creation error](#) if `mask` is a [const-expression](#).
- It is a [pipeline-creation error](#) if `mask` is an [override-expression](#).

A [subgroup_uniformity diagnostic](#) is [triggered](#) if `mask` is not a [uniform value](#). An [indeterminate value](#) is returned if `mask` does not select an [active](#) invocation or if `mask` is not a uniform value within the subgroup.

§ 17.12.13. *subgroupXor*

Overload

```
@must_use fn subgroupXor(e : T) -> T
```

Preconditions `T` is `i32`, `u32`, `vecN<i32>`, or `vecN<u32>`

Description Reduction operation.

Returns the bitwise xor (^) of `e` among all [active](#) invocations in the [subgroup](#).

§ 17.13. Quad Operations

See [§ 15.6.4 Quad Operations](#).

Calls to these functions:

- [Must](#) only be used in a [fragment](#) or [compute](#) shader stage.
- [Trigger](#) a [subgroup_uniformity diagnostic](#) if [uniformity analysis](#) cannot prove the call is in [uniform control flow](#).

NOTE: For the [compute shader stage](#), the scope of uniform control flow is the [workgroup](#). For the [fragment shader stage](#), the scope of uniform control flow is the [draw command](#). Both of these scopes are larger than [quad](#).

§ 17.13.1. quadBroadcast

Overload

```
@must_use fn quadBroadcast(e : T, id : I) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)
 I is [u32](#) or [i32](#)

Description Returns the value of e from the invocation whose [quad invocation ID](#) matches id in the quad to all [active](#) invocations in the quad.

id [must](#) be a [const-expression](#) in the range [0, 4).

An [indeterminate value](#) is returned if id does not select an [active](#) invocation.

NOTE: Unlike [subgroupBroadcast](#), there is currently no non-constant alternative.

§ 17.13.2. quadSwapDiagonal

Overload

```
@must_use fn quadSwapDiagonal(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description Returns the value of e from the invocation in the [quad](#) with the opposite coordinates. That is:

- IDs 0 and 3 swap.
- IDs 1 and 2 swap.

§ 17.13.3. quadSwapX

Overload

```
@must_use fn quadSwapX(e : T) -> T
```

Preconditions T is [concrete numeric scalar](#) or [numeric vector](#)

Description	Returns the value of <code>e</code> from invocation in the quad sharing the same X dimension. That is: - IDs 0 and 1 swap. - IDs 2 and 3 swap.
-------------	---

§ 17.13.4. quadSwapY

Overload

```
@must_use fn quadSwapY(e : T) -> T
```

Preconditions	<code>T</code> is concrete numeric scalar or numeric vector
---------------	---

Description	Returns the value of <code>e</code> from invocation in the quad sharing the same Y dimension. That is: - IDs 0 and 2 swap. - IDs 1 and 3 swap.
-------------	---

§ 17.14. Buffer View Built-in Functions

These functions all require the [buffer_view](#) language feature.

These functions operate on a pointer to a [buffer](#). [bufferView](#) and [bufferArrayView](#) reinterpret the opaque data in the buffer into another [host-shareable](#) data type. The resulting pointer can then be used like other pointers.

Define *ArrayOffset*(*T*) as:

- [OffsetOfMember](#)(*T*, *lastMemberIndex*), if *T* is a [structure](#) whose last member is a [runtime-sized array](#) (`array<E>`)
- 0, otherwise

Define *MinTypeSize*(*T*) as:

- [SizeOf](#)(*T*), if *T* has a [fixed footprint](#)
- [StrideOf](#)(*T*), if *T* is a [runtime-sized array](#)
- [ArrayOffset](#)(*T*) + [StrideOf](#)(`array<E>`) if *T* is a [structure](#) whose last member is a [runtime-sized array](#)

Since buffer pointer arguments to [function calls](#) can undergo [automatic conversion](#) to the [parameter's](#) type, implementations [will](#) use interprocedural analysis to check for both [invalid memory references](#) and ensure correct results. The buffer size used for any buffer view built-in function is the minimum buffer size encountered for the particular [originating variable](#) used with the call. Because this is a dynamic concept, implementations [will](#) ensure that this value is correctly determined for a particular instance of the built-in function. The size will be the minimum of:

- The size of the [originating variable](#), or
- The minimum [fixed-size buffer](#) encountered as a [formal parameter](#) in the call stack.

NOTE: If the implementation does not support the [unrestricted_pointer_parameters](#) language feature then the [root identifier](#) of the argument to the built-in function must be the [originating variable](#).

NOTE: Buffer sizes cannot be made larger as they are passed to functions.

§ 17.14.1. `bufferView`

Overloads

```
@must_use fn bufferView<T>(p : ptr<AS, buffer, AM>, offset: I) -> ptr<AS, T, AM>
```

Preconditions

AS is one of [storage](#), [uniform](#), or [workgroup](#).

AM is a valid [access mode](#) for AS.

T is a [host-shareable](#) type.

T **must not** be or contain an [atomic](#) or [buffer](#) type.

T **must** satisfy the [address space layout constraints](#) for AS.

I is [u32](#) or [i32](#).

Description

Reinterpret the [memory view](#) offset bytes from the start of p as the type T. That is, if the [memory locations](#) associated with p are in the range [0, `bufferLength(p)`), then the resulting [memory locations](#) associated with the result are in the range [`offset`, `bufferLength(p)`) with the store type T.

If I is [i32](#), then:

- It is a [shader-creation error](#) if offset is a [const-expression](#) < 0.
- It is a [pipeline-creation error](#) if offset is an [override-expression](#) < 0.
- Otherwise, an [indeterminate value](#) for I may be used.

An [invalid pointer](#) is returned if `MinTypeSize(T) + offset > bufferLength(p)`.

If offset % `RequiredAlignOf(T, AS) != 0`, then:

- It is a [shader-creation error](#) if offset is a [const-expression](#).
- It is a [pipeline-creation error](#) if offset is an [override-expression](#).
- A value of offset & `~(RequiredAlignOf(T, AS) - 1)` **will** be used for offset.

NOTE: This interacts with the [uniform_buffer_standard_layout](#) language feature.

Overloads

```
@must_use fn bufferView<T>(p : ptr<AS, buffer<N>, AM>, offset: I) -> ptr<AS, T, AM>
```

Preconditions

AS is one of [storage](#), [uniform](#), or [workgroup](#).

AM is a valid [access mode](#) for AS.

T is a [host-shareable](#) type.

T **must not** be or contain an [atomic](#) or [buffer](#) type.

T **must** satisfy the [address space layout constraints](#) for AS.

I is [u32](#) or [i32](#).

Description Reinterpret the [memory view](#) offset bytes from the start of p as the type T. That is, if the [memory locations](#) associated with p are in the range [0, bufferLength(p)), then the resulting [memory locations](#) associated with the result are in the range [offset, bufferLength(p)) with the store type T.

If I is [i32](#), then:

- It is a [shader-creation error](#) if offset is a [const-expression](#) < 0.
- It is a [pipeline-creation error](#) if offset is an [override-expression](#) < 0.
- Otherwise, an [indeterminate value](#) for I may be used.

An [invalid memory reference](#) is returned if [MinTypeSize](#)(T) + offset > N.

- It is a [shader-creation error](#) if [MinTypeSize](#)(T) > N.
- It is a [shader-creation error](#) if offset is a [const-expression](#).
- It is a [pipeline-creation error](#) if offset is an [override-expression](#).

If offset % [RequiredAlignOf](#)(T, AS) != 0, then:

- It is a [shader-creation error](#) if offset is a [const-expression](#).
- It is a [pipeline-creation error](#) if offset is an [override-expression](#).
- A value of offset & ~([RequiredAlignOf](#)(T, AS) - 1) [will](#) be used for offset.

NOTE: This interacts with the [uniform_buffer_standard_layout](#) language feature.

EXAMPLE: WGLSL BUFFERVIEW USAGE

```
requires buffer_view;

@group(0) @binding(0) var<storage> in : buffer;
@group(0) @binding(1) var<storage, read_write> out : buffer;

struct S {
    a : mat3x3f,
    b : vec2f,
}

fn foo(offsets : array<u32, 4>) {
    // Read and write some of the data as vec4u.
    let p1 = bufferView<vec4u>(&in, offsets[0]);
    *bufferView<vec4u>(&out, offsets[1]) = *p1;

    // Read and write some of the other data via a structure.
    let p2 = bufferView<array<S, 2>>(&in, offsets[2]);
    let v = (*p2)[1].a;
    *bufferView<mat3x3f>(&out, offsets[3]) = v;
}
```

§ 17.14.2. `bufferArrayView`

Overloads

```
@must_use fn bufferArrayView<T>(p : ptr<AS, buffer, AM>,
                                  offset: OI, size: SI) -> ptr<AS, T, AM>
```

Preconditions

AS is one of [storage](#), [uniform](#), or [workgroup](#).

AM is a valid [access mode](#) for AS.

T is a [host-shareable](#) type without a [fixed footprint](#).

T [must not](#) be nor contain an [atomic](#) or [buffer](#) type.

T [must](#) satisfy the [address space layout constraints](#) for AS.

OI is [u32](#) or [i32](#).

SI is [u32](#) or [i32](#).

Description

Reinterpret the [memory view](#) offset bytes from the start of p for size bytes as the type T. That is, if the [memory locations](#) associated with p are in the range [0, `bufferLength(p)`), then the resulting [memory locations](#) associated with the result are in the range [offset, offset + size) with the store type T.

If OI is [i32](#), then:

- It is a [shader-creation error](#) if offset is a [const-expression](#) < 0.
- It is a [pipeline-creation error](#) if offset is an [override-expression](#) < 0.
- Otherwise, an [indeterminate value](#) for OI may be used.

If SI is [i32](#), then:

- It is a [shader-creation error](#) if size is a [const-expression](#) < 0.
- It is a [pipeline-creation error](#) if size is an [override-expression](#) < 0.
- Otherwise, an [indeterminate value](#) for SI may be used.

An [invalid memory reference](#) is returned if `MinTypeSize(T) > size`.

- It is a [shader-creation error](#) if size is a [const-expression](#) (using 0 for offset if it is not).
- It is a [pipeline-creation error](#) if size is an [override-expression](#) (using 0 for offset if it is not).

An [invalid memory reference](#) is returned if `offset + size > bufferLength(p)`.

If `offset % RequiredAlignOf(T, AS) != 0`, then:

- It is a [shader-creation error](#) if offset is a [const-expression](#).
- It is a [pipeline-creation error](#) if offset is an [override-expression](#).
- A value of offset & $\sim(\text{RequiredAlignOf}(T, AS) - 1)$ [will](#) be used for offset.

NOTE: This interacts with the [uniform buffer standard layout](#) language feature.

If `(size - ArrayOffset(T)) % StrideOf(array<E>) != 0`, then:

- It is a [shader-creation error](#) if size is a [const-expression](#).
- It is a [pipeline-creation error](#) if size is an [override-expression](#).

- A value of `roundDown(StrideOf(array<E>), size - ArrayOffset\(T\)) + ArrayOffset\(T\)` will be used for size.

Overloads

```
@must_use fn bufferArrayView<T>(p : ptr<AS, buffer<N>, AM>,
                                offset: OI, size: SI) -> ptr<AS, T, AM>
```

Preconditions	AS is one of storage, uniform, or workgroup. AM is a valid access mode for AS. T is a host-shareable type without a fixed footprint. T <u>must not</u> be nor contain an atomic or buffer type. T <u>must</u> satisfy the address space layout constraints for AS. OI is u32 or i32. SI is u32 or i32.
Description	Reinterpret the memory view offset bytes from the start of p for size bytes as the type T. That is, if the memory locations associated with p are in the range [0, bufferLength(p)), then the resulting memory locations associated with the result are in the range [offset, offset + size) with the store type T. If OI is i32, then: • It is a shader-creation error if offset is a const-expression < 0. • It is a pipeline-creation error if offset is an override-expression < 0. • Otherwise, an indeterminate value for OI may be used. If SI is i32, then: • It is a shader-creation error if size is a const-expression < 0. • It is a pipeline-creation error if size is an override-expression < 0. • Otherwise, an indeterminate value for SI may be used. An invalid memory reference is returned if MinTypeSize(T) > size. • It is a shader-creation error if size is a const-expression (using 0 for offset if it is not). • It is a pipeline-creation error if size is an override-expression (using 0 for offset if it is not). An invalid memory reference is returned if offset + size > N. • It is a shader-creation error if either offset or size is a const-expression (using 0 if a parameter is not). • It is a pipeline-creation error if either offset or size is an override-expression (using 0 if a parameter is not). If MinTypeSize(T) > N, then: • It is a shader-creation error if N is a const-expression. • It is a pipeline-creation error if N is an override-expression.

If $\text{offset} \% \text{RequiredAlignOf}(T, AS) \neq 0$, then:

- It is a [shader-creation error](#) if offset is a [const-expression](#).
- It is a [pipeline-creation error](#) if offset is an [override-expression](#).
- A value of offset $\& \sim(\text{RequiredAlignOf}(T, AS) - 1)$ [will](#) be used for offset.

NOTE: This interacts with the [uniform_buffer_standard_layout](#) language feature.

If $(\text{size} - \text{ArrayOffset}(T)) \% \text{StrideOf}(\text{array}\langle E \rangle) \neq 0$, then:

- It is a [shader-creation error](#) if size is a [const-expression](#).
- It is a [pipeline-creation error](#) if size is an [override-expression](#).
- A value of $\text{roundDown}(\text{StrideOf}(\text{array}\langle E \rangle), \text{size} - \text{ArrayOffset}(T)) + \text{ArrayOffset}(T)$ [will](#) be used for size.

EXAMPLE: WGSL BUFFERARRAYVIEW USAGE

```
requires buffer_view;

const stride = 32;
const N = 4;
const size = 2048;
const_assert stride * N < size;

struct strided_u32 {
    @size(stride) a : u32,
}

@group(0) @binding(0) var<uniform> in : buffer<size>;
@group(0) @binding(1) var<storage, read_write> out: array<u32, N>;

fn foo(offsets : u32) {
    // Read a strided array from in and copy it unstrided to out.
    var tmp : array<u32, N>;
    let view = bufferArrayView<array<strided_u32>>(&in, offset, stride * N);
    for (var i = 0; i < N; i++) {
        tmp[i] = (*view)[i];
    }
    out = tmp;
}
```

§ 17.14.3. bufferLength

Overloads

```
@must_use fn bufferLength(p : ptr<AS, buffer, AM>) -> u32
@must_use fn bufferLength(p : ptr<AS, buffer<N>, AM>) -> u32
```

Preconditions AS is one of [storage](#), [uniform](#), or [workgroup](#).
AM is a valid [access mode](#) for AS.

Description	Returns the minimum size of the buffer pointed to by <code>p</code> encountered during the interprocedural analysis, or the size of the WebGPU GPUBuffer if only runtime-sized buffers were encountered.
-------------	--

EXAMPLE: BUFFERLENGTH VALUES

```
requires buffer_view;
requires unrestricted_pointer_parameters;

@group(0) @binding(0) var<storage> b1 : buffer<2048>;
@group(0) @binding(0) var<storage> b2 : buffer<4096>;

fn foobar(p : ptr<storage, buffer>) -> u32 {
    return bufferLength(p);
}

fn bar(p : ptr<storage, buffer<1024>>) -> u32 {
    return foobar(p);
}

fn foo(p : ptr<storage, buffer<256>>) -> u32 {
    return bufferLength(p);
}

@compute @workgroup_size(1)
fn main() {
    let len1 = bufferLength(&b1); // 2048
    let len2 = bufferLength(&b2); // 4096

    let len3 = foo(&b1); // 256
    let len4 = foo(&b2); // 256

    let len5 = bar(&b1); // 1024
    let len6 = bar(&b2); // 1024

    let len7 = foobar(&b1); // 2048
    let len8 = foobar(&b2); // 4096
}
```

§ 18. Grammar for Recursive Descent Parsing

This section is non-normative.

The WGSL grammar is specified in a form suitable for an LALR(1) parser. An implementation may want to use a recursive-descent parser instead.

The normative grammar cannot be used directly in a recursive-descent parser, because several of its rules are left-recursive. A grammar rule is directly left-recursive when the nonterminal being defined appears first in one of its productions.

The following is the WGSL grammar, but mechanically transformed to:

- Eliminate direct and indirect left-recursion.
- Avoid empty productions. (That is, avoid epsilon-rules.)
- Bring together common prefixes among sibling productions.

However, it is not LL(1). For some nonterminals, several productions have common lookahead sets. For example, all productions for the `attribute` nonterminal start with the `attr` token. A more subtle example is `global_decl`, where three productions start with an `attribute *` phrase, but then are distinguished by tokens `fn`, `override`, and `var`.

For the sake of brevity, many token definitions are not repeated. Use token definitions from the main part of the specification.

additive_operator:

'+'
| '-'

argument_expression_list:

'(' (*expression* (',' *expression*)* ',' ?)? ')'

assignment_statement/0.1:

compound_assignment_operator
| '='

attribute:

compute_attr
| *const_attr*
| *fragment_attr*
| *interpolate_attr*
| *invariant_attr*
| *must_use_attr*
| *vertex_attr*
| *workgroup_size_attr*
| '@' *ident_pattern_token* ('(' (*expression* (',' *expression*)* ',' ?)? ') ?)?
| '@' 'align' '(' *expression* ',' ? ')'
| '@' 'binding' '(' *expression* ',' ? ')'
| '@' 'blend_src' '(' *expression* ',' ? ')'
| '@' 'builtin' '(' *builtin_value_name* ',' ? ')'
| '@' 'diagnostic' *diagnostic_control*
| '@' 'group' '(' *expression* ',' ? ')'
| '@' 'id' '(' *expression* ',' ? ')'
| '@' 'location' '(' *expression* ',' ? ')'
| '@' 'size' '(' *expression* ',' ? ')'
| '@' 'subgroup_size' '(' *expression* ',' ? ')'

bitwise_expression.post.unary_expression:

'&' *unary_expression* ('&' *unary_expression*)*
| '^' *unary_expression* ('^' *unary_expression*)*
| '|' *unary_expression* ('|' *unary_expression*)*

bool_literal:

'false'

| 'true'

builtin_value_name: [ident_pattern_token](#)

call_expression.post.ident:

[template_elaborated_ident.post.ident](#) '(' ([expression](#) (',' [expression](#))* ',' ?)? ')'

case_selector:

[expression](#)

| 'default'

component_or_swizzle_specifier:

'.' [member_ident](#) [component_or_swizzle_specifier](#) ?

| '.' [swizzle_name](#) [component_or_swizzle_specifier](#) ?

| '[' [expression](#) ']' [component_or_swizzle_specifier](#) ?

compound_assignment_operator:

[shift_left_assign](#)

| [shift_right_assign](#)

| '%='

| '&='

| '*='

| '+='

| '-='

| '/='

| '^='

| '|='

compound_statement:

[attribute](#) * '{' [statement](#) * '}'

compute_attr:

'@' 'compute'

const_attr:

'@' 'const'

core_lhs_expression:

[ident](#)

| [ident](#) [call_expression.post.ident](#)

| '(' [lhs_expression](#) ')'

decimal_int_literal:

/0[iu]?/

| /[1-9][0-9]*[iu]?/

diagnostic_control:

'(' *ident_pattern_token* ',' *diagnostic_rule_name* ',' ? ')'

diagnostic_rule_name:

ident_pattern_token
| *ident_pattern_token* '.' *ident_pattern_token*

expression:

unary_expression *bitwise_expression.post.unary_expression*
| *unary_expression* *relational_expression.post.unary_expression*
| *unary_expression* *relational_expression.post.unary_expression* '&&' *unary_expression*
relational_expression.post.unary_expression ('&&' *unary_expression* *relational_expression.post.unary_expression*
) *
| *unary_expression* *relational_expression.post.unary_expression* '||' *unary_expression*
relational_expression.post.unary_expression ('||' *unary_expression* *relational_expression.post.unary_expression*
) *

float_literal:

decimal_float_literal
| *hex_float_literal*

for_init:

ident *call_expression.post.ident*
| *variable_or_value_statement*
| *variable_updating_statement*

for_update:

ident *call_expression.post.ident*
| *variable_updating_statement*

fragment_attr:

'@' 'fragment'

global_assert:

'const_assert' ';' ;

global_decl:

attribute * 'fn' *ident* '(' (*attribute* * *ident* ':' *type_specifier* (',' *param*) * ',' ?) ? ')' ('-'> *attribute* *
ident *template_elaborated_ident.post.ident*) ? *attribute* * '{' *statement* * '}'
| *attribute* * 'var' (*template_args_start* *expression* (',' *expression*) * ',' ? *template_args_end*) ?
optionally_typed_ident ('=' *expression*) ? ';' ;
| *global_value_decl* ' ;'
| 'alias' *ident* '=' *ident* *template_elaborated_ident.post.ident* ' ;'
| 'struct' *ident* '{' *attribute* * *member_ident* ':' *type_specifier* (',' *attribute* * *member_ident* ':'
type_specifier) * ',' ? '}'

global_directive:

'diagnostic' '(' *ident_pattern_token* ',' *diagnostic_rule_name* ',' ? ')' ';' ;
| 'enable' *ident_pattern_token* (',' *ident_pattern_token*) * ',' ? ';' ;

| 'requires' *ident_pattern_token* (',' *ident_pattern_token*)* ',' ? ';'

global_value_decl:

attribute * 'override' *optionally_typed_ident* ('=' *expression*)?
| 'const' *optionally_typed_ident* '=' *expression*

ident:

ident_pattern_token

int_literal:

decimal_int_literal
| *hex_int_literal*

interpolate_attr:

'@' 'interpolate' '(' *ident_pattern_token* ',' ? ')'
| '@' 'interpolate' '(' *ident_pattern_token* ',' *ident_pattern_token* ',' ? ')'

invariant_attr:

'@' 'invariant'

lhs_expression:

core_lhs_expression *component_or_swizzle_specifier* ?
| '&' *lhs_expression*
| '*' *lhs_expression*

literal:

bool_literal
| *float_literal*
| *int_literal*

member_ident: *ident_pattern_token*

multiplicative_operator:

'%'
| '*'
| '/'

must_use_attr:

'@' 'must_use'

optionally_typed_ident:

ident (':' *type_specifier*)?

param:

attribute * *ident* ':' *type_specifier*

primary_expression:

ident *template_elaborated_ident*.*post.ident*

| ident template_elaborated_ident.post.ident argument_expression_list
 | literal
 | '(' expression ')'

relational_expression.post.unary_expression:

shift_expression.post.unary_expression
 | shift_expression.post.unary_expression greater_than unary_expression shift_expression.post.unary_expression
 | shift_expression.post.unary_expression greater_than_equal unary_expression
shift_expression.post.unary_expression
 | shift_expression.post.unary_expression less_than unary_expression shift_expression.post.unary_expression
 | shift_expression.post.unary_expression less_than_equal unary_expression shift_expression.post.unary_expression
 | shift_expression.post.unary_expression '!=' unary_expression shift_expression.post.unary_expression
 | shift_expression.post.unary_expression '==' unary_expression shift_expression.post.unary_expression

shift_expression.post.unary_expression:

(multiplicative_operator unary_expression) * (additive_operator unary_expression (multiplicative_operator unary_expression) *) *
 | shift_left unary_expression
 | shift_right unary_expression

statement:

attribute * 'for' '(' for_init ? ';' expression ? ';' for_update ? ')' compound_statement
 | attribute * 'if' expression compound_statement ('else' 'if' expression compound_statement) * ('else' compound_statement) ?
 | attribute * 'loop' attribute * '{' statement * ('continuing' attribute * '{' statement * ('break' 'if' expression ';') ? '}') ? '}'
 | attribute * 'switch' expression attribute * '{' switch_clause * '}'
 | attribute * 'while' expression compound_statement
compound_statement
 | ident template_elaborated_ident.post.ident argument_expression_list ';' ;
 | variable_or_value_statement ';' ;
 | variable_updating_statement ';' ;
 | assert_statement ';' ;
 | break_statement ';' ;
 | continue_statement ';' ;
 | ';' ;
 | 'discard' ';' ;
 | 'return' expression ? ';' ;

switch_clause:

'case' case_selector (',' case_selector) * ',' ? ':' ? compound_statement
 | 'default' ':' ? compound_statement

swizzle_name:

/[rgba]/
 |/[rgba][rgba]/
 |/[rgba][rgba][rgba]/
 |/[rgba][rgba][rgba][rgba]/
 |/[xyzw]/
 |/[xyzw][xyzw]/

| / [xyzw] [xyzw] [xyzw] /
 | / [xyzw] [xyzw] [xyzw] [xyzw] /

template_arg_expression: [expression](#)

template_elaborated_ident.post.ident:

([_template_args_start](#) [template_arg_expression](#) (' , ' [expression](#)) * ' , ' ? [_template_args_end](#))?

translation_unit:

[global_directive](#) * ([global_decl](#) | [global_assert](#) | ' ; ') *

translation_unit/0.1/0/0.0:

[global_assert](#)
 | [global_decl](#)
 | ' ; '

type_specifier:

[ident](#) ([_template_args_start](#) [template_arg_expression](#) (' , ' [expression](#)) * ' , ' ? [_template_args_end](#))?

unary_expression:

[primary_expression](#) [component_or_swizzle_specifier](#) ?
 | ' ! ' [unary_expression](#)
 | ' & ' [unary_expression](#)
 | ' * ' [unary_expression](#)
 | ' - ' [unary_expression](#)
 | ' ~ ' [unary_expression](#)

variable_decl:

' var ' ([_template_args_start](#) [expression](#) (' , ' [expression](#)) * ' , ' ? [_template_args_end](#))?
[optionally_typed_ident](#)

variable_or_value_statement:

[variable_decl](#)
 | [variable_decl](#) '=' [expression](#)
 | ' const ' [optionally_typed_ident](#) '=' [expression](#)
 | ' let ' [optionally_typed_ident](#) '=' [expression](#)

variable_updating_statement:

[lhs_expression](#) ('=' | [compound_assignment_operator](#)) [expression](#)
 | [lhs_expression](#) ' ++ '
 | [lhs_expression](#) ' -- '
 | ' _ ' '=' [expression](#)

vertex_attr:

' @ ' ' vertex '

workgroup_size_attr:

' @ ' ' workgroup_size ' '(' [expression](#) ' , ' ? ')' '

```
| '@' 'workgroup_size' '(' expression ',' expression ',' ? ')'
| '@' 'workgroup_size' '(' expression ',' expression ',' expression ',' ? ')'

```

§ Appendix A: The text/wgsl Media Type

The Internet Assigned Numbers Authority (IANA) maintains a registry of media types, at [\[IANA-MEDIA-TYPES\]](https://www.iana.org/assignments/media-types/text/wgsl).

The following is the definition of the text/wgsl media type for WGSL modules. It has been registered at IANA, appearing at <https://www.iana.org/assignments/media-types/text/wgsl>.

Type name

text

Subtype name

wgsl

Required parameters

N/A

Optional parameters

None

Encoding considerations

binary

WGSL is Unicode text using the UTF-8 encoding, with no byte order mark (BOM). See [§ 3 Textual Structure](#).

Security considerations:

WebGPU Shading Language (WGSL) is a programming language for GPU code to be executed in the context of the WebGPU API. For security considerations, see [\[WebGPU\]](#) Section 2.1 Security Considerations. For privacy considerations, see [\[WebGPU\]](#) Section 2.2 Privacy Considerations.

Interoperability considerations:

Implementations of WebGPU may have different capabilities, and these differences may affect what features may be exercised by WGSL programs. See [\[WebGPU\]](#) Section 3.6 Optional capabilities, and [§ 4.1.2 Language Extensions](#).

It is expected that implementations will behave as if this registration applies to later editions of WGSL, and its published specification references may be updated accordingly from time to time. Although this expectation is unusual among media type registrations, it matches widespread industry conventions.

Published specification:

WebGPU Shading Language

Applications that use this media type:

Implementations of WebGPU. This is expected to include web browsers.

Fragment identifier considerations

None

Additional information:

Magic number(s): None

File extension(s): .wgsl

Macintosh file type code(s): TEXT

Person & email address to contact for further information:

David Neto, dneto@google.com, or the Editors listed in WGSL.

Intended usage

COMMON

Author

W3C. See the Editors listed in WGSL.

Change controller

W3C

Normative References[\[WebGPU\]](https://w3.org/TR/webgpu) W3C, "WebGPU" W3C Working Draft, January 2023. <https://w3.org/TR/webgpu>

Webgpu Shading Language W3C, "WebGPU Shading Language" W3C Working Draft, January 2023.

<https://w3.org/TR/WGSL>§ **Conformance**§ **Document conventions**

Conformance requirements are expressed with a combination of descriptive assertions and RFC 2119 terminology. The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in the normative parts of this document are to be interpreted as described in RFC 2119. However, for readability, these words do not appear in all uppercase letters in this specification.

All of the text of this specification is normative except sections explicitly marked as non-normative, examples, and notes.

[\[RFC2119\]](#)

Examples in this specification are introduced with the words “for example” or are set apart from the normative text with `class="example"`, like this:

EXAMPLE 127

This is an example of an informative example.

Informative notes begin with the word “Note” and are set apart from the normative text with `class="note"`, like this:

Note, this is an informative note.

§ **Conformant Algorithms**

Requirements phrased in the imperative as part of algorithms (such as "strip any leading space characters" or "return false and abort these steps") are to be interpreted with the meaning of the key word ("must", "should", "may", etc) used in introducing the algorithm.

Conformance requirements phrased as algorithms or specific steps can be implemented in any manner, so long as the end result is equivalent. In particular, the algorithms defined in this specification are intended to be easy to understand and are not intended to be performant. Implementers are encouraged to optimize.

§ **Index**§ **Terms defined by this specification**[64-bit integer](#), in § 14.4.4[abstract](#), in § 6.2.1[AbstractFloat](#), in § 6.2.1[AbstractInt](#), in § 6.2.1[abstract numeric types](#), in § 6.2.1[AccessibleBytes](#), in § 14.4[AccessibleSlots](#), in § 14.4[access mode](#), in § 14.2[active](#), in § 15.5[additive_expression](#), in § 8.18

additive_operator	attribute	buffer_view , in § 4.1.2
dfn for recursive descent syntax , in § 18	definition of , in § 12	builtin , in § 12.4
dfn for syntax , in § 8.18	dfn for recursive descent syntax , in § 18	builtin_attr , in § 12.4
address-of , in § 8.13	dfn for syntax , in § 12	built-in functions , in § 17
address spaces , in § 14.3	bang , in § 16.3	builtin functions that compute a derivative , in § 15.6.2
affected range , in § 2.3.3	base , in § 8.5	built-in input value , in § 13.3.1.1
alias	behavior , in § 9.7.1	built-in output value , in § 13.3.1.1
definition of , in § 11.4.1.2	Behavioral requirements , in § 1.1	builtin_value_name
dfn for syntax_kw , in § 16.1	bgra8unorm , in § 6.6.1	dfn for recursive descent syntax , in § 18
align , in § 12.1	binary16 , in § 15.7.1	dfn for syntax , in § 3.8.2
align_attr , in § 12.1	binary32 , in § 15.7.1	built-in value name-token , in § 3.8.2
alignment , in § 14.4.1	binary64 , in § 15.7.1	built-in values , in § 13.3.1.1
AlignOf , in § 14.4	binary_and_expression , in § 8.18	byte-size , in § 14.4.1
AlignOfMember , in § 14.4	binary_or_expression , in § 8.18	called function , in § 11.2
AllTypes , in § 6.7	binary_xor_expression , in § 8.18	callee , in § 11.2
and , in § 16.3	binding , in § 12.2	caller , in § 11.2
and_and , in § 16.3	binding_attr , in § 12.2	call_expression , in § 8.18
and_equal , in § 16.3	bitwise_expression , in § 8.18	call_expression.post.ident , in § 18
Angles , in § 1.3	bitwise_expression.post.unary_expression , in § 18	calling function , in § 11.2
arg_i , in § 15.2.7	Blankspace , in § 3.2	call_phrase , in § 8.18
argument_expression_list	blend_src , in § 12.3	call site , in § 11.2
dfn for recursive descent syntax , in § 18	blend_src_attr , in § 12.3	CallSiteNoRestriction , in § 15.2.3
dfn for syntax , in § 8.18	block comment , in § 3.3	CallSiteRequiredToBeUniform.S , in § 15.2.3
array , in § 6.2.9	bool , in § 6.2.2	call site tag , in § 15.2.3
arrayed , in § 6.6	boolean literal , in § 3.5	case , in § 16.1
ArrayOffset , in § 17.14	bool_literal	case clause , in § 9.4.2
array size , in § 6.6	dfn for recursive descent syntax , in § 18	case_clause , in § 9.4.2
arrow , in § 16.3	dfn for syntax , in § 3.5.1	case_selector
assertion , in § 10	brace_left , in § 16.3	dfn for recursive descent syntax , in § 18
assert_statement , in § 10.1	brace_right , in § 16.3	dfn for syntax , in § 9.4.2
assignment , in § 9.2	bracket_left , in § 16.3	case_selectors , in § 9.4.2
assignment_statement , in § 9.2	bracket_right , in § 16.3	ceiling expression , in § 1.3
assignment_statement/0.1 , in § 18	break	center , in § 13.3.1.4
atomic modification , in § 6.2.8	dfn for statement , in § 9.4.6	centroid , in § 13.3.1.4
atomic type , in § 6.2.8	dfn for syntax_kw , in § 16.1	CF_start , in § 15.2.3
attr , in § 16.3	break-if , in § 9.4.7	channel format , in § 6.6.1
	break_if_statement , in § 9.4.7	Channel Formats , in § 6.6.1
	break_statement , in § 9.4.6	
	buffer , in § 6.3	

- [channels](#), in § 6.6.1
- [channel transfer function](#), in § 6.6.1
- [clip_distances](#)
 - [dfn for built-in values](#), in § 13.3.1.1.1
 - [dfn for extension](#), in § 4.1.1
- [colon](#), in § 16.3
- [comma](#), in § 16.3
- [comment](#), in § 3.3
- [component_or_swizzle_specifier](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 8.18
- [components](#), in § 6.2.11
- [component type](#), in § 6.2.6
- [component-wise](#), in § 6.2.6
- [composite](#), in § 6.2.11
- [compound assignment](#), in § 9.2.3
- [compound_assignment_operator](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 9.2.3
- [compound statement](#), in § 9.1
- [compound_statement](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 9.1
- [compute](#)
 - [definition of](#), in § 13.1
 - [dfn for attribute](#), in § 12.16.3
- [compute_attr](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 12.16.3
- [compute shader grid](#), in § 15.3
- [compute shader stage](#), in § 13.1
- [concrete](#), in § 6.2.1
- [concretization](#), in § 6.1.2
- [concretization of a value](#), in § 6.1.2
- [conflict](#), in § 2.3.3
- [const](#)
 - [dfn for attribute](#), in § 12.5
 - [dfn for syntax_kw](#), in § 16.1
- [const_assert](#)
 - [dfn for syntax](#), in § 10
 - [dfn for syntax_kw](#), in § 16.1
- [const_attr](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 12.5
- [const-declaration](#), in § 7.2.1
- [const-expressions](#), in § 8.1.1
- [const-functions](#), in § 11.3
- [constructible](#), in § 6.2.12
- [context-dependent name](#), in § 3.8
- [continue](#)
 - [dfn for statement](#), in § 9.4.8
 - [dfn for syntax_kw](#), in § 16.1
- [continue_statement](#), in § 9.4.8
- [continuing](#)
 - [dfn for statement](#), in § 9.4.9
 - [dfn for syntax_kw](#), in § 16.1
- [continuing_compound_statement](#), in § 9.4.9
- [continuing_statement](#), in § 9.4.9
- [control barrier](#), in § 15.6.1
- [ConversionRank](#), in § 6.1.2
- [core_lhs_expression](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 8.18
- [correctly rounded](#), in § 15.7.4
- [creation-fixed footprint](#), in § 6.2.13
- [decimal floating point literal](#), in § 3.5.2
- [decimal_float_literal](#), in § 3.5.2
- [decimal_int_literal](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 3.5.2
- [declaration](#), in § 5
- [decrement statement](#), in § 9.3
- [decrement_statement](#), in § 9.3
- [default](#), in § 16.1
- [default-alone clause](#), in § 9.4.2
- [default_alone_clause](#), in § 9.4.2
- [default clause](#), in § 9.4.2
- [default initial value](#), in § 7.3
- [Denormalized](#), in § 15.7.1
- [depth texture](#), in § 6.6.6
- [derivative_uniformity](#), in § 2.3.2
- [diagnostic](#)
 - [definition of](#), in § 2.3
 - [dfn for attribute](#), in § 12.6
 - [dfn for syntax_kw](#), in § 16.1
- [diagnostic_attr](#), in § 12.6
- [diagnostic_control](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 12.6
- [diagnostic_directive](#), in § 4.2
- [diagnostic filter](#), in § 2.3.3
- [diagnostic name-token](#), in § 3.8.3
- [diagnostic_name_token](#), in § 3.8.3
- [diagnostic_rule_name](#)
 - [dfn for recursive descent syntax](#), in § 18
 - [dfn for syntax](#), in § 2.3
- [dimensionality](#), in § 6.6
- [directive](#), in § 4
- [_disambiguate_template](#), in § 16.3
- [discard](#)
 - [dfn for statement](#), in § 9.4.11
 - [dfn for syntax_kw](#), in § 16.1
- [dispatch command](#), in § 1.1
- [dispatch size](#), in § 15.3
- [Division by zero](#), in § 15.7.1
- [division_equal](#), in § 16.3
- [domain](#), in § 15.7.1
- [draw command](#), in § 1.1
- [Draw uniformity scope](#), in § 15.2.1
- [dual_source_blending](#), in § 4.1.1
- [dynamic context](#), in § 6.1
- [dynamic error](#), in § 2.2
- [dynamic statement instance](#), in § 15.1
- [effective-value-type](#), in § 7
- [either](#), in § 13.3.1.4
- [element count](#), in § 6.2.9

[element stride](#), in § 14.4[else](#), in § 16.1[else_clause](#), in § 9.4.1[else_if_clause](#), in § 9.4.1[enable](#), in § 16.1[enable_directive](#), in § 4.1.1[enable_directive](#), in § 4.1.1[enable-extension](#), in § 4.1.1[enable_extension_list](#), in § 4.1.1[enable_extension_name](#), in § 3.8.5[entry point](#), in § 13[enumerant](#), in § 6.4[enumeration](#), in § 6.4[equal](#), in § 16.3[equal_equal](#), in § 16.3[error](#), in § 2.3[exceptions](#), in § 15.7.1[execution scope](#), in § 14.5.3[exponent bias](#), in § 15.7.1[exponent field](#), in § 15.7.1[expression](#)[definition of](#), in § 6.1[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 8.18[expression_comma_list](#), in § 8.18[extended real](#), in § 1.3[extension](#), in § 4.1[External texture](#), in § 6.6.4[f16](#)[definition of](#), in § 6.2.4[dfn for extension](#), in § 4.1.1[f32](#), in § 6.2.4[false](#), in § 16.1[feasible automatic conversion](#), in § 6.1.2[Finite Math Assumption](#), in § 15.7.2[finite range](#), in § 15.7.1[first](#), in § 13.3.1.4[fixed footprint](#), in § 6.2.13[fixed-size array](#), in § 6.2.9[fixed-size buffer](#), in § 6.3[flat](#), in § 13.3.1.4[Floating point interpretation of bits](#), in § 15.7.1[floating point literal](#), in § 3.5.2[float_literal](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 3.5.2[floor expression](#), in § 1.3[flush to zero](#), in § 15.7.2[fn](#), in § 16.1[for](#)[dfn for statement](#), in § 9.4.4[dfn for syntax kw](#), in § 16.1[for_header](#), in § 9.4.4[for_init](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 9.4.4[formal parameter](#), in § 11.1[for_statement](#), in § 9.4.4[for_update](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 9.4.4[forward_slash](#), in § 16.3[frag_depth](#), in § 13.3.1.1.2[fragment](#)[definition of](#), in § 13.1[dfn for attribute](#), in § 12.16.2[fragment_attr](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 12.16.2[fragment shader stage](#), in § 13.1[front_facing](#), in § 13.3.1.1.3[full assignment](#), in § 15.2.5[full pointer](#), in § 15.2.5[full reference](#), in § 15.2.5[fully elaborated](#), in § 6.1[func_call_statement](#), in § 9.5[function](#)[dfn for address spaces](#), in § 14.3[dfn for function](#), in § 11[function body](#), in § 11.1[function call](#), in § 11.2[function_decl](#), in § 11.1[function declaration](#), in § 11.1[function_header](#), in § 11.1[function scope](#), in § 7[functions in a shader stage](#), in § 13.2[function tag](#), in § 15.2.3[global_assert](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 10[global_decl](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 2[global diagnostic filter](#), in § 4.2[global_directive](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 4[global invocation ID](#), in § 15.3[global_invocation_id](#), in § 13.3.1.1.4[global invocation index](#), in § 15.3[global_invocation_index](#), in § 13.3.1.1.5[global_value_decl](#)[dfn for recursive descent syntax](#), in § 18[dfn for syntax](#), in § 7.4[global_variable_decl](#), in § 7.4[GPUComputePipeline](#), in § 13.1[GPURenderPipeline](#), in § 13.1[_greater_than](#), in § 16.3[greater_than](#), in § 16.3[_greater_than_equal](#), in § 16.3[greater_than_equal](#), in § 16.3[group](#), in § 12.7[group_attr](#), in § 12.7[group_count_x](#), in § 15.3

[group_count_y](#), in § 15.3
[group_count_z](#), in § 15.3
[handle](#), in § 14.3
[helper invocation](#), in § 15.4
[hexadecimal floating point literal](#), in § 3.5.2
[hex_float_literal](#), in § 3.5.2
[hex_int_literal](#), in § 3.5.2
[host-shareable](#), in § 6.5.2
[hyperbolic angle](#), in § 1.3
[i32](#), in § 6.2.3
[id](#), in § 12.8
[id_attr](#), in § 12.8
[ident](#)
 [dfn for recursive descent syntax](#), in § 18
 [dfn for syntax](#), in § 3.7
[identifier](#), in § 3.7
[ident_pattern_token](#), in § 3.7
[if](#)
 [dfn for statement](#), in § 9.4.1
 [dfn for syntax_kw](#), in § 16.1
[if_clause](#), in § 9.4.1
[if_statement](#), in § 9.4.1
[immediate](#), in § 14.3
[immediate_address_space](#), in § 4.1.2
[immediate data](#), in § 7.3
[implied from linear terms](#), in § 15.7.7
[inactive](#), in § 15.5
[in-bounds index](#), in § 8.5
[increment statement](#), in § 9.3
[increment_statement](#), in § 9.3
[indeterminate value](#), in § 8.2
[indexing expression](#), in § 8.5
[indirection](#), in § 8.14
[Inexact](#), in § 15.7.1
[info](#), in § 2.3
[inherited from](#), in § 15.7.4
[in scope](#), in § 5
[instance_index](#), in § 13.3.1.1.6
[integer literal](#), in § 3.5.2
[integer scalar](#), in § 6.2.5
[interface of a shader](#), in § 13.3
[interior nodes](#), in § 15.2.3
[intermediate result](#), in § 15.7.1
[interpolate](#), in § 12.9
[interpolate_attr](#)
 [dfn for recursive descent syntax](#), in § 18
 [dfn for syntax](#), in § 12.9
[interpolate_sampling_name](#), in § 3.8.7
[interpolate_type_name](#), in § 12.9
[interpolation sampling](#), in § 13.3.1.4
[interpolation sampling name-token](#), in § 3.8.7
[interpolation type](#), in § 13.3.1.4
[interpolation type name-token](#), in § 3.8.6
[interval](#), in § 1.3
[int_literal](#)
 [dfn for recursive descent syntax](#), in § 18
 [dfn for syntax](#), in § 3.5.2
[Invalid Load](#), in § 6.5.6
[invalid memory reference](#), in § 6.5.4
[Invalid operation](#), in § 15.7.1
[invalid pointer](#), in § 6.5.4
[Invalid Store](#), in § 6.5.6
[invariant](#), in § 12.10
[invariant_attr](#)
 [dfn for recursive descent syntax](#), in § 18
 [dfn for syntax](#), in § 12.10
[inverse channel transfer function](#), in § 6.6.1
[invocations](#), in § 1.1
[IO attributes](#), in § 13.3.1
[iteration](#), in § 9.4.3
[keyword](#), in § 3.6
[language extension](#), in § 4.1.2
[language_extension_list](#), in § 4.1.2
[language_extension_name](#), in § 3.8.5
[layout attributes](#), in § 14.4.2
[left-hand side](#), in § 9.2
[_less_than](#), in § 16.3
[less_than](#), in § 16.3
[_less_than_equal](#), in § 16.3
[less_than_equal](#), in § 16.3
[let](#), in § 16.1
[let-declaration](#), in § 7.2.3
[lhs_expression](#)
 [dfn for recursive descent syntax](#), in § 18
 [dfn for syntax](#), in § 8.18
[LHSValue](#), in § 15.2.6
[lifetime](#), in § 7.3
[linear](#), in § 13.3.1.4
[linear_indexing](#), in § 4.1.2
[line break](#), in § 3.2
[line-ending comment](#), in § 3.3
[literal](#)
 [definition of](#), in § 3.5
 [dfn for recursive descent syntax](#), in § 18
 [dfn for syntax](#), in § 3.5
[Load Rule](#), in § 6.5.7
[local invocation ID](#), in § 15.3
[local_invocation_id](#), in § 13.3.1.1.7
[local invocation index](#), in § 15.3
[local_invocation_index](#), in § 13.3.1.1.8
[location](#), in § 12.11
[location_attr](#), in § 12.11
[logical texel address](#), in § 6.6
[loop](#)
 [dfn for statement](#), in § 9.4.3
 [dfn for syntax_kw](#), in § 16.1
[loop body](#), in § 9.4.3
[loop_statement](#), in § 9.4.3
[mat2x2f](#), in § 6.2.7
[mat2x2h](#), in § 6.2.7
[mat2x3f](#), in § 6.2.7
[mat2x3h](#), in § 6.2.7
[mat2x4f](#), in § 6.2.7

mat2x4h , in § 6.2.7	must_use , in § 12.12	overload candidates , in § 6.1.3
mat3x2f , in § 6.2.7	must_use_attr	overload resolution , in § 6.1.3
mat3x2h , in § 6.2.7	dfn for recursive descent syntax , in § 18	override , in § 16.1
mat3x3f , in § 6.2.7	dfn for syntax , in § 12.12	override-declaration , in § 7.2.2
mat3x3h , in § 6.2.7	name , in § 5	override-expressions , in § 8.1.2
mat3x4f , in § 6.2.7	named component expression , in § 8.5	packed_4x8_integer_dot_product , in § 4.1.2
mat3x4h , in § 6.2.7	NaN , in § 15.7.1	param
mat4x2f , in § 6.2.7	nearest enclosing diagnostic filter , in § 2.3.3	dfn for recursive descent syntax , in § 18
mat4x2h , in § 6.2.7	Negative infinity , in § 1.3	dfn for syntax , in § 11.1
mat4x3f , in § 6.2.7	negative zero , in § 15.7.1	ParameterContentsRequiredToBeUniform.S , in § 15.2.3
mat4x3h , in § 6.2.7	nesting depth , in § 6.2.11	parameterized , in § 6.1
mat4x4f , in § 6.2.7	NoRestriction , in § 15.2.3	ParameterNoRestriction , in § 15.2.3
mat4x4h , in § 6.2.7	normal , in § 15.7.1	ParameterRequiredToBeUniform.S , in § 15.2.3
matrix , in § 6.2.7	not_equal , in § 16.3	ParameterReturnContentsRequiredToBeUniform , in § 15.2.3
MaybeNonUniform , in § 15.2.3	NRuntime , in § 13.3.4	ParameterReturnNoRestriction , in § 15.2.3
member , in § 6.2.10	numeric literal , in § 3.5	parameter return tag , in § 15.2.3
member_ident	numeric scalar , in § 6.2.5	parameter tag , in § 15.2.3
dfn for recursive descent syntax , in § 18	numeric scalar conversion to floating point , in § 15.7.6	param_i , in § 15.2.3
dfn for syntax , in § 3.7	numeric vector , in § 6.2.6	param_i_contents , in § 15.2.3
memory access , in § 14.2	num_subgroups , in § 13.3.1.1.20	param_list , in § 11.1
memory footprint , in § 6.2.13	num_workgroups , in § 13.3.1.1.9	paren_expression , in § 8.18
memory layout , in § 14.4	off , in § 2.3	paren_left , in § 16.3
memory locations , in § 14.1	OffsetOfMember , in § 14.4	paren_right , in § 16.3
memory scope , in § 14.5.3	opaque , in § 6	partial assignment , in § 15.2.5
memory view , in § 6.5	optionally_typed_ident	partial derivative , in § 15.6.2
MinTypeSize , in § 17.14	dfn for recursive descent syntax , in § 18	partial reference , in § 15.2.5
minus , in § 16.3	dfn for syntax , in § 7.4	period , in § 16.3
minus_equal , in § 16.3	or , in § 16.3	perspective , in § 13.3.1.4
minus_minus , in § 16.3	or_equal , in § 16.3	phony assignment , in § 9.2.2
Mip level , in § 6.6	originating variable , in § 6.5.5	pipeline , in § 13.1
mip level count , in § 6.6	or_or , in § 16.3	pipeline constant ID , in § 7.2.2
module scope , in § 5	out-of-bounds access , in § 6.5.6	Pipeline creation , in § 2.1
modulo , in § 16.3	out-of-bounds index , in § 8.5	pipeline-creation error , in § 2.2
modulo_equal , in § 16.3	Overflow , in § 15.7.1	pipeline-overridable , in § 7.2.2
multiplicative_expression , in § 8.18	overlap , in § 14.1	
multiplicative_operator	overload , in § 6.1	
dfn for recursive descent syntax , in § 18		
dfn for syntax , in § 8.18		
multisampled texture , in § 6.6.3		

pipeline-overridable constant default value , in § 7.2.2	r8uint , in § 6.6.1	ReturnValueMayBeNonUniform , in § 15.2.3
pipeline-overridable constant identifier string , in § 7.2.2	r8unorm , in § 6.6.1	rg11b10float , in § 6.6.1
plain type , in § 6.2	range diagnostic filter , in § 2.3.3	rg16float , in § 6.6.1
plus , in § 16.3	rasterization fragments , in § 13.1	rg16sint , in § 6.6.1
plus_equal , in § 16.3	read , in § 14.2	rg16snorm , in § 6.6.1
plus_plus , in § 16.3	read access , in § 14.2	rg16uint , in § 6.6.1
pointer_composite_access , in § 4.1.2	readonly_and_readwrite_storage_textures , in § 4.1.2	rg16unorm , in § 6.6.1
PointerParameterMayBeNonUniform , in § 15.2.3	read-only storage texture , in § 6.6.5	rg32float , in § 6.6.1
PointerParameterNoRestriction , in § 15.2.3	read_write , in § 14.2	rg32sint , in § 6.6.1
pointer parameter tag , in § 15.2.3	read-write storage texture , in § 6.6.5	rg32uint , in § 6.6.1
pointer type , in § 6.5.3	Reassociation , in § 15.7.5	rg8sint , in § 6.6.1
position , in § 13.3.1.1.10	reference type , in § 6.5.3	rg8snorm , in § 6.6.1
Positive infinity , in § 1.3	relational_expression , in § 8.18	rg8uint , in § 6.6.1
potential-trigger-set , in § 15.2.2	relational_expression.post.unary_expression , in § 18	rg8unorm , in § 6.6.1
predeclared , in § 5	RequiredAlignOf , in § 14.4.5	rgb10a2uint , in § 6.6.1
preferable candidate , in § 6.1.3	RequiredToBeUniform.error , in § 15.2.3	rgb10a2unorm , in § 6.6.1
primary_expression	RequiredToBeUniform.info , in § 15.2.3	rgba16float , in § 6.6.1
dfn for recursive descent syntax , in § 18	RequiredToBeUniform.S , in § 15.2.3	rgba16sint , in § 6.6.1
dfn for syntax , in § 8.18	RequiredToBeUniform.warning , in § 15.2.3	rgba16snorm , in § 6.6.1
primitive_index	requires , in § 16.1	rgba16uint , in § 6.6.1
dfn for built-in values , in § 13.3.1.1.11	requires-directive , in § 4.1.2	rgba16unorm , in § 6.6.1
dfn for extension , in § 4.1.1	requires_directive , in § 4.1.2	rgba32float , in § 6.6.1
private , in § 14.3	_reserved , in § 16.2	rgba32sint , in § 6.6.1
program_error , in § 2.2	reserved word , in § 16.2	rgba32uint , in § 6.6.1
quad , in § 15.4	resolves , in § 5	rgba8sint , in § 6.6.1
quad invocation ID , in § 15.4	resource , in § 13.3.2	rgba8snorm , in § 6.6.1
r16float , in § 6.6.1	resource interface of a shader , in § 13.3.2	rgba8uint , in § 6.6.1
r16sint , in § 6.6.1	Result , in § 15.2.7	rgba8unorm , in § 6.6.1
r16snorm , in § 6.6.1	return	RHSValue , in § 15.2.6
r16uint , in § 6.6.1	dfn for statement , in § 9.4.10	right-hand side , in § 9.2
r16unorm , in § 6.6.1	dfn for syntax_kw , in § 16.1	root identifier , in § 11.4.1.1
r32float , in § 6.6.1	returns , in § 11.2	roundDown , in § 1.3
r32sint , in § 6.6.1	return_statement , in § 9.4.10	Rounding , in § 15.7.1
r32uint , in § 6.6.1	return type , in § 11.1	rounding mode , in § 15.7.1
r8sint , in § 6.6.1	return value , in § 9.4.10	roundUp , in § 1.3
r8snorm , in § 6.6.1		runtime expression , in § 8
		runtime-sized , in § 6.2.9
		runtime-sized buffer , in § 6.3

sample , in § 13.3.1.4	shift_right_assign , in § 16.3	subgroup_id dfn for built-in values , in § 13.3.1.1.19 dfn for language extension , in § 4.1.2
sample count , in § 6.6	short_circuit_and_expression , in § 8.18	subgroup invocation ID , in § 15.5
sampled texture , in § 6.6.2	short_circuit_or_expression , in § 8.18	subgroup_invocation_id , in § 13.3.1.1.17
sampled type , in § 6.6.2	sign field , in § 15.7.1	subgroups , in § 4.1.1
sample_index , in § 13.3.1.1.12	significant , in § 3.5.2	subgroup_size , in § 15.5
sample_mask , in § 13.3.1.1.13	simple assignment , in § 9.2.1	subgroup_size dfn for attribute , in § 12.14 dfn for built-in values , in § 13.3.1.1.18
sampler definition of , in § 6.6.7 dfn for type , in § 6.6.7	singular_expression , in § 8.18	subgroup_size_attr , in § 12.14
sampler_comparison , in § 6.6.7	size dfn for attribute , in § 12.13 dfn for texture , in § 6.6	subgroup_size_control , in § 4.1.1
sampler_resource , in § 7.3	size_attr , in § 12.13	subgroup_uniformity dfn for language extension , in § 4.1.2 dfn for trigger , in § 2.3.2
sampler types , in § 6.6.7	SizeOf , in § 14.4	Subgroup uniformity scope , in § 15.2.1
scalar , in § 6.2.5	SizeOfMember , in § 14.4	subnormal , in § 15.7.1
scalar conversion , in § 6.2.5	star , in § 16.3	substitution , in § 6.1
scalar floating point to integral conversion , in § 15.7.6	statement definition of , in § 9 dfn for recursive descent syntax , in § 18 dfn for syntax , in § 9.6	switch dfn for statement , in § 9.4.2 dfn for syntax_kw , in § 16.1
scope , in § 5	statically accessed , in § 13.3	switch_body , in § 9.4.2
semantics of an expression , in § 6.1.1	statically typed language , in § 6.1	switch_clause dfn for recursive descent syntax , in § 18 dfn for syntax , in § 9.4.2
semicolon , in § 16.3	static context , in § 6.1	switch_statement , in § 9.4.2
severity , in § 2.3	static type , in § 6.1	swizzle , in § 8.5.1
severity_control_name , in § 3.8.4	storable , in § 6.5.1	swizzle_name dfn for recursive descent syntax , in § 18 dfn for syntax , in § 3.8.8
shader , in § 1.1	storage , in § 14.3	syntactic phrase , in § 6.1
shader-creation error , in § 2.2	storage buffer , in § 7.3	syntactic token , in § 16.3
Shader execution end , in § 2.1	storage-textel-formats , in § 6.6.1	template_arg_comma_list , in § 3.9
Shader execution start , in § 2.1	storage texture , in § 6.6.5	template_arg_expression dfn for recursive descent syntax , in § 18 dfn for syntax , in § 3.9
Shader module creation , in § 2.1	store type , in § 6.5	_template_args_end , in § 16.3
shader stage attributes , in § 12.16	StrideOf , in § 14.4	
shader stage input , in § 13.3.1	struct , in § 16.1	
shader stage output , in § 13.3.1	struct_body_decl , in § 6.2.10	
shader stages , in § 13.1	struct_decl , in § 6.2.10	
shift_expression , in § 8.18	struct_member , in § 6.2.10	
shift_expression.post.unary_expression , in § 18	structure , in § 6.2.10	
_shift_left , in § 16.3	subexpressions , in § 6.1	
shift_left , in § 16.3	subgroup , in § 15.5	
_shift_left_assign , in § 16.3	subgroup ID , in § 15.5	
shift_left_assign , in § 16.3		
_shift_right , in § 16.3		
shift_right , in § 16.3		
_shift_right_assign , in § 16.3		

_template_args_start , in § 16.3	top-level expression , in § 6.1	uniform buffer , in § 7.3
template_elaborated_ident , in § 6.9	trailing significand field , in § 15.7.1	uniform_buffer_standard_layout , in § 4.1.2
template_elaborated_ident.post.ident , in § 18	translation_unit dfn for recursive descent syntax , in § 18 dfn for syntax , in § 2	uniform control flow , in § 15.2.1
template_list , in § 3.9	translation_unit/0.1/0/0.0 , in § 18	uniformity analysis , in § 15.2
template_list , in § 3.9	transpose , in § 1.3	uniformity failure , in § 15.2
Template list discovery , in § 3.9	Trap , in § 6.5.6	uniformity scope , in § 15.2.1
Template parameterization , in § 3.9	triggered , in § 2.3	uniform value , in § 15.2.1
template_parameters , in § 3.9	triggering location , in § 2.3	uniform variable , in § 15.2.1
texel , in § 6.6	triggering rule , in § 2.3	unrestricted_pointer_parameters , in § 4.1.2
texel format , in § 6.6.1	true , in § 16.1	use dual source blending , in § 13.3.1.3
texture , in § 6.6	truncate , in § 1.3	user-defined function , in § 11.1
texture_1d , in § 6.6.2	type , in § 6	user-defined input datum , in § 13.3.1.2
texture_2d , in § 6.6.2	type alias , in § 6.8	user-defined output datum , in § 13.3.1.2
texture_2d_array , in § 6.6.2	type alias_decl , in § 6.8	valid pointer , in § 6.5.4
texture_3d , in § 6.6.2	type assertion , in § 6.1	valid reference , in § 6.5.4
texture_and_sampler_let , in § 4.1.2	Type checking , in § 6.1	value constructor , in § 17.1
texture_cube , in § 6.6.2	type error , in § 6.1	value declaration , in § 7
texture_cube_array , in § 6.6.2	type-generator , in § 6	Value_return , in § 15.2.3
texture_depth_2d , in § 6.6.6	type parameters , in § 6.1	Value_return_i_contents , in § 15.2.3
texture_depth_2d_array , in § 6.6.6	type rule , in § 6.1	var , in § 16.1
texture_depth_cube , in § 6.6.6	type rule applies to a syntactic phrase , in § 6.1	variable , in § 7.3
texture_depth_cube_array , in § 6.6.6	type rule conclusion , in § 6.1	variable_decl dfn for recursive descent syntax , in § 18 dfn for syntax , in § 7.4
texture_depth_multisampled_2d , in § 6.6.3	type rule preconditions , in § 6.1	variable_declaration , in § 7
texture_external , in § 6.6.4	type rule tables , in § 6.1.1	variable_or_value_statement dfn for recursive descent syntax , in § 18 dfn for syntax , in § 7.4
texture_formats_tier1 , in § 4.1.2	type_specifier dfn for recursive descent syntax , in § 18 dfn for syntax , in § 6.9	variable_updating_statement dfn for recursive descent syntax , in § 18 dfn for syntax , in § 9.6
texture_gather , in § 17.7.2	u32 , in § 6.2.3	vec2f , in § 6.2.6
texture_gather_compare , in § 17.7.3	ULP , in § 15.7.4	vec2h , in § 6.2.6
texture_multisampled_2d , in § 6.6.3	unary_expression dfn for recursive descent syntax , in § 18 dfn for syntax , in § 8.18	vec2i , in § 6.2.6
texture_resource , in § 7.3	uncategorized error , in § 2.2	vec2u , in § 6.2.6
texture_storage_1d , in § 6.6.5	Underflow , in § 15.7.1	
texture_storage_2d , in § 6.6.5	underscore , in § 16.3	
texture_storage_2d_array , in § 6.6.5	uniform , in § 14.3	
texture_storage_3d , in § 6.6.5		
texture_types , in § 6.6		
tilde , in § 16.3		
times_equal , in § 16.3		
token , in § 3.4		
token candidate , in § 3.1		

vec3f , in § 6.2.6	vertex_index , in § 13.3.1.1.14	workgroup_index , in § 13.3.1.1.16
vec3h , in § 6.2.6	vertex_shader_stage , in § 13.1	workgroup_size , in § 12.15
vec3i , in § 6.2.6	warning , in § 2.3	<code>workgroup_size_attr</code>
vec3u , in § 6.2.6	well-typed , in § 6.1	dfn for recursive descent syntax , in § 18
vec4f , in § 6.2.6	<code>while</code>	dfn for syntax , in § 12.15
vec4h , in § 6.2.6	dfn for statement , in § 9.4.5	Workgroup uniformity scope , in § 15.2.1
vec4i , in § 6.2.6	dfn for syntax kw , in § 16.1	write , in § 14.2
vec4u , in § 6.2.6	while_statement , in § 9.4.5	write access , in § 14.2
vector , in § 6.2.6	<code>workgroup</code>	write-only storage texture , in § 6.6.5
<code>vertex</code>	dfn for address spaces , in § 14.3	xor , in § 16.3
definition of , in § 13.1	dfn for compute shader stage , in § 15.3	xor_equal , in § 16.3
dfn for attribute , in § 12.16.1	workgroup_grid , in § 15.3	zero value , in § 17.1.1
<code>vertex_attr</code>	workgroup_ID , in § 15.3	
dfn for recursive descent syntax , in § 18	workgroup_id , in § 13.3.1.1.15	
dfn for syntax , in § 12.16.1	workgroup_index , in § 15.3	

§ Terms defined by reference

[UAX14] defines the following terms:

UAX14 LB4

UAX14 LB5

UAX14 Section 6.1 Non-tailorable
Line Breaking Rules

[UAX31] defines the following terms:

UAX31 Grammar

UAX31 Lexical Classes

Unicode Standard Annex #31 for
Unicode Version 14.0.0

[UNICODE CHARACTER DATABASE FOR UNICODE VERSION 14.0.0]

defines the following terms:

Unicode Character Database for
Unicode Version 14.0.0

[UnicodeVersion14] defines the
following terms:

code point

[VULKAN] defines the following
terms:

memory model atomic operation
memory model memory operation
memory model memory semantics
memory model non-private
memory model reference
memory model scope

[WebGPU] defines the following

terms:

"1d"

"2d"

"2d-array"

"3d"

"comparison"

"cube"

"cube-array"

"depth"

"filtering"

"float"

"non-filtering"

"primitive-index"

"read-only"

"read-only-storage"

"read-write"

"sint"

"storage"

"subgroup-size-control"

"uint"

"unfilterable-float"

"uniform"

"write-only"

GPU

GPUAddressMode

GPUBuffer

GPUBufferBindingType

GPUCompareFunction

GPUCompilationInfo

GPUComputePassEncoder

GPUDevice

GPUFeatureName

GPUFilterMode

GPUProgrammableStage

GPUSampler

GPUSamplerBindingType

GPUStorageTextureAccess

GPUTexture

GPUTextureBindingLayout

GPUTextureFormat

GPUTextureSampleType

GPUTextureViewDimension

Interstage interface validation

STORAGE_BINDING

[[viewport]]

binding member

binding type

buffer

clip position

clip space coordinates

count

createBindGroupLayout(descriptor)

createComputePipeline(descriptor)

createRenderPipeline(descriptor)

createShaderModule(descriptor)

device loss

effective buffer binding size

externalTexture

format

fragmentdestination-position

framebuffer

front-facing

get as buffer binding

GPU command

immediateSize

maxComputeWorkgroupStorageSize

maxImmediateSize

messages

normalized device coordinates

primitive restart value

RasterizationPoint

rasterizationpoint-coveragemask

rasterizationpoint-depth

rasterizationpoint-destination

rasterizationpoint-perspectivedivisor

resource

sampleType

sampler

setImmediates

shader-output mask

storageTexture

subgroupMaxSize

subgroupMinSize

subgroupMaxSize

subgroupMinSize

subgroups-feature

texture

texture view swizzle

validating GPUProgrammableStage

validating shader binding

wgslLanguageFeatures

§ References

§ Normative References

[DeRemer1969]

Practical Translators for LR(k) Languages. 24 October 1969. URL: <http://publications.csail.mit.edu/lcs/pubs/pdf/MIT-LCS-TR-065.pdf>

[ECMAScript]

ECMAScript Language Specification. URL: <https://tc39.es/ecma262/multipage/>

[IEEE-754]

IEEE Standard for Floating-Point Arithmetic. 29 August 2008. URL: <http://ieeexplore.ieee.org/servlet/opac?punumber=4610933>

[Muller2005]

On the definition of ulp(x). February 2005. URL: <https://inria.hal.science/inria-00070503>

[RFC2119]

S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. March 1997. Best Current Practice. URL: <https://datatracker.ietf.org/doc/html/rfc2119>

[UAX14]

Robin Leroy. *Unicode Line Breaking Algorithm*. 5 September 2025. Unicode Standard Annex #14. URL: <https://www.unicode.org/reports/tr14/tr14-55.html>

[UAX31]

Mark Davis; Robin Leroy. *Unicode Identifiers and Syntax*. 20 August 2025. Unicode Standard Annex #31. URL: <https://www.unicode.org/reports/tr31/tr31-43.html>

[UnicodeVersion14]

The Unicode Standard, Version 14.0.0. URL: <http://www.unicode.org/versions/Unicode14.0.0/>

[VanWyk2007]

Eric R. Van Wyk; August C. Schwerdfeger. *Context-Aware Scanning for Parsing Extensible Languages*. 2007. URL: <https://dl.acm.org/doi/10.1145/1289971.1289983>

[VulkanMemoryModel]

Jeff Bolz; et al. *Vulkan Memory Model*. URL: <https://www.khronos.org/registry/vulkan/specs/1.2-extensions/html/vkspec.html#memory-model>

[WebGPU]

Kai Ninomiya; Brandon Jones; Myles C. Maxfield. *WebGPU*. Working Draft. URL: <https://w3.org/TR/webgpu>

§ Non-Normative References

[CHARMOD-NORM]

Addison Phillips; et al. *Character Model for the World Wide Web: String Matching*. 16 July 2026. FPWD. URL: <https://www.w3.org/TR/charmod-norm/>

[IANA-MEDIA-TYPES]

Media Types. URL: <https://www.iana.org/assignments/media-types/>

[Jeannerod2013]

Claude-Pierre Jeannerod; Nicolas Louvet; Jean-Michel Muller. *Further Analysis of Kahan's Algorithm for the Accurate Computation of 2x2 Determinants*. URL: <https://www.ams.org/journals/mcom/2013-82-284/S0025-5718-2013-02679-8/S0025-5718-2013-02679-8.pdf>

[WASM-CORE-2]

Andreas Rossberg. *WebAssembly Core Specification*. 27 May 2026. CRD. URL: <https://www.w3.org/TR/wasm-core-2/>